CPE342

Machine Learning

Study Book

หนังสืออ่านสอบฉบับอธิบายละเอียด

Midterm รวม Lecture 1–8

คำอธิบายแนวคิดและที่มาของสูตร พร้อมตัวอย่างคำนวณ

สรุปสิ่งที่ต้องจำ พร้อมจุดสับสนและคำถามทบทวน

Lecture 1–8 พร้อมคำอธิบายเสริมจากตำรา

An Introduction to Statistical Learning with Applications in Python (ISLP)

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

11 ตุลาคม 2026

ชุดแนวข้อสอบและเฉลยแยกเป็นอีกไฟล์

เนื้อหาเพิ่มเติมและแนวข้อสอบเป็นเอกสารฝึกที่เรียบเรียงใหม่ ไม่ใช่ข้อสอบจริงหรือประกาศสอบใหม่ของอาจารย์

แนวทางในการอ่านเนื้อหาเรียนไฟล์นี้และแนวทางข้อสอบ

ขอบเขตและรูปแบบสอบที่ได้รับแจ้ง: อ่าน Lecture 1–8. ข้อสอบมี 3 ส่วน ได้แก่ True/False 40 ข้อ, multiple choice 12 ข้อ และ short answer 5 ข้อ รวม 40 คะแนน ใช้เวลา 3 ชั่วโมง ไม่มีหักคะแนนเมื่อตอบผิด ไม่มีสูตรหรือสมการให้ในข้อสอบ และอนุญาต cheat sheet ที่เขียนด้วยมือ 1 แผ่น. คะแนนย่อยของแต่ละส่วนไม่ได้ระบุ จึงอย่าใช้สัดส่วนคะแนนใน Mock เป็นสัดส่วนทางการ

แนวทางอ่านสูตรให้เข้าใจ

1. เริ่มจากโจทย์: ระบุ input, target, งานที่ต้องทำ และปริมาณที่โมเดลต้องทำนาย
2. อ่านสมการ: บอกความหมายและขนาดของตัวแปร เงื่อนไข และที่มาของสูตรก่อนแทนค่า
3. ทำตามขั้น: เขียนค่าที่แทนลงทีละบรรทัด คำนวณใหม่เมื่อพารามิเตอร์เปลี่ยน แล้วแปลคำตอบกลับเป็นภาษาของโจทย์
4. ทวนความเข้าใจ: อธิบายว่าเมื่อเปลี่ยนข้อมูลหรือค่าตั้งของโมเดล ผลใดควรเปลี่ยนและมีข้อจำกัดอะไร

แผนที่เนื้อหาและสิ่งที่ต้องอธิบายได้

Lecture	เนื้อหาหลักที่ควรอธิบายหรือคำนวณได้
1	T, P, E; experience equation; Bayes; LDA/QDA; confusion matrix และ metrics
2	OLS, normal equation, Cramer, gradient descent, learning rate และ logistic loss
3	functional margin กับ geometric distance, slack, C, KKT และ kernel
4	L1/L2, interaction, Poisson regression, censoring และ Kaplan–Meier
5	entropy, information gain แบบถ่วง child, Gini, bootstrap/OOB, Random Forest และ boosting
6	forward pass, backpropagation, activation/loss, optimizer และ regularization
7	convolution, output shape, parameter count, pooling, receptive field และ transfer learning
8	centered PCA, covariance, eigenvectors, score, PVE, reconstruction, MDS, LLE และ ISOMAP

ใช้สีและเลขหน้าอ้างอิง

แหล่งเนื้อหา: L1–L8 คือ lecture; p คือหน้า PDF ต้นฉบับ ไม่ใช่เลขหน้าที่พิมพ์บนสไลด์

อ่านเพิ่มจากตำรา อธิบายเสริมจาก textbook หรือขยายเหตุผลของสไลด์

ฝึกให้ทำได้ ใช้ฝึกคิดตามเงื่อนไข ไม่ใช่คำยืนยันว่าข้อสอบจริงจะถามเหมือนกัน

> **ต้องจำ** สรุปสิ่งที่ต้องจำหลังจากเข้าใจเหตุผลและตัวอย่าง ไม่ควรท่องกล่องสูตรโดยข้ามเงื่อนไข

Mock แยกเป็นอีกไฟล์และมีเวลา 180 นาที. เวลาแนะนำใน Mock เป็นเพียงวิธีแบ่งเวลาฝึก ไม่ใช่เวลาทางการของแต่ละส่วน

สารบัญ

แนวทางในการอ่านเนื้อหาเรียนไฟล์นี้และแนวทางข้อสอบ i

1 Introduction to ML & Statistical Learning 1

2 Training Models 21

3 Support Vector Machine 35

4 Regression, Shrinkage & Survival 47

5 Tree-Based & Ensemble Models 63

6 Neural Network Basics 80

7 Convolutional Neural Networks 95

8 Dimensionality Reduction 111

เมื่อเงื่อนไขเปลี่ยน ให้คาดผลของโมเดลอย่างไร 127

แผนที่ตัวแปรและเงื่อนไขของสมการ 128

ดัชนีเทียบต้นฉบับและหัวข้ออ่าน 132

แผนที่สมการสำคัญ Lecture 1–2

หน้านี้ใช้ทวนสูตรหลังจากอ่านที่มาและตัวอย่างในบทแล้ว จุดที่ต้องจำให้ครบมีทั้งความหมายของตัวแปร เงื่อนไข และเหตุผลที่เลือกสูตร

Lecture 1: Task, Performance, Experience

Task ระบุสิ่งที่ต้องทำ เช่น ทำนาย y จาก x; Performance คือเกณฑ์วัดผล; Experience คือข้อมูลฝึกที่ใช้เลือกพารามิเตอร์ w.

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(x_i^\top w - y_i)^2 = \frac{1}{m}\|Xw - y\|_2^2$$

ให้ $r = Xw - y$ เป็น residual. อนุพันธ์ของกำลังสองให้ตัวคูณ 2 และการคูณผ่านเมทริกซ์ให้ $X^\top$:

$$\nabla_w J(w) = \frac{2}{m}X^\top(Xw - y)$$

Gradient Descent (GD) ใช้ gradient ณ ค่าปัจจุบันแล้วก้าวไปทิศตรงข้าม:

$$w_{k+1} = w_k - \eta\,\frac{2}{m}X^\top(Xw_k - y)$$

m คือจำนวนตัวอย่าง, d คือจำนวน features, $X \in \mathbb{R}^{m\times d}$, $w \in \mathbb{R}^d$, $y \in \mathbb{R}^m$ และ $\eta > 0$ คือ learning rate. ถ้ามี intercept ให้เพิ่มคอลัมน์หนึ่งใน X.

Ordinary Least Squares: เงื่อนไขของคำตอบปิดรูป

ตั้ง gradient เป็นศูนย์แล้วจัดรูป:

$$X^\top(Xw - y) = 0 \quad\Longrightarrow\quad X^\top Xw = X^\top y$$

ถ้า X มี full column rank จึงเขียนได้ว่า

$$\hat{w} = (X^\top X)^{-1}X^\top y.$$

ถ้า rank ไม่เต็ม เมทริกซ์ผกผันนี้ใช้ไม่ได้ ต้องใช้วิธีที่รองรับ เช่น pseudoinverse หรือ regularization. เงื่อนไข $X^\top r = 0$ บอกว่า residual ตั้งฉากกับคอลัมน์ของ X ไม่ได้บอกว่า residual ทุกตัวเป็นศูนย์.

Bayes, LDA และ QDA

สำหรับ class C_k และข้อมูลที่สังเกตได้ x:

$$P(C_k \mid x) = \frac{p(x \mid C_k)\,\pi_k}{\sum_j p(x \mid C_j)\,\pi_j}$$

$\pi_k = P(C_k)$ คือ prior หรือความน่าจะเป็นตั้งต้น; $p(x \mid C_k)$ คือ likelihood หรือโอกาสเห็นข้อมูล x เมื่ออยู่ใน class C_k. ตัวส่วนทำให้ posterior ของทุก class รวมกันได้ 1.
Linear Discriminant Analysis (LDA) สมมติว่าแต่ละ class เป็น Gaussian และใช้ covariance ร่วมกัน Σ; score ที่ตัดพจน์ร่วมซึ่งไม่ขึ้นกับ class ได้คือ

$$\delta_k(x) = x^\top\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^\top\Sigma^{-1}\mu_k + \ln\pi_k.$$

Quadratic Discriminant Analysis (QDA) ใช้ covariance แยก Σ_k:

$$\delta_k(x) = -\frac{1}{2}\ln|\Sigma_k| - \frac{1}{2}(x-\mu_k)^\top\Sigma_k^{-1}(x-\mu_k) + \ln\pi_k.$$

μ_k คือ ค่า เฉลี่ย ของ class k; shared covariance ทำให้ขอบเขต LDA เป็นเส้นตรง ส่วน QDA ยอมให้ขอบเขตโค้งได้.

Logistic Regression: probability และ loss

สำหรับ label $y_i \in \{0, 1\}$:

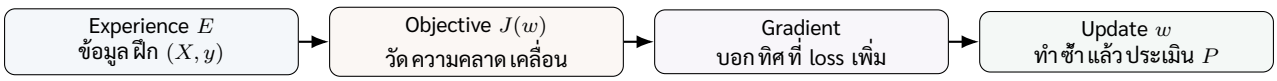
$$z_i = \beta_0 + x_i^\top\beta, \qquad p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}.$$

Binary Cross-Entropy (BCE) เฉลี่ยและ gradient คือ

$$J(\beta) = -\frac{1}{m}\sum_i\left[y_i\ln p_i + (1-y_i)\ln(1-p_i)\right],$$

$$\nabla_\beta J = \frac{1}{m}X^\top(p - y).$$

p_i คือความน่าจะเป็นที่แบบจำลองให้กับ class 1; y_i คือ label จริง. ถ้าไม่ได้รวม intercept เป็นคอลัมน์ใน X ให้ใช้ $\partial J/\partial\beta_0 = \frac{1}{m}\sum_i(p_i - y_i)$. ใช้ probability คำนวณ loss ก่อน แล้วจึงค่อยใช้ threshold แปลงเป็น class.



แผนที่สมการสำคัญ Lecture 3–4

ในบท SVM ให้แยกคะแนน ระยะ และ slack ออกจากกัน ส่วนบท Regression และ Survival ให้ดูหน่วยของผลลัพธ์และช่วงเวลาที่ข้อมูลยังอยู่ในกลุ่มเสี่ยง

Support Vector Machine (SVM): margin

คะแนนและระยะจากจุด x ถึงเส้นแบ่ง:

$$f(x) = w^\top x + b, \qquad d(x, \text{boundary}) = \frac{|f(x)|}{\|w\|_2}.$$

เมื่อ $t_i \in \{-1, +1\}$ ปริมาณ $t_i f(x_i)$ แสดงว่าจุดอยู่ฝั่งถูกหรือไม่; canonical margin ต้องมี $t_i f(x_i) \geq 1$. Soft-margin SVM แก้

$$\min_{w,b,\xi} \frac{1}{2}\|w\|_2^2 + C\sum_i \xi_i$$

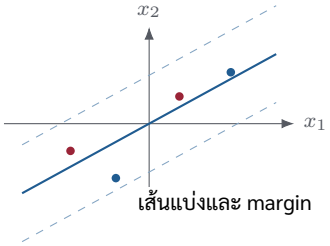
โดยมีเงื่อนไข $t_i(w^\top x_i + b) \geq 1 - \xi_i$ และ $\xi_i \geq 0$. $C > 0$ คุมโทษของการละเมิด margin; ξ_i คือ slack ของตัวอย่างที่ i. Full margin กว้าง $2/\|w\|_2$.

Regularization: ขนาด penalty มีผลต่อ coefficients

ให้ MSE (Mean Squared Error) เป็น ความคลาด เคลื่อนเฉลี่ย, β เป็น coefficients ของ features และ $\lambda \geq 0$ เป็นน้ำหนัก penalty:

$$J_{\text{Ridge}} = \text{MSE} + \lambda\sum_j \beta_j^2, \qquad J_{\text{Lasso}} = \text{MSE} + \lambda\sum_j |\beta_j|.$$

Ridge มักย่อ coefficients ให้เล็กลง; Lasso สามารถทำบาง coefficients เป็นศูนย์ได้. โดยทั่วไปไม่ลงโทษ intercept. เมื่อ λ เพิ่ม การยอมให้ coefficients ใหญ่มีต้นทุนมากขึ้น จึงต้องปรับเทียบด้วย validation data.



เส้นแบ่งและ margin

Poisson Regression: ค่าเฉลี่ยของจำนวนเหตุการณ์

ถ้า Y_i เป็นจำนวนเหตุการณ์และมีค่าเฉลี่ย μ_i:

$$P(Y_i = y_i \mid x_i) = \frac{e^{-\mu_i}\mu_i^{y_i}}{y_i!},$$
$$\ln \mu_i = x_i^\top \beta, \qquad \mu_i = e^{x_i^\top \beta}.$$

x_i คือชุด features, β คือ coefficients และ $\mu_i > 0$ คือจำนวนที่คาดหมาย. เมื่อ feature j เพิ่มหนึ่งหน่วย โดยตัวแปรอื่นคงเดิม ค่า e^{β_j} เป็น rate ratio.

Survival Analysis: Kaplan–Meier (KM)

ที่เวลา event t_j:

$$\widehat{S}(t) = \prod_{t_j \leq t}\left(1 - \frac{d_j}{n_j}\right) = \prod_{t_j \leq t}\frac{n_j - d_j}{n_j}.$$

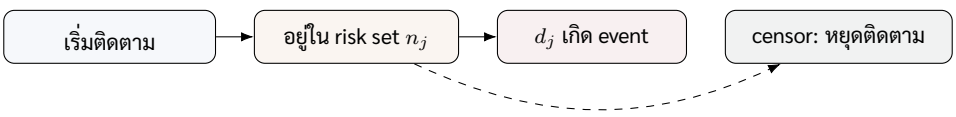
n_j คือจำนวนที่ยังอยู่ในกลุ่มเสี่ยงก่อนเวลา t_j; d_j คือจำนวน event ที่เวลานั้น. คูณโอกาสที่ยังไม่เกิด event ในแต่ละเวลา จึงได้ survival curve ที่คงเดิมระหว่าง event และลดลงเมื่อเกิด event. การ censoring บอกว่าติดตามถึงเวลาหนึ่งโดยยังไม่เห็น event; ไม่ได้นับเป็น event.

Kernel: คำนวณความคล้ายโดยไม่สร้างพิกัดใหม่

Radial Basis Function (RBF) kernel ที่ใช้บ่อย:

$$K(x, z) = \exp\left(-\gamma\|x - z\|_2^2\right), \qquad \gamma > 0.$$

เมื่อ x กับ z อยู่ใกล้กัน ค่า kernel เข้าใกล้ 1; เมื่อห่างกัน ค่าจะเข้าใกล้ 0. γ คุมระยะที่นับว่าใกล้ จึงต้องพิจารณาคู่กับ scale ของ features และพารามิเตอร์ C.



แผนที่สมการสำคัญ Lecture 5–8

สูตรกลุ่มนี้ใช้กับการแบ่งข้อมูล การปรับแบบจำลองหลายรอบ การส่ง gradient ย้อนกลับ และการฉายข้อมูลลงมิติที่น้อยลง

Lecture 5: Entropy และ Information Gain

ถ้า node มี class K กลุ่ม และสัดส่วน class k คือ p_k:

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k, \qquad Gini = 1 - \sum_{k=1}^{K} p_k^2.$$

Information Gain (IG) ของ split:

$$IG = H(\text{parent}) - \sum_{c} \frac{N_c}{N} H(c).$$

N คือจำนวนตัวอย่างใน parent และ N_c คือจำนวนใน child c. ต้องถ่วง impurity ของ child ด้วยสัดส่วนข้อมูลก่อนนำมาหัก.

AdaBoost: เพิ่มน้ำหนักจุดที่ทายผิด

กำหนด $y_i, h_m(x_i) \in \{-1, +1\}$ และ weighted error err_m:

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \text{err}_m}{\text{err}_m},$$
$$D_{m+1}(i) = \frac{D_m(i) e^{-\alpha_m y_i h_m(x_i)}}{Z_m}.$$

$D_m(i)$ คือน้ำหนักตัวอย่างก่อนรอบ m; Z_m ทำให้ผลรวมน้ำหนักใหม่เท่ากับ 1. ถ้าทายถูกจะลดน้ำหนักสัมพัทธ์; ถ้าทายผิดจะเพิ่ม. เมื่อ error ต่ำกว่า 0.5, $\alpha_m > 0$.

Gradient Boosting: เพิ่มผู้เรียนทีละรอบ

หลังรอบ $m-1$ ให้ $F_{m-1}(x)$ เป็นผลรวมปัจจุบัน และให้ h_m เรียน pseudo-residual:

$$r_{im} = -\left.\frac{\partial \ell(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)},$$
$$F_m(x) = F_{m-1}(x) + \nu \rho_m h_m(x).$$

ν คือ learning rate; ρ_m คือ ขนาด ก้าว ที่ เลือก ให้ loss ลดลง. สำหรับ squared error, pseudo-residual เท่ากับ $y_i - F_{m-1}(x_i)$.

Lecture 6: Neural Network และ backpropagation

สำหรับ layer ℓ:

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \sigma(z^{(\ell)}).$$

เมื่อใช้ softmax กับ cross-entropy, $\delta^{(L)} = a^{(L)} - y$. ส่ง gradient กลับ:

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}),$$
$$\frac{\partial J}{\partial W^{(\ell)}} = \frac{1}{m} \sum_i \delta_i^{(\ell)} (a_i^{(\ell-1)})^\top.$$

W คือน้ำหนัก, b คือ bias, z คือค่าก่อน activation, a คือผลหลัง activation, m คือจำนวนตัวอย่างใน batch และ $\odot$ คือคูณสมาชิกต่อสมาชิก. คำนวณ gradients ให้ครบก่อนปรับพารามิเตอร์.

Lecture 7: Convolutional Neural Network (CNN)

ขนาด output ต่อหนึ่งแกน:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

จำนวนพารามิเตอร์ convolution:

$$N_{\text{param}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}.$$

H, W คือ ความสูง และ กว้าง input; K คือ kernel, P คือ padding, S คือ stride; C_{in} และ C_{out} คือจำนวน input และ output channels. สูตรขนาดสมมติ dilation เท่ากับ 1; ค่า $+1$ ในสูตรพารามิเตอร์นับ bias หนึ่งค่าต่อ output channel. ตัว กรอง ใช้ weights ชุด เดิม ทุก ตำแหน่ง จึง ไม่ คูณ จำนวน ตำแหน่งภาพในสูตรพารามิเตอร์.

Lecture 8: Principal Component Analysis (PCA)

ให้ $X \in \mathbb{R}^{m \times d}$ มี m แถวข้อมูลและ d features. ลบค่าเฉลี่ย μ ของแต่ละ feature ก่อน:

$$X_c = X - \mathbf{1}\mu^\top, \qquad S = \frac{X_c^\top X_c}{m-1}, \qquad S v_j = \lambda_j v_j.$$

เลือก eigenvectors $V_r = [v_1, \ldots, v_r]$ ที่มี eigenvalues มากสุด:

$$Z = X_c V_r, \qquad \hat{X} = Z V_r^\top + \mathbf{1}\mu^\top,$$
$$\text{PVE}(r) = \frac{\sum_{j=1}^{r} \lambda_j}{\sum_{j=1}^{d} \lambda_j}.$$

Z คือ scores ในมิติใหม่; การคืนข้อมูลต้องบวกค่าเฉลี่ยกลับ. PVE (Proportion of Variance Explained) คือสัดส่วนความแปรปรวนที่เก็บไว้ ไม่ใช่ accuracy.

Lecture 1 Introduction to ML & Statistical Learning

แหล่งเนื้อหา: L1 PDF p.1–30 | ขอบเขตมิดเทอม

คำย่อที่ใช้ในบทนี้

ML คือ Machine Learning หรือการเรียนรู้ของเครื่อง, **AI** คือ Artificial Intelligence และ **DL** คือ Deep Learning. **T, P, E** ย่อจาก Task, Performance และ Experience ตามลำดับ. **LDA** คือ Linear Discriminant Analysis ส่วน **QDA** คือ Quadratic Discriminant Analysis. **CV** คือ Cross-validation ในตัววัดผล **TP, FP, FN, TN** คือ True Positive, False Positive, False Negative และ True Negative. **ROC** คือ Receiver Operating Characteristic และ **AUC** คือ Area Under the Curve. **MSE** คือ Mean Squared Error, **MAE** คือ Mean Absolute Error และ **RMSE** คือ Root Mean Squared Error. **TPR** และ **FPR** คือ True Positive Rate และ False Positive Rate. การเรียนรู้ของเครื่อง (Machine Learning) คือการปรับแบบจำลองจากข้อมูล เพื่อให้ทำงานตามโจทย์และเกณฑ์ที่กำหนด คำว่า “เรียนรู้” ไม่ได้แปลว่าคอมพิวเตอร์เลือกทุกอย่างเอง ผู้พัฒนายังเป็นผู้กำหนดโจทย์ เลือกข้อมูลและตัวแปร เลือกวิธีเรียนรู้ และกำหนดวิธีวัดผล ส่วนสิ่งที่แบบจำลองเรียนจากข้อมูลคือรูปแบบหรือค่าพารามิเตอร์ที่ใช้สร้างคำทำนาย

Task, Performance, Experience

กรอบ T, P, E ช่วยแยกสามคำถามออกจากกัน สำหรับการกรองอีเมลขยะ T คือการจำแนกอีเมลใหม่, E คือชุดอีเมลที่มีป้ายกำกับและใช้ฝึกแบบจำลอง, ส่วน P คือเกณฑ์วัดผล เช่น precision หรือ recall ทั้งสามอย่างจึงมีหน้าที่ต่างกัน และไม่ได้หมายถึง accuracy เหมือนกันทั้งหมด

ประเด็น	Classification	Regression
เป้าหมาย	class และ category	ค่าตัวเลขต่อเนื่อง
ชนิดผลลัพธ์	$\{1, \ldots, k\}$	$\mathbb{R}$
ตัวอย่าง	spam, ประเภทภาพ	ราคา, อายุ, ยอดขาย

Task อื่นที่สไลด์กล่าวถึง

การจัดกลุ่ม (clustering) แบ่งตัวอย่างตามความคล้ายกันโดยไม่มีป้ายคำตอบ การถอดเสียง (transcription) เปลี่ยนเสียงพูดให้เป็นข้อความ ส่วนการแปลภาษา (translation) เปลี่ยนข้อความจากภาษาหนึ่งเป็นอีกภาษา การตรวจจับความผิดปกติ (anomaly detection) หาแถวที่แตกต่างจากรูปแบบปกติ และการสร้างหรือสุ่มตัวอย่าง (synthesis and sampling) สร้างข้อมูลใหม่ที่มีลักษณะใกล้กับการกระจายของข้อมูลที่ใช้ฝึก

> **ต้องจำ** ตัวเลขเป็น class code ได้ เช่น 0/1 ไม่ได้ทำให้ task เป็น regression ต้องดูความหมายของ target และเกณฑ์การตัดสิน

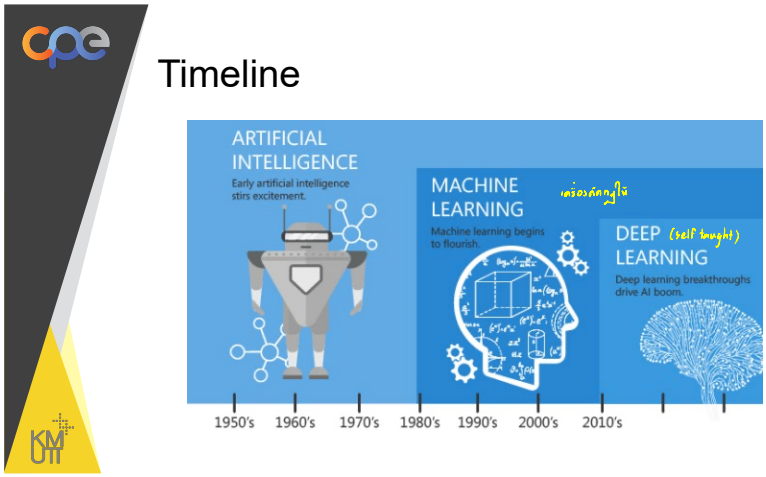
ฝึกให้ทำได้ ได้รับสถานการณ์แล้วระบุ T, P, E และ task type พร้อมเหตุผลหนึ่งประโยค อย่าตอบจากชื่อ application อย่างเดียว

อ่านเพิ่มจากตำรา ISLP Ch.2 และ Géron Ch.1: การอัตโนมัติและ scale เป็นแรงจูงใจ ไม่ใช่ข้อรับประกันว่า ML จะแม่นกว่า expert ทุกสถานการณ์

1.1 ภาพรวมงาน ML และ Performance ที่หน้า 31–32

แหล่งเนื้อหา: L1 PDF p.8–13,20–35

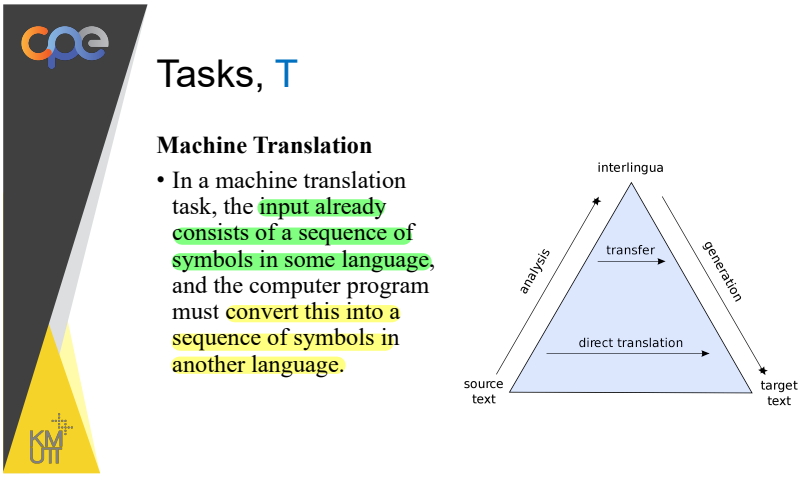
AI เป็นขอบเขตใหญ่. ML เป็นแนวทางที่อาศัยการเรียนรู้จากข้อมูล. Deep Learning เป็นกลุ่มวิธี ML ที่ใช้ neural networks หลายชั้น การเห็นผลิตภัณฑ์ที่ใช้ AI ไม่ได้แปลว่าทุกขั้นในผลิตภัณฑ์เป็น ML ความเร็ว ความแม่นยำ และการจัดการข้อมูลจำนวนมากเป็นเหตุผลที่อยากใช้ ML แต่ยังต้องทดสอบว่าระบบจริงทำได้ตามเป้าหมายหรือไม่



รูป 1.1: Timeline ตัวอย่างการพัฒนาแนวคิด AI และ ML จากหน้า 11

อ่านภาพทีละจุด **1.** ภาพจัดแนวคิดและการประยุกต์ตามช่วงเวลา เพื่อให้เห็นบริบทของการพัฒนา AI และ Deep Learning
2. อ่านภาพควบคู่กับนิยาม: regression และ decision tree เป็น ML ได้โดยไม่ต้องเป็น deep network. timeline ไม่ได้กำหนดว่าระบบทุกระบบต้องมีวิธีเดียวกัน

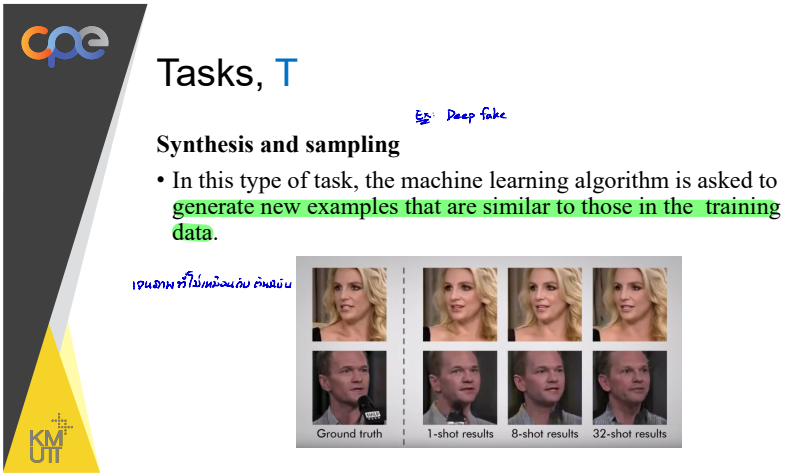
งาน	สิ่งที่ให้เป็น input	สิ่งที่ต้องคืน และ วิธีประเมิน
Classification	feature ของตัวอย่าง	class หรือ probability. accuracy, precision, recall
Regression	feature ของตัวอย่าง	ค่าต่อเนื่อง. MAE, MSE, R^2
Transcription	เสียงหรือข้อมูลที่ต้องถอด	ลำดับสัญลักษณ์ และ ข้อความ ไม่ใช่ class เดียวทั้งไฟล์
Translation	ข้อความภาษาต้นทาง	ข้อความปลายทาง ความหมายและลำดับสำคัญ
Anomaly detection	ตัวอย่างเทียบ pattern ปกติ	ความผิดปกติหรือ anomaly score
Synthesis และ sampling	เงื่อนไขหรือ noise	ตัวอย่างใหม่จาก model distribution
Missing-value imputation	ข้อมูลที่มีช่องหาย	ค่าที่ประมาณของช่องหาย ต้องแยกจากค่าจริง



รูป 1.2: ตัวอย่าง tasks หน้า 28: input และ output มีโครงสร้าง

อ่านภาพทีละจุด **1.** งานเสียง และ ข้อความอาจมี output เป็นลำดับ จึงไม่ควรบังคับทุกงานให้เป็น vector label ตัวเดียว

2. ชื่อ task ไม่ได้กำหนด learning algorithm อัตโนมัติ ต้องระบุข้อมูลฝึกและ metric ด้วย



รูป 1.3: ตัวอย่าง synthesis และ sampling และ missing values หน้า 30

อ่านภาพทีละจุด **1.** สร้างข้อมูลใหม่ต่างจากทำนายคำตอบของตัวอย่างเดิม ต้องตรวจคุณภาพข้อมูลที่สร้างด้วย

2. การ impute ก่อน split โดยเรียนค่าเฉลี่ยจากทั้ง dataset ทำให้ข้อมูลทดสอบรั่วเข้า preprocessing

สำหรับ regression ให้ $RSS = \sum(y_i - \hat{y}_i)^2$, $SST = \sum(y_i - \bar{y})^2$ จึง $MAE = \frac{1}{m}\sum|y_i - \hat{y}_i|$, $R^2 = 1 - RSS/SST$ เมื่อ $SST > 0$ ถ้า model แย่กว่าคำทำนาย $\bar{y}$ บนชุดที่ประเมิน R^2 เป็นลบได้ ไม่ได้จำกัด 0–1 เสมอ หาก y_i เท่ากันหมด denominator เป็นศูนย์ต้องระบุ convention แยก

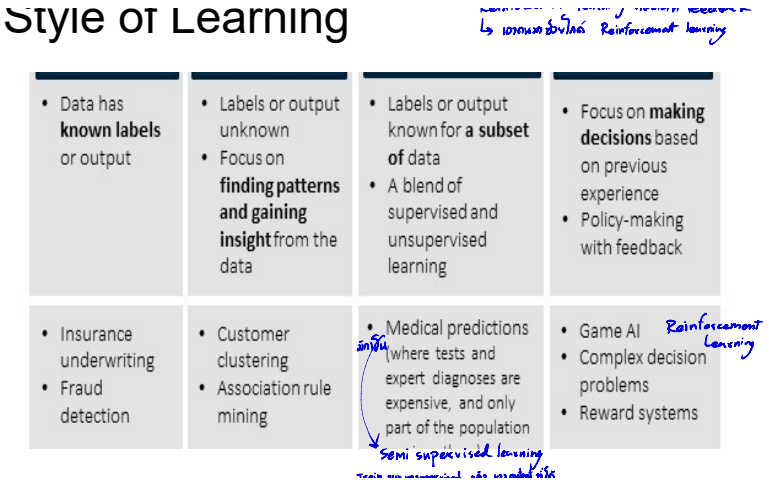
ระวัง: คำย่อ SSR มีสองธรรมเนียม: บางแหล่งหมายถึง residual sum of squares บางแหล่งหมายถึง explained regression sum of squares สูตร $R^2 = SSR/SST$ ใช้ได้ในความหมายหลังเมื่อ OLS พร้อม intercept ให้ decomposition ที่เหมาะสม อย่าสลับกับ $1 - RSS/SST$ โดยดูชื่อย่ออย่างเดียว

> **ต้องจำ** Metric ต้องเข้ากับ task ค่า error ต่ำของ train ไม่รับรอง P บนข้อมูลใหม่ และ R^2 ไม่ใช่เปอร์เซ็นต์ classification ถูก

1.2 รูปแบบการเรียนรู้และการเลือกปัญหา

แหล่งเนื้อหา: L1 PDF p.12–13, 20–30

Supervised Learning มีคู่ (x, y) ให้เรียนความสัมพันธ์และประเมินคำตอบ. Unsupervised มี x แต่ไม่มี ground-truth target ที่ใช้สอน. Semi-supervised ใช้ทั้งข้อมูลที่มีและไม่มี label. Reinforcement Learning ใช้การกระทำและ reward เพื่อเรียน policy ในสภาพแวดล้อม



รูป 1.4: การแบ่ง style of learning ตามข้อมูลป้อนกลับในสไลด์ (L1 p.13)

อ่านภาพทีละจุด **1.** ดูว่าข้อมูลฝึกมี target label หรือไม่: supervised มีคู่ input และ target ส่วน unsupervised ค้นโครงสร้างของ input

2. กิ่ง reinforcement ใช้ reward และ interaction ตามเวลา ต่างจาก label ที่กำหนดให้ทุก sample เลือกกิ่งจากข้อมูลป้อนกลับ ไม่เลือกจากชื่อ algorithm อย่างเดียว

แยก clustering กับ classification

ทั้งสองอาจ output หมายเลขกลุ่มเหมือนกัน แต่ classification ฝึกจาก class ที่มีความหมายกำหนดไว้แล้ว ขณะที่ clustering สร้างการแบ่งกลุ่มจากความคล้ายคลึง หมายเลข cluster จึงสลับได้โดยไม่เปลี่ยนคำตอบเชิงโครงสร้าง

เลือก model จากข้อมูลและสิ่งที่ต้องตอบ

ถ้าต้องการค่าจำนวนเหตุการณ์ ใช้ count model ที่มี support เหมาะสม หากต้องการเวลาจนเกิดเหตุการณ์และมี censoring ใช้ survival formulation หากต้องการ class จากภาพ model ที่ใช้ spatial structure เช่น CNN มี inductive bias สอดคล้องกับ input มากกว่า flatten โดยไม่คิด

> **ต้องจำ** Supervised และ Unsupervised เป็นการแบ่งวิธีใช้ข้อมูล ส่วน Classification และ Regression เป็นชนิด task ไม่ใช่แกนเดียวกันทุกกรณี

ทดสอบตัวเอง: ระบบจัดกลุ่มลูกค้าจาก recency-frequency-monetary โดยไม่มี label เป็น clustering ระบบทำนายว่าลูกค้าจะลาออกจากข้อมูลที่มี label เป็น classification

1.3 Statistical Learning: $Y = f(X) + \varepsilon$

แหล่งเนื้อหา: L1 PDF p.14–18

ให้ $X = (X_1, \ldots, X_p)$ เป็น predictors, Y เป็น response และ f เป็นความสัมพันธ์ที่ไม่ทราบ เราสังเกตข้อมูลจริงที่มี random error:

$$Y = f(X) + \varepsilon, \quad E[\varepsilon] = 0, \qquad \hat{Y} = \hat{f}(X).$$

f คือ systematic signal ส่วน ε รวมสิ่งที่ input ปัจจุบันอธิบายไม่ได้ เช่นตัวแปรที่ไม่ได้วัดและความแปรผันที่ไม่สามารถคาดการณ์ การ fit model คือประมาณ f ไม่ใช่ทำให้ทุกค่า ε หายไป

Reducible กับ irreducible error

ส่วนที่เกิดจาก $\hat{f}$ ไม่ตรงกับ f ลดได้ด้วยข้อมูล และ วิธีเรียนที่เหมาะสม ส่วน variance ของ noise ภายใต้ชุด feature ที่กำหนดเป็น irreducible error เพิ่มความซับซ้อนเพียงอย่างเดียวไม่ทำให้ noise กลายเป็น signal

อ่านเพิ่มจากตำรา ISLP §2.1.1, printed p.18 (PDF p.28): สำหรับ $X, \hat{f}$ คงที่ ให้ noise มี $E[\varepsilon \mid X] = 0$ แล้วขยายกำลังสองและใช้ความคาดหมายทีละเทอม

$$\begin{aligned} E[(Y - \hat{f}(X))^2 \mid X] &= E[((f(X) - \hat{f}(X)) + \varepsilon)^2 \mid X] \\ &= (f(X) - \hat{f}(X))^2 + 2(f(X) - \hat{f}(X))E[\varepsilon \mid X] + E[\varepsilon^2 \mid X] \\ &= (f(X) - \hat{f}(X))^2 + \mathrm{Var}(\varepsilon \mid X). \end{aligned}$$

สูตรนี้แยกความคลาดเคลื่อนในการประมาณ function จากความสุ่มของ observation ไม่ใช่ train MSE ของทุก dataset โดยอัตโนมัติ

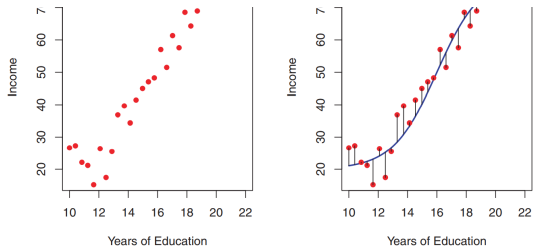


FIGURE 2.2. *The* `Income` *data set.* Left: *The red dots are the observed values of* `income` *(in tens of thousands of dollars) and* `years of education` *for* 30 *individuals.* Right: *The blue curve represents the true underlying relationship between* `income` *and* `years of education`*, which is generally unknown (but is known in this case because the data were simulated). The black lines represent the error associated with each observation. Note that some errors are positive (if an observation lies above the blue curve) and some are negative (if an observation lies below the curve). Overall, these errors have approximately mean zero.*

รูป 1.5: ข้อมูลอาจสัมพันธ์แบบโค้ง: ตั้งสมมติฐาน linear ผิดตั้งแต่ต้นอาจทำให้ estimate f ได้ไม่ดี (L1 p.16)

อ่านภาพทีละจุด **1.** แต่ละจุดคือ observation ส่วนพื้นผิว และ แนวโค้งคือความสัมพันธ์ที่ model พยายามประมาณ
2. ความต่างระหว่างจุดกับฟังก์ชันไม่ใช่ parameter อีกตัวเสมอ อาจมี noise หากบังคับเส้นตรงอาจเกิด approximation bias แม้มีข้อมูลมาก

> **ต้องจำ** การที่ training error เป็นศูนย์ไม่ได้แปลว่าไม่มี irreducible error หรือว่า model เข้าใจ function จริง

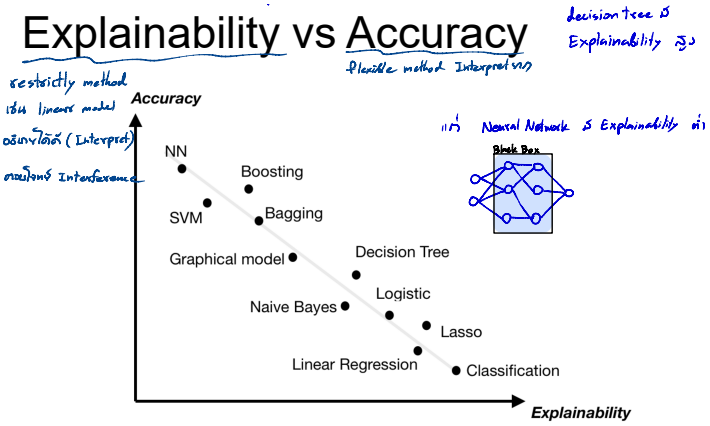
1.4 Prediction, Inference และความอธิบายได้

แหล่งเนื้อหา: L1 PDF p.17–19

Prediction เน้นทำนาย response ของตัวอย่างใหม่ได้ดี ส่วน **Inference** เน้นความสัมพันธ์: predictors ใดเกี่ยวกับ response, ทิศและขนาดของความสัมพันธ์ และจำเป็นต้องใช้ nonlinear relationship หรือไม่ งานเดียวอาจต้องการทั้งสองอย่าง

ประเด็น	Prediction	Inference
คำถาม	Y ของข้อมูลใหม่เท่าไร และ เป็น class ใด	ตัวแปรใดสัมพันธ์กับ Y และอย่างไร
model	อาจยอมรับ black box ถ้าตรวจได้	ต้องเข้าใจรูปแบบและข้อสมมติ
การวัด	test performance	การตีความและความสมเหตุสมผล

Parametric method เลือกรูป function ก่อนแล้วประมาณจำนวน parameter ที่กำหนด เช่น linear model ขณะที่ nonparametric ไม่ล็อครูปแบบง่ายเช่นนั้นไว้ล่วงหน้า แต่ยังมี hyperparameter และสมมติฐานของ algorithm ไม่ได้หมายความว่า “ไม่มี parameter เลย”



Duval, Alexandre. (2019). Explainable Artificial Intelligence (XAI). 10.13140/RG.2.2.24722.09929.

รูป 1.6: ความยืดหยุ่นและ explainability ของ model เป็น trade-off เชิงแนวคิด ภาพไม่ใช่ลำดับความแม่นยำตายตัวสำหรับทุก dataset (L1 p.19)

อ่านภาพทีละจุด **1.** ตำแหน่ง model ในภาพเป็นการเทียบ flexibility กับ interpretability เชิงแนวคิด

2. การย้ายไป model ที่ flexible ขึ้นช่วยประมาณรูปแบบซับซ้อน แต่ต้องตรวจ validation เพื่อดูว่าจำ noise หรือไม่ ภาพไม่ได้ให้ accuracy ของ dataset ใดโดยตรง

ระวัง: coefficient หรือ feature importance แสดง association ใน model ไม่ใช่หลักฐาน causal effect หากไม่มีการออกแบบและสมมติฐานเชิงเหตุผลรองรับ

> **ต้องจำ** ตอบ inference ต้องบอกเงื่อนไข “เมื่อ predictor อื่นคงที่” และระวัง interaction ทำให้ผลของตัวแปรเปลี่ยนตามอีกตัวแปร

1.5 Training, Validation, Test และ Cross-validation

แหล่งเนื้อหา: L1 PDF p.31–35 (performance). split และ CV อธิบายเพิ่มจาก ISLP Ch.5. เชื่อม L6 p.45

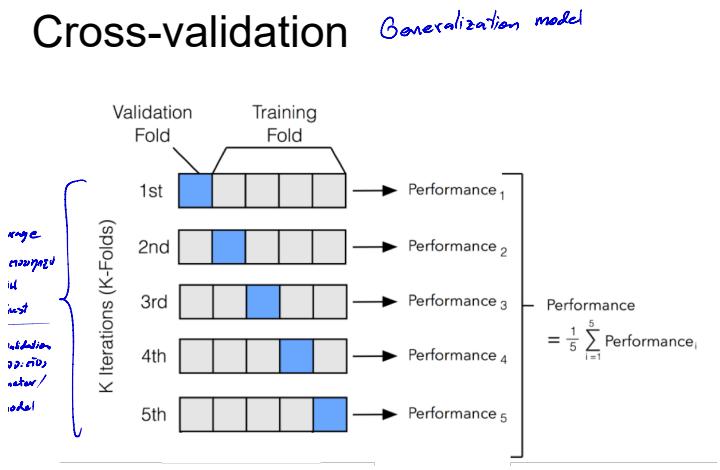
Training set ใช้ประมาณ parameters. Validation set ใช้เลือก model และ hyperparameter และ threshold. Test set ใช้ประเมินครั้งสุดท้ายหลังตัดสินใจแล้ว หากนำ test score มาเลือกหลาย model ซ้ำ เราเริ่มเรียนจาก test โดยอ้อม ทำให้คะแนน optimistic

k-fold cross-validation

แบ่งข้อมูลสำหรับการพัฒนาเป็น k fold แต่ละรอบ train ด้วย $k-1$ fold และ validate ด้วย fold ที่เหลือ รวมเป็น k ผล:

$$\text{CV} = \frac{1}{k}\sum_{j=1}^{k} \text{Score}_j.$$

หาก fold size ต่างกันควรพิจารณาการถ่วงน้ำหนักตามจำนวน sample การใช้ CV ไม่ได้ทำให้ sample ใน fold เดียวกันเป็นข้อมูลอิสระ ถ้าผู้ใช้หรือช่วงเวลาซ้ำกัน ต้องแบ่งตามหน่วยจริง



รูป 1.7: ในแต่ละรอบ validation fold สลับ แต่การ fit model ต้องเริ่มจาก training fold ของรอบนั้น (L1 p.35)

อ่านภาพทีละจุด **1.** หนึ่งแถว และ รอบเลือก fold หนึ่งเป็น validation ส่วนที่เหลือเป็น training

2. ไล่แต่ละรอบแล้วสังเกตว่า sample สลับบทบาท ค่า scaler, PCA และ model ต้อง fit ใหม่จาก training ของรอบนั้น แม้รูปไม่ได้วาด preprocessing ทุกขั้น

อ่านเพิ่มจากตำรา ISLP Ch.5: preprocessing ที่เรียนค่า mean และ std หรือ selection ต้องอยู่ใน training fold ด้วย ไม่ fit ทุกข้อมูลแล้วค่อยแบ่ง fold เพราะทำให้ leakage

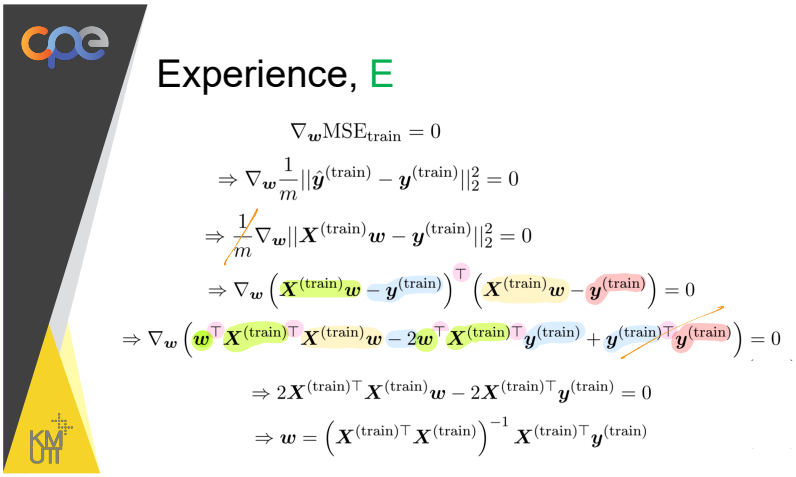
> **ต้องจำ** Test set ไม่ใช่ชุดฝึกอีกชุด. CV ไม่ใช่ train model เพียงหนึ่งตัวแล้วใช้คะแนน training ทุก fold แทน validation

ฝึกให้ทำได้ ได้รับ workflow แล้วชี้ได้ว่าขั้นใดเกิด leakage และเสนอขั้นตอนที่ถูกต้องโดยเก็บ test ไว้หลังการเลือก model

1.6 Experience E: สมการหน้า 39 และการเรียนรู้ที่เกิดขึ้นจริง

แหล่งเนื้อหา: L1 PDF p.36–41

ตัวอย่างนี้แยกสามสิ่งให้ชัด: **Task T** คือรับ feature vector x แล้วทำนายค่าต่อเนื่องด้วย $\hat{y} = x^T w$. **Performance P** คือวัดความผิดพลาดกับคำตอบจริง เช่น MSE บนข้อมูลทดสอบ. **Experience E** คือข้อมูลฝึกที่นำมาใช้เลือก w ให้ training error ต่ำลง การมีสูตร $\hat{y} = x^T w$ อย่างเดียวยังไม่ได้ระบุว่าเรียนรู้จากข้อมูลอย่างไร หน้า 39 จึงเป็นขั้นเชื่อมระหว่างข้อมูลกับน้ำหนักที่ได้



รูป 1.8: สมการ Experience E ที่ตกหล่น: ต้นฉบับ Lecture 1 หน้า PDF 39

อ่านภาพทีละจุด **1.** ไล่จาก gradient ของ training MSE ที่บรรทัดบน ลงมาถึง normal equation ด้านล่าง ทุกบรรทัดใช้ objective เดียวกัน
2. ตัวประกอบ $1/m$ ไม่เปลี่ยนตำแหน่ง minimum ของ least squares นี้ แต่ไม่ควรตัดทิ้งเมื่อจะคำนวณ gradient descent ด้วย learning rate ที่กำหนด

กำหนดขนาดก่อนคูณเมทริกซ์

มีข้อมูล m ตัวอย่าง d features ให้ $X \in \mathbb{R}^{m \times d}$ แต่ละแถวเป็น x_i^T, $w \in \mathbb{R}^d$, $y \in \mathbb{R}^m$ จึง Xw เป็นเวกเตอร์คำทำนายยาว m หากต้องการ intercept ให้เพิ่มคอลัมน์ 1 ใน X และรวม intercept เป็น coefficient ตัวแรก ห้ามบวก intercept อีกครั้งหลังคูณแล้ว

ขยายสมการทุกขั้น

$$
\begin{aligned}
J(w) &= \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2 = \frac{1}{m} (Xw - y)^T (Xw - y) \\
&= \frac{1}{m} \left[w^T X^T X w - w^T X^T y - y^T X w + y^T y \right] \\
&= \frac{1}{m} \left[w^T X^T X w - 2 w^T X^T y + y^T y \right].
\end{aligned}
$$

สอง cross terms เท่ากันเพราะเป็น scalar และ scalar เท่ากับ transpose ของตนเอง เทอม $y^T y$ ไม่ขึ้นกับ w จึง derivative เป็นศูนย์. $X^T X$ สมมาตรทำให้ derivative ของ quadratic term เป็น $2X^T Xw$

$$
\begin{aligned}
\nabla_w J &= \frac{2}{m} X^T (Xw - y), \\
\nabla_w J = 0 &\Longrightarrow X^T X w = X^T y, \\
w^* &= (X^T X)^{-1} X^T y \quad \text{เมื่อ } X \text{ มี full column rank เท่านั้น.}
\end{aligned}
$$

Hessian $\nabla^2 J = 2X^T X / m$ เป็น positive semidefinite เพราะ $v^T X^T X v = \|Xv\|^2 \geq 0$ จึง stationary solution เป็น global minimum ในปัญหานี้ ถ้า columns พึ่งพากัน inverse ใช้ไม่ได้ แต่ยังหา least-squares solution ด้วย pseudoinverse ได้ Coefficients อาจไม่ unique แม้ fitted vector Xw unique

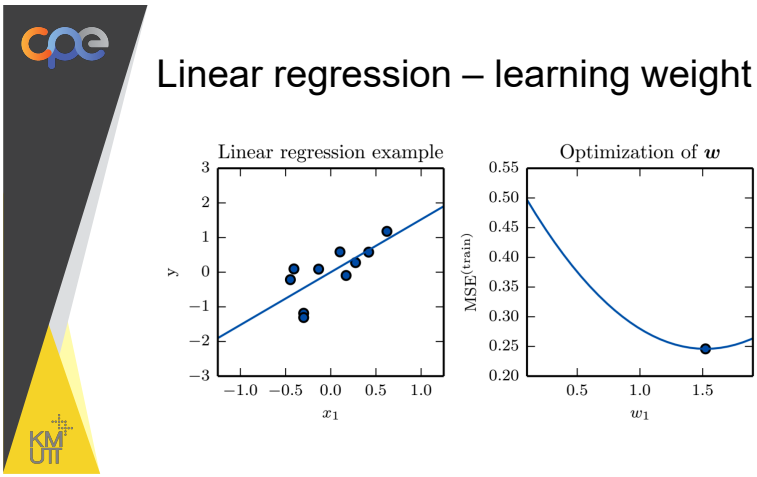
ตัวอย่างคำนวณทีละขั้น

ใช้ $x = (1, 2, 3)$ และ $y = (2, 2, 5)$ โดยมี intercept b และ slope w

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad X^T X = \begin{bmatrix} 1+1+1 & 1+2+3 \\ 1+2+3 & 1^2+2^2+3^2 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 2+2+5 \\ 1(2)+2(2)+3(5) \end{bmatrix} = \begin{bmatrix} 9 \\ 21 \end{bmatrix}.$$

1. Normal equations คือ $3b + 6w = 9$ และ $6b + 14w = 21$
2. คูณสมการแรกด้วย 2 ได้ $6b + 12w = 18$ นำสมการที่สองลบสมการนี้ได้ $2w = 3$ ดังนั้น $w = 1.5$
3. แทนกลับ: $3b + 6(1.5) = 9$ จึง $3b = 0$ และ $b = 0$
4. ทำนายทีละแถวได้ $0 + 1.5(1) = 1.5, 0 + 1.5(2) = 3, 0 + 1.5(3) = 4.5$ จึง residual $y - \hat{y} = (0.5, -1, 0.5)$
5. SSE $= 0.5^2 + (-1)^2 + 0.5^2 = 0.25 + 1 + 0.25 = 1.5$ และ MSE $= 1.5/3 = 0.5$
6. ตรวจ gradient ด้วย $r = \hat{y} - y = (-0.5, 1, -0.5)$: $\sum r_i = -0.5 + 1 - 0.5 = 0$ และ $\sum x_i r_i = -0.5 + 2 - 1.5 = 0$ จึง $X^T r = (0, 0)^T$

ได้ gradient ศูนย์แม้ MSE ยังเป็น 0.5 เพราะเส้นตรงที่ดีที่สุดไม่จำเป็นต้องผ่านข้อมูลทุกจุด



รูป 1.9: กราฟหลังฝึกในหน้า 41: อย่าสับสนแกน feature กับแกน parameter

อ่านภาพทีละจุด **1.** ภาพเส้นทำนายแสดงการเปลี่ยนคำทำนายตาม x เมื่อเลือก w แล้ว
2. ภาพ error เทียบกับน้ำหนักแสดง objective ตาม w และจุดต่ำสุด เป็นคนละความสัมพันธ์กับกราฟ x–y

> **ต้องจำ** จำ T และ P และ E พร้อมหน้าที่: T ทำนาย, P ประเมิน, E ใช้ข้อมูลเลือก parameters เขียน MSE จากนั้น expand จากนั้น gradient จากนั้น normal equation ได้เอง และกล่าวเงื่อนไข inverse

1.7 Cross-validation แบบติดตามทุกขั้น: กัน leakage ที่ซ่อนอยู่

แหล่งเนื้อหา: L1 PDF p.31–35 (evaluation). workflow CV เสริมจาก ISLP Ch.5. เชื่อม L8 pipeline

มี 12 samples แบ่ง 3 folds กลุ่มละ 4 และเปรียบเทียบ pipeline สองแบบ แต่ละรอบใช้ train 8 และ validation 4 ค่าเฉลี่ยต่อรอบเหมาะเมื่อ fold เท่ากัน ถ้า fold ต่างขนาดและต้องการ per-observation loss ควรถ่วงตามจำนวน observations

รอบ	Train folds	Validation	Fit ใหม่จาก train	ใช้กับ validation
1	2,3	1	mean, scaler, PCA, model	transform ด้วยค่าจาก train
2	1,3	2	mean, scaler, PCA, model	ไม่ refit PCA จาก fold 2
3	1,2	3	mean, scaler, PCA, model	คำนวณ loss ที่แยกไว้

Mean Squared Error (MSE) ของ model A ในสาม fold เป็น $(0.6, 0.9, 0.9)$ จึง $CV_A = (0.6+0.9+0.9)/3 = 2.4/3 = 0.8$ ส่วน B ได้ $(0.4, 1.0, 1.3)$ จึง $CV_B = (0.4 + 1.0 + 1.3)/3 = 2.7/3 = 0.9$ เลือก A จาก criterion ที่กำหนด ไม่เลือก B เพราะมีรอบหนึ่งดีมาก แล้ว refit pipeline A บน training pool ทั้งหมดก่อนประเมิน test ที่กันไว้ ไม่เอา test มาสลับเป็น fold เพิ่มเพื่อให้ครบจำนวน

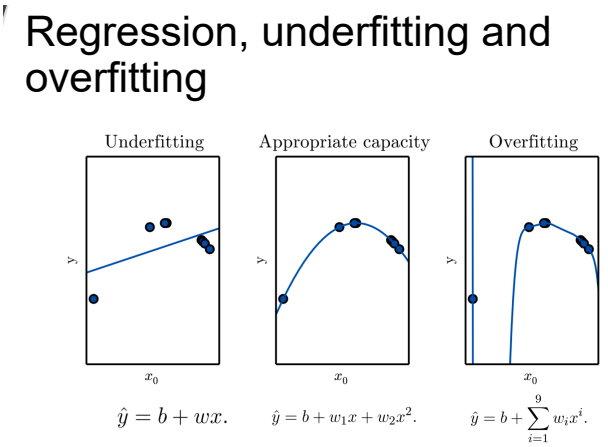
ระวัง: Fit scaler และ PCA จากทั้ง 12 samples ก่อนทำ CV คือ leakage แม้ไม่ใช้ labels เพราะ validation มีอิทธิพลต่อ representation ที่ใช้ฝึก

> **ต้องจำ** CV ใช้เลือก model และ hyperparameters. test ใช้ประเมินหลังเลือกแล้ว การลองค่าซ้ำจน test ดีเท่ากับใช้ test เป็น validation โดยอ้อม

1.8 Underfitting, Overfitting และ Bias–Variance

แหล่งเนื้อหา: L1 PDF p.42–43. L5 p.33. L6 p.67–73

Underfitting คือ model ไม่พอจะจับ pattern สำคัญ จึงมักมี error สูงทั้ง train และข้อมูลใหม่ Overfitting คือ fit รายละเอียดเฉพาะ training รวมถึง noise ทำให้ train ดีแต่ generalize ไม่ดี ต้องดู performance ของข้อมูลใหม่ร่วมด้วย ไม่ตัดสินจากความซับซ้อนอย่างเดียว



รูป 1.10: เส้นตรงอาจง่ายเกิน ส่วนเส้นโค้งซับซ้อนที่ผ่านแทบทุกจุดอาจจำ noise ต้องเลือก capacity ให้สอดคล้องกับ signal (L1 p.42)

อ่านภาพทีละจุด **1.** เทียบเส้นที่ง่ายเกินกับเส้นที่ผ่านจุดฝึกละเอียด: ทั้งสองใช้ observations ชุดเดียวกัน

2. เส้นเรียบพลาด signal เป็น underfit เส้นคดตาม noise เสี่ยง overfit ต้องอ่าน error บนข้อมูลที่ไม่ใช้ฝึกเพื่อเลือก capacity

อ่านเพิ่มจากตำรา ISLP §2.2.2, printed p.32 (PDF p.42):

$$\begin{aligned} E[(y_0 - \hat{f}(x_0))^2] &= E[(f(x_0) - \hat{f}(x_0))^2] + \sigma_\varepsilon^2 \\ &= E[(f(x_0) - a + a - \hat{f}(x_0))^2] + \sigma_\varepsilon^2 \\ &= (f(x_0) - a)^2 + 2(f(x_0) - a)E[a - \hat{f}(x_0)] \\ &\quad + E[(\hat{f}(x_0) - a)^2] + \sigma_\varepsilon^2 \\ &= \text{Bias}^2[\hat{f}(x_0)] + \text{Var}[\hat{f}(x_0)] + \sigma_\varepsilon^2. \end{aligned}$$

ให้ $a = E[\hat{f}(x_0)]$ โดยความคาดหมายนี้ครอบคลุม training sets ที่สุ่มต่างกัน และ noise ของจุดทดสอบเป็นอิสระจากการฝึก เทอมไขว้เป็นศูนย์เพราะ $E[a - \hat{f}] = a - a = 0$ จึงเหลือ bias squared, variance ของตัวประมาณ และ noise variance. Bias สะท้อนความคลาดเคลื่อนเชิงระบบ ส่วน variance สะท้อนความไวต่อชุดฝึก

การแก้ที่ตรงเหตุ

ถ้า underfit อาจเพิ่ม feature และ capacity หรือลด regularization ถ้า overfit อาจเพิ่มข้อมูล, ลด capacity, regularize, ใช้ early stopping และ ensemble ตามชนิด model ถ้า optimizer ยังไม่ converge ต้องแก้การ training ก่อนตีความว่า underfit จาก capacity

> **ต้องจำ** Bias ไม่ใช่ intercept ของ model. Variance ใน trade-off ไม่ใช่ variance ของ feature อย่างเดียว. regularization มักเพิ่ม bias เพื่อลด variance

1.9 Probability ที่ต้องใช้ก่อน Bayes

แหล่งเนื้อหา: L1 PDF p.49–54

ความน่าจะเป็นแบบมีเงื่อนไข $P(A \mid B)$ คือโอกาสที่ A จะเกิด เมื่อเรารู้แล้วว่า B เกิดขึ้น จึงคำนวณภายในกลุ่มกรณีที่ B เป็นจริง และต้องมี $P(B) > 0$:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(A \cap B) = P(A \mid B)P(B) = P(B \mid A)P(A).$$

หาก $F_1, \ldots, F_K$ แบ่ง sample space แบบ mutually exclusive และ exhaustive จะมี law of total probability:

$$P(E) = \sum_{k=1}^{K} P(E \mid F_k)P(F_k).$$

ที่มาของ Bayes' theorem

เทียบ joint probability สองรูปแล้วหารด้วย evidence:

$$P(F_k \mid E) = \frac{P(E \mid F_k)P(F_k)}{\sum_j P(E \mid F_j)P(F_j)}.$$

ในสมการนี้ $P(F_k)$ คือ prior หรือโอกาสของ class F_k ก่อนเห็นข้อมูล ส่วน $P(E \mid F_k)$ คือ likelihood หรือโอกาสพบหลักฐาน E เมื่อ class เป็น F_k ผลคูณสองค่านี้เป็นน้ำหนักของ class นั้น ตัวส่วนรวมทุก class ที่เป็นไปได้เพื่อปรับน้ำหนักให้เป็น posterior ซึ่งรวมกันได้ 1

ตัวอย่างคำนวณทีละขั้น

กำหนด $P(A) = 0.2$, $P(B \mid A) = 0.8$ และ $P(B \mid \bar{A}) = 0.1$ ต้องการหาความน่าจะเป็นของ A เมื่อทราบว่า B เกิดขึ้น

1. หาส่วนเติมเต็มก่อน: $P(\bar{A}) = 1 - 0.2 = 0.8$
2. คำนวณสองเส้นทางที่ทำให้เกิด B: $P(A \cap B) = 0.8(0.2) = 0.16$ และ $P(\bar{A} \cap B) = 0.1(0.8) = 0.08$
3. รวมเป็น evidence: $P(B) = 0.16 + 0.08 = 0.24$
4. ใช้ Bayes: $P(A \mid B) = P(A \cap B)/P(B) = 0.16/0.24 = 2/3 \approx 0.6667$

ความน่าจะเป็นหลังเห็น B จึงประมาณ 66.67% ต่างจาก prior 20% เพราะข้อมูล B พบในกลุ่ม A ได้มากกว่า

> **ต้องจำ** จำ "posterior = likelihood × prior และ evidence" พร้อมแปลทิศของ conditional ให้ถูก $P(A \mid B)$ ไม่เท่ากับ $P(B \mid A)$ โดยทั่วไป

ฝึกให้ทำได้ วาดสอง branch ตามสถานะจริงก่อน branch ผลทดสอบ แล้วรวมเส้นทาง evidence เดียวกันในตัวส่วน

1.10 Bayes: ตัวอย่างโรคหายากของอาจารย์

แหล่งเนื้อหา: L1 PDF p.55–58. เป็นโจทย์ probability เชิงการเรียน

ให้ D เป็นมีโรคและ $+$ เป็นผลบวก: $P(D) = 10^{-5}$, $P(+ \mid D) = 0.99$, $P(- \mid \bar{D}) = 0.995$ จึงมี $P(+ \mid \bar{D}) = 0.005$ ไม่ใช่ 0.995

$$\begin{aligned} P(\bar{D}) &= 1 - 10^{-5} = 0.99999, \\ P(+ \mid \bar{D}) &= 1 - 0.995 = 0.005, \\ P(D \cap +) &= P(+ \mid D)P(D) = 0.99(10^{-5}) = 0.0000099, \\ P(\bar{D} \cap +) &= 0.005(0.99999) = 0.00499995, \\ P(+) &= 0.0000099 + 0.00499995 = 0.00500985, \\ P(D \mid +) &= \frac{0.0000099}{0.00500985} \approx 0.001976107 = 0.1976107\%. \end{aligned}$$

ผลบวกยังมี posterior ต่ำเพราะโรคหายากมาก และกลุ่มไม่มีโรคมีจำนวนมหาศาล แม้ false-positive rate ต่ำก็สร้างผลบวกปลอมมากกว่าบวกจริงในตัวอย่างนี้

ถ้าผลลบ

$$\begin{aligned} P(- \mid D) &= 1 - 0.99 = 0.01, \\ P(-) &= 0.01(10^{-5}) + 0.995(0.99999) = 0.0000001 + 0.99499005 = 0.99499015, \\ P(D \mid -) &= 0.0000001/0.99499015 \approx 1.005035 \times 10^{-7}. \end{aligned}$$

จึงได้ $P(\bar{D} \mid -) \approx 0.9999998995$ เป็นคำตอบของ “ผลลบแล้วไม่มีโรค” ไม่ใช่ sensitivity ของ test

วิธีตรวจความสมเหตุสมผล

ลองสมมติประชากร 100,000 คน: มีโรคเฉลี่ย 1 คน มีผลบวกจริงประมาณ 0.99 คน แต่กลุ่มไม่มีโรคให้บวกปลอมประมาณ 499.995 คน จึงมีส่วนบวกจริงประมาณ $0.99/500.985$ เท่ากับ Bayes ข้างต้น

> **ต้องจำ** ความแม่นของ test ไม่ใช่ posterior โดยตรง ต้องพิจารณา prior และ base rate ทุกครั้ง

ระวัง: ตัวอย่างนี้ใช้เพื่อทำความเข้าใจ conditional probability ไม่ใช่คำแนะนำในการตัดสินใจทางการแพทย์

1.11 Bayes classifier และ Spam filter

แหล่งเนื้อหา: L1 PDF p.45–47, 59–67

Bayes classifier เลือก class ที่ posterior สูงที่สุด:

$$\hat{y}(x) = \arg\max_k P(Y = k \mid X = x).$$

กฎนี้ลด expected misclassification error ภายใต้ zero-one loss ไม่ใช่กฎที่เหมาะกับต้นทุน error ทุกแบบ ถ้า cost ของ FN และ FP ต่างกัน threshold ที่เหมาะสมอาจไม่เป็น 0.5

ตัวอย่าง Rolex ของสไลด์

มีอีเมลขยะ 2,000 ฉบับ พบ Rolex 250 ฉบับ จึงมี $p(w) = 0.125$. อีเมลปกติ 1,000 ฉบับพบ 5 ฉบับ จึงมี $q(w) = 0.005$ ภายใต้ prior เท่ากัน:

$$r(w) = \frac{p(w)}{p(w) + q(w)} = \frac{0.125}{0.130} \approx 0.96154.$$

ถ้า threshold 0.9 ให้จัดเป็น spam จำนวนอีเมลในข้อมูลสอง class ไม่เท่ากัน แต่โจทย์ระบุ prior 0.5 จึงต้องใช้ prior ตามโจทย์ ไม่แทน $2000/3000$ โดยอัตโนมัติ

หลายคำ

สำหรับ “stock” กับ “undervalued” ในสไลด์ มี likelihood $(0.2, 0.1)$ ใน spam และ $(0.06, 0.025)$ ใน non-spam:

$$r = \frac{0.2(0.1)}{0.2(0.1) + 0.06(0.025)} \approx 0.93023.$$

สูตรคูณต้องอาศัย conditional independence given class หากคำสัมพันธ์กันมากสมมติฐานนี้เป็น approximation อย่าสับสนกับ independent แบบไม่ condition

> **ต้องจำ** คูณ likelihood ภายใน class แล้วคูณ prior ก่อน normalize. posterior ของคำแต่ละคำไม่ใช่สิ่งที่เอามาคูณตรง ๆ โดยไม่คิด

1.12 Naive Bayes: จากตารางนับสู่ posterior โดยไม่ข้ามขั้น

แหล่งเนื้อหา: L1 PDF p.59–67. ตัวเลขฝึกใหม่และ smoothing เสริมจาก ISLP Ch.4

สมมติข้อความฝึก 10 ฉบับเป็น Spam 4 และ Ham 6 ให้ A หมายถึงพบคำว่า “offer” และ B หมายถึงพบคำว่า “meeting” ตารางนี้เป็น counts ของเหตุการณ์ binary ไม่ใช่จำนวนคำรวมทั้ง corpus

Class	จำนวนฉบับ	พบ A	พบ B
Spam	4	3	1
Ham	6	1	4

1. **Prior:** $\pi_S = 4/10 = 0.4$, $\pi_H = 6/10 = 0.6$ ค่านี้ไม่ใช่ความถี่พบคำ
2. **Conditional probabilities:** $P(A|S) = 3/4$, $P(B|S) = 1/4$. $P(A|H) = 1/6$, $P(B|H) = 4/6$
3. ข้อความใหม่พบ A และไม่พบ B ต้องใช้ $P(\bar{B}|k) = 1 - P(B|k)$ ไม่ใช้ $P(B|k)$ ตรง ๆ
4. หาความน่าจะเป็นของการไม่พบ B: $P(\bar{B} \mid S) = 1 - 1/4 = 3/4$ และ $P(\bar{B} \mid H) = 1 - 4/6 = 2/6$
5. ภายใต้สมมติฐาน conditional independence given class คะแนน Spam คือ $u_S = P(S)P(A \mid S)P(\bar{B} \mid S) = (4/10)(3/4)(3/4) = 36/160 = 9/40 = 0.225$
6. คะแนน Ham คือ $u_H = (6/10)(1/6)(2/6) = 12/360 = 1/30 \approx 0.033333$
7. รวมคะแนนเป็น evidence $u_S + u_H = 9/40 + 1/30 = 27/120 + 4/120 = 31/120$
8. Normalize: $P(S \mid A, \bar{B}) = (9/40)/(31/120) = (27/120)/(31/120) = 27/31 \approx 0.870968$
9. $P(H \mid A, \bar{B}) = (1/30)/(31/120) = 4/31 \approx 0.129032$ ผลรวมคือ 1 จึงทาย Spam เมื่อค่าความผิดพลาดสอง class เท่ากัน

การคูณ likelihood เป็นสมมติฐานของ Naive Bayes ไม่ตามจาก Bayes theorem เพียงอย่างเดียว หากคำทั้งสองพึ่งพากันภายใน class ผลคำนวณนี้อาจคลาดเคลื่อน แต่ classifier ยังใช้งานได้เมื่อประเมินด้วยข้อมูลใหม่

Zero frequency และ Laplace smoothing

ถ้า Spam ไม่มีข้อความที่พบ B เลย empirical $P(B|S) = 0$ จะทำให้ผลคูณของข้อความที่พบ B เป็นศูนย์ทั้งหมด สำหรับ Bernoulli feature ที่มีสองผลลัพธ์และ pseudocount 1 ใช้ $P(B|S) = (0 + 1)/(4 + 2) = 1/6$ ตัวส่วนบวก 2 เพราะเพิ่มหนึ่งให้ทั้ง “พบ” และ “ไม่พบ” ไม่ใช่บวก 1 ให้ตัวส่วนอย่างเดียว

ต้องจำ ต้องแยก prior, likelihood, unnormalized score และ posterior ข้อความไม่พบคำเป็นข้อมูลด้วยใน Bernoulli Naive Bayes และ smoothing convention ต้องตรงชนิด model

ทดสอบตัวเอง: ถ้าข้อความใหม่พบทั้ง A, B จากตารางเดิม posterior Spam เท่ากับเท่าไร? ตรวจ: $0.075/(0.075 + 1/15) = 9/17$ ไม่ใช่ $27/31$

1.13 LDA: Gaussian model และ discriminant score

แหล่งเนื้อหา: L1 PDF p.69–77

LDA เป็น generative approach: ประมาณ distribution ของ $X \mid Y = k$ ก่อน แล้วใช้ Bayes หาความน่าจะเป็น $Y \mid X = x$ ให้ prior π_k, mean μ_k และ variance ร่วม σ^2

$$f_k(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu_k)^2/(2\sigma^2)}, \quad P(Y = k \mid x) = \frac{\pi_k f_k(x)}{\sum_j \pi_j f_j(x)}.$$

เริ่มเปรียบเทียบ $\pi_k f_k(x)$ เพราะ evidence ในตัวส่วน posterior เหมือนกันทุก class จากนั้นใช้ log ซึ่งรักษาลำดับคะแนน

$$\begin{aligned} \ln[\pi_k f_k(x)] &= \ln \pi_k - \tfrac{1}{2}\ln(2\pi\sigma^2) - \frac{(x-\mu_k)^2}{2\sigma^2} \\ &= \ln \pi_k - \tfrac{1}{2}\ln(2\pi\sigma^2) - \frac{x^2 - 2x\mu_k + \mu_k^2}{2\sigma^2} \\ &= \underbrace{-\tfrac{1}{2}\ln(2\pi\sigma^2) - \frac{x^2}{2\sigma^2}}_{\text{เท่ากันทุก class}} + \frac{x\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \ln \pi_k. \end{aligned}$$

เมื่อตัดเฉพาะเทอมที่เหมือนกันทุก class จึงเหลือ discriminant score

$$\delta_k(x) = \frac{x\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log \pi_k.$$

เลือก class ที่ δ_k สูงสุด คำว่า linear มาจาก score เป็น linear ใน x เมื่อ covariance ร่วม ไม่ได้หมายความว่า Gaussian density เป็นเส้นตรง ค่าที่ต้อง estimate ตามสไลด์: $\hat{\pi}_k = n_k/n$, $\hat{\mu}_k = \frac{1}{n_k}\sum_{i:y_i=k} x_i$ และ pooled within-class variance $\hat{\sigma}^2 = \frac{1}{n-K}\sum_k \sum_{i:y_i=k}(x_i - \hat{\mu}_k)^2$

ตัวอย่างคำนวณทีละขั้น

ให้ $\mu_1 = 0, \mu_2 = 2, \sigma^2 = 1$ และ $\pi_1 = \pi_2 = 0.5$

1. แทนค่าของ class 1: $\delta_1(x) = x(0) - 0^2/2 + \ln 0.5 = \ln 0.5$
2. แทนค่าของ class 2: $\delta_2(x) = x(2) - 2^2/2 + \ln 0.5 = 2x - 2 + \ln 0.5$
3. หา boundary โดยตั้งคะแนนเท่ากัน: $2x - 2 = 0$ จึง $x = 1$
4. ที่ $x = 1.5$: $\delta_1 = -0.693147$ และ $\delta_2 = 3 - 2 - 0.693147 = 0.306853$ จึงเลือก class 2 เพราะคะแนนสูงกว่า

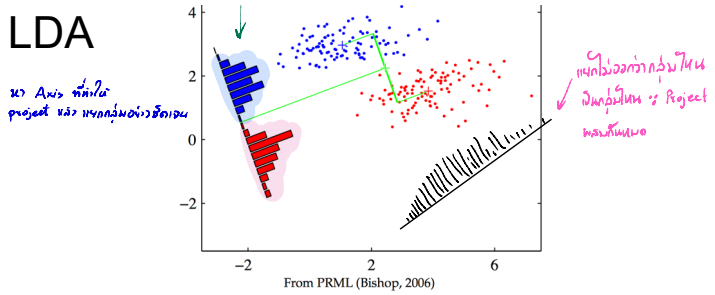
ต้องจำ Gaussian class means ต่างกันได้. LDA บังคับ covariance ร่วม ไม่ได้บังคับ means หรือ priors เท่ากัน

ฝึกให้ทำได้ อธิบายได้ว่าตัด term ใดจาก log-posterior และทำไม แล้วคำนวณ score และ decision boundary จาก parameter ที่ให้

1.14 LDA projection, multivariate LDA และ QDA

แหล่งเนื้อหา: L1 PDF p.70–81

อีกมุมของ LDA คือหา projection ที่กลุ่มต่าง class ห่างกัน แต่แต่ละ class กระจายน้อย เมื่อ project บนแกนที่ไม่เหมาะสม กลุ่มจะทับกันแม้ในข้อมูลเดิมแยกได้ดี แนวคิดนี้เกี่ยวกับ Fisher discriminant และไม่เหมือน PCA ที่มอง variance โดยไม่ใช้ label



- Separate samples of distinct groups by projecting them onto a space that
  - Maximize their between-class separability while
  - Minimize their within-class variability

รูป 1.11: ทิศของ projection สำคัญ: LDA ใช้ข้อมูล class เพื่อเน้น between-class separation และลด within-class variation (L1 p.71)

อ่านภาพทีละจุด 1. ดูสี และ class เดิมก่อนอ่านตำแหน่งที่ฉาย: ระยะในแกนฉายคือ coordinates ใหม่

2. ทิศที่กระจายรวมมากอาจทำให้สอง class ทับกัน ส่วน LDA เลือกทิศให้กลุ่มห่างกันเทียบกับ within-class spread จึงใช้ labels ต่างจาก PCA

เมื่อมีหลาย features ให้ x และ μ_k เป็นเวกเตอร์ขนาด p และ Σ เป็น covariance matrix ขนาด $p \times p$ ที่ใช้ร่วมกันทุก class เริ่มจาก log ของ Gaussian density คูณ prior แล้วขยาย quadratic form

$$\log[\pi_k f_k(x)] = \log \pi_k - \frac{p}{2}\log(2\pi) - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k),$$
$$(x-\mu_k)^T\Sigma^{-1}(x-\mu_k) = x^T\Sigma^{-1}x - 2x^T\Sigma^{-1}\mu_k + \mu_k^T\Sigma^{-1}\mu_k.$$

ใช้สมมาตรของ Σ^{-1} เพื่อรวมสอง cross terms เมื่อนำไปแทนบรรทัดแรก เทอม $-p\log(2\pi)/2$, $-\log|\Sigma|/2$ และ $-x^T\Sigma^{-1}x/2$ เท่ากันทุก class จึงตัดออกจากการเปรียบเทียบคะแนนได้ เหลือ score คือ

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log\pi_k.$$

QDA อนุญาต Σ_k ต่างกัน จึงได้

$$\delta_k(x) = -\tfrac{1}{2}\log|\Sigma_k| - \tfrac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k) + \log\pi_k.$$

QDA มี boundary quadratic โดยทั่วไปและต้อง estimate covariance มากกว่า LDA จึง flexible แต่ variance สูงขึ้นเมื่อข้อมูลน้อย LDA เป็น special case ของ QDA เมื่อ shared covariance

ระวัง: L1 p.81 มีข้อความ “LDA is a special case of LDA” ซึ่งเป็น typo ควรเป็น QDA ตามสูตรและ ISLP §4.4.3. ไม่ใช้ typo นี้เป็นนิยาม

> **ต้องจำ** LDA ใช้ label. PCA ไม่ใช้ label. QDA ไม่ใช่เพิ่ม polynomial features ให้ LDA โดยตรง แต่เปลี่ยน covariance assumption

1.15 อ่าน density, covariance และขอบเขต LDA และ QDA จากภาพ

แหล่งเนื้อหา: L1 PDF p.45–48,78–81

Bayes classifier เลือก class ที่ posterior มากสุด ไม่ได้เลือก density สูงสุดเสมอโดยไม่คูณ prior ภาพสอง classes แสดงทั้งการประมาณ distribution และเส้นแบ่ง class จึงต้องแยก density model ออกจาก decision boundary

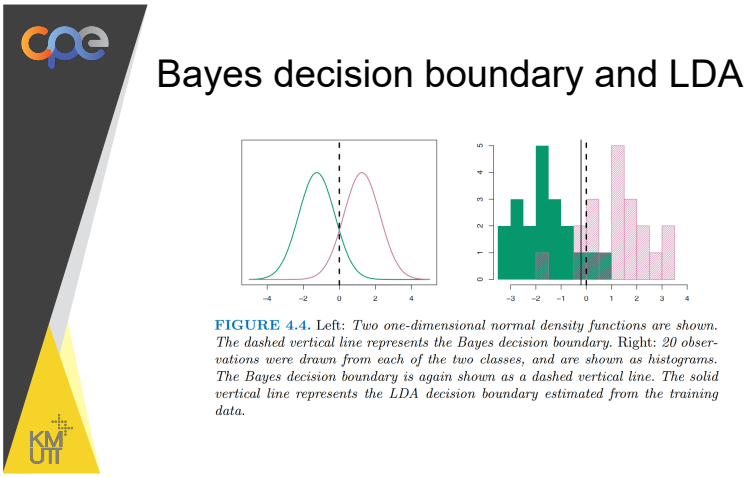


FIGURE 4.4. Left: *Two one-dimensional normal density functions are shown. The dashed vertical line represents the Bayes decision boundary.* Right: *20 observations were drawn from each of the two classes, and are shown as histograms. The Bayes decision boundary is again shown as a dashed vertical line. The solid vertical line represents the LDA decision boundary estimated from the training data.*

รูป 1.12: การประมาณ Gaussian density และเส้นแบ่งในหน้า 78

อ่านภาพทีละจุด **1.** เส้นของ density อธิบาย likelihood ภายใน class. prior ยังมีผลต่อ posterior

2. Boundary เกิดตรง discriminant scores เท่ากัน รูปทรง boundary ไม่ใช่ contour ของ class ใด class หนึ่ง

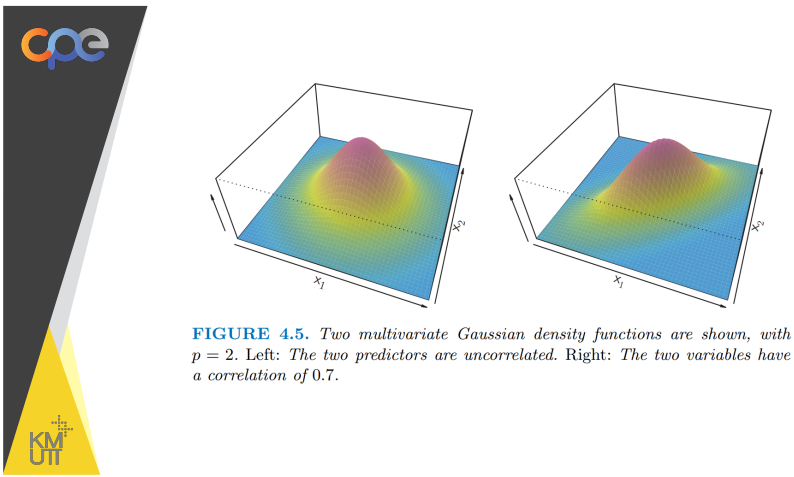


FIGURE 4.5. *Two multivariate Gaussian density functions are shown, with* $p = 2$. Left: *The two predictors are uncorrelated.* Right: *The two variables have a correlation of* 0.7.

รูป 1.13: Covariance แสดงความกระจายและแนวเอียงของกลุ่ม

อ่านภาพทีละจุด **1.** Eigenvectors กำหนดแนวแกน ellipse. eigenvalues กำหนดความแปรปรวนในแนวนั้น จึงไม่ดูแค่ความกว้างแกนนอน และ ตั้ง

2. LDA ใช้ covariance ร่วมทุก class ส่วน QDA ยอมให้ covariance ต่างกัน จึงมี quadratic terms ไม่หักล้าง

สำหรับ Gaussian class k เขียน log-posterior ถึงค่าคงที่ที่เหมือนกันทุก class ได้

$$\delta_k^{QDA}(x) = -\frac{1}{2}\log|\Sigma_k| - \frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k) + \log\pi_k.$$

เมื่อ $\Sigma_k = \Sigma$ ทุก class เทอม $-x^T\Sigma^{-1}x/2$ และ $-\log|\Sigma|/2$ เท่ากันจึงตัดในการเปรียบเทียบ เหลือ $\delta_k^{LDA} = x^T\Sigma^{-1}\mu_k - \mu_k^T\Sigma^{-1}\mu_k/2 + \log\pi_k$ ซึ่ง linear ใน x ถ้า priors เปลี่ยน threshold เลื่อน แม้ covariance และ means คงเดิม

> **ต้องจำ** อธิบายว่าทำไม LDA เป็น linear จากการตัดเทอมร่วม ไม่ใช่จำแค่ว่า LDA เส้นตรง และ QDA เส้นโค้ง. covariance ต่างกันทำให้ quadratic term ของแต่ละ class ต่างกัน

1.16 Confusion Matrix, metrics และ ROC

แหล่งเนื้อหา: L1 PDF p.82–87. เชื่อม L5 p.28–30

ก่อนใช้สูตร ให้กำหนด positive เป็น class ที่เราสนใจ: TP คือข้อมูล positive ที่ทายถูกว่าเป็น positive, FP คือข้อมูล negative ที่ทายผิดเป็น positive, FN คือข้อมูล positive ที่ทายผิดเป็น negative และ TN คือข้อมูล negative ที่ทายถูกว่าเป็น negative ควรอ่านชื่อแกน actual กับ predicted ใน confusion matrix ทุกครั้ง เพราะการวางแกนอาจต่างกันระหว่างภาพ

$$\text{Acc} = \frac{TP+TN}{TP+FP+FN+TN}, \quad P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}.$$

Accuracy วัดสัดส่วนคำทำนายที่ถูกทั้งหมด ส่วน precision ตอบว่า “ในข้อมูลที่ทายเป็น positive มีสัดส่วนที่ถูกเท่าไร” จึงใช้ $TP+FP$ เป็นตัวส่วน ส่วน recall ตอบว่า “ในข้อมูล positive จริง แบบจำลองหาเจอเท่าไร” จึงใช้ $TP+FN$ เป็นตัวส่วน F_1 เป็น harmonic mean ของ precision กับ recall ค่าจึงถูกดึงลงเมื่อค่าใดค่าหนึ่งต่ำ ไม่ใช่ค่าเฉลี่ยเลขคณิต $(P+R)/2$

ตัวอย่างคำนวณทีละขั้น

ให้ $TP=30, FP=10, FN=20, TN=40$ รวม $n=30+10+20+40=100$ ตัวอย่าง

1. Accuracy $= (TP+TN)/n = (30+40)/100 = 0.70$
2. Precision $= TP/(TP+FP) = 30/(30+10) = 30/40 = 0.75$
3. Recall $= TP/(TP+FN) = 30/(30+20) = 30/50 = 0.60$
4. $F_1 = 2PR/(P+R) = 2(0.75)(0.60)/(0.75+0.60) = 0.90/1.35 = 2/3 \approx 0.6667$

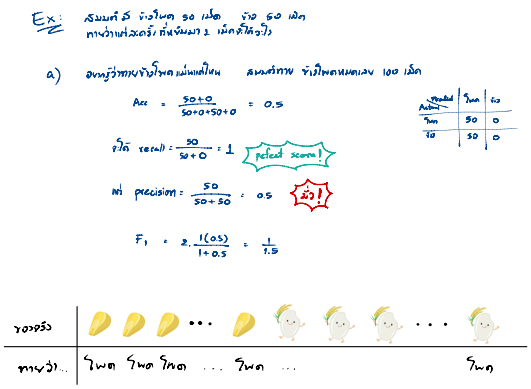
ตรวจอีกทางได้ $F_1 = 2TP/(2TP+FP+FN) = 60/90 = 2/3$ คำตอบทั้งสองวิธีต้องตรงกัน ROC plot ใช้ $TPR = R$ บนแกนตั้ง และ $FPR = FP/(FP+TN)$ บนแกนนอนโดยเปลี่ยน threshold หากลด threshold จะทายบวกมากขึ้น ทำให้ TP และ FP ไม่ลดลงบนชุดคะแนนเดิม AUC สรุปการจัดอันดับข้าม threshold ไม่ใช่ accuracy ที่ threshold หนึ่ง

> **ต้องจำ** Class imbalance ทำให้ accuracy ลวงได้: ถ้าบวกจริงเพียง 1% การทายลบทั้งหมดได้ Acc 99% แต่ Recall 0

ระวัง: ROC-AUC ไม่ใช่ precision และไม่ได้เลือก threshold ให้โดยอัตโนมัติ ข้อความระดับคุณภาพในภาพ L1 p.87 เป็นแนวคร่าว ๆ ไม่ใช่มาตรฐานทุก application

1.17 ตัวอย่างข้าวโพด และ หัวไชเท้า: recall 1 ยังผิดได้ครึ่งหนึ่ง

แหล่งเนื้อหา: L1 PDF p.83–85. อ่านตัวอย่างลายมือที่มองเห็นชัด



รูป 1.14: ตัวอย่างเดิม 50 ข้าวโพด + 50 หัวไชเท้า ทายทุกเม็ดเป็นข้าวโพด

อ่านภาพทีละจุด **1.** กำหนดข้าวโพดเป็น positive ก่อนอ่านตาราง แถว Actual และคอลัมน์ Predict ห้ามสลับ
2. ทุกคำทำนายเป็น positive จึง FN=TN=0 แต่ยังมี FP=50. recall เต็มไม่ได้แปลว่า classifier ดีทุกด้าน

ได้ $TP = 50, FN = 0, FP = 50, TN = 0$ จึง accuracy $= 0.5$, recall $= 50/50 = 1$, precision $= 50/100 = 0.5$ และ $F_1 = 2TP/(2TP + FP + FN) = 100/150 = 2/3$ แหล่งเดิมเขียน $1/1.5$ ซึ่งเท่ากัน ถ้ากำหนด positive เป็นหัวไชเท้า ต้องตั้ง confusion matrix ใหม่ อย่าเพียงสลับคำบรรยายแล้วใช้ TP เดิม

> **ต้องจำ** อ่านชื่อแกน Actual และ Predict ก่อนทุกครั้ง การทาย positive ทั้งหมดทำ recall สูงได้โดยมี false positives มาก

1.18 ฝึก LDA และผลของ prior แบบครบขั้น

แหล่งเนื้อหา: L1 PDF p.69–81. ตัวเลขฝึกแต่งขึ้นใหม่

อ่านเพิ่มจากตำรา ISLP §4.4: ตัวอย่างต่อไปนี้ใช้สูตร discriminant เดิมของ lecture เพื่อแยก mean, variance และ prior ออกจากกัน ไม่ใช่ข้อมูลที่อาจารย์กำหนดให้ท่อง

ให้สอง class มี $\mu_1 = 0, \mu_2 = 2$, variance ร่วม $\sigma^2 = 1$ และ prior เท่ากัน 0.5 คะแนนเป็น

$$\delta_1(x) = \log 0.5, \qquad \delta_2(x) = 2x - 2 + \log 0.5.$$

เมื่อเปรียบเทียบ $\delta_2 - \delta_1 = 2x - 2$ class 2 ชนะเมื่อ $x > 1$ ดังนั้น boundary อยู่กึ่งกลาง means ในกรณีที่ variance ร่วมและ priors เท่ากันนี้ หาก $x = 0.75$ จะได้ score difference $= -0.5$ จึงเลือก class 1

เปลี่ยน prior โดยไม่เปลี่ยน likelihood

ให้ $\pi_1 = 0.8, \pi_2 = 0.2$ จะได้

$$\delta_2 - \delta_1 = 2x - 2 + \log(0.2/0.8) = 2x - 2 - \log 4.$$

class 2 ชนะเมื่อ $x > 1 + \frac{1}{2}\log 4 \approx 1.6931$ Boundary จึงขยับไปทาง mean ของ class 2 เพราะต้องการ evidence มากขึ้นก่อนเลือก class ที่พบได้น้อย ที่ $x = 1.5$ ถ้า priors เท่ากันเลือก class 2 แต่ priors แบบใหม่เลือก class 1

ตรวจจาก posterior อีกวิธี

Likelihood ratio ของ Gaussian ทั้งสองคือ $f_2(x)/f_1(x) = e^{2x-2}$ ที่ $x = 1.5$ ได้ e^1 Posterior odds จึงเป็น $e^1(0.2/0.8) \approx 0.67957$ และ $P(Y = 2 \mid x) = 0.67957/(1 + 0.67957) \approx 0.40461$ ซึ่งต่ำกว่า 0.5 ตรงกับการเปรียบเทียบ discriminants

ต้องจำ Discriminant score ไม่ใช่ probability โดยตรง แต่ score ที่ต่างกันคือ log ของ posterior odds สำหรับสอง class ใน model นี้ การเปลี่ยน prior เปลี่ยน boundary ได้แม้ distribution ภายใน class ไม่เปลี่ยน

ทดสอบตัวเอง: เปลี่ยน prior ของ class 2 เป็น 0.8,class 1 เป็น 0.2 จะได้ boundary $1 - \frac{1}{2}\log 4 \approx 0.3069$ ลองอธิบายทิศทางก่อนคำนวณ

1.19 Threshold, ROC และ cost: เลือกจุดใช้งาน

แหล่งเนื้อหา: L1 PDF p.82–87. intuition เสริมจาก ISLP §4.4

กำหนด label positive เป็น class ที่ต้องการตรวจ Model ให้ probability p แต่การทาย class ต้องเลือก threshold τ การลด threshold ทำให้ กลุ่มที่ทายเป็น positive เพิ่ม ในข้อมูลชุดเดียวกัน TP และ FP จึงไม่ลดลงตามการลด threshold แต่ precision อาจขึ้นหรือลงตามกลุ่มที่เพิ่มเข้ามา

threshold	TP	FP	FN	TN	TPR และ FPR
0.7	20	5	30	45	0.40 / 0.10
0.5	30	10	20	40	0.60 / 0.20
0.3	40	20	10	30	0.80 / 0.40

ทุกแถวมี positive จริง 50 และ negative จริง 50 ROC วาง FPR แกนนอนและ TPR แกนตั้ง แถว threshold ต่ำให้ recall สูงขึ้นแต่ false alarms มากขึ้น AUC วัด ranking across thresholds ไม่ได้เลือก threshold ใช้งานให้โดยอัตโนมัติ และไม่แทน cost-sensitive evaluation ทุกกรณี

เมื่อ error สองแบบมีต้นทุนต่างกัน

อ่านเพิ่มจากตำรา ตัวอย่าง cost ต่อไปนี้ใช้ conditional expected loss เพื่อเข้าใจ threshold ไม่ใช่สูตรที่ annotation ระบุว่าออกสอบ

ถ้า ต้นทุน false positive เป็น c_{FP} และ ต้นทุน false negative เป็น c_{FN} โดยทายถูก cost 0 expected loss เมื่อทาย positive คือ $c_{FP}(1-p)$. เมื่อทาย negative คือ $c_{FN}p$ จึงเลือก positive เมื่อ

$$c_{FP}(1-p) < c_{FN}p \quad \Longleftrightarrow \quad p > \frac{c_{FP}}{c_{FP}+c_{FN}}.$$

ถ้า FN แพงกว่า FP9 เท่า threshold เชิงทฤษฎีเป็น 0.1 ภายใต้ probabilities ที่ calibrated และ cost ที่ระบุ ไม่ใช่ 0.5 ตลอด ตัวอย่างตารางข้างต้นถ้า $c_{FN} = 4, c_{FP} = 1$ cost รวมเป็น 125,90,60 สำหรับ threshold 0.7,0.5,0.3 ตามลำดับ แต่ต้องเลือก threshold ด้วย validation ที่เหมาะกับ ประชากรที่จะนำไปใช้ แล้วเก็บ test ไว้ประเมินสุดท้าย

ระวัง: ค่าทำนายที่อยู่[0,1]ไม่รับประกัน calibration. precision ขึ้นกับ prevalence เมื่อใช้ model กับประชากรใหม่จึงอาจต่างจากชุด validation

ต้องจำ ROC ใช้ TPR กับ FPR ส่วน precision ใช้ TP กับ FP ไม่มี TN ใน denominator และเลือก threshold ไม่เหมือนเลือก model capacity

1.20 Lecture 1: ทบทวนเชื่อมและ Checklist

แหล่งเนื้อหา: L1 PDF p.9–87

คำตอบสั้นที่ควรพูดได้

ML เรียน mapping หรือ pattern จากข้อมูล ภายใต้ task และ metric. Statistical Learning ประมาณ f ใน $Y = f(X) + \varepsilon$. Prediction เน้นข้อมูลใหม่ ส่วน Inference เน้นความสัมพันธ์. Train และ Validation และ Test แยกหน้าที่เพื่อวัด generalization Bayes อัปเดต prior ด้วย evidence. Bayes classifier เลือก posterior มากสุด. LDA และ QDA ประมาณ Gaussian class distributions แล้วใช้ Bayes. metrics ตรวจชนิด error. ROC ดูพฤติกรรมตลอดหลาย threshold

Checklist

- ระบุ supervised และ unsupervised และ classification และ regression จากสถานการณ์ได้
- แยก reducible กับ irreducible และ bias กับ variance ได้
- อธิบาย train และ test leakage และ k-fold ได้
- เขียน Bayes ทั้ง binary และหลาย class ได้
- ทำโจทย์โรคหายากและ spam filter ตาม prior ที่ให้ได้
- หา LDA score และ boundary พร้อม covariance assumption ได้
- เปรียบเทียบ LDA และ QDA และ Logistic แบบ generative และ discriminative ได้
- คำนวณ Accuracy, Precision, Recall, F1, FPR ได้

ต้องจำ สูตรที่ต้องจำ: conditional probability, Bayes, total probability, δ_k ของ LDA และ metrics สมมติฐานที่ต้องจำ: shared covariance และ conditional independence ใน spam example

สะพานไปบทถัดไป

Linear model ของ L1 จากนั้น OLS และ GD ใน L2. Linear decision boundary ของ LDA และ Logistic จากนั้น Maximum margin ใน L3. Bias–Variance จากนั้น Shrinkage ใน L4 และ Ensemble ใน L5. Train และ Validation และ Test จากนั้น NN และ CNN training ใน L6–7

อ่านเพิ่มจากตำรา อ่านต่อ: ISLP Ch.2, §4.4, Ch.5. ไม่เพิ่ม KNN และ advanced Bayesian theory เป็นขอบเขตบังคับเพียงเพราะปรากฏใน textbook

Lecture 2 Training Models

แหล่งเนื้อหา: L2 PDF p.2–3 | ขอบเขตอ่านสอบ Lecture 1–8

คำย่อที่ใช้ในบทนี้

OLS คือ Ordinary Least Squares หรือวิธีกำลังสองน้อยที่สุด. **SSE** คือ Sum of Squared Errors, **RSS** คือ Residual Sum of Squares และ **MSE** คือ Mean Squared Error. SSE กับ RSS ใช้เรียกผลรวมกำลังสองของ residual ในบริบทนี้ ส่วน MSE หารด้วยจำนวนแถว. **GD** คือ Gradient Descent และ **SGD** คือ Stochastic Gradient Descent. **BCE** คือ Binary Cross-Entropy. **SVD** คือ Singular Value Decomposition ส่วน QR เป็นการแยกเมทริกซ์เป็นเมทริกซ์ตั้งฉาก Q กับเมทริกซ์สามเหลี่ยมบน R. การฝึกแบบจำลอง (model) คือการหาค่าพารามิเตอร์ (parameter) ที่ทำให้คำทำนายสอดคล้องกับข้อมูลตามเกณฑ์ที่กำหนด คำว่า “เรียนรู้” ในที่นี้หมายถึงการปรับค่าที่ใช้คำนวณคำทำนาย ไม่ใช่แก้คำตอบของแต่ละแถวให้ตรงกับเฉลย บทนี้อธิบายสองวิธีหาพารามิเตอร์ ได้แก่ การแก้สมการโดยตรง (closed-form) และการปรับค่าซ้ำตาม gradient (iterative optimization)

Closed-form กับ iterative ต่างกันตรงไหน

วิธี closed-form คำนวณคำตอบจากสมการชุดหนึ่ง เช่น แก้ normal equation ของ OLS ส่วนวิธี iterative เริ่มจากค่าตั้งต้น คำนวณ gradient แล้วปรับพารามิเตอร์ซ้ำ ทั้งสองวิธีอาจมุ่งลด objective เดียวกัน แต่ใช้เวลาและมีเงื่อนไขการลู่เข้าต่างกัน

ประเด็น	Direct approach	Iterative approach
วิธีหา parameter	แก้สมการ และ least-squares	ปรับค่าเป็นรอบตาม gradient
การหยุด	เมื่อคำนวณคำตอบเสร็จ	ใช้ stopping criterion
จุดต้องระวัง	rank, conditioning, จำนวน feature	learning rate, scaling, convergence

> **ต้องจำ** แยก model, objective และ optimizer: Linear Regression คือ model. MSE คือ objective. OLS และ Gradient Descent เป็นวิธีหา parameter ที่ทำให้ objective ต่ำ

ฝึกให้ทำได้ อธิบายได้ว่าทำไม OLS และ GD อาจได้คำตอบเดียวกัน แต่ GD ไม่ได้เท่ากับ Linear Regression เพราะใช้กับ objective อื่นได้ด้วย

อ่านเพิ่มจากตำรา Géron Ch.4, Training Models: OLS แบบ full-rank มี minimum ของ convex quadratic เดียวกันกับที่ GD มุ่งไปเมื่อกำหนด step อย่างเหมาะสม

2.1 สมการ Linear Regression และ residual

แหล่งเนื้อหา: L2 PDF p.4–6

สไลด์ใช้ $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$ โดย x_j คือตัวแปรอธิบายตัวที่ j, θ_j คือ coefficient ของตัวแปรนั้น และ θ_0 คือ intercept สำหรับ simple regression เขียนเป็น $Y_i = \alpha + \beta X_i + \varepsilon_i$: Y_i เป็นค่าจริงของแถวที่ i, X_i เป็นค่าป้อน, α เป็น intercept, β เป็น slope และ ε_i เป็น error ที่แบบจำลองอธิบายไม่ได้ ระวังว่า n อาจหมายถึงจำนวน features แต่ m ในผลรวมถัดไปหมายถึงจำนวนตัวอย่าง

$$\hat{y}_i = \alpha + \beta x_i, \qquad e_i = y_i - \hat{y}_i, \qquad S(\alpha, \beta) = \sum_{i=1}^{m} (y_i - \alpha - \beta x_i)^2.$$

ε_i คือ error ในแบบจำลองทางสถิติ ส่วน e_i คือ residual ที่คำนวณได้จากข้อมูลหลังหาค่า $\hat{y}_i$ แล้ว ทั้งสองเกี่ยวข้องกัน แต่ไม่ใช่สิ่งเดียวกัน การยกกำลังสองทำให้ residual บวกกับลบไม่หักล้างกัน และทำให้ความผิดพลาดขนาดใหญ่มีผลต่อ

objective มากขึ้น สไลด์ระบุ error แบบ normal, independent และ variance คงที่ใน simple regression สมมติฐานนี้ช่วยอธิบายโมเดลทางสถิติ ส่วนการ minimize squared error เป็นปัญหา optimization ที่คำนวณได้แม้เราไม่ได้พิสูจน์ว่า error จริงเป็น normal

ตัวอย่างคำนวณทีละขั้น

ใช้ $\hat{y} = 2 + 3x$ กับข้อมูลสองแถวที่ $x = 2$ โดยมี $y_1 = 9, y_2 = 6$

1. คำทำนายทั้งสองแถวคือ $2 + 3(2) = 8$
2. Residual แถวแรก $e_1 = 9 - 8 = 1$ และ $e_1^2 = 1$ ส่วนแถวที่สอง $e_2 = 6 - 8 = -2$ และ $e_2^2 = 4$
3. ผลรวมกำลังสอง SSE $= 1 + 4 = 5$ แล้วหารจำนวนแถวเพื่อหา MSE $= 5/2 = 2.5$

> **ต้องจำ** SSE และ RSS คือผลรวม. MSE คือผลเฉลี่ย การหารด้วยจำนวน sample ไม่เปลี่ยน OLS minimizer เมื่อไม่มี penalty แต่เปลี่ยนขนาด gradient

ทดสอบตัวเอง: ถ้าใช้ $(\hat{y} - y)$ แทน $(y - \hat{y})$ ใน squared loss จะต่างหรือไม่? ไม่ต่างสำหรับค่าของ loss แต่ตอนหา derivative ต้องรักษาเครื่องหมายให้สอดคล้อง

2.2 ที่มาของ normal equation

แหล่งเนื้อหา: L2 PDF p.7–10

เริ่มจากผลรวมกำลังสองของ residual $S = \sum_i (y_i - \alpha - \beta x_i)^2$ โดยข้อมูล x_i, y_i เป็นค่าที่ทราบแล้ว สิ่งที่ต้องหา คือ intercept α และ slope β จึงหาอนุพันธ์ย่อยเทียบกับพารามิเตอร์สองตัวนี้ แล้วตั้งให้เป็นศูนย์

$$\frac{\partial S}{\partial \alpha} = -2 \sum_i (y_i - \alpha - \beta x_i) = 0,$$
$$\frac{\partial S}{\partial \beta} = -2 \sum_i x_i (y_i - \alpha - \beta x_i) = 0.$$

เมื่อขยายอนุพันธ์ จะได้ผลรวมของ residual และผลรวม residual ที่คูณด้วย x_i จากนั้นย้ายพจน์ที่มี α, β ไปทางซ้าย และคงผลรวมจากข้อมูลไว้ทางขวา:

$$m\alpha + \beta \sum_i x_i = \sum_i y_i, \qquad \alpha \sum_i x_i + \beta \sum_i x_i^2 = \sum_i x_i y_i.$$

$$\begin{bmatrix} m & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}.$$

จึงได้ระบบสมการเชิงเส้นสองสมการสำหรับหาพารามิเตอร์สองตัว แม้ฟังก์ชันเป้าหมายเริ่มต้นจะเป็นกำลังสองก็ตาม เมื่อจัดอนุพันธ์ให้เท่ากับศูนย์แล้ว เราแก้ระบบนี้เพื่อหาค่า intercept และ slope ได้โดยตรง

> **ต้องจำ** ต้องจำ derivative และ normal equations เพราะเป็นสะพานระหว่าง “least squares” กับสูตรสำเร็จ จำแต่ slope formula แล้วอธิบายที่มาไม่ได้ยังไม่พอ

ฝึกให้ทำได้ หากถาม derivation: เขียน objective, หา partial derivative, ตั้งศูนย์, จัดรูป และระบุเงื่อนไขว่าข้อมูลต้องมีความแปรผันใน x

ระวัง: stationary point ไม่จำเป็นต้องเป็น minimum สำหรับทุก function แต่ OLS objective เป็น convex quadratic จึงใช้โครงสร้างนี้ได้

2.3 สูตร intercept และ slope และ Cramer's Rule

แหล่งเนื้อหา: L2 PDF p.10–12

กำหนด $D = m\sum x_i^2 - (\sum x_i)^2$ เป็น determinant ของระบบ normal equations ถ้า $D \neq 0$ จะได้คำตอบ unique:

$$\beta = \frac{m\sum x_i y_i - (\sum x_i)(\sum y_i)}{D}, \quad \alpha = \frac{(\sum y_i)(\sum x_i^2) - (\sum x_i)(\sum x_i y_i)}{D}.$$

จาก normal equation แรก หารด้วย m ได้ $\alpha + \beta\bar{x} = \bar{y}$ จึง $\alpha = \bar{y} - \beta\bar{x}$ แทนในสมการที่สองแล้วเก็บเทอม β

$$\begin{aligned}(\bar{y} - \beta\bar{x})\sum_i x_i + \beta\sum_i x_i^2 &= \sum_i x_i y_i,\\ \beta\left[\sum_i x_i^2 - m\bar{x}^2\right] &= \sum_i x_i y_i - m\bar{x}\bar{y},\\ \beta &= \frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sum_i (x_i-\bar{x})^2}.\end{aligned}$$

ตัวส่วนต้องเป็นบวก กล่าวคือค่า x ไม่เท่ากันทั้งหมด อีกวิธีที่สั้นกว่าคือ $\beta = \sum_i (x_i-\bar{x})(y_i-\bar{y}) / \sum_i (x_i-\bar{x})^2$ และ $\alpha = \bar{y} - \beta\bar{x}$ ซึ่งได้ค่าเดียวกัน สูตรแรกตรงกับเส้นทาง Cramer's Rule ในสไลด์ สูตรที่สองช่วยตรวจคำตอบ

ตัวอย่างคำนวณทีละขั้น

ใช้ข้อมูล $(1,2),(2,3),(3,5)$ สร้างตารางก่อนแทนสูตร

x	y	x^2	xy
1	2	1	2
2	3	4	6
3	5	9	15
6	10	14	23

1. $m = 3$ และ $D = m\sum x^2 - (\sum x)^2 = 3(14) - 6^2 = 42 - 36 = 6$
2. $\beta = [3(23) - 6(10)]/6 = (69-60)/6 = 1.5$
3. $\alpha = [10(14) - 6(23)]/6 = (140-138)/6 = 1/3$ จึง $\hat{y} = 1/3 + 1.5x$
4. ที่ $x = 1,2,3$ ได้ $\hat{y} = (11/6, 10/3, 29/6)$ ตามลำดับ จึง $e = (2-11/6, 3-10/3, 5-29/6) = (1/6, -1/3, 1/6)$
5. $\text{SSE} = (1/6)^2 + (-1/3)^2 + (1/6)^2 = 1/36 + 4/36 + 1/36 = 1/6$ และ $\text{MSE} = (1/6)/3 = 1/18$

ตรวจด้วย $\sum e_i = 1/6 - 1/3 + 1/6 = 0$ และ $\hat{y}(\bar{x}) = 1/3 + 1.5(2) = 10/3 = \bar{y}$

> **ต้องจำ** ทำตาราง x, y, x^2, xy ก่อนแทนสูตร จะลดการตกหล่นของ summation ตรวจด้วยการแทน parameter กลับไปคำนวณ prediction

ระวัง: หากทุก x_i เท่ากัน จะได้ $D = 0$ ไม่มีข้อมูลให้แยก slope ออกจาก intercept ไม่ใช่ปัญหาที่แก้ได้ด้วยการหารเลขเดิมซ้ำ

2.4 Matrix OLS และ pseudo inverse

แหล่งเนื้อหา: L2 PDF p.13–15

สไลด์ใช้ $Y = AB$ โดย A ขนาด $N \times M$ เป็น design matrix, Y ขนาด $N \times 1$ เป็น response และ B ขนาด $M \times 1$ เป็น parameter vector คอลัมน์ของ 1 แทน intercept จะวางแรกหรือท้ายก็ได้ แต่ลำดับ parameter ต้องตามคอลัมน์

$$S(B) = \|AB - Y\|_2^2, \qquad \nabla_B S = 2A^T(AB - Y).$$

ตั้ง gradient เป็นศูนย์ได้ $A^TAB = A^TY$ ถ้า A มี full column rank:

$$\hat{B} = (A^TA)^{-1}A^TY.$$

ตรวจมิติ: $(A^TA)^{-1}$ มีขนาด $M \times M$, A^T มีขนาด $M \times N$, Y มีขนาด $N \times 1$ จึงได้คำตอบ $M \times 1$ ในกรณีมีข้อมูลมากกว่า parameter เราแก้ $AB = Y$ แบบ exact ไม่ได้เสมอ จึงหาคำตอบที่ลด residual norm

อ่านเพิ่มจากตำรา Géron Ch.4: Moore–Penrose pseudo inverse A^+ มีความหมายทั่วไปกว่าสูตร $(A^TA)^{-1}A^T$ เมื่อ rank ไม่เต็ม ใช้ $\hat{B} = A^+Y$ ได้ ในงานจริง QR และ SVD ช่วยลดปัญหา numerical conditioning ไม่ควรจำว่า pseudo inverse ทุกกรณีคำนวณโดย inverse ได้

> **ต้องจำ** Normal equation เป็นเงื่อนไข. inverse formula ต้องมีเงื่อนไข rank ส่วน pseudo inverse ใช้ได้กว้างกว่า

ฝึกให้ทำได้ เขียน matrix จากโจทย์แล้วบอกมิติแต่ละตัว ตรวจว่าคอลัมน์ bias อยู่ตรงไหน และอธิบายได้ว่าทำไม least-squares residual ไม่ต้องเป็นศูนย์ทุกแถว

2.5 Matrix OLS: ขยายการคูณและตรวจคำตอบจาก residual

แหล่งเนื้อหา: L2 PDF p.7–16. ตัวเลขฝึกใหม่

ให้ $(x, y) = (0, 1), (1, 2), (2, 4)$ และลำดับ coefficient เป็น $\theta = (b, w)^T$ ดังนั้น design matrix มี intercept เป็น column แรก

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad A^TA = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad A^Ty = \begin{bmatrix} 7 \\ 10 \end{bmatrix}.$$

1. ช่อง $(1,1)$ เป็นผลรวม $1^2 + 1^2 + 1^2 = 3$. ช่อง $(1,2)$ และ $(2,1)$ คือ $\sum x = 3$. ช่อง $(2,2)$ คือ $\sum x^2 = 5$
2. $\det(A^TA) = 3(5) - 3(3) = 6 \neq 0$ จึง inverse ได้ในตัวอย่างนี้ ไม่สมมติว่าทุกข้อมูล inverse ได้
3. $(A^TA)^{-1} = \frac{1}{6}\begin{bmatrix} 5 & -3 \\ -3 & 3 \end{bmatrix}$ จากสูตร inverse 2×2
4. $\theta = \frac{1}{6}(35 - 30, -21 + 30)^T = (5/6, 3/2)^T$ จึง $\hat{y} = 5/6 + 1.5x$
5. แทน $x = 0, 1, 2$ ใน $\hat{y} = 5/6 + (3/2)x$ ได้ $(5/6, 14/6, 23/6) = (5/6, 7/3, 23/6)$
6. Residual $e = y - \hat{y} = (1 - 5/6, 2 - 7/3, 4 - 23/6) = (1/6, -1/3, 1/6)$
7. $RSS = (1/6)^2 + (-1/3)^2 + (1/6)^2 = 1/36 + 4/36 + 1/36 = 1/6$ และ $MSE = RSS/3 = 1/18$
8. ตรวจ $A^Te = (\sum e, \sum xe)^T = (0, 0)^T$ นี่คือ orthogonality จาก normal equation ไม่ใช่ residual ทุกตัวต้องเป็น 0

เมื่อ A^TA singular ควรตอบอย่างไร

ถ้า predictor $x_2 = 2x_1$ ทุกแถว columns พึ่งพากัน จึงมีหลาย coefficients ที่ให้ prediction เดียวกัน เช่น $w_1x_1 + w_2x_2 = (w_1 + 2w_2)x_1$ จะระบุ w_1, w_2 แยกอย่าง unique ไม่ได้จากข้อมูลนี้ ใช้ pseudo inverse สำหรับ least squares หรือ regularization ที่ระบุชัด ไม่แก้ด้วยการเปลี่ยนตัวเลข determinant ตามใจ

> **ต้องจำ** ก่อน inverse ต้องตรวจ rank และ เงื่อนไข. coefficients เปลี่ยนตามลำดับ columns แต่ predictions ต้องไม่เปลี่ยนเมื่อจัดลำดับสอดคล้องกัน

2.6 ตัวอย่าง Multiple Regression ของอาจารย์

แหล่งเนื้อหา: L2 PDF p.13–16. คำนวณซ้ำจากข้อมูลต้นฉบับ

$$A=\begin{bmatrix}1&2&1\\2&1&1\\3&2&1\\1&1&1\end{bmatrix},\qquad Y=\begin{bmatrix}12\\9\\19\\8\end{bmatrix},\qquad B=\begin{bmatrix}c_1\\c_2\\c_3\end{bmatrix}.$$

ใช้ $y=c_1x_1+c_2x_2+c_3$ จึงวาง intercept เป็นคอลัมน์สุดท้าย คูณ matrix ได้

$$A^TA=\begin{bmatrix}15&11&7\\11&10&6\\7&6&4\end{bmatrix},\quad A^TY=\begin{bmatrix}95\\79\\48\end{bmatrix}.$$

ตั้งระบบสามสมการแล้วกำจัดตัวแปร โดยให้ (1),(2),(3) เป็นแต่ละแถวของ normal equations

$$\begin{aligned}
15c_1+11c_2+7c_3&=95 && (1)\\
11c_1+10c_2+6c_3&=79 && (2)\\
7c_1+6c_2+4c_3&=48 && (3)\\
4(1)-7(3):\quad 11c_1+2c_2&=44 && (4)\\
2(2)-3(3):\quad c_1+2c_2&=14 && (5)\\
(4)-(5):\quad 10c_1&=30 \quad\Rightarrow c_1=3.\\
3+2c_2&=14 \quad\Rightarrow c_2=11/2=5.5,\\
7(3)+6(5.5)+4c_3&=48 \quad\Rightarrow 4c_3=48-21-33=-6 \quad\Rightarrow c_3=-1.5.
\end{aligned}$$

ตรวจกลับในสมการแรกได้ $15(3)+11(5.5)+7(-1.5)=45+60.5-10.5=95$ แล้วคำนวณค่าทำนายและ residual ต่อดังตาราง

x_1	x_2	y	$\hat{y}$	$(y-\hat{y})^2$
1	2	12	12.5	0.25
2	1	9	10	1
3	2	19	18.5	0.25
1	1	8	7	1

SSE $=2.5$ และ MSE $=2.5/4=0.625$ ค่าทำนายแถวสุดท้ายต้องเป็น $3+5.5-1.5=7$ ไม่ใช่ 6.5 แม้ตัวเลขลายมือบางส่วนในหน้า 16 จะอ่านได้ไม่ชัด

> **ต้องจำ** จำโครงสร้างการทำ ไม่ต้องจำชุดตัวเลข: สร้าง A จากนั้น หา A^TA, A^TY จากนั้น แก้ parameter จากนั้น predict จากนั้น residual จากนั้น MSE

ทดสอบตัวเอง: ตรวจ $A^T(Y-A\hat{B})=0$ จะได้ทั้งสามองค์ประกอบเป็นศูนย์ เป็นเงื่อนไข OLS ที่ใช้ตรวจคำตอบได้อีกทาง

2.7 Gradient Descent: ทิศและขนาดของการก้าว

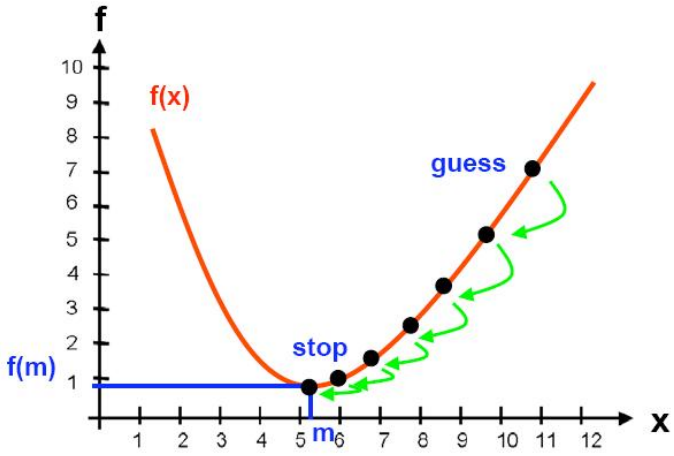
แหล่งเนื้อหา: L2 PDF p.17–30

เกรเดียนต์ (gradient) คือเวกเตอร์ที่รวบรวมอนุพันธ์ย่อยของฟังก์ชันเป้าหมายตามพารามิเตอร์แต่ละตัว ณ จุดปัจจุบัน มันชี้ไปทางที่ค่าฟังก์ชันเพิ่มเร็วที่สุดในระยะสั้น ดังนั้นถ้าต้องการลดค่าเป้าหมาย เราจึงก้าวไปทางตรงข้ามกับ gradient

และคำนวณ gradient ใหม่ทุกครั้งหลังพารามิเตอร์เปลี่ยน

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta J(\theta^{(t)}), \qquad \eta > 0.$$

ในสูตรนี้ $\theta^{(t)}$ คือพารามิเตอร์ก่อนก้าวที่ t, J คือค่าฟังก์ชันเป้าหมาย และ η คือ learning rate ซึ่งกำหนดขนาดก้าว ส่วน $\nabla_\theta J$ คือ gradient ที่คำนวณจากพารามิเตอร์ชุดเดิม ต้องคำนวณทุกองค์ประกอบก่อน แล้วจึงอัปเดตพร้อมกัน เมื่อ gradient มีขนาดเล็ก การเปลี่ยนพารามิเตอร์ในรอบนั้นก็มักเล็กลงด้วย อย่างไรก็ตาม ฟังก์ชันที่ไม่เป็น convex อาจมี local minimum หรือ saddle point จึงสรุปไม่ได้จาก gradient ที่เล็กเพียงอย่างเดียวว่าได้ global minimum แล้ว



รูป 2.1: ลำดับ GD ในสไลด์: ประเมิน gradient ใหม่ทุกจุด แล้วก้าวในทิศตรงข้ามจนเข้าใกล้ minimum (L2 p.24)

อ่านภาพทีละจุด **1.** แต่ละตำแหน่งบนผิว และ เส้น loss คือชุด parameters ไม่ใช่ sample หนึ่งคน

2. ลูกศรการก้าวต้องเป็น negative gradient ที่ตำแหน่งปัจจุบัน ความยาวขึ้นกับ learning rate ต้องประเมิน gradient ใหม่ ไม่ใช้ลูกศรแรกซ้ำจนจบ

ตัวอย่างคำนวณทีละขั้น

ให้ $J(w) = (w-3)^2$, $w_0 = 0$ และ $\eta = 0.1$ อนุพันธ์คือ $J'(w) = 2(w-3)$

1. ก่อนฝึก $J(w_0) = (0-3)^2 = 9$
2. รอบแรก $g_0 = 2(0-3) = -6$ จึง $w_1 = w_0 - \eta g_0 = 0 - 0.1(-6) = 0.6$ และ $J(w_1) = (0.6-3)^2 = (-2.4)^2 = 5.76$
3. ประเมิน gradient ใหม่ที่ w_1: $g_1 = 2(0.6-3) = -4.8$ จึง $w_2 = 0.6 - 0.1(-4.8) = 1.08$
4. ตรวจ loss หลังรอบที่สอง: $J(w_2) = (1.08-3)^2 = (-1.92)^2 = 3.6864$ ซึ่งลดลงจาก 5.76

> **ต้องจำ** ใน update มีเครื่องหมายลบ. gradient เป็นลบทำให้ parameter เพิ่ม เพราะต้องเดินไปทางขวาเพื่อลด cost

2.8 Learning rate, scaling และ stopping

แหล่งเนื้อหา: L2 PDF p.17–33. Explanation เสริมจาก Géron Ch.4

Learning rate เล็กทำให้ก้าวช้าและอาจหยุดก่อนเข้าใกล้ optimum ถ้าจำกัดจำนวนรอบ Learning rate ใหญ่อาจกระโดดข้าม minimum, oscillate หรือ diverge การตั้ง η มากขึ้นจึงไม่ได้รับประกันว่าจบเร็วขึ้น สำหรับ $J(w) = (w-3)^2$ เขียน error จาก optimum เป็น $d_t = w_t - 3$ จะได้ $d_{t+1} = (1-2\eta)d_t$ จึงลู่เข้าเมื่อ $|1-2\eta| < 1$, หรือ $0 < \eta < 1$ เงื่อนไขนี้ใช้เฉพาะ function ตัวอย่าง ไม่ใช่สูตร learning rate ที่ดีของทุก model Feature ต่าง scale ทำให้ contour ยืดยาวและการก้าว zigzag การ standardize มักช่วย conditioning ของ optimizer ต้อง fit mean และ std จาก training set แล้วใช้ค่าเดิมกับ validation และ test เพื่อไม่ให้ข้อมูลอนาคตรั่วไหล

Stopping criterion ที่อธิบายได้

หยุดเมื่อ gradient norm ต่ำ, loss เปลี่ยนน้อย, ถึงจำนวนรอบสูงสุด หรือ validation performance ไม่ดีขึ้น การตั้ง stop จาก training loss เพียงอย่างเดียวไม่ตรวจ overfitting

> **ต้องจำ** แยก optimization failure กับ generalization failure: loss ไม่ลงอาจเกี่ยวกับ η. training loss ลงแต่ validation แย่ลงอาจเป็น overfitting

ฝึกให้ทำได้ ดูกราฟ loss แล้วอธิบายเหตุ: ลดช้ามาก, แกว่ง, พุ่งขึ้น หรือ train และ validation แยกกัน ต้องระบุการแก้ที่ตรงกับปัญหา

ระวัง: สไลด์ที่กล่าวว่า GD ลงได้เสมอ ต้องอ่านภายใต้ step ที่เหมาะสม สำหรับ SGD loss ราย step อาจขึ้นลงเพราะ gradient มี noise

2.9 Batch, Stochastic และ Mini-batch GD

แหล่งเนื้อหา: L2 PDF p.31–33

สำหรับ linear regression ที่ใช้ MSE แบบไม่มี $1/2$:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\theta^T x^{(i)} - y^{(i)})^2, \quad \frac{\partial J}{\partial \theta_j} = \frac{2}{m}\sum_{i}(\hat{y}^{(i)} - y^{(i)})x_j^{(i)}.$$

ให้ $x_0^{(i)} = 1$ จะใช้สูตรเดียวกับ bias ได้ Batch ใช้ทุกตัวอย่างก่อน update. SGD ใช้หนึ่งตัวอย่าง. Mini-batch ใช้ b ตัวอย่างและคำนวณเฉลี่ย gradient ของ batch นั้น ต้อง update parameter ทั้งหมดจาก gradient ที่คำนวณด้วยค่าเก่าชุดเดียวกัน

ประเด็น	Batch GD	SGD และ Mini-batch
ข้อมูลต่อ step	ทุก sample	1 / b samples
เส้นทาง	เรียบกว่า	noisy และ ปานกลาง
ต้นทุนต่อ step	สูงเมื่อข้อมูลมาก	ต่ำกว่าและ vectorize mini-batch ได้

ตัวอย่างคำนวณทีละขั้น

ใช้ $(x, y) = (1, 2), (2, 4)$ เริ่ม $\theta_0 = \theta_1 = 0$ และ $\eta = 0.1$ โดยใช้ MSE ที่ไม่มีตัวคูณ $1/2$

1. คำทำนายเก่าทั้งสองแถวเป็น 0 จึง $r = \hat{y} - y = (-2, -4)$
2. Bias gradient: $g_0 = (2/2)[(-2)(1) + (-4)(1)] = -6$
3. Slope gradient: $g_1 = (2/2)[(-2)(1) + (-4)(2)] = -2 - 8 = -10$
4. อัปเดตพร้อมกัน: $\theta_0' = 0 - 0.1(-6) = 0.6$ และ $\theta_1' = 0 - 0.1(-10) = 1$
5. คำทำนายใหม่เป็น $(0.6 + 1(1), 0.6 + 1(2)) = (1.6, 2.6)$ จึง MSE ใหม่ $= [(1.6 - 2)^2 + (2.6 - 4)^2]/2 = (0.16 + 1.96)/2 = 1.06$ ลดจาก $(4 + 16)/2 = 10$

> **ต้องจำ** SGD ไม่ใช่สุ่มค่า parameter ไปเรื่อย ๆ แต่สุ่ม sample เพื่อประมาณ gradient. “batch size” ไม่ใช่ learning rate

ทดสอบตัวเอง: ถ้า objective เปลี่ยนเป็น $\frac{1}{2m}\sum e_i^2$ gradient จะลดครึ่งหนึ่ง ต้องดู normalization ในโจทย์ก่อนใช้สูตร

2.10 Logistic Regression: จาก score เป็น probability

แหล่งเนื้อหา: L2 PDF p.34–35

แม้ชื่อมี Regression แต่ Logistic Regression ใช้กับ classification เพราะ output เป็น probability ของ class ที่กำหนด ให้ score $z = \theta_0 + \sum_j \theta_j x_j$ แล้วแปลงด้วย sigmoid:

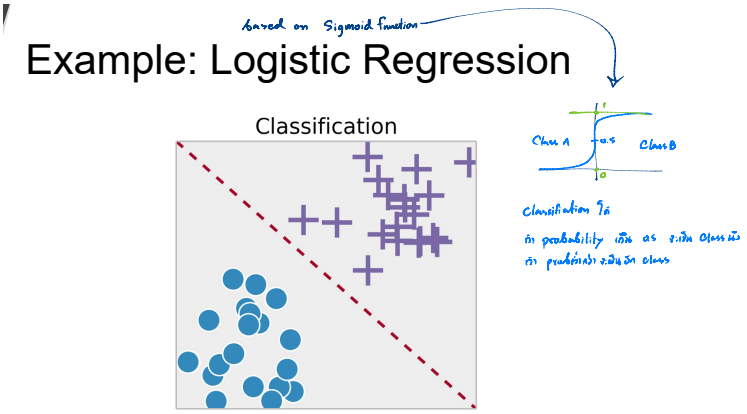
$$H_\theta(x) = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad \log \frac{H_\theta(x)}{1-H_\theta(x)} = z.$$

Score อยู่ได้ทุกจำนวนจริง. probability อยู่ระหว่าง 0 กับ 1. class เกิดจาก threshold ทั้งสามสิ่งเป็นคนละชนิดคำตอบ ถ้า threshold 0.5 จะมี $H_\theta(x) \geq 0.5$ เมื่อ $z \geq 0$ จึงได้ linear decision boundary $\theta_0 + \theta^T x = 0$ แม้ sigmoid เป็น nonlinear function

ตัวอย่างคำนวณทีละขั้น

ให้ $z = -1 + 2x$ และทาย class 1 เมื่อ $p \geq 0.5$

1. ที่ $x = 1$: $z = -1 + 2(1) = 1$, $e^{-z} = e^{-1} \approx 0.367879$ จึง $p = 1/(1 + 0.367879) \approx 0.731059$ และทาย class 1
2. ที่ $x = 0$: $z = -1 + 2(0) = -1$, $e^{-z} = e^{1} \approx 2.718282$ จึง $p = 1/(1 + 2.718282) \approx 0.268941$ และทาย class 0



รูป 2.2: ข้อมูล classification ต้องแปลง score เป็น probability ก่อนใช้ threshold ไม่ใช่แทน output แบบ OLS โดยตรง (L2 p.34)
อ่านภาพทีละจุด **1.** score ของเส้น regression มีช่วงไม่จำกัด แต่ probability สำหรับ class ต้องอยู่ระหว่าง 0 กับ 1

2. ตามเส้น sigmoid จากค่า score ไปแกน probability แล้วจึงใช้ threshold. threshold ไม่ใช่ขั้นสร้างสมการ linear score และไม่ใช่ loss

> **ต้องจำ** จำ sigmoid, log-odds และความสัมพันธ์ $z = 0 \Leftrightarrow p = 0.5$. probability 0.8 ไม่ได้หมายความว่ามี class “0.8”

อ่านเพิ่มจากตำรา ISLP §4.3: coefficient ของ Logistic อธิบายการเปลี่ยน log-odds หาก x_j เพิ่มหนึ่งหน่วย odds คูณด้วย e^{θ_j} เมื่อ feature อื่นคงที่

2.11 Logistic loss และ gradient

แหล่งเนื้อหา: L2 PDF p.36–38

Binary cross-entropy และ negative log-likelihood ใช้ label $y_i \in \{0, 1\}$ และ probability $p_i = H_\theta(x_i)$:

$$J(\theta) = -\frac{1}{m} \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

ถ้า $y = 1$ จะเหลือ $-\log p$. ถ้า $y = 0$ จะเหลือ $-\log(1-p)$ การทำนายผิดอย่างมั่นใจมี loss สูงกว่าไม่มั่นใจ เช่น $y = 1, p = 0.9$ ได้ 0.1054 แต่ $p = 0.1$ ได้ 2.3026 (ใช้ natural log) Sigmoid derivative คือ $p(1-p)$ แต่เมื่อใช้คู่กับ cross-entropy แล้ว chain rule จะยุบให้ gradient ง่าย:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m}\sum_i (p_i - y_i)x_{ij}, \quad \theta_j \leftarrow \theta_j - \eta\frac{1}{m}\sum_i (p_i - y_i)x_{ij}.$$

ทำไม sigmoid คู่ cross-entropy จึงเหลือ $p - y$

สำหรับหนึ่งแถว $L = -[y \ln p + (1-y)\ln(1-p)]$ และ $p = \sigma(z)$ โดย $0 < p < 1$

$$\begin{aligned}
\frac{\partial L}{\partial p} &= -\frac{y}{p} + \frac{1-y}{1-p} = \frac{-y(1-p) + (1-y)p}{p(1-p)} = \frac{p-y}{p(1-p)}, \\
\frac{dp}{dz} &= \frac{e^{-z}}{(1+e^{-z})^2} = p(1-p), \\
\frac{\partial L}{\partial z} &= \frac{p-y}{p(1-p)}p(1-p) = p - y, \\
\frac{\partial L}{\partial w_j} &= (p-y)x_j, \qquad \frac{\partial L}{\partial b} = p - y.
\end{aligned}$$

Mean loss ของ m แถวจึงนำ gradients รายแถวมารวมแล้วหาร m ใช้ค่าของทุก parameter ก่อน update ชุดเดียวกัน ไม่คูณ derivative ของ sigmoid ซ้ำหลังได้ $p - y$ แล้ว

ตัวอย่างคำนวณทีละขั้น

ใช้ $x = (0, 1)$, $y = (0, 1)$ เริ่ม $(\theta_0, \theta_1) = (0, 0)$ และ $\eta = 0.2$

1. $z_1 = z_2 = 0$ จึง $p_1 = p_2 = \sigma(0) = 0.5$
2. ค่าต่าง $p - y$ คือ $(0.5 - 0, 0.5 - 1) = (0.5, -0.5)$
3. Bias gradient $g_0 = (0.5 - 0.5)/2 = 0$ ส่วน slope gradient $g_1 = [0(0.5) + 1(-0.5)]/2 = -0.25$
4. อัปเดต $\theta_0' = 0 - 0.2(0) = 0$, $\theta_1' = 0 - 0.2(-0.25) = 0.05$
5. คำทำนายใหม่เป็น $\sigma(0) = 0.5$ และ $\sigma(0.05) \approx 0.512497$ ค่า mean BCE ใหม่ $= [-\ln(1-0.5) - \ln 0.512497]/2 \approx 0.680803$ ลดจาก $-\ln 0.5 \approx 0.693147$

> **ต้องจำ** MSE gradient มี factor 2 ตาม convention ของ L2. Logistic cross-entropy gradient ไม่มี factor 2 สูตรคล้ายกันแต่ไม่เหมือนกันทั้งหมด

ฝึกให้ทำได้ อธิบายความหมายของ loss, คำนวณค่าจาก probability, หา gradient และ update หนึ่งรอบ โดยยังไม่เปลี่ยน probability เป็น hard class ก่อนหา loss

2.12 Batch Logistic GD: มี intercept และคำนวณทุก sample

แหล่งเนื้อหา: L2 PDF p.34–38. ตัวเลขฝึกใหม่

ใช้ $x = (0, 1, 2)$, $y = (0, 0, 1)$, $b = w = 0$, mean BCE และ $\eta = 0.3$ โดย $z_i = b + wx_i$ ทุก sample เริ่ม $p_i = 0.5$

x_i	y_i	p_i	$p_i - y_i$	$(p_i - y_i)x_i$
0	0	0.5	0.5	0
1	0	0.5	0.5	0.5
2	1	0.5	-0.5	-1

$$\frac{\partial J}{\partial b} = \frac{0.5 + 0.5 - 0.5}{3} = \frac{1}{6}, \quad \frac{\partial J}{\partial w} = \frac{0 + 0.5 - 1}{3} = -\frac{1}{6}.$$

อัปเดตจากค่าเก่าพร้อมกันแล้วตรวจ forward ใหม่

$$b^{new} = 0 - 0.3(1/6) = -0.05, \qquad w^{new} = 0 - 0.3(-1/6) = 0.05,$$
$$z_i^{new} = -0.05 + 0.05x_i,$$
$$z^{new} = (-0.05, 0, 0.05),$$
$$p^{new} = (\sigma(-0.05), \sigma(0), \sigma(0.05)) \approx (0.487503, 0.5, 0.512497),$$
$$J_{new} = \frac{-\ln(1 - 0.487503) - \ln(1 - 0.5) - \ln 0.512497}{3}$$
$$\approx \frac{0.668460 + 0.693147 + 0.668460}{3} = 0.676689.$$

ก่อนอัปเดตทุก probability เท่ากับ 0.5 จึง $J_{old} = -\ln 0.5 \approx 0.693147$ ค่า loss ลดลงจริง

อ่านทิศการเปลี่ยนแบบมีเหตุผล

มี negative สองคนและ positive หนึ่งคน จึง gradient ของ bias เป็นบวกทำให้ bias ลด ส่วน positive อยู่ที่ x สูงจึง gradient ของ slope เป็นลบทำให้ slope เพิ่ม ไม่ได้หมายความว่า gradient ของทุก parameter ต้องมี sign เดียวกัน

ระวัง: BCE+sigmoid gradient เป็น $(p-y)x$ ไม่คูณ $p(1-p)$ ซ้ำ เพราะยกเลิกกันใน chain rule แล้ว ส่วน MSE+sigmoid มี derivative อีกแบบ

> **ต้องจำ** เขียนตาราง $z, p, p-y, (p-y)x$ แล้วรวมตาม convention การทาย hard class ก่อนคำนวณ gradient ทำให้เสียข้อมูล probability

2.13 Gradient Descent หลายก้าวและผลของ normalization

แหล่งเนื้อหา: L2 PDF p.17–33. ตัวอย่างฝึกแต่งขึ้นใหม่

ใช้ข้อมูล $(x, y) = (0, 1), (1, 3)$ กับ $\hat{y} = wx + b$ และ $J = \frac{1}{2}\sum_i (\hat{y}_i - y_i)^2$ ซึ่งเท่ากับ MSE เพราะ $n = 2$ เริ่ม $w = b = 0$, learning rate 0.1:

$$\partial_w J = \sum_i (\hat{y}_i - y_i)x_i, \qquad \partial_b J = \sum_i (\hat{y}_i - y_i).$$

ก้าว 1: prediction $(0, 0)$ และ prediction ลบ actual $= (-1, -3)$ ได้ gradients $(-3, -4)$ จึงอัปเดตพร้อมกันจากค่าชุดเดิมเป็น $w = 0.3, b = 0.4$ Prediction ใหม่ $(0.4, 0.7)$, loss $= (0.36 + 5.29)/2 = 2.825$ ลดจาก 5 **ก้าว 2:** ที่ $w = 0.3, b = 0.4$ residual แบบ prediction ลบ actual เป็น $(-0.6, -2.3)$ ได้ gradients $(-2.3, -2.9)$ จึง $w = 0.53, b = 0.69$ Prediction ใหม่ $(0.69, 1.22)$ และ loss $= (0.0961 + 3.1684)/2 = 1.63225$

ข้อผิดพลาดที่พบได้ง่าย

ถ้าอัปเดต w ก่อนแล้วใช้ w ใหม่หาค่า gradient ของ b จะไม่ได้เป็น simultaneous GD ตามสูตรเดียวกัน ต้องเก็บ gradients ทั้งหมดจาก parameters ก่อน update ในงาน training แต่ละ step ใช้ค่าชุดเดียวกันคำนวณ gradient vector แล้วจึงเปลี่ยน parameters

SSE, MSE และ half MSE ต่างกันอย่างไร

Multiplying objective ด้วย positive constant ไม่เปลี่ยน minimizer แต่ scale ของ gradient เปลี่ยน สำหรับข้อมูล n observations:

$$\nabla \text{MSE} = \frac{2}{n}A^T(AB - Y), \qquad \nabla\left(\tfrac{1}{2}\text{MSE}\right) = \frac{1}{n}A^T(AB - Y).$$

ใช้ learning rate เท่ากันจะก้าวคนละขนาด ถ้า lecture ตัวอย่างมี factor 2 ต่างกันให้ตรวจนิยาม loss ก่อนสรุปว่าสูตรผิด และถ้า objective มี regularization ด้วย การ scale เฉพาะ data-loss โดยไม่ scale ของ penalty จะเปลี่ยน optimum ได้

ต้องจำ GD ลด objective ด้วย negative gradient เขียน loss ให้ชัดก่อน derive ทุก parameter ต้อง update จาก gradient ที่คำนวณที่จุดเดียวกัน

ทดสอบตัวเอง: แก้ closed form ของข้อมูลนี้ได้ $w = 2, b = 1$ ลองเช็กว่าค่า loss ที่ optimum เป็น 0 และ gradient ทั้งสองเป็น 0

2.14 Logistic likelihood สู่ cross-entropy และ gradient

แหล่งเนื้อหา: L2 PDF p.34–38. derivation เสริมจาก Géron Ch.4

สำหรับ binary label $y_i \in \{0, 1\}$ ให้ $p_i = \sigma(w^T x_i + b)$ และสมมติ observations เป็น independent conditional on features Likelihood ของ sample คือ $p_i^{y_i}(1 - p_i)^{1-y_i}$ จึงมี negative average log likelihood

$$J = -\frac{1}{n} \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

ถ้า $y = 1$ และ p ใกล้ 0 loss จะสูงมาก ถ้า $y = 0$ และ p ใกล้ 1 loss สูงมากเช่นกัน จึง penalize confident wrong predictions อย่างแรง สิ่งที่ optimize คือ continuous loss ไม่ใช่ accuracy ที่เป็น threshold decision ไม่เรียบ

Derive gradient สำหรับหนึ่ง sample

ใช้ score z กับ $p = \sigma(z)$:

$$\begin{aligned}
\frac{\partial \ell}{\partial p} &= -\frac{y}{p} + \frac{1 - y}{1 - p}, \\
\frac{\partial p}{\partial z} &= p(1 - p), \\
\frac{\partial \ell}{\partial z} &= \left(-\frac{y}{p} + \frac{1 - y}{1 - p} \right) p(1 - p) = p - y.
\end{aligned}$$

ดังนั้น $\partial \ell / \partial w = (p - y)x$, $\partial \ell / \partial b = p - y$ และ average gradient รวม samples ด้วย $1/n$ การยุบเป็น $p - y$ เกิดจาก sigmoid จับคู่ cross-entropy ไม่ใช่สูตร universal ของทุก activation และ loss

คำนวณหนึ่ง update และตรวจทิศ

ให้ $x = 2, y = 1, w = b = 0$ ได้ $z = 0, p = 0.5, \ell = \log 2$ Gradients เป็น -1 กับ -0.5 ใช้ $\eta = 0.1$ จะได้ $w = 0.1, b = 0.05$ Score ใหม่ 0.25, probability ≈ 0.56218, loss ≈ 0.57594 ลดจาก 0.69315 เพราะ model เพิ่ม probability ของ class ที่ถูก

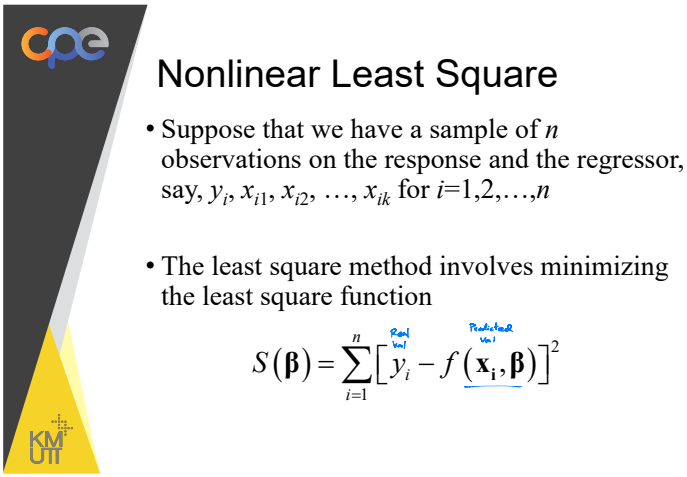
ระวัง: กรณีข้อมูล linearly separable และไม่มี regularization logistic coefficients อาจโตไม่จำกัดเพื่อให้ loss เข้าใกล้ 0 การหยุด training และ regularization จึงสำคัญ อย่าใช้ inverse OL Sformula แก้ BCE โดยตรง

ต้องจำ Likelihood แล้ว log likelihood แล้ว negative average log likelihood = BCE. gradient เทียบ score เป็น $p - y$ แล้ว chain ต่อกับ feature และ bias

2.15 Nonlinear least squares และคำว่า “linear in parameters”

แหล่งเนื้อหา: L2 PDF p.17–19

Polynomial $\beta_0 + \beta_1 x + \beta_2 x^2$ โค้งตาม x แต่ยัง linear ใน parameters β จึงใช้ design matrix columns $(1, x, x^2)$ กับ OLS ได้ ส่วน $f(x; a, b) = ae^{bx}$ ไม่ linear ใน b ต้องหา optimum ของ nonlinear objective โดยวิธีเหมาะสม



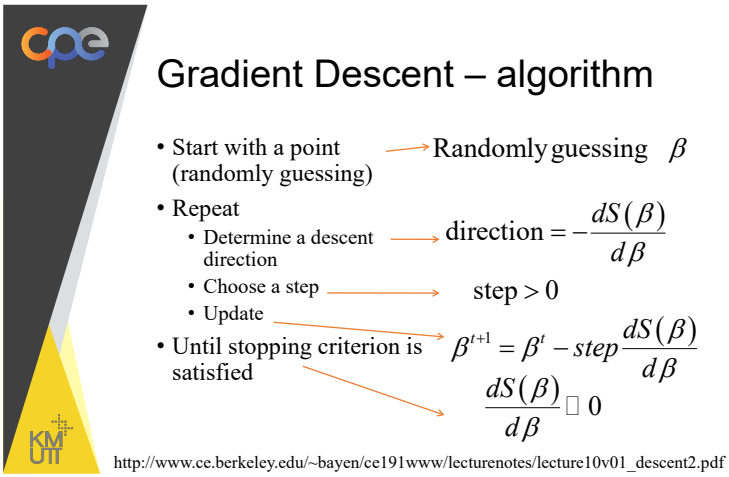
รูป 2.3: Nonlinear least-squares objective ในหน้า 18

อ่านภาพทีละจุด **1.** เลือก parameters ที่ทำให้ผลรวม squared residuals ต่ำสุด คำว่า nonlinear เกี่ยวกับการขึ้นกับ parameters **2.** การตั้ง derivative เป็นศูนย์ให้ candidate สำหรับ nonlinear objective ต้องพิจารณา curvature และ หลายจุด stationary เพิ่ม ให้ $r_i = y_i - f(x_i; \beta)$ และ $J = \sum_i r_i^2$ ใช้ chain rule ได้ $\partial J/\partial \beta_j = -2\sum_i r_i \partial f(x_i;\beta)/\partial \beta_j$ เช่น $\partial f/\partial a = e^{bx}$ และ $\partial f/\partial b = axe^{bx}$ จึงหา gradient ทั้งสองแล้ว update จากค่าเก่าพร้อมกัน Gradient นี้ไม่เท่ากับ normal equation ของ model linear ใน parameters

ระวัง: การมี minimum ไม่ได้แปลว่าทุกปัญหาต้องมี finite stationary point: ตัวอย่าง e^{-x} ในหน้า 19 ลดลงตลอด มี infimum 0 เมื่อ $x \to \infty$ แต่ไม่ถึง 0 ที่ finite x เงื่อนไข derivative ศูนย์ใช้กับ interior differentiable minimum ที่มีจริง

2.16 อ่านการก้าว GD และ logistic boundary จากภาพต้นฉบับ

แหล่งเนื้อหา: L2 PDF p.25–33,39

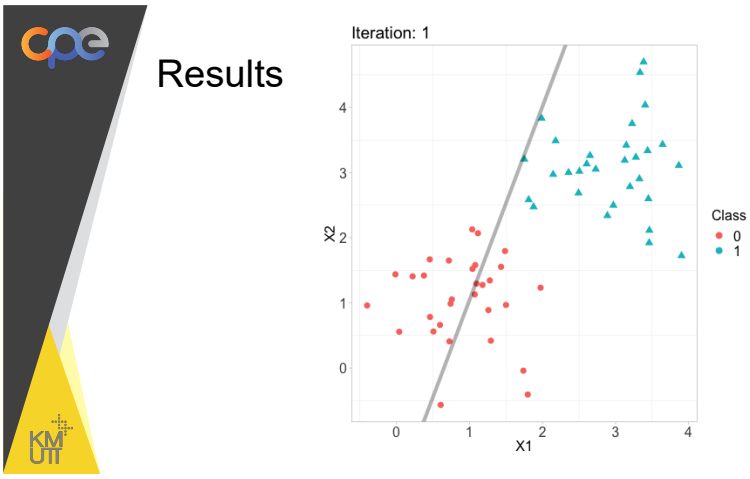


รูป 2.4: ขั้นตอน GD หน้า 30: จากตำแหน่งปัจจุบันถึงก้าวใหม่

อ่านภาพทีละจุด **1.** ประเมิน gradient ที่ parameters ปัจจุบัน แล้วก้าวทิศตรงข้าม gradient ไม่ใช่ก้าวหาเส้นกราฟด้วยตา **2.** ทำซ้ำ ตรวจ loss และ gradient และ จำนวนรอบตาม stopping rule ถ้า overshoot ให้ตรวจ learning rate และ scaling

สำหรับ mini-batch B ขนาด b ให้ $X_B \in \mathbb{R}^{b\times d}$ และ mean MSE จะได้ $g = 2X_B^T(X_B w - y_B)/b$ ไม่หารด้วยจำนวนทั้ง dataset อีกครั้ง Sampling เปลี่ยนความแปรปรวนของ gradient. batch เล็กให้ gradient noise มากขึ้นโดยทั่วไป ไม่ได้

รับประกัน converge เร็วกว่าในทุก metric หากเปลี่ยน SSE เป็น MSE ต้องคำนึงถึง gradient scale และ learning rate ด้วย



รูป 2.5: Logistic regression หน้า 39: probability กับ class boundary

อ่านภาพทีละจุด **1.** Model ให้ probability ผ่าน sigmoid ของ linear score z. ความน่าจะเป็นไม่ใช่เลข class

2. Threshold 0.5 ตรง $z = 0$ แต่ threshold อื่นย้าย boundary ไป $z = \log[t/(1-t)]$ และ metric เปลี่ยนตาม threshold

> **ต้องจำ** ตอบให้ครบ model จากนั้น loss จากนั้น optimizer จากนั้น threshold จากนั้น metric. Logistic ใช้ linear log-odds ไม่ได้หมายถึง probability เป็น linear function

2.17 Lecture 2: สรุป สูตร และสิ่งที่ต้องทำได้

แหล่งเนื้อหา: L2 PDF p.3–39

การเชื่อมแนวคิด

Data + model กำหนด prediction. prediction + target กำหนด loss. optimizer เปลี่ยน parameter เพื่อลด loss จากนั้น validation และ test ตรวจว่าพฤติกรรมที่เรียนรู้ใช้กับข้อมูลใหม่ได้หรือไม่

1. OLS: squared residual จากนั้น derivative จากนั้น normal equation จากนั้น parameter จากนั้น prediction จากนั้น MSE
2. GD: initialize จากนั้น evaluate gradient จากนั้น choose positive step จากนั้น update opposite gradient จากนั้น repeat
3. Logistic: linear score จากนั้น sigmoid probability จากนั้น cross-entropy จากนั้น gradient จากนั้น threshold สำหรับตัดสิน class

สูตรที่ต้องเขียนจากความจำ

$$\beta = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \quad \alpha = \bar{y} - \beta\bar{x}, \quad \hat{B} = (A^T A)^{-1} A^T Y.$$

$$\theta^{new} = \theta - \eta \nabla J, \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \nabla J_{logistic} = \frac{1}{m} X^T (p - y).$$

Checklist ก่อนเข้าสอบ

- อธิบาย parameter กับ hyperparameter และ direct กับ iterative ได้

- หา α, β จาก 3–4 sample และตรวจคำตอบด้วย residual ได้
- สร้าง design matrix รวม intercept และบอกมิติได้
- คำนวณ MSE และแยก SSE และ mean normalization ได้
- ทำ Batch และ SGD update โดยใช้ค่าเก่าพร้อมกันได้
- แยก score, probability, threshold และ class ได้
- อธิบายผลของ learning rate และ feature scale ได้

ระวัง: ชื่อ “End of Lecture 3” ใน L2 หน้า 40 เป็นข้อความท้ายสไลด์ที่ไม่ตรงกับชื่อไฟล์ ไม่ใช้ข้อความนี้เปลี่ยนหมายเลขบท

อ่านเพิ่มจากตำรา อ่านต่อ: Géron Ch.4 สำหรับ training และ optimization. ISLP Ch.3 และ §4.3 สำหรับ regression และ logistic และการตีความ coefficient

Lecture 3 Support Vector Machine

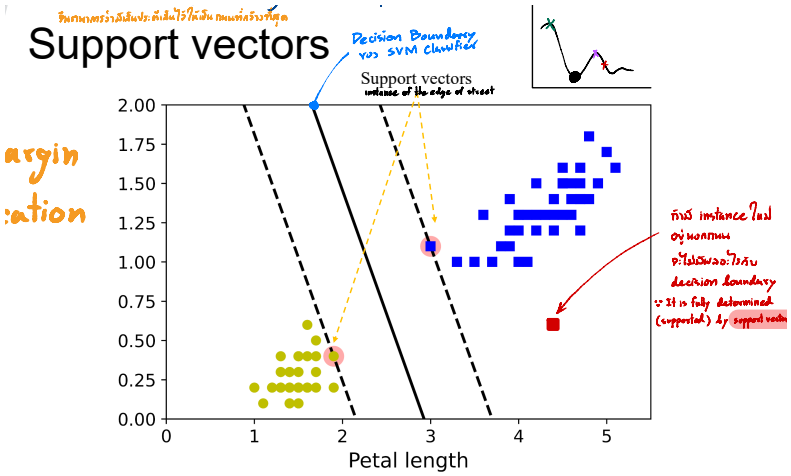
แหล่งเนื้อหา: L3 PDF p.1–8 | ขอบเขตมิดเทอม

คำย่อที่ใช้ในบทนี้

SVM คือ Support Vector Machine และ **SV** คือ Support Vector. **RBF** คือ Radial Basis Function. **KKT** คือ Karush–Kuhn–Tucker ซึ่งเป็นเงื่อนไข optimum ที่เชื่อม primal กับ dual สัญลักษณ์ L1 และ L2 ใช้ระบุชนิด norm ไม่ใช่คำย่อของหมายเลข Lecture เมื่ออยู่ในสมการ penalty. SVM เลือก decision boundary ที่มี margin กว้าง ภายใต้ข้อจำกัดของการจำแนก แนวคิดไม่ได้หยุดที่ “ลากเส้นแยก class” เพราะมีเส้นที่แยก training data ได้หลายเส้น แต่เส้นที่อยู่ใกล้กลุ่มใดมากเกินไปอาจไวต่อข้อมูลใหม่ SVM จึงเลือกโดยเกณฑ์ maximum margin

$$f(x) = w^T x + b, \qquad \hat{y} = \begin{cases} 1 & f(x) \geq 0 \\ 0 & f(x) < 0. \end{cases}$$

w เป็น normal vector ตั้งฉากกับ hyperplane. b เลื่อนตำแหน่ง. decision boundary คือ $f(x) = 0$ ถ้า input มี d dimensions, boundary มี $d-1$ dimensions เช่น input 2D มี boundary เป็นเส้น



รูป 3.1: Support vectors อยู่ที่ขอบถนนใน hard-margin illustration และเป็นจุดที่กำหนด boundary มากที่สุด (L3 p.6)

อ่านภาพทีละจุด **1.** เส้นกลางคือ decision boundary ส่วนเส้นคู่ขนานสองข้างคือขอบ margin

2. จุดบนขอบเป็น support vectors ใน hard-margin ภาพนี้ ระยะเต็มระหว่างขอบคือ 2/||w|| ไม่ใช่ระยะจากจุดหนึ่งถึงเส้นกลาง

> **ต้องจำ** SVM score $f(x)$ ไม่ใช่ probability โดยตรง. sign ใช้ตัดสิน class ส่วนขนาดต้องพิจารณา normalization หากจะตีความเป็นระยะ

ฝึกให้ทำได้ ดูภาพแล้วระบุ boundary, margin boundaries, support vectors และอธิบายว่าเส้นที่แยก train ได้ทุกจุดยังมีคุณภาพไม่เท่ากัน

3.1 Functional margin และ geometric distance

แหล่งเนื้อหา: L3 PDF p.8–10, 13

เพื่อเขียนเงื่อนไขของ SVM ใช้ label $t_i \in \{-1, +1\}$ แม้โจทย์บางข้อจะเข้ารหัสเป็น 0/1 ก็ตาม ให้ x_i เป็น feature vector, w เป็นเวกเตอร์ตั้งฉากกับเส้นแบ่ง และ b เป็นค่าที่เลื่อนเส้นแบ่ง เมื่อแปลง label แล้ว ผลคูณ $t_i f(x_i)$ จะเป็นบวกเมื่ออยู่ฝั่งถูก และเป็นลบเมื่ออยู่ฝั่งผิด

$$\text{signed distance} = \frac{t_i(w^T x_i + b)}{\|w\|}, \qquad \text{distance to boundary} = \frac{|w^T x + b|}{\|w\|}.$$

โดย t_i คือ label -1 หรือ $+1$, $f(x_i) = w^T x_i + b$ คือ score ของแถวที่ i และ $\|w\|$ คือความยาวของเวกเตอร์ตั้งฉากกับ boundary การหารด้วย $\|w\|$ ทำให้ได้ระยะเชิงเรขาคณิต ส่วน signed distance เก็บเครื่องหมายไว้เพื่อบอกว่าจุดอยู่ฝั่งถูกหรือผิด ถ้ากำหนด margin boundaries เป็น $w^T x + b = \pm 1$, ระยะจาก boundary ถึงแต่ละขอบเท่ากับ $1/\|w\|$ และความกว้างเต็มของถนนเท่ากับ $2/\|w\|$ ต้องบอกว่าคำว่า “margin” ในคำตอบหมายถึง half-width หรือ full-width

ตัวอย่างคำนวณทีละขั้น

ให้ $w = (3, 4)^T$, $b = -10$ และ $x = (2, 2)^T$

1. $\|w\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$ จึง full margin $2/\|w\|_2 = 2/5 = 0.4$
2. Score $f(x) = 3(2) + 4(2) - 10 = 6 + 8 - 10 = 4$ จึงทาย class $+1$
3. ระยะถึง boundary $|f(x)|/\|w\|_2 = |4|/5 = 0.8$
4. ถ้า label จริง $t = +1$ จะมี signed distance $tf/\|w\| = 4/5 = 0.8$ แต่ถ้า $t = -1$ จะได้ $-4/5 = -0.8$ และทายผิด

ทำไมต้อง canonical normalization

การคูณ w, b ด้วย constant บวกไม่เปลี่ยนเส้น $f(x) = 0$ แต่เปลี่ยน score หากไม่มี constraint ที่กำหนด scale เราทำให้ $\|w\|$ เล็กด้วยการหารทุกอย่างแล้วอ้างว่า margin กว้างขึ้นไม่ได้ SVM ล็อค scale ด้วย $t_i f(x_i) \geq 1$

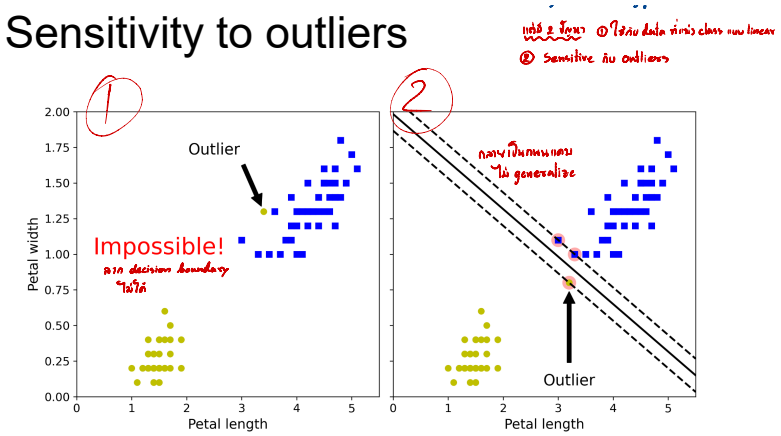
> **ต้องจำ** ลด $\|w\|$ พร้อม constraint ที่กำหนดแล้วจึงเท่ากับเพิ่ม margin ไม่ใช่ลด weight อย่างไม่มีข้อจำกัด

3.2 Hard-margin objective และข้อจำกัด

แหล่งเนื้อหา: L3 PDF p.9–11

$$\min_{w,b} \frac{1}{2}\|w\|^2 \qquad \text{subject to } t_i(w^T x_i + b) \geq 1 \quad \forall i.$$

การใช้ squared norm กับ $1/2$ ไม่เปลี่ยน minimizer แต่ differentiate ง่ายเป็น w Objective ต้องการถนนกว้าง ส่วน constraint บังคับทุกตัวอย่างอยู่ฝั่งถูกและไม่เข้า margin ถ้าข้อมูลแยกด้วย hyperplane ไม่ได้ ปัญหา hard margin ไม่มี feasible solution



รูป 3.2: Outlier อาจทำให้แยกเชิงเส้นไม่ได้ หรือบังคับ margin ให้แคบลงมาก (L3 p.11)

อ่านภาพทีละจุด **1.** เทียบกรณีมี outlier กับกรณีเดิมว่าเส้นแยกจำเป็นต้องเอียง และ แคบลงหรือแยกไม่ได้
2. Hard margin บังคับทุก signed score อย่างน้อย 1 จึงอ่อนไหวต่อจุดผิดปกติ Soft margin ยอม violation โดยคิดค่าใช้จ่าย ไม่ใช่ลบ outlier โดยอัตโนมัติ

อ่าน constraint อย่างไร

สำหรับ positive class $f(x_i) \geq 1$. negative class $f(x_i) \leq -1$ จึงไม่ได้บังคับว่า score ของทุกจุดต้องมากกว่า 1 แต่บังคับ signed score ซึ่งรวม label แล้ว

ตัวอย่างคำนวณทีละขั้น

ตรวจ hard-margin constraint โดยคำนวณ tf ก่อนเทียบกับ 1

1. $t = +1, f = 0.7$: $tf = 1(0.7) = 0.7 < 1$ ทายถูกฝั่งแต่ยังไม่ผ่าน margin
2. $t = -1, f = -2$: $tf = (-1)(-2) = 2 \geq 1$ ทายถูกและผ่าน margin
3. $t = -1, f = 0.3$: $tf = (-1)(0.3) = -0.3 < 1$ ทายผิดและไม่ผ่าน margin

> **ต้องจำ** Hard margin ไม่อนุญาต margin violation ไม่ใช่เพียงไม่มี misclassification จุดฝั่งถูกแต่ใกล้เส้นเกินก็ถือว่าไม่ผ่าน

ฝึกให้ทำได้ เขียน objective กับ constraints แยกกัน และอธิบายให้ได้ว่า outlier ทำอะไรต่อ feasibility และ width

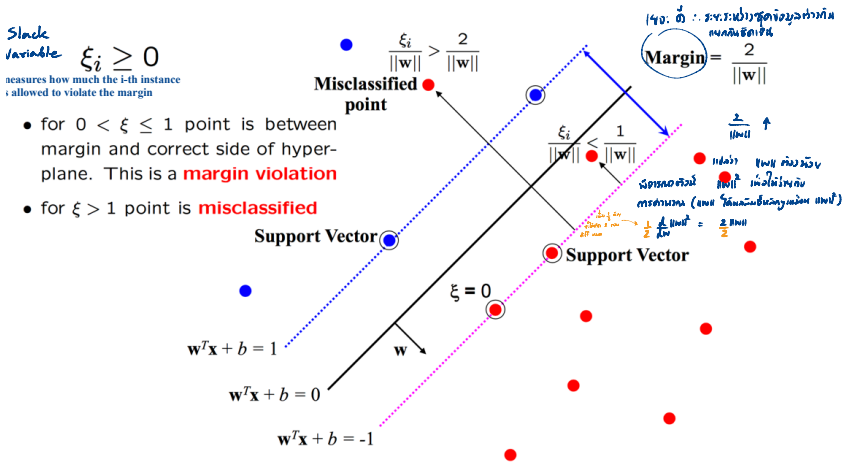
3.3 Soft margin, slack และ C

แหล่งเนื้อหา: L3 PDF p.12–15

Soft margin เพิ่มตัวแปร $\xi_i \geq 0$ เพื่อยอมให้ตัวอย่างละเมิดขอบถนน และลงโทษตามขนาด:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i, \quad t_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

ที่ optimum เมื่อไม่เพิ่ม slack เกินจำเป็น จะมี $\xi_i = \max(0, 1 - t_i f(x_i))$ ดังนั้น $\xi = 0$ ผ่านขอบ. $0 < \xi < 1$ อยู่ใน margin แต่ยังถูกฝั่ง. $\xi = 1$ อยู่บน decision boundary. $\xi > 1$ อยู่ผิดฝั่ง



รูป 3.3: Slack วัด margin violation ไม่ใช่เพียงจำนวนจุดผิด และไม่ได้มีหน่วยเดียวกับ geometric distance โดยอัตโนมัติ (L3 p.13)

อ่านภาพทีละจุด 1. อ่านตำแหน่งจุดเทียบเส้น f=0 และ f=±1 เป็นสองเรื่อง: class ถูกหรือผิด กับอยู่ใน margin หรือไม่

2. จุดถูก class แต่อยู่ใน margin มี 0<slack<1. จุดผิดมี slack>1 เมื่อใช้ minimal slack การวัดระยะด้วยสายตาต้องไม่สับสนกับ functional slack

C ใหญ่ เพิ่มน้ำหนักลงโทษ violation จึงมักพยายาม fit มากและ regularize น้อย. **C เล็ก** ยอม violation มากขึ้นเพื่อ margin ที่กว้าง และ weight ที่เล็ก ไม่ได้แปลว่า C เล็กดีกว่าเสมอ เพราะเล็กเกินอาจ underfit

ต้องจำ จำทิศ C: ใหญ่ จากนั้น violation แพง เล็ก จากนั้น weight penalty มีอิทธิพลสัมพัทธ์มาก อย่าสลับกับ λ ของ Ridge

ระวัง: $C \to \infty$ จะเข้าใกล้ hard-margin formulation เมื่อปัญหา hard margin feasible ข้อมูลที่แยกไม่ได้ไม่กลายเป็น separable เพียงเพิ่ม C

3.4 Hinge loss และตัวอย่างตรวจ margin

แหล่งเนื้อหา: L3 PDF p.34. เชื่อม p.13–15

Hinge loss รวม constraint slack เป็น loss ของ signed score:

$$\ell_i = \max\{0, 1 - t_i(w^T x_i + b)\}, \qquad J = \tfrac{1}{2}\|w\|^2 + C\sum_i \ell_i.$$

ถ้า signed score ตั้งแต่ 1 ขึ้นไป loss เป็น 0. ถ้าต่ำกว่า 1 เส้น loss ลดด้วย slope -1 ตาม signed score ที่จุด 1 ไม่ differentiable แบบปกติ ใช้ subgradient ได้ ห้ามสรุปว่า derivative เป็น -1 ทุกจุด

t_i	$f(x_i)$	$t_i f$	hinge	ความหมาย
1	2	2	0	ถูกและพ้น margin
1	0.4	0.4	0.6	ถูกแต่อยู่ใน margin
-1	0.5	-0.5	1.5	ผิดฝั่ง
-1	-1	1	0	บน margin boundary

การเปรียบเทียบกับ Logistic

Hinge ต้องการ signed score ถึง margin แล้วหยุดลงโทษ Logistic cross-entropy ใช้ probability และยังลดเมื่อทำนาย class ถูกแต่ไม่มั่นใจพอ ทั้งสองมี boundary linear ได้ แต่ objective ต่างกัน จึงไม่รับประกัน parameter เดียวกัน

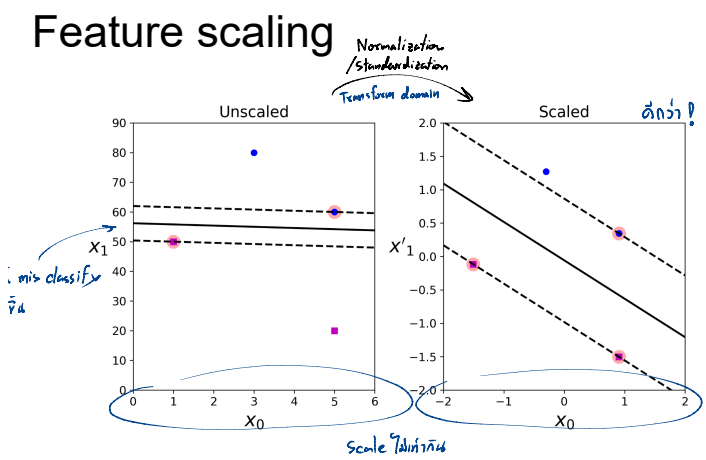
ต้องจำ Hinge ของจุดที่ทาย class ถูกอาจยังเป็นบวก เพราะ criterion ไม่ใช่เพียง sign แต่เป็น signed score ≥ 1

ทดสอบตัวเอง: ให้ $t = -1, f = -0.2$ จะได้ $tf = 0.2$ และ loss 0.8. ถ้าแทน label 0 จะได้ loss 1 ตลอด ซึ่งแสดงว่าใช้ label encoding ผิด

3.5 Feature scaling และความไวของ SVM

แหล่งเนื้อหา: L3 PDF p.7, 14, 25–27

SVM ใช้ geometry ของ feature space ดังนั้นหน่วยของ feature มีผลต่อระยะและ margin หาก feature หนึ่งมี scale มากกว่ามาก การมองความกว้างของถนนและ RBF distance จะถูกครอบงำโดย feature นั้น การ scale เปลี่ยนปัญหาที่ model มอง ไม่ใช่เปลี่ยนแค่หน้าตากราฟ



รูป 3.4: เมื่อ scale ต่างกัน margin ที่ดูเหมาะสมในแกนหนึ่งอาจไม่สะท้อนโครงสร้าง class ต้องกำหนดหน่วยให้เหมาะ (L3 p.7)

อ่านภาพทีละจุด **1.** สังเกตหน่วย และ ช่วงของทั้งสองแกนก่อนเทียบความกว้างของถนน margin

2. เปลี่ยนสเกลแกนหนึ่งเปลี่ยน geometry ใน optimization ดังนั้น standardization ต้อง fit จาก train และใช้สเกลเดิมกับข้อมูลใหม่

Workflow ที่ใช้ได้กับข้อมูลใหม่

เริ่มจากแบ่งข้อมูลเป็น train, validation และ test แล้ว fit scaler ด้วย train เท่านั้น จากนั้นใช้ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานชุดเดิมแปลงทั้งสามส่วน ฝึก SVM ด้วย train เลือก C และ kernel จากผลบน validation หรือ cross-validation แล้วจึงประเมิน test หลังตัดสินใจเลือกโมเดลแล้ว หาก fit scaler จากข้อมูลทั้งหมดก่อนแบ่ง ค่า test จะมีส่วนกำหนดสเกลที่ใช้ฝึก จึงเกิด data leakage

การทำนาย

Linear SVM เก็บ w, b ใช้ dot product ได้รวดเร็ว ส่วน kernel SVM ใช้ similarities กับ support vectors ความเร็วจึงขึ้นกับจำนวน support vectors และต้นทุน kernel

> **ต้องจำ** SVM sensitive ต่อ scale. Decision Tree ที่ใช้ threshold ราย feature ไม่ได้ต้องการ scale แบบเดียวกัน อย่าทำ preprocessing เพราะชื่อเป็น ML เหมือนกันเท่านั้น

ฝึกให้ทำได้ ถ้าคะแนนไม่ดีหลังรวม feature ต่างหน่วย ให้อธิบายทั้ง scale และ hyperparameter แทนสรุปว่า kernel ไม่ดีทันที

3.6 Lagrange multiplier: ตัวอย่างในสไลด์

แหล่งเนื้อหา: L3 PDF p.17–18. จุดจำจาก annotation L2 p.1

โจทย์ minimize $f(x, y) = x^2 + 2y$ subject to $3x + 2y + 1 = 0$ สร้าง Lagrangian ตามเครื่องหมายสไลด์:

$$g(x, y, \alpha) = x^2 + 2y - \alpha(3x + 2y + 1).$$

$$\begin{aligned} \partial_x g &= 2x - 3\alpha = 0, \\ \partial_y g &= 2 - 2\alpha = 0, \\ \partial_\alpha g &= -(3x + 2y + 1) = 0. \end{aligned}$$

จากสมการที่สอง $\alpha = 1$ จึงได้ $x = 3/2$ และ $y = -11/4$ โดย $f = 9/4 - 11/2 = -13/4$

ตรวจว่าเป็น minimum จริง

แทน constraint เป็น $y = -(3x+1)/2$ จะได้ $f = x^2 - 3x - 1 = (x-1.5)^2 - 3.25$ จึงมี minimum ที่ $x = 1.5$ ตรงกัน นี่เป็นการตรวจที่ดีกว่าพบ derivative ศูนย์แล้วประกาศ global optimum ทุกกรณี

สิ่งที่ต้องเข้าใจ

Multiplier ใช้รวม constraint เข้า objective เพื่อเขียน stationary conditions แต่ยังต้องตรวจ constraint และเงื่อนไข optimum Lagrangian ไม่ใช่ตัด constraint ทิ้งแล้ว optimize ทุกตัวแบบ unconstrained minimization ธรรมดา

> **ต้องจำ** จำสามขั้น: เขียน g จากนั้น partial derivative ตามทุกตัวรวม multiplier จากนั้น แก้พร้อม constraint ใช้เครื่องหมาย $+$ หรือ $-$ ได้ถ้าทำสอดคล้องกัน แต่ค่า multiplier อาจกลับ sign

ฝึกให้ทำได้ โจทย์นี้เป็นแนวคำนวณที่มีตัวอย่างใน Lecture และ L2 p.1 เขียนให้จำ Lagrange จึงควรทำได้ด้วยมือ

3.7 Lagrange: หา candidate แล้วพิสูจน์ว่าเป็น minimum

แหล่งเนื้อหา: L3 PDF p.17–18. ตัวอย่างฝึกใหม่

ต้องการ minimize $f(x,y) = x^2 + y^2$ เมื่อ $x + y = 3$ กำหนด $g = x + y - 3 = 0$ และ $\mathcal{L} = x^2 + y^2 + \lambda(x + y - 3)$

$$2x + \lambda = 0, \quad 2y + \lambda = 0, \quad x + y - 3 = 0.$$

สองสมการแรกให้ $x = y$ แทน constraint ได้ $x = y = 1.5$ และ $\lambda = -3$ ตรวจ objective $f = 4.5$ จะเปลี่ยนเครื่องหมายหน้า λg ก็ได้แต่ multiplier เปลี่ยน sign. x, y ไม่ควรเปลี่ยน

ตรวจชนิดของจุดโดยลดมิติ

ใช้ $y = 3 - x$ จะได้ $f = x^2 + (3-x)^2 = 2(x-1.5)^2 + 4.5$ จึงเป็น global minimum อย่างแน่นอน ไม่ใช่เห็น derivative ศูนย์แล้วสรุป minimum ทุกปัญหาโดยไม่มีเหตุผล

ทดสอบตัวเอง: ถ้า constraint เปลี่ยนเป็น $x + y = a$ จะได้ $(a/2, a/2)$ และ $f_{min} = a^2/2$ นี่ช่วยเช็กโจทย์ใหม่ได้โดยไม่จำเฉพาะค่าตัวอย่าง

3.8 Primal สู่ Dual ของ hard-margin SVM

แหล่งเนื้อหา: L3 PDF p.19–24

เขียน constraint เป็น $t_i(w^T x_i + b) - 1 \geq 0$ และใช้ $\alpha_i \geq 0$:

$$\mathcal{L}(w, b, \alpha) = \tfrac{1}{2} w^T w - \sum_i \alpha_i [t_i(w^T x_i + b) - 1].$$

หา stationary conditions ตาม w, b:

$$\nabla_w \mathcal{L} = w - \sum_i \alpha_i t_i x_i = 0, \qquad \partial_b \mathcal{L} = -\sum_i \alpha_i t_i = 0.$$

จึงได้ $w = \sum_i \alpha_i t_i x_i$ และ $\sum_i \alpha_i t_i = 0$ ให้ $s = \sum_i \alpha_i t_i x_i$ จาก stationary conditions ได้ $w = s$ และ $\sum_i \alpha_i t_i = 0$ จึง

แทนกลับทีละส่วน

$$\begin{aligned}\mathcal{L}(s,b,\alpha) &= \tfrac{1}{2}s^Ts - \sum_i \alpha_i t_i x_i^T s - b\sum_i \alpha_i t_i + \sum_i \alpha_i \\ &= \tfrac{1}{2}s^Ts - s^Ts - 0 + \sum_i \alpha_i \\ &= \sum_i \alpha_i - \tfrac{1}{2}s^Ts,\end{aligned}$$

$$s^Ts = \left(\sum_i \alpha_i t_i x_i\right)^T \left(\sum_j \alpha_j t_j x_j\right) = \sum_i\sum_j \alpha_i\alpha_j t_i t_j x_i^T x_j.$$

ดังนั้น dual อยู่ในรูป maximize

$$\max_\alpha \sum_i \alpha_i - \tfrac{1}{2}\sum_i\sum_j \alpha_i\alpha_j t_i t_j x_i^T x_j.$$

subject to $\alpha_i \geq 0$ และ $\sum_i \alpha_i t_i = 0$ สไลด์ p.23 เขียนเป็น minimize ของ negative objective ซึ่งเท่ากัน ต้องไม่เอาเครื่องหมาย maximize ไปใช้กับ objective ฝั่ง minimize โดยไม่กลับ sign

Recover primal solution

ใช้ w ตามผลรวมด้านบน และสำหรับ support vector ที่อยู่บน margin ใช้ $b = t_s - w^Tx_s$ เฉลี่ยหลายตัวเพื่อลด numerical error ใน soft margin ต้องเลือกตัวที่ $0 < \alpha_s < C$ สำหรับสูตรนี้ ไม่ใช้จุดผิดฝั่งทุกตัวแทนได้ทันที

> **ต้องจำ** Dual ใช้ dot product ของคู่ sample จุดที่ $\alpha_i > 0$ มีบทบาทใน decision function และเชื่อมกับ support vectors

ที่มาของขอบเขต dual ใน soft margin

ให้ $\xi_i \geq 0$ เป็น slack ของจุดที่ i, $C > 0$ เป็นค่าน้ำหนัก penalty, $\alpha_i \geq 0$ เป็น multiplier ของ margin constraint และ $\nu_i \geq 0$ เป็น multiplier ของ $\xi_i \geq 0$. Lagrangian เป็น

$$\mathcal{L} = \tfrac{1}{2}w^Tw + C\sum_i \xi_i - \sum_i \alpha_i[t_i(w^Tx_i + b) - 1 + \xi_i] - \sum_i \nu_i\xi_i,$$

$$\frac{\partial\mathcal{L}}{\partial\xi_i} = C - \alpha_i - \nu_i = 0 \quad\Longrightarrow\quad \nu_i = C - \alpha_i \geq 0 \quad\Longrightarrow\quad 0 \leq \alpha_i \leq C.$$

เงื่อนไขจาก w และ b ยังเหมือน hard margin เมื่อแทนกลับ เทอม slack หายไปเพราะ coefficient $C - \alpha_i - \nu_i$ เป็นศูนย์ จึงได้ dual objective เดิมพร้อมขอบเขตใหม่ $0 \leq \alpha_i \leq C$. Complementary slackness ให้

$$\alpha_i[t_i(w^Tx_i + b) - 1 + \xi_i] = 0, \qquad (C - \alpha_i)\xi_i = 0.$$

ถ้า $0 < \alpha_i < C$ สมการที่สองบังคับ $\xi_i = 0$ และสมการแรกบังคับ signed score เท่ากับ 1 จึงใช้จุดนั้นหา b ได้ ถ้า $\alpha_i = C$ ไม่อาจสรุปว่า slack ต้องเป็นบวกเสมอ.

อ่านเพิ่มจากตำรา Géron Ch.5 อธิบายความสัมพันธ์ของ soft-margin primal และ dual นี้

3.9 Soft-margin Dual: ติดตาม C, support vectors และ KKT

แหล่งเนื้อหา: L3 PDF p.19–24,34. เงื่อนไข KKT เสริมเพื่อเข้าใจ

ใช้จุด $x_1 = -1, t_1 = -1$ และ $x_2 = 1, t_2 = 1$ จากตัวอย่างสองจุดเดิม ใช้สมมาตรเลือก $b = 0$ เป็น optimum หนึ่งตัว และ $\alpha_1 = \alpha_2 = \alpha$ Constraint soft margin คือ $0 \le \alpha_i \le C$ ต่างจาก hard margin ที่ไม่มี upper bound

$$D(\alpha) = 2\alpha - 2\alpha^2, \quad D'(\alpha) = 2 - 4\alpha, \quad \alpha^* = \min(C, 0.5), \quad w = 2\alpha^*.$$

ถ้า $C = 0.2$: $\alpha = 0.2$, $w = 0.4$, signed margins ทั้งคู่ 0.4, slack ทั้งคู่ 0.6 Primal objective $= 0.5(0.4)^2 + 0.2(0.6 + 0.6) = 0.32$ ตรงกับ Dual $= 0.4 - 0.08 = 0.32$ หาก $C = 1$ ได้ $\alpha = 0.5$, $w = 1$, slack 0 และ objective 0.5

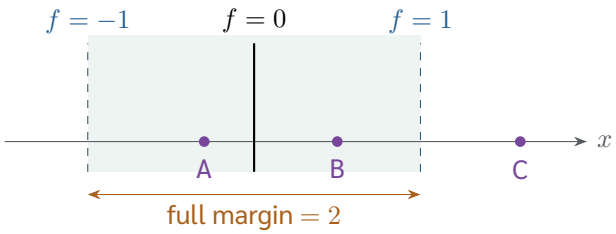
กรณี dual	เงื่อนไขที่ optimum	อ่านความหมาย
$\alpha_i = 0$	โดยทั่วไป $t_i f_i \ge 1$	ไม่ร่วมกำหนด w ผ่านผลรวม dual อาจเป็น boundary ที่ multiplier 0 ในกรณี degenerate
$0 < \alpha_i < C$	$t_i f_i = 1, \xi_i = 0$	support vector บนขอบ margin ใช้หา $b = t_i - w^T x_i$ ได้
$\alpha_i = C$	$t_i f_i \le 1$	อาจอยู่บนขอบ ใน margin หรือผิด class ไม่ใช่ผิด class เสมอ

ระวัง: ถ้าใช้ support vector ที่มี $\xi > 0$ หา b ด้วยสมการ $t_i f_i = 1$ จะผิด เพราะจุดนั้นมี $t_i f_i = 1 - \xi_i$ อย่าใช้สูตรโดยไม่ตรวจกรณี

ต้องจำ คำว่า support vector ใน soft margin ไม่ได้แปลว่าทุกจุดอยู่บนเส้น $f = \pm 1$. C กำหนดเพดาน dual weight และความแพงของ violation ใน primal

ชี้ margin กับ slack บนภาพหนึ่งมิติ

ใช้ $f(x) = x$ จึง boundary ที่ 0 ขอบ margin ที่ $x = \pm 1$ สมมติทุกจุดที่วาดมี label $t = +1$ เพื่อแยกตำแหน่งกับ class จริง



รูป 3.5: A ผิด class และ slack 1.3. B ถูก class แต่ใน margin และ slack 0.5. C ถูก class อยู่นอก margin และ slack 0

อ่านเส้น 0 เพื่อตัดสิน class แล้วอ่านขอบ+1 เพื่อคำนวณ violation ของ label+1 ถ้า label เปลี่ยนเป็น−1 ต้องใช้ signed score tf ใหม่ ไม่ใช้ slack เดิมจากภาพโดยไม่ดู label

3.10 Nonlinear features และ Kernel trick

แหล่งเนื้อหา: L3 PDF p.25–33

ข้อมูลแยกไม่ได้ใน input space อาจแยกเชิงเส้นได้หลัง feature map $\phi(x)$ เช่นข้อมูล 1D ที่ class กลางต่างจากปลายสองด้าน เพิ่ม x^2 แล้ว hyperplane ใน space ใหม่อาจแก้ได้ แต่การสร้าง feature จำนวนมากมีต้นทุน Kernel trick คำนวณ $K(a, b) = \phi(a)^T \phi(b)$ โดยไม่ materialize ϕ ทุกครั้ง สิ่งที่แทนคือ dot product ใน dual ไม่ใช่ใช้ kernel แทน classifier ทั้งระบบ

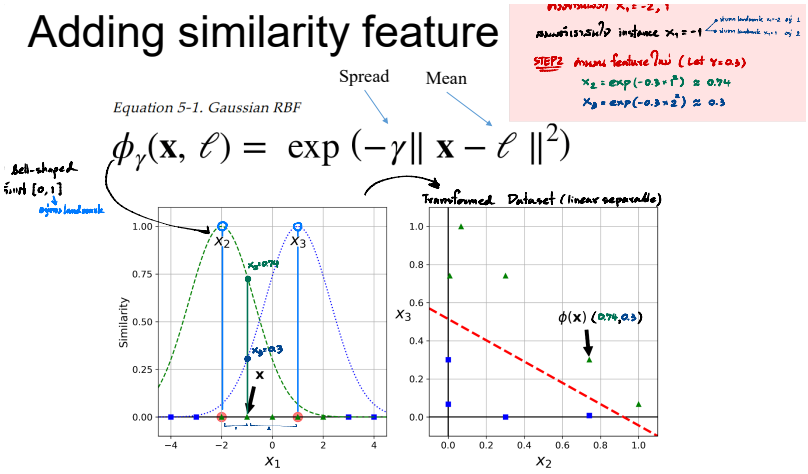
$$K_{linear}(a, b) = a^T b, \quad K_{poly}(a, b) = (\gamma a^T b + r)^d, \quad K_{RBF}(a, b) = e^{-\gamma \|a-b\|^2}.$$

สไลด์ยังแสดง sigmoid kernel $\tanh(\gamma a^T b + r)$ โดยเงื่อนไขความเป็น valid kernel ขึ้นกับ parameters ไม่ใช่ activation function ใด ๆ จะใช้เป็น kernel ได้เสมอ

Prediction ใน kernelized SVM

$$f(x) = \sum_{i \in SV} \alpha_i t_i K(x_i, x) + b.$$

RBF γ ใหญ่ให้ similarity ลดเร็ว จากนั้น ขอบเขต local มากและเสี่ยง overfit. γ เล็กให้ similarity กว้างและ pattern เรียบกว่า แต่ต้องเลือกคู่กับ C



รูป 3.6: Similarity feature เทียบกับ landmark: ใกล้ landmark ให้ค่า RBF สูง ไม่ใช่ค่าระยะสูง (L3 p.27)

อ่านภาพทีละจุด **1.** Landmark เป็นจุดอ้างอิงเพื่อสร้าง similarity feature ใกล้ landmark ให้ค่า RBF สูง
2. ไล่ input เดิมไป feature ใหม่แต่ละตัวก่อนอ่าน boundary ใน feature space Gamma สูงทำให้ความคล้ายลดเร็วตามระยะ ไม่ได้ทำให้จุดไกลคล้ายมากขึ้น

ระวัง: L3 p.28 มีลายมือ “ไม่สอบ” บน derivation feature map second-degree polynomial ผมแยกเป็นส่วนเสริม ไม่ขยาย derivation นี้เป็นโจทย์บังคับ แต่แนวคิด kernel ยังอยู่ในเนื้อหาบท

3.11 Soft margin: แยกสี่กรณีจาก signed score

แหล่งเนื้อหา: L3 PDF p.12–15,34. ตัวเลขฝึกแต่งขึ้นใหม่

กำหนด $t \in \{-1, +1\}$, score $s = w^T x + b$ และ signed score $m = ts$ slack ต่ำสุด ที่ทำให้ $m \geq 1 - \xi$ และ $\xi \geq 0$ คือ $\xi = \max(0, 1 - m)$ ซึ่งเท่ากับ hinge loss ใน convention นี้

m	minimum ξ	ความหมาย
1.5	0	ทายถูกและอยู่นอก margin
1	0	บน margin boundary
0.4	0.6	ทายถูกแต่เข้า margin
-0.3	1.3	ทายผิดฝั่ง จึง slack มากกว่า 1

ที่ $m = 0$ จุดอยู่บน decision boundary และ slack 1 การทาย class อาจขึ้นกับ tie convention ดังนั้นไม่เขียนว่าจุด slack 1 ทุกจุดผิดแน่นอน ส่วน $0 < \xi < 1$ ใน คำตอบที่ใช้ slack ต่ำสุด คือถูกฝั่งแต่เข้า margin และ $\xi > 1$ คือ ฝั่งที่ทายผิด

Trade-off ของ C จาก candidates สมมติ

สมมติ candidate A มี $\frac{1}{2}\|w\|^2 = 1$, ผลรวม slack 2. candidate B มี เทอม norm 4,ผลรวม slack 0.2 Objectives เป็น $1 + 2C$ กับ $4 + 0.2C$ เมื่อ $C = 0.5$ A ได้ 2 และ B ได้ 4.1 จึง A ดีกว่า เมื่อ $C = 5$ A ได้ 11 และ B ได้ 5 จึง B ดีกว่า จุดเปลี่ยนคือ $1.8C = 3$ หรือ $C = 5/3$ แสดง โทษของการละเมิด margin แพงขึ้นแล้วเลือก model ที่แก้ slack มากขึ้นได้ แม้ norm ใหญ่ขึ้น

> **ต้องจำ** Soft margin ใช้ slack ยอมให้ violation. slack ไม่ใช่จำนวนจุดผิดอย่างเดียว เพราะ correct points inside margin มี slack ได้

ทดสอบตัวเอง: ถ้า score เป็น-0.8 และ label ลบ signed score เป็น 0.8 ไม่ใช่-0.8 จึง hinge 0.2 และทายถูก ลองสลับ label แล้วคำนวณใหม่

3.12 Dual SVM สองจุด: คำนวณจนได้ margin

แหล่งเนื้อหา: L3 PDF p.19–24. ตัวอย่างใหม่เพื่อทบทวนสมการ lecture

ให้ ข้อมูลหนึ่งมิติ $x_1 = -1, t_1 = -1$ และ $x_2 = 1, t_2 = 1$ Hard margin primal มี constraints $w - b \geq 1$, $w + b \geq 1$ ดังนั้น optimal $w = 1, b = 0$ ได้ เทอม norm 0.5 และ ความกว้าง margin 2 จาก dual constraint $-\alpha_1 + \alpha_2 = 0$ ให้ทั้งคู่เป็น $a \geq 0$ และ $w = \sum_i \alpha_i t_i x_i = 2a$ Dual objective เป็น

$$D(a) = 2a - \frac{1}{2}(2a)^2 = 2a - 2a^2.$$

Maximum เกิดเมื่อ $D'(a) = 2 - 4a = 0$ จึง $a = 0.5$ แล้ว $w = 1$ ใช้ support vector $(x_2, t_2) = (1, 1)$ ที่ margin: $1(w \cdot 1 + b) = 1$ ได้ $b = 0$ ค่า dual objective 0.5 เท่ากับ เทอม norm ใน primal ตรงตาม optimality ของ convex feasible problem นี้

เปลี่ยนให้เป็น soft margin

เพิ่ม box constraint $0 \leq a \leq C$ จะได้ $a = \min(0.5, C)$ สำหรับข้อมูลนี้ หาก $C = 0.2$ ได้ $w = 0.4$ ใช้ symmetry เลือก $b = 0$ จุดทั้งสองมี signed score 0.4 และ minimum slack 0.6 Objective primal $= 0.5(0.4)^2 + 0.2(1.2) = 0.32$. dual $= 2(0.2) - 2(0.2)^2 = 0.32$ เท่ากัน

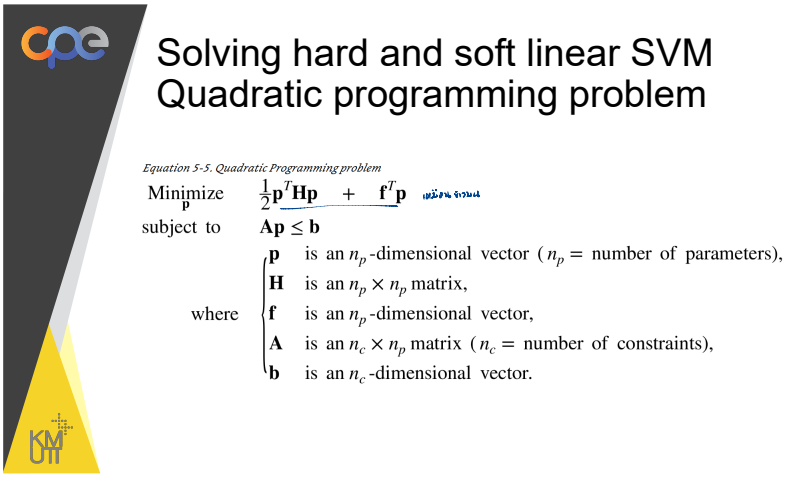
ระวัง: Soft margin จุดบน boundary ไม่ได้ทำให้ bias ที่ unique ทุก dataset ตัวอย่าง $b = 0$ นี้เป็น choice ตาม symmetry ที่ optimal ไม่ใช่สูตรว่า soft margin bias ต้อง 0 เสมอ

Kernel hyperparameter แบบคำนวณง่าย

สำหรับ RBF $K(x, z) = \exp[-\gamma\|x - z\|^2]$ ถ้า distance 2 จะได้ $K = e^{-4\gamma}$ เมื่อ $\gamma = 0.1$ ได้ 0.6703. $\gamma = 1$ ได้ 0.0183 จึงเห็น gamma ที่ใหญ่ ทำ similarity ลดเร็วตาม distance Scaling input สองเท่าทำ squared distance สี่เท่า จึงเปลี่ยน kernel แม้ gamma เดิม

3.13 SVM เป็น Quadratic Program อย่างไร

แหล่งเนื้อหา: L3 PDF p.16,19–24



รูป 3.7: รูปแบบ QP ในหน้า 16

อ่านภาพทีละจุด **1.** Quadratic term กำหนด curvature และ linear term กำหนด cost ของ slack

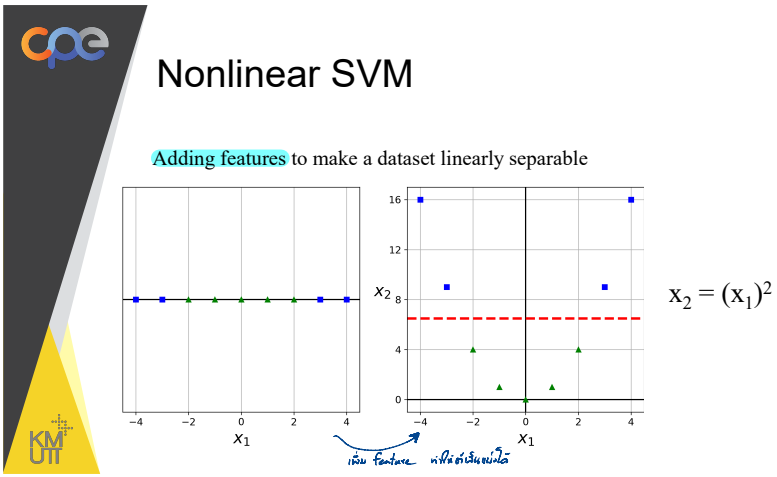
2. ข้อจำกัดเขียนเป็น linear inequalities ได้ แม้ objective มี quadratic term

ใช้ตัวแปรรวม $u = (w, b, \xi_1, \ldots, \xi_m)$ เขียน min $\frac{1}{2}u^T Hu + q^T u$ โดย $H = \text{diag}(I_d, 0, 0_m)$ และ $q = (0_d, 0, C\mathbf{1}_m)$ จึง objective เท่ากับ $\|w\|^2/2 + C\sum \xi_i$ Bias ไม่มี quadratic penalty ใน SVM formulation นี้ Constraints คือ $-t_i w^T x_i - t_i b - \xi_i \le -1$ และ $-\xi_i \le 0$ แต่ละแถวเป็น linear ในตัวแปรรวมและ H เป็น PSD จึงเป็น convex QP Hard margin ไม่มี slack เมื่อข้อมูล separable ส่วน soft-margin dual มี $0 \le \alpha_i \le C$ และ $\sum \alpha_i t_i = 0$ หากใช้ dual แบบ minimization ในแหล่งหนึ่งและ maximization อีกแหล่ง objective จะต่างกันด้วยเครื่องหมาย อย่าบวกข้อจำกัดเหมือน hard margin แล้วลืม upper bound C

> **ต้องจำ** แยก quadratic objective กับ linear constraints. C ลงโทษ slack ไม่ใช่ bias และไม่ได้เปลี่ยน feature dimension

3.14 Explicit feature map, kernels และข้อจำกัดที่ต้องจำ

แหล่งเนื้อหา: L3 PDF p.25–33



รูป 3.8: เพิ่ม feature x^2 ในหน้า 25 เพื่อแยกกลุ่มที่เส้นเดียวแยกไม่ได้

อ่านภาพทีละจุด **1.** บนแกน x original จุดสองปลายอาจเป็น class เดียวกัน. mapping (x, x^2) ทำให้ separator linear ในพื้นที่ใหม่

2. เส้น $ax + bx^2 + c = 0$ กลับมาเป็น nonlinear boundary ใน original input space ไม่ใช่เปลี่ยน labels ของจุด

เช่น $\phi(x) = (x, x^2)$ และ decision $x^2 - 1 > 0$ แยก $|x| > 1$ จาก $|x| < 1$ ได้ แม้ไม่ใช่ threshold เดียวบน x Kernel คำนวณ inner product ของ mapped vectors โดยไม่สร้างทุก dimension อย่างชัดแจ้ง Polynomial degree 2 เมื่อ $K(x, z) = (x^T z + 1)^2$ มี cross terms และ scaling ต้องสอดคล้อง ไม่ใช่ square features ทีละตัวแล้วละทิ้ง interaction

รูป 3.9: ตาราง kernel functions หน้า 31

อ่านภาพทีละจุด **1.** Linear และ polynomial และ RBF ให้ similarity ตามรูปแบบที่กำหนด. RBF เพิ่ม γ แล้ว similarity ของจุดต่างกันลดลง
2. ชื่อ “sigmoid kernel” ไม่รับรอง positive-semidefinite สำหรับทุก parameter ต้องมีเงื่อนไขที่เหมาะสม ไม่ถือว่า activation function ทุกแบบเป็น valid kernel

ระวัง: บันทึก “ไม่สอบ” ในสไลด์เป็นข้อมูลของรอบสอบแยก ไม่ใช้ตัดเนื้อหาจากขอบเขต Lecture 1–8 โดยไม่มีประกาศเฉพาะ การขยาย polynomial map ใช้เพื่อเข้าใจ ไม่ใช่ยืนยันว่าจะต้องพิสูจน์ทุก kernel

3.15 Lecture 3: สรุปและ Checklist

แหล่งเนื้อหา: L3 PDF p.3–34

ประเด็น	Hard margin	Soft margin
constraint	ทุกจุดต้อง $t_i f_i \geq 1$	ยอม slack และลงโทษด้วย C
ข้อมูล	ต้อง linearly separable	รองรับ overlap และ outlier มากขึ้น
failure	outlier ทำ infeasible และ ถนนแคบ	C ไม่เหมาะทำ under และ overfit

สูตรและความหมาย

$f = w^T x + b$ คือ score. distance $= |f|/\|w\|$. full margin $= 2/\|w\|$. soft objective $= \frac{1}{2}\|w\|^2 + C\sum\xi_i$. hinge $= \max(0, 1 - t_i f_i)$. kernel decision $= \sum_{SV} \alpha_i t_i K(x_i, x) + b$

Checklist

- ระบุ w, b, hyperplane dimension และ margin ได้
- แปลง 0/1 เป็น $-1/+1$ ก่อนใช้ constraint ได้
- คำนวณ score, distance, slack และ hinge ได้
- แยก correctly classified ใน margin กับ misclassified ได้
- อธิบายทิศของ C และ γ โดยมี caveat ได้
- ทำ Lagrange ตัวอย่างสองตัวแปรทีละขั้นได้
- แยก primal objective, dual objective และ sign convention ได้
- อธิบายว่า kernel trick แทน dot product และทำนายผ่าน SV ได้

ต้องจำ อย่าจำ SVM เป็น probability model อย่าสลับ full กับ half margin อย่าใช้ C แบบเดียวกับ λ. อย่าเรียก convolution filter ว่า SVM kernel trick

Lecture 4 Regression, Shrinkage & Survival

แหล่งเนื้อหา: L4 PDF p.1–5 | ขอบเขตมิดเทอม

คำย่อที่ใช้ในบทนี้

GLM คือ Generalized Linear Model. **MLE** คือ Maximum Likelihood Estimation. **KM** คือ Kaplan–Meier estimator. RSS, MSE และ OLS ใช้ชื่อเต็มตามบทที่ 2. CDF คือ Cumulative Distribution Function หรือฟังก์ชันแจกแจงสะสม ส่วน PDF ในสูตร probability หมายถึง Probability Density Function ต่างจากคำว่า PDF ที่ใช้ระบุหน้าไฟล์เอกสาร. บทนี้เริ่มจากการถดถอยเชิงเส้น แล้วต่อด้วยแบบจำลองที่ให้ผลลัพธ์เป็นความน่าจะเป็น จำนวนเหตุการณ์ หรือเวลาจนเกิดเหตุการณ์ การเลือกแบบจำลองจึงต้องพิจารณาทั้งชนิดคำตอบและข้อสมมติ ไม่ใช่เลือกจากเส้นที่เข้ากับข้อมูลที่สุดอย่างเดียว

$$f(x) = w_0 + w_1 x_1 + w_2 x_2 + \cdots .$$

ใน simple regression, w_1 บอกว่าค่าเฉลี่ยที่แบบจำลองทำนายเปลี่ยนเท่าไรเมื่อ x เพิ่มหนึ่งหน่วย ใน multiple regression สัมประสิทธิ์ของตัวแปรหนึ่งบอกความเปลี่ยนแปลงเมื่อคงตัวแปรอื่นไว้เท่าเดิม หาก features มี correlation สูง ค่าสัมประสิทธิ์อาจแกว่งและตีความได้ยาก แม้แบบจำลองจะทำนายได้ดี

Linear ใน parameter ไม่จำเป็นต้อง linear ใน input

แบบจำลอง $w_0 + w_1 x + w_2 x^2$ มีรูปเป็นเส้นโค้งเมื่อมองตาม x แต่ยังเป็นเชิงเส้นในพารามิเตอร์ w เราจึงเพิ่ม feature x^2 แล้วใช้ OLS ได้ ส่วน nonlinear least-squares หมายถึงสมการไม่เป็นเชิงเส้นในพารามิเตอร์ จึงแก้ด้วย closed-form ของ OLS แบบเดิมไม่ได้

ประเด็น	Simple	Multiple
predictors	หนึ่ง	หลายตัว
coefficient	slope ของ x	effect conditional on predictors อื่น
solution	OLS scalar และ matrix	OLS matrix เมื่อ rank เหมาะสม

> **ต้องจำ** อย่าสรุปว่า coefficient บวกแปลว่าเหตุเชิง causal บวก หรือว่าทุกคนที่ x มากจะมี y มากเพราะยังมี predictors อื่นและ error

ฝึกให้ทำได้ อ่านข้อความโจทย์แล้วเลือก output และ support ก่อนเลือกสมการ อธิบาย coefficient พร้อมเงื่อนไขที่ทำให้ความหมายถูกต้อง

4.1 Interaction: ผลของตัวแปรขึ้นกับอีกตัวแปร

แหล่งเนื้อหา: L4 PDF p.5

หาก model มี interaction:

$$f(x) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1 x_2, \qquad \frac{\partial f}{\partial x_1} = w_1 + w_3 x_2.$$

Effect ของ x_1 จึงไม่คงที่ทุกค่า x_2 Coefficient w_1 เป็น effect ของ x_1 เมื่อ $x_2 = 0$ ซึ่งอาจอยู่นอกช่วงข้อมูล การแปลว่า “เพิ่ม x_1 หนึ่งหน่วยแล้ว y เพิ่ม w_1 เสมอ” จึงผิด

ตัวอย่างคำนวณทีละขั้น

ให้ $f(x_1, x_2) = 2 + 3x_1 + 4x_2 - 2x_1x_2$ ตรึง $x_2 = 2$ และเพิ่ม x_1 จาก 1 เป็น 2

1. ค่าเดิม $f(1, 2) = 2 + 3(1) + 4(2) - 2(1)(2) = 2 + 3 + 8 - 4 = 9$
2. ค่าใหม่ $f(2, 2) = 2 + 3(2) + 4(2) - 2(2)(2) = 2 + 6 + 8 - 8 = 8$
3. ความต่าง $\Delta f = 8 - 9 = -1$ ตรวจกับ slope ตามเงื่อนไขนี้ได้ $3 - 2x_2 = 3 - 2(2) = -1$

ถ้าเปลี่ยนเงื่อนไขเป็น $x_2 = 0$ จะได้ slope $3 - 2(0) = 3$ ดังนั้นผลของ x_1 ไม่คงที่ทุกกลุ่ม

การทำ design matrix

สร้างคอลัมน์ $1, x_1, x_2, x_1x_2$ แล้ว estimate weights ตามลำดับ เห็นว่า interaction เป็น feature เพิ่มหนึ่งคอลัมน์ ไม่ได้เปลี่ยน OLS เป็นอีก algorithm การมี x_1x_2 อาจเพิ่ม flexibility และเพิ่มโอกาส overfit จึงต้องตรวจข้อมูลใหม่

Multiple regression with interaction

- Adding interaction terms to a regression model can greatly expand understanding of the relationships among the variables in the model
- This occurs when two or more variables depend on one another for the outcome

$$f(\boldsymbol{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2 + \cdots$$

รูป 4.1: Interaction term ในสไลด์เพิ่มความสามารถอธิบายร่วมของ predictors ต้องอ่าน coefficient แบบ conditional (L4 p.5)

อ่านภาพทีละจุด **1.** เทียบผลของ predictor หนึ่งภายใต้ค่าของอีก predictor: interaction ทำให้ slope เปลี่ยนตามกลุ่ม

2. อ่าน partial effect จาก coefficient หลักบวก coefficient interaction คูณค่าตัวแปรที่ตรึงไว้ ไม่อ่าน coefficient หลักเป็นผลคงที่ทุก observation

> **ต้องจำ** จำ derivative และ finite difference ของ model interaction และตรวจว่าตัวแปรใดถูกคงไว้ก่อนคำนวณ effect

4.2 Shrinkage: ทำไมต้องลงโทษ weight

แหล่งเนื้อหา: L4 PDF p.6–8, 12

OLS ให้ความสำคัญกับ fit training residual แต่เมื่อ feature เยอะหรือสัมพันธ์กันมาก coefficient อาจใหญ่และแปรผันตาม sample การเพิ่ม penalty ทำให้ยอม fit แย่ลงบางส่วนเพื่อให้ solution เสถียรขึ้น

$$J(w) = \text{RSS}(w) + \lambda\,\text{Penalty}(w), \quad \lambda \geq 0.$$

$\lambda = 0$ กลับเป็น unregularized objective. λ เพิ่มทำให้การใช้ weight ใหญ่แพงขึ้น ผลทั่วไปคือ bias เพิ่ม, variance ลด แต่ถ้าลงโทษแรงเกินอาจ underfit การ regularize ไม่ใช่การแก้ optimizer ที่ยังไม่ converge

ทำไมไม่ลงโทษ intercept ตามสไลด์

สูตร L4 p.8 sum penalty เริ่มจาก $j = 1$ จึงไม่รวม w_0 เพราะ intercept เป็น baseline ขณะที่ weight ของ predictor คุมความไวของ response ต่อ feature ถ้าบางโจทย์ใช้ convention ต่างกันต้องอ่าน objective ตามนั้น

ความสำคัญของ scale

ถ้าเปลี่ยนหน่วย x_j เป็น 100 เท่า coefficient ที่ให้ prediction เดิมลดเหลือหนึ่งในร้อย Penalty บน weights จึงเปลี่ยนแม้ function เท่าเดิม การ standardize ทำให้การเปรียบเทียบ shrinkage ของ predictor เหมาะสมขึ้น

> **ต้องจำ** Regularization ควบคุม complexity และ generalization ผ่าน objective ส่วน Gradient Descent เป็น optimization ทั้งสองใช้ร่วมกันได้

ฝึกให้ทำได้ ดู training และ validation behavior แล้วอธิบายว่าควรเพิ่มหรือลด λ พร้อมเหตุ ไม่เลือกจาก training score เพียงอย่างเดียว

4.3 Ridge, Lasso และ Elastic Net

แหล่งเนื้อหา: L4 PDF p.8–13

$$J_{ridge} = \text{RSS} + \lambda \sum_{j=1}^{p} w_j^2, \qquad J_{lasso} = \text{RSS} + \lambda \sum_{j=1}^{p} |w_j|.$$

Ridge ใช้ squared L2 penalty ช่วย shrink coefficients แต่โดยทั่วไปไม่ทำให้เป็นศูนย์ exact. Lasso ใช้ L1 และทำให้หลาย coefficients เป็นศูนย์ได้ จึงทำ embedded feature selection ไม่ใช่ทุก feature จะหายแน่นอนหรือเลือกชุดเดียวกันเมื่อ predictors highly correlated

ประเด็น	Ridge และ L2	Lasso และ L1
penalty	$\sum w_j^2$	$\sum \lvert w_j \rvert$
coefficient	หดแต่โดยทั่วไปไม่ exact zero	บางตัว exact zero ได้
constraint geometry	smooth sphere และ circle	diamond มี corners บนแกน
จุดเด่น	เสถียรเมื่อ predictors สัมพันธ์	sparse model และ selection

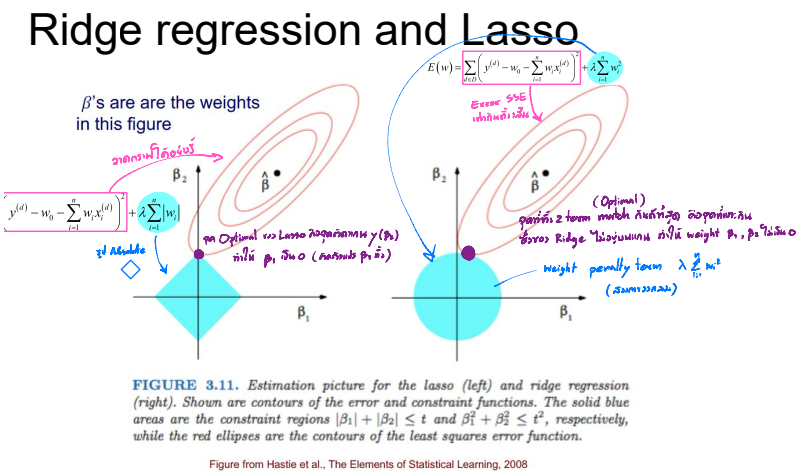


FIGURE 3.11. *Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \le t$ and $\beta_1^2 + \beta_2^2 \le t^2$, respectively, while the red ellipses are the contours of the least squares error function.*

Figure from Hastie et al., The Elements of Statistical Learning, 2008

รูป 4.2: Contour ของ RSS แตะ constraint: corner ของ L1 ทำให้มีโอกาสแตะแกนที่ coefficient เป็น 0 (L4 p.11)

อ่านภาพทีละจุด **1.** Contour คือค่า RSS เท่ากัน ส่วนพื้นที่ constraint จำกัดขนาด coefficients

2. จุดสัมผัส L1 มักพบ corner บนแกนจึงมี coefficient ศูนย์. L2 ขอบโค้งมัก shrink ต่อเนื่อง รูปเป็น geometry ของ objective ไม่ใช่เส้นแบ่ง classes

Elastic Net รวม L1/L2. ในสไลด์มี $l1_ratio$ ซึ่ง 0 คือ L2 และ 1 คือ L1 ภายใต้ implementation นั้น ในโน้ตนี้ไม่ถือค่าตัวอย่างในโค้ดเป็น hyperparameter ที่ดีที่สุดทุกงาน

> **ต้องจำ** จำ penalty ไม่ใช่แค่ชื่อ. L2 norm จริงคือ $\sqrt{\sum w^2}$ แต่ Ridge objective ในสไลด์ใช้ squared L2 norm $\sum w^2$

4.4 Constrained form และตัวอย่าง regularization

แหล่งเนื้อหา: L4 PDF p.9–13. Explanation เสริม

Lasso constrained form คือ minimize RSS subject to $\sum_j |w_j| \le t$ เมื่อ t เล็ก constraint แรง ส่วน penalized form ใช้ λ ซึ่งใหญ่แล้ว penalty แรง สอง hyperparameters มีทิศกลับกันแต่สัมพันธ์ผ่าน optimum ไม่ใช่สูตรง่าย $t = 1/\lambda$ ทุก dataset

ตัวอย่างคำนวณทีละขั้น

เปรียบเทียบ candidate A ที่ RSS $= 8, w = (3, 0)$ กับ B ที่ RSS $= 9, w = (1, 1)$ ใช้ $\lambda = 1$

1. Ridge penalty ของ A คือ $3^2 + 0^2 = 9$ และของ B คือ $1^2 + 1^2 = 2$
2. Ridge objective: $J_A = 8 + 1(9) = 17$, $J_B = 9 + 1(2) = 11$ จึงเลือก B ระหว่างสอง candidate
3. Lasso penalty ของ A คือ $|3| + |0| = 3$ และของ B คือ $|1| + |1| = 2$
4. Lasso objective: $J_A = 8 + 1(3) = 11$, $J_B = 9 + 1(2) = 11$ จึงเสมอกันตาม objective นี้

ผลนี้เปรียบเทียบเฉพาะสอง candidate ไม่ใช่การพิสูจน์ว่า B เป็น optimum ของ parameter ทุกค่าที่เป็นไปได้

อนุพันธ์สำหรับ Ridge

ถ้าใช้ RSS และไม่ penalty intercept:

$$\nabla_w J = 2X^T(Xw - y) + 2\lambda D w,$$

โดย D เป็น diagonal matrix ที่ตำแหน่ง intercept เป็น 0 และ weight อื่นเป็น 1 สำหรับ full-rank และ positive-definite ที่เหมาะสมจะได้ $(X^TX + \lambda D)\hat{w} = X^Ty$

อ่านเพิ่มจากตำรา ISLP §6.2: gradient ของ Lasso ใช้ sign ของ w_j เมื่อ $w_j \neq 0$ และใช้ subgradient ที่ 0 ไม่ใช่ differentiate absolute value เป็น 1 ทุกจุด

> **ต้องจำ** เมื่อเปลี่ยน RSS เป็น MSE ต้องพิจารณา relative scale ของ λ ด้วย มิฉะนั้น “ค่า regularization เท่ากัน” จะไม่ให้ objective เดียวกัน

ระวัง: L4 p.13 แสดง software convention C ซึ่งสัมพันธ์กับ strength แบบผกผัน ต้องอ่าน function และ parameter ที่ใช้ ไม่สลับกับ λ ในสูตร Ridge

4.5 Ridge และ Lasso หนึ่ง coefficient: เห็น shrinkage จากสมการจริง

แหล่งเนื้อหา: L4 PDF p.8–13. derivation ฝึกเสริม

ถือว่า centered data และไม่ penalize intercept ตั้ง $s = \sum x_i^2$, $q = \sum x_i y_i$ แล้วเขียน RSS เป็น $\sum y_i^2 - 2qw + sw^2$ ทำให้เห็นผลของ penalty โดยไม่ต้อง invert matrix ใหญ่

Ridge ใช้ $RSS + \lambda w^2$

ขยาย objective ก่อนแล้วหา derivative ตาม w

$$\begin{aligned} J(w) &= \sum_i (y_i - wx_i)^2 + \lambda w^2 = \sum_i y_i^2 - 2w \sum_i x_i y_i + w^2 \sum_i x_i^2 + \lambda w^2 \\ &= \sum_i y_i^2 - 2qw + (s+\lambda) w^2, \\ J'(w) &= -2q + 2(s+\lambda)w = 0 \quad \Rightarrow \quad (s+\lambda)w = q \quad \Rightarrow \quad \hat{w} = \frac{q}{s+\lambda}. \end{aligned}$$

ถ้า $s = 4, q = 6$, OLS ใช้ $\lambda = 0$ ให้ $6/4 = 1.5$ ส่วน Ridge ที่ $\lambda = 2$ ให้ $6/(4+2) = 1$ เนื่องจาก $q \neq 0$ และ denominator มีค่าจำกัด จึงยังไม่เป็นศูนย์ exact

Lasso ใช้ $RSS + \lambda|w|$

เมื่อ $w > 0$: $-2q + 2sw + \lambda = 0$ ให้ $w = (q - \lambda/2)/s$. เมื่อ $w < 0$: $w = (q + \lambda/2)/s$ รวมกับ subgradient ที่ 0 ได้

$$\hat{w} = \frac{\text{sign}(q)}{s} \max(|q| - \lambda/2, 0).$$

ที่ $w = 0$ subgradient ของ $|w|$ อยู่ใน $[-1, 1]$ เงื่อนไข optimum คือ $0 \in -2q + \lambda[-1, 1]$ ซึ่งเป็นไปได้ก็ต่อเมื่อ $|2q| \leq \lambda$ หรือ $|q| \leq \lambda/2$ จึงได้กรณีศูนย์ในสูตรข้างบน ใช้ $s = 4, q = 6$: ถ้า $\lambda = 2$, $w = (6 - 2/2)/4 = (6-1)/4 = 1.25$ แต่ถ้า $\lambda = 12$ จะมี $|q| = 6 \leq 12/2$ ทำให้ $w = \max(6-6, 0)/4 = 0$ ถ้า objective ใช้ $\frac{1}{2} RSS$ แทน RSS threshold จะเป็น λ จึงต้องอ่าน convention ก่อนท่อง soft-threshold

> **ต้องจำ** L1 มี corner และ subgradient ทำให้ coefficient เป็นศูนย์ได้. L2 shrink ต่อเนื่อง สูตรเฉพาะหนึ่ง predictor นี้ไม่ใช่ใช้ทีละ column แยกได้ทันทีเมื่อ predictors correlated

4.6 Generalized Linear Model และ Poisson

แหล่งเนื้อหา: L4 PDF p.17–22

เริ่มจาก linear predictor $\eta = \beta_0 + \sum_j \beta_j x_j$ โดย x_j เป็นตัวแปรอธิบาย และ β_j เป็น coefficient ของตัวแปรนั้น จากนั้นเลือก link ให้ตรงกับชนิดคำตอบ: Linear Regression ใช้ mean $\mu = \eta$, Logistic ใช้ log-odds $\log[p/(1-p)] = \eta$, ส่วน Poisson ใช้ log-mean $\log \mu = \eta$

$$Y \mid x \sim \text{Poisson}(\mu(x)), \qquad P(Y = y \mid x) = \frac{e^{-\mu(x)}\mu(x)^y}{y!}, \quad y = 0, 1, 2, \ldots.$$

ค่า count ที่สังเกตได้เป็นจำนวนเต็มไม่ติดลบ ส่วนค่าเฉลี่ยที่แบบจำลองทำนาย μ เป็นจำนวนจริงบวก จึงไม่จำเป็นต้องเป็นจำนวนเต็ม ความหมายต่างจาก class code และจากผลทำนายต่อเนื่องของ OLS ซึ่งอาจติดลบได้

$$\log \mu(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \quad \mu(x) = e^{\eta(x)} > 0.$$

เหตุผลที่ใช้ log link

Linear predictor อยู่ได้ทุกค่าจริง แต่ count mean ต้องไม่ติดลบ การ exponentiate ทำให้ mean positive เสมอ และทำให้ coefficient ตีความแบบ multiplicative

ตัวอย่างคำนวณทีละขั้น

ให้ $\beta_0 = \ln 2, \beta_1 = \ln 1.5$ ดังนั้น $\mu(x) = e^{\ln 2 + x \ln 1.5} = 2(1.5)^x$

1. $x = 0$: $\mu(0) = 2(1.5)^0 = 2$
2. $x = 1$: $\mu(1) = 2(1.5)^1 = 3$
3. $x = 2$: $\mu(2) = 2(1.5)^2 = 2(2.25) = 4.5$

ผลต่างเป็น $3 - 2 = 1$ แล้ว $4.5 - 3 = 1.5$ แต่ ratio ทั้งสองเป็น 1.5 เท่ากัน จึงตีความเป็นตัวคูณ ไม่ใช่ผลบวกคงที่

> **ต้องจำ** Log link ใช้กับ mean ไม่ใช่ใช้ $\log y_i$ กับทุก observation แล้ว OLS โดยตรง เพราะ $y_i = 0$ เกิดได้

อ่านเพิ่มจากตำรา ISLP §4.6: Poisson model มี mean=variance ภายใต้ distribution นี้ ถ้าข้อมูล overdispersed อาจต้อง model อื่น แต่เป็นส่วนเสริม ไม่ขยายเป็นข้อสอบบังคับ

4.7 Poisson coefficient และ Maximum Likelihood

แหล่งเนื้อหา: L4 PDF p.23–25

หาก predictor x_j เพิ่มหนึ่งหน่วยโดยอื่นคงที่:

$$\frac{\mu(x_j + 1)}{\mu(x_j)} = e^{\beta_j}.$$

$\beta_j > 0$ mean เพิ่ม. $\beta_j < 0$ mean ลด. $\beta_j = 0$ mean ไม่เปลี่ยน หาก $\beta_j = \log 1.2$ แปลว่า mean คูณ 1.2 หรือเพิ่ม 20% ไม่ใช่เพิ่ม 0.1823 หน่วย

MLE คือเลือก parameter ให้ observed data plausible ที่สุด

สำหรับ independent observations ใช้ likelihood

$$L(\beta) = \prod_i \frac{e^{-\mu_i}\mu_i^{y_i}}{y_i!}, \quad \ell(\beta) = \sum_i [y_i \eta_i - e^{\eta_i} - \log(y_i!)].$$

ขั้นที่ขยาย log ใช้ $\ln(\prod_i a_i) = \sum_i \ln a_i$, $\ln e^{-\mu_i} = -\mu_i$ และ $\ln \mu_i = \eta_i$ จึงได้ $\ell = \sum_i [-\mu_i + y_i \ln \mu_i - \ln(y_i!)]$ แล้วแทน $\mu_i = e^{\eta_i}$ เป็นสมการด้านบน สำหรับ mean ร่วมกัน μ ให้ $s = \sum_i y_i$ จะมี $\ell'(\mu) = s/\mu - n = 0$ จึง $\hat{\mu} = s/n$ เมื่อ $s > 0$ และ $\ell''(\mu) = -s/\mu^2 < 0$ ถ้า $s = 0$ ทุก count เป็นศูนย์ คำตอบอยู่ที่ boundary $\mu = 0$ ซึ่งเป็น limit ของ log link เมื่อ intercept ลดลงโดยไม่มีขอบเขต Maximize L กับ maximize ℓ ได้คำตอบเดียวกันเพราะ log monotone ตัวอย่างที่สไลด์ใช้คือ accidents per mine ในช่วง 3 เดือนโดยมี predictors INB, EXTRP, AHS, AGE จึงต้องเชื่อมชนิด count กับช่วงสังเกต ไม่สับสนกับ probability ของการเกิดอุบัติเหตุอย่างน้อยหนึ่งครั้ง

ตัวอย่างคำนวณทีละขั้น

ใช้ $P(Y = y) = e^{-\mu}\mu^y/y!$ และ $\mu = 2$

1. ที่ $y = 0$ ใช้ $2^0 = 1$ และ $0! = 1$: $P(Y = 0) = e^{-2}(1)/1 \approx 0.135335$
2. ที่ $y = 2$ ใช้ $2^2 = 4$ และ $2! = 2$: $P(Y = 2) = e^{-2}(4)/2 = 2e^{-2} \approx 0.270671$

Mean เท่ากับ 2 จึงไม่ได้หมายความว่าทุก observation ต้องมี count เท่ากับ 2

อ่านเพิ่มจากตำรา การขยาย log-likelihood นี้เป็น Explanation จาก distribution ใน L4 p.21 และ ISLP §4.6 ช่วยเข้าใจ MLE ขอบเขตสูตรที่อาจารย์ต้องการจำให้ยึด annotation และ ประกาศสอบ

> **ต้องจำ** เปรียบเทียบ Logistic กับ Poisson: e^{β_j} คูณ odds ใน Logistic แต่คูณ mean ใน Poisson

4.8 Interaction และ Poisson: ตีความผลร่วมแบบไม่สลับ scale

แหล่งเนื้อหา: L4 PDF p.5,17–25. ตัวเลขฝึกใหม่

ให้ expected count เป็น $\mu = e^{\beta_0+\beta_1 x+\beta_2 z+\beta_3 xz}$ ดังนั้นผลของเพิ่ม x หนึ่งหน่วยเมื่อ z คงที่คือ

$$\mu(x + 1, z)/\mu(x, z) = e^{\beta_1+\beta_3 z}.$$

ถ้า $\beta_0 = \log 2$, $\beta_1 = \log 1.5$, $\beta_2 = \log 2$, $\beta_3 = \log(2/3)$: กลุ่ม $z = 0$ มี multiplier ของ x เท่ากับ 1.5 แต่กลุ่ม $z = 1$ มี $1.5(2/3) = 1$ จึงไม่มีผลของ x ต่อ mean ในกลุ่มนี้ แม้ $\beta_1 > 0$

z	$\mu(0, z)$	$\mu(1, z)$	อ่านผล
0	2	3	เพิ่ม mean 50%
1	4	4	mean ไม่เปลี่ยนเมื่อเพิ่ม x

ระวัง: Poisson mean เป็นจำนวนจริงบวกได้ ไม่ต้องเป็น integer ทุกค่าที่ทำนาย. observed count เป็น integer ส่วน β_1 เปลี่ยน log-mean ไม่ใช่เพิ่ม count ทีละ β_1 โดยตรง

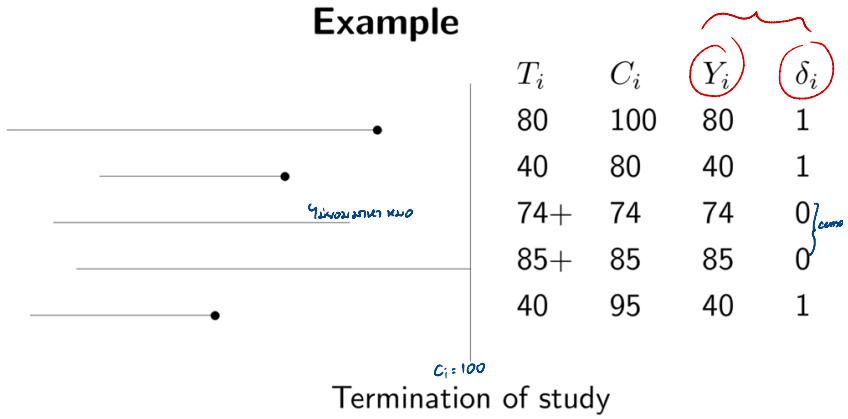
4.9 Survival Analysis และ censored observation

แหล่งเนื้อหา: L4 PDF p.26–33, 38–39

Survival analysis สนใจเวลาจนเกิด event เช่น recurrence, machine failure หรือการใช้บัตรครั้งถัดไป คำว่า survival ไม่จำเป็นต้องเป็นการเสียชีวิต Logistic บอกมี และ ไม่มี event ภายในเงื่อนไขที่กำหนด แต่ไม่ได้แทน time-to-event และ censoring โดยอัตโนมัติ ให้ T_i เป็น event time, C_i เป็น censoring time และ observation:

$$Y_i = \min(T_i, C_i), \quad \delta_i = \begin{cases} 1 & T_i \le C_i \text{ (observed event)} \\ 0 & T_i > C_i \text{ (right-censored).} \end{cases}$$

Right censoring แปลว่าทราบว่า $T_i > Y_i$ แต่ไม่ทราบเวลาเกิด event ที่แท้จริง เช่นจบการศึกษาขณะยังไม่เกิด event คนนี้ไม่ได้มี event ที่วันจบศึกษา และไม่ได้แปลว่าจะไม่มี event ตลอดไป



รูป 4.3: การบันทึกเวลาที่เห็นกับ event indicator: censored rows ยังมีข้อมูลเรื่องการอยู่รอดถึงเวลาที่ติดตาม (L4 p.39)

อ่านภาพทีละจุด **1.** หนึ่งแถวมีเวลาที่สังเกตและ indicator ว่า event เกิดจริงหรือยังไม่เห็น

2. กรณี censor เวลาเป็นขอบล่างของ event time ที่ไม่ทราบ ไม่ใส่ event=1 เพียงเพราะการติดตามหยุด และไม่ทิ้งแถวนี้จาก survival analysis

> **ต้องจำ** Censored ไม่ใช่ negative label ถาวร และไม่ควรทิ้งทุกคนที่ censored เพราะเขามีข้อมูลใน risk set ก่อนหลุดการติดตาม

ระวัง: สไลด์ระบุ independent censoring เป็นสมมติฐานของวิธีวิเคราะห์ ถ้าผู้เสี่ยงสูงหลุดติดตามเป็นพิเศษอาจเกิด bias ไม่ได้แก้ด้วยเขียน indicator อย่างเดียว

ฝึกให้ทำได้ ให้ timeline แล้วเขียน (Y_i, δ_i) ถูกและบอกความรู้ที่ได้จาก censored row

4.10 Survival function, density, hazard

แหล่งเนื้อหา: L4 PDF p.34–37

$$F(t) = P(T \le t), \quad S(t) = P(T > t) = 1 - F(t), \quad f(t) = F'(t).$$

S ไม่เพิ่มตามเวลา อยู่ระหว่าง 0 กับ 1 ใน formulation ของสไลด์เริ่มที่ 1 และไป 0 เมื่อเวลามากพอ แต่ S ไม่ใช่จำนวนคนดิบหรือ probability ที่ event เกิด ณ จุดเวลาแบบ continuous

$$h(t) = \lim_{\Delta t \to 0^+} \frac{P(t < T \le t + \Delta t \mid T > t)}{\Delta t} = \frac{f(t)}{S(t)}.$$

Hazard เป็น instantaneous conditional rate ต่อเวลา ไม่ใช่ probability ที่ต้องอยู่ระหว่าง 0 กับ 1 และไม่ใช่ $1 - S(t)$ ซึ่งเป็น cumulative event probability

$$h(t) = \frac{f(t)}{S(t)} = -\frac{S'(t)}{S(t)} = -\frac{d}{dt} \ln S(t),$$

$$H(t) = \int_0^t h(u)\, du = -[\ln S(u)]_0^t = -\ln S(t) + \ln S(0) = -\ln S(t),$$

$$S(t) = e^{-H(t)}, \qquad h(t) = -\frac{d}{dt} \log S(t).$$

ระวัง: L4 PDF p.37 พิมพ์ $h(t) = \partial \log S(t)/\partial t$ โดยขาดเครื่องหมายลบ หาก S ลด derivative เป็นลบทั้งที่ hazard ไม่ติดลบ จึงใส่เครื่องหมายลบตามนิยาม p.36 และ ISLP §11.5.1 (printed p.476–478) พร้อมบันทึก discrepancy

ตัวอย่างคำนวณทีละขั้น

ให้ $S(t) = e^{-0.2t}$

1. Cumulative hazard $H(t) = -\ln S(t) = -\ln(e^{-0.2t}) = 0.2t$
2. Density $f(t) = -S'(t) = -[-0.2e^{-0.2t}] = 0.2e^{-0.2t}$
3. Hazard $h(t) = f(t)/S(t) = 0.2e^{-0.2t}/e^{-0.2t} = 0.2$ ต่อหน่วยเวลา
4. ที่ $t = 5$: $S(5) = e^{-0.2(5)} = e^{-1} \approx 0.367879$ ดังนั้นโอกาสเกิด event ภายในเวลา 5 คือ $1 - S(5) \approx 0.632121$

Hazard 0.2 จึงมีความหมายต่างจาก event probability ในช่วงเวลานี้

> **ต้องจำ** จำ $S = 1 - F$, $h = f/S$, $H = -\log S$, $S = e^{-H}$ และเครื่องหมายลบใน derivative

4.11 Kaplan–Meier และ risk set

แหล่งเนื้อหา: L4 PDF p.40–43

Kaplan–Meier เป็น nonparametric survival estimate สำหรับ right-censored data ให้ distinct event times t_j, risk set size ก่อน event n_j และจำนวน event d_j:

$$\hat{S}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right).$$

ในแต่ละเวลา ถ้าไม่มี event แต่มีคน censored survival ไม่ลด ณ จุดนั้น เพียงลด risk set สำหรับเวลาถัดไป จึงห้ามนับ censored เป็น death และห้ามลืมหักออกจาก denominator ใน event ถัดไป

ตัวอย่างคำนวณทีละขั้น

เริ่มติดตาม 5 คน ให้ event ที่เวลา 2,4,5 และ censor อีกหนึ่งคนที่เวลา 3

เวลา	n_j ก่อน	d_j	censor	การคำนวณ $\hat{S}$
2	5	1	0	$1(1 - 1/5) = 4/5 = 0.8$
3	4	0	1	0.8 คงเดิม แล้วเหลือ 3 คน
4	3	1	0	$(4/5)(1 - 1/3) = (4/5)(2/3) = 8/15$
5	2	1	0	$(8/15)(1 - 1/2) = (8/15)(1/2) = 4/15$

จึง $\hat{S}(4) \approx 0.533333$ และ $\hat{S}(5) \approx 0.266667$ ทุก factor ใช้จำนวนที่ยังเสี่ยงก่อนเวลานั้น

	at risk			survival
59	26	1	0	$25/26 = 0.962$
115	25	1	0	$24/25 \times 0.962 = 0.923$
156	24	1	0	$23/24 \times 0.923 = 0.885$
268	23	1	0	$22/23 \times 0.885 = 0.846$
329	22	1	0	$21/23 \times 0.846 = 0.808$
353	21	1	0	$20/21 \times 0.808 = 0.769$
365	20	0	1	$20/20 \times 0.769 = 0.769$
377	19	0	1	$19/19 \times 0.769 = 0.769$
421	18	0	1	$18/18 \times 0.769 = 0.769$
431	17	1	0	$16/17 \times 0.769 = 0.688$
⋮				⋮
⋮				⋮

รูป 4.4: ตารางต้นทาง (แก้ตัวเลขที่ไม่สอดคล้องในหัวข้อ KM ต้นทางด้านถัดไป): ที่มี censored แต่ไม่มี death cumulative survival คงเดิม ต้อง update risk set แทน (L4 p.42)

อ่านภาพทีละจุด 1. อ่าน risk set ก่อนเวลาที่มี event และจำนวน events ที่เวลานั้นก่อนแทน factor

2. แถว censor-only คง survival เดิมแต่จำนวนเสี่ยงถัดไปลด ตัวส่วนไม่ใช่ขนาด dataset เดิมทุกแถวและไม่เอา cumulative deaths มาแทน event count รอบนั้น

สิ่งที่ต้องสังเกตในกราฟ

เส้น KM เป็นขั้นและลดที่ event times ส่วนเครื่องหมาย censor แสดงเวลาหยุดติดตาม ไม่ใช่จุดที่ curve ต้องตก ช่วงปลายมีคนเหลือน้อย estimate จึงอาจไม่เสถียรเท่าต้นช่วง การอ่านเส้นสูงกว่าควรพิจารณาเวลาและข้อมูล ไม่สรุปสาเหตุจากสีของเส้น

> **ต้องจำ** KM คูณ conditional survival ของแต่ละ event time ไม่ใช่ average ของ survival fractions ทุกเวลา

อ่านเพิ่มจากตำรา ISLP §11.3: ตัวอย่างนี้สร้างเพิ่มเพื่อฝึก risk-set arithmetic ใช้ convention จัดการ ties ที่โจทย์ระบุเมื่อ event และ censor เกิดเวลาเดียวกัน

4.12 Poisson MLE แบบ intercept-only และการตีความ

แหล่งเนื้อหา: L4 PDF p.17–25. ตัวอย่างฝึกใหม่

ให้ count observations $(0, 1, 2, 5)$ และ model $Y_i \sim \text{Poisson}(\mu)$ มี mean ร่วมกัน $\mu > 0$ log likelihood เป็น

$$\ell(\mu) = \sum_i [y_i \log \mu - \mu - \log(y_i!)] = 8 \log \mu - 4\mu - \sum_i \log(y_i!).$$

Derivative เป็น $8/\mu - 4 = 0$ จึง $\hat{\mu} = 2$ และ second derivative $-8/\mu^2 < 0$ จึงเป็น maximum ถ้าใช้ loglink $\log \mu = \beta_0$ จะได้ $\hat{\beta}_0 = \log 2 \approx 0.6931$ ไม่ใช่ 2

เมื่อเพิ่ม predictor

ให้ $\log \mu = \log 2 + \log(1.5)x$ เมื่อ $x = 0, 1, 2$ จะได้ค่าเฉลี่ยเท่ากับ $2, 3, 4.5$ ตามลำดับ จำนวนเหตุการณ์ที่สังเกตได้ยังเป็นจำนวนเต็ม แต่ค่าเฉลี่ยไม่จำเป็นต้องเป็นจำนวนเต็ม และ $P(Y = 0 \mid x = 0) = e^{-2} \approx 0.1353$ ดังนั้นค่าเฉลี่ยต้องเป็นบวก แต่จำนวนเหตุการณ์ที่สังเกตได้อาจเป็นศูนย์

อ่านเพิ่มจากตำรา ISLP §4.6: Poisson distribution กำหนด variance เท่ากับ mean conditional on features หาก data มี variance มากกว่านี้มากจะมี overdispersion เป็นเหตุให้ตรวจ model assumption ไม่ใช่สรุปว่า formula ของ mean ผิด

ระวัง: ถ้า observations ทุกตัวเป็น 0 intercept-only likelihood มี supremum เมื่อ $\mu \to 0$ หรือ $\beta_0 \to -\infty$ จึงไม่มี finiteMLE ใน log parameterization นี้ อย่าหารโดย numerator 0 แล้วประกาศ log 0 เป็น finite parameter

ต้องจำ Count mean กับ count observation ต่างกัน. Poisson coefficient อยู่ log-mean scale และ e^{β_j} เป็น multiplier เมื่อ feature เพิ่มหนึ่งหน่วยและตรึงตัวอื่น

4.13 Survival likelihood: ใช้ censored data คำนวณได้

แหล่งเนื้อหา: L4 PDF p.26–39. Extra Reference: ISLP §11.5.1

อ่านเพิ่มจากตำรา ตัวอย่าง exponential survival นี้ขยาย intuition ของ likelihood ใน textbook ไม่อ้างว่า annotation กำหนดให้ออกโจทย์ MLE นี้

ให้ $\delta_i = 1$ เมื่อ observed event และ 0 เมื่อ right censored y_i เป็น เวลาที่ติดตามและสังเกตได้ ภายใต้ independent non-informative censoring contribution ที่เกี่ยวกับ event-timemodel เป็น

$$L_i = f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}.$$

กรณีที่พบ event ใช้ density ณ เวลาที่เกิดเหตุการณ์ ส่วนกรณีที่ถูก censor ใช้ความน่าจะเป็นที่ยังไม่เกิด event จนถึงเวลาหยุดติดตาม ห้ามใช้ $f(y_i)$ กับข้อมูลที่ถูก censor เพราะไม่ทราบเวลาเกิด event จริง เมื่อใช้ exponential model ที่มี hazard คงที่ λ จะได้ $S(t) = e^{-\lambda t}$ และ $f(t) = \lambda e^{-\lambda t}$ ดังนั้น

$$L = \lambda^{\sum_i \delta_i} e^{-\lambda \sum_i y_i}, \qquad \ell = (\sum_i \delta_i) \log \lambda - \lambda \sum_i y_i.$$

เมื่อมี observed events อย่างน้อยหนึ่งตัว maximum ให้ $\hat{\lambda} = (\sum \delta_i)/(\sum y_i)$ = จำนวน events และ total follow-up time

ตัวอย่างสามคน

ถ้า A เกิด event ที่เวลา 2, B ถูก censor ที่เวลา 4 และ C เกิด event ที่เวลา 6 จะมี event ทั้งหมด 2 ครั้ง และเวลาติดตามรวม 12 หน่วย จึงได้ $\hat{\lambda} = 2/12 = 1/6$ หากนับ B เป็น event จะคำนวณผิดเป็น $3/12 = 1/4$ และประเมิน hazard สูงเกินไป จากนั้น $S(3) = e^{-3/6} \approx 0.6065$ คือความน่าจะเป็นที่ยังไม่เกิด event จนถึงเวลา 3 ไม่ใช่ค่า hazard ณ เวลา 3 ซึ่งเท่ากับ $1/6$ ต่อหน่วยเวลา

ต้องจำ Censored sample ยังให้ exposure time และ survival information. hazard เป็น rate มี units หนึ่ง และเวลา ไม่จำกัด[0,1] และไม่เท่ากับ probability ของ event ทันที

4.14 Kaplan–Meier แบบตาราง: event กับ censor ต่างหน้าที่

แหล่งเนื้อหา: L4 PDF p.40–43. ข้อมูลตัวเลขฝึกแต่งขึ้นใหม่

ติดตาม 6 คนพร้อมกัน: Aevent ที่ 1, Bcensored ที่ 2, Cevent ที่ 3, Devent ที่ 3, Ecensored ที่ 4, Fevent ที่ 5 โดยไม่มี tiedevent และ censoring ต้องคำนวณ risk set ก่อนเวลา event และรวม event ที่เวลาเดียวกัน

eventtime	risk set n_j	events d_j	factor $1 - d_j/n_j$	$\hat{S}$ หลังเวลา
1	6	1	$5/6$	$5/6$
3	4	2	$1/2$	$5/12$
5	1	1	0	0

ก่อน 3 มี C,D,E,F เพราะ Aevent ไปแล้วและ B ออกจาก risk set ที่ 2 จึง $n_3 = 4$ ไม่ใช่ 5 ที่ 3 มีสอง events ต้อง factor $1 - 2/4$ แล้ว E ออกที่ 4 จึงเหลือ F หนึ่งคนก่อน 5 Curve ไม่ตกที่ censoring times 2,4 แต่ risk set ของเวลาถัดไปเปลี่ยน

อ่าน survival ที่เวลาระหว่าง event

$\hat{S}(2.5) = 5/6$ เพราะยังไม่มี event ใหม่. $\hat{S}(4.5) = 5/12$. หลัง 5 estimate เป็น 0 เนื่องจากคนสุดท้ายใน risk set เกิด event แต่ไม่ได้แปลว่าประชากรจริงทุกคนต้องเกิด event ภายใน 5 เพราะ estimate จาก sample เล็กและ risk set ช่วงท้าย เหลือน้อยมาก

เมื่อมี tied event กับ censoring

ตัวอย่างนี้หลีกเลี่ยง ties ถ้ามี event และ censoring เวลาเดียวกันต้องระบุ convention โดยมักถือ event เกิดก่อน censoring ณเวลานั้นเพื่อรวมคน censored ใน risk set ก่อน event อย่าเดาลำดับจาก ลำดับตัวอักษร ของชื่อคน

ระวัง: KM ตีความเป็น survival estimate ได้ภายใต้ assumptions เรื่อง censoring ที่เหมาะสม ถ้าคนที่ความเสี่ยงสูงหายจาก follow-up มากกว่าโดยไม่มีการจัดการ การใช้สูตรอย่างเดียวอาจให้ estimate biased

> **ต้องจำ** ขั้นคำนวณ survival คูณเฉพาะ event times. event ลด survival และ risk set. censoring ลด risk set ในอนาคตแต่ไม่เป็น eventfactor

4.15 Embedded feature selection: อ่าน Logistic code ที่ใช้ L1 penalty

แหล่งเนื้อหา: L4 PDF p.13–16

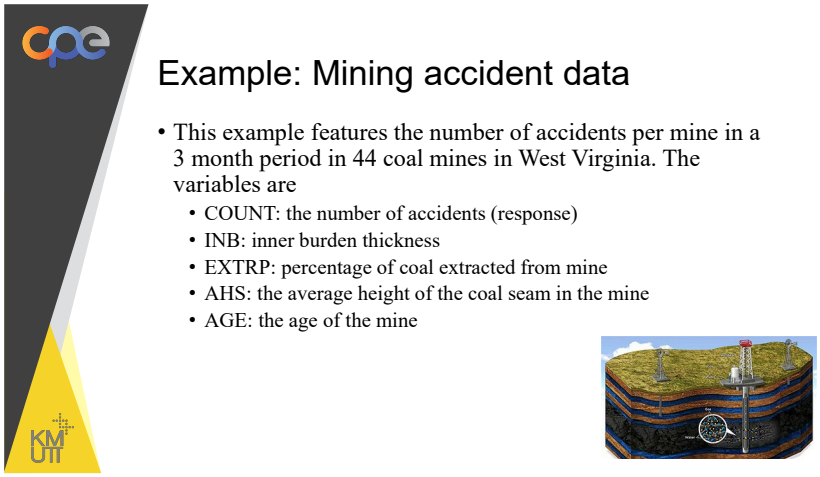
รูป 4.5: Feature mask จาก coefficients ของ L1 Logistic Regression

อ่านภาพทีละจุด **1.** Fit model กับ training data ก่อน แล้วดู coefficient ที่ไม่ใกล้ศูนย์ ไม่ใช้ class labels ของ test เลือก features **2.** Threshold $|w_j| > 0.001$ เป็นเกณฑ์คัดใน code ไม่ใช่นิยาม objective ของ Lasso และไม่ได้แปลว่า feature ที่เหลือเป็นสาเหตุของ outcome

ต้นฉบับใช้ penalty L1 กับ $C = 0.1$ เพื่อเลือก feature ของงาน classification เป็น **embedded method** เพราะการเลือกเกิดผ่านการ fit penalized model ไม่ใช่ feature ranking แยกนอก model C ที่นี่เป็น inverse strength: เล็กลงเพิ่มแรง regularization ต่างจาก λ ใน $RSS + \lambda\|w\|_1$ ที่ใหญ่ลงโทษมากขึ้น ตัวอย่างฝึก coefficients $(0.40, 0, -0.0005, -0.20)$ จะได้ mask $(1, 0, 0, 1)$ เพราะใช้ absolute value ไม่ใช่เลือกเฉพาะ positive coefficients จากนั้นฝึก และ evaluate pipeline ด้วย selected features ในแต่ละ training fold. fit mask ทั้งชุดก่อน cross-validation ทำ leakage ค่าประเมิน AUC ในตัวอย่างสไลด์เป็นผลของข้อมูล และ การแบ่งชุดนั้น ไม่ใช่ performance รับประกันของ L1

4.16 Poisson ของอาจารย์: count และ covariates ในเหมือง

แหล่งเนื้อหา: L4 PDF p.23–25



รูป 4.6: ตัวอย่าง mine accidents: count ในช่วงติดตาม กับ features ของเหมือง

อ่านภาพทีละจุด **1.** COUNT เป็นจำนวนอุบัติเหตุ ไม่ใช่ probability และไม่ใช่ time until event

2. INB, EXTR, AHS และ AGE เป็นตัวแปรอธิบาย อ่านหน่วยก่อนตีความ coefficient. expected count ไม่จำเป็นต้องเป็นจำนวนเต็ม

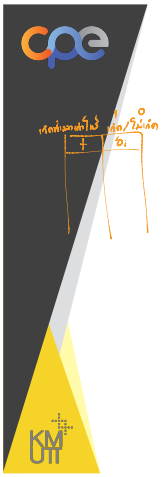
ตัวอย่างประกอบด้วย 44 เหมือง ติดตามอุบัติเหตุใน 3 เดือน. INB คือ innerburden thickness, EXTR คือ percentage extraction of coal, AHS คือ average height of coal seam, AGE คืออายุเหมือง ใช้ $\log \mu_i = \beta_0 + \sum_j \beta_j x_{ij}$ เมื่อเพิ่ม x_j หนึ่งหน่วยโดยตรึงตัวอื่น expected count คูณ e^{β_j} ถ้าช่วงติดตามไม่เท่ากัน ต้องพิจารณา exposure เช่น offset $\log e_i$ ในโมเดล rate ส่วนนี้เป็นคำอธิบายเสริม ไม่ใช่บังคับเปลี่ยนข้อมูลตัวอย่าง 3 เดือนเดิม

4.17 Censoring records และ KM ต้นทาง: ตรวจเลขทุกแถว

แหล่งเนื้อหา: L4 PDF p.38–43

ในหน้า 38–39 ค่า $74+$ และ $85+$ หมายถึงรู้เพียงว่ายังไม่เกิด event ตอนจบการติดตาม ไม่ได้รู้ event time ที่แน่นอน ให้ $Y = \min(T, C)$ และ $\delta = I(T \le C)$ ภายใต้ convention ที่ระบุ

คน	ข้อมูล T ที่ทราบ	C	(Y, δ)	ความหมาย
1	80	100	$(80, 1)$	พบ event
2	40	80	$(40, 1)$	พบ event
3	$74+$	74	$(74, 0)$	รู้เพียงว่า $T > 74$
4	$85+$	85	$(85, 0)$	รู้เพียงว่า $T > 85$
5	40	95	$(40, 1)$	พบ event



t	No. subjects at risk	Deaths	Censored	Cumulative survival
59	26	1	0	$25/26 = 0.962$
115	25	1	0	$24/25 \times 0.962 = 0.923$
156	24	1	0	$23/24 \times 0.923 = 0.885$
268	23	1	0	$22/23 \times 0.885 = 0.846$
329	22	1	0	$21/23 \times 0.846 = 0.808$
353	21	1	0	$20/21 \times 0.808 = 0.769$
365	20	0	1	$20/20 \times 0.769 = 0.769$
377	19	0	1	$19/19 \times 0.769 = 0.769$
421	18	0	1	$18/18 \times 0.769 = 0.769$
431	17	1	0	$16/17 \times 0.769 = 0.688$
⋮	⋮	⋮	⋮	⋮

รูป 4.7: ตาราง KM 26 คนในหน้า 42: มีตัวเลขที่ต้องแก้ตาม risk set

อ่านภาพทีละจุด **1.** แถว $t = 329$ risk set 22 จึง factor ต้อง $21/22$ แม้ในสูตรพิมพ์ $21/23$. ค่า 0.808 สอดคล้องกับ factor ที่แก้ **2.** แถว $t = 431$ ใช้ $16/17$ คูณ survival เดิม จะได้ 0.724 ไม่ใช่ 0.688 ที่พิมพ์ด้านขวา

t	n_j	Deaths	Censored	$\hat{S}(t)$ ที่คำนวณใหม่
59	26	1	0	$25/26 = 0.961538$
115	25	1	0	$24/26 = 0.923077$
156	24	1	0	$23/26 = 0.884615$
268	23	1	0	$22/26 = 0.846154$
329	22	1	0	$21/26 = 0.807692$
353	21	1	0	$20/26 = 0.769231$
365	20	0	1	$20/26$
377	19	0	1	$20/26$
421	18	0	1	$20/26$
431	17	1	0	$(20/26)(16/17) = 160/221 = 0.723982$

อย่าปัด cumulative survival ทุกขั้นก่อนนำไปคูณครั้งถัดไป และไม่ extrapolate ตารางส่วนที่ภาพไม่ได้ให้ไว้ คุณสมบัติ $S(0) = 1$ ต้องไม่มี mass ของ event ที่เวลา 0 ส่วน $S(t) \to 0$ ต้องเป็น proper finite-event-time distribution หากมี cure fraction อาจเหลือ positive limit ได้ Hazard เป็น instantaneous rate ไม่ใช่ probability ที่จำกัดไม่เกิน 1

ต้องจำ KM ลดเฉพาะ event แต่ censor เปลี่ยน denominator ในอนาคต ใช้จำนวน risk ก่อน event และตรวจเลขต้นทางด้วยสูตร ไม่จำค่าพิมพ์ผิด

4.18 Kaplan–Meier ที่มีเวลาผูกกัน: risk set ก่อนและหลังเหตุการณ์

แหล่งเนื้อหา: L4 PDF p.40–43. convention ฝึกเสริม

ข้อมูลห้าคน $(time, event) = (2,1),(2,0),(3,1),(4,0),(5,1)$ ใช้ convention ว่า event เกิดก่อน censor ที่เวลาเดียวกัน ผู้ที่ censor ที่ 2 จึงยังอยู่ใน risk set ก่อน event ที่ 2

เวลา	n_j ก่อน	events	censors	$\hat{S}$ หลัง	risk set ถัดไป
2	5	1	1	$1(4/5) = 0.8$	3
3	3	1	0	$0.8(2/3) = 8/15$	2
4	2	0	1	$8/15$	1
5	1	1	0	0	0

1. เริ่มทุกคนอยู่ใน risk set: 5 คน ไม่หัก censor ที่เวลา 2 ก่อน denominator
2. Event เปลี่ยน survival factor. censor เปลี่ยนจำนวนคนที่ติดตามในเวลาต่อไป
3. ช่วง $3 \le t < 5$ survival เท่ากับ $8/15$ แม้มี censor ที่ 4
4. Survival ลดเป็น 0 ที่ 5 เพราะคนที่ยังเสี่ยงคนสุดท้ายเกิด event ไม่ได้แปลว่าผู้ censored เสียชีวิตที่เวลานี้

ต้องจำ เมื่อเวลาผูกต้องระบุ convention และทำให้ denominator สอดคล้อง. KM ต้องไม่เพิ่มและ censor ไม่ทำให้เส้นกระโดดลงเอง

ทำไมต้องเทียบ scale ก่อนดู Ridge และ Lasso coefficients

สไลด์ L4 p.6–12 แสดงการหด coefficients: penalty ขึ้นกับ ตัวเลขของ coefficients จึงไวต่อหน่วยของ feature สมมติ prediction เดิมคือ $2x$. หากบันทึก feature ใหม่เป็น $x' = 10x$, coefficient ที่ให้ prediction เดิมคือ $0.2x'$. สำหรับ Ridge penalty λw^2 ค่าเดิมเป็น 4λ แต่หลังเปลี่ยนหน่วยเป็น 0.04λ ทั้งที่คำทำนายเท่าเดิม จึงไม่ควรเปรียบเทียบขนาด weights จาก features ต่างหน่วยโดยไม่จัดการ scale. Fit ค่า mean และ std สำหรับ standardization เฉพาะ training fold แล้วนำค่าเดิมไป transform validation และ test ปกติแยก intercept ไม่ลง penalty. Lasso ก็มีปัญหาเรื่องหน่วยเช่นกัน เพราะ $|2| \neq |0.2|$ แต่มีมุมของ L1 ที่ทำให้ coefficients บางตัวเป็นศูนย์ได้

อ่านเพิ่มจากตำรา ISLP §6.2, printed p.241–245 (PDF p.249–253): เส้นทาง coefficients เมื่อ λ เปลี่ยนและเหตุผลที่ standardize ก่อนเปรียบ shrinkage ตัวอย่างเปลี่ยนหน่วยด้านบนแต่งใหม่เพื่อแสดงผลของ penalty ไม่ใช่เลขในสไลด์

4.19 Lecture 4: เลือก model และ Checklist

แหล่งเนื้อหา: L4 PDF p.3–43

Model	output ที่ model อธิบาย	สิ่งที่ต้องจำ
Linear	continuous mean	identity link, coefficient additive
Logistic	class probability	logit link, odds multiplicative
Poisson	count mean	log link, mean positive
Survival	time-to-event distribution	censoring, S, h, H และ risk set

Quick checklist

- ตีความ simple และ multiple coefficient เมื่อ predictor อื่นคงที่ได้

- คำนวณ interaction และ effect แบบขึ้นกับอีกตัวแปรได้
- เขียน Ridge และ Lasso objective โดยไม่ penalty intercept ผิดได้
- เปรียบเทียบ λ ใหญ่กับ constraint radius t เล็กได้
- เลือก candidate จาก RSS+penalty และแยก L1/L2 ได้
- คำนวณ Poisson mean, probability และ multiplicative effect ได้
- บันทึก event และ censoring และอธิบาย right-censoring ได้
- แยก survival probability กับ hazard rate และใช้ minus sign ถูกได้
- ทำ KM หลาย event times พร้อม censored observation ได้

ต้องจำ Regularization ไม่ใช่ชนิด response. survival ไม่ใช่การทิ้ง censored records. Poisson ไม่ใช่ linear regression หลัง log ค่า count ทุกแถว

Cross-lecture connection

Shrinkage $\leftrightarrow$ SVM C $\leftrightarrow$ NN L1/L2. Poisson log link $\leftrightarrow$ Logistic log-odds. Survival likelihood $\leftrightarrow$ importance ของ observation mechanism. generalization $\leftrightarrow$ split และ CV ที่เรียนใน L1

อ่านเพิ่มจากตำรา Textbook ที่ใช้เพิ่ม intuition: ISLP §3.3.2, §4.6, §6.2 และ Ch.11 โดยไม่เพิ่ม Cox และ log-rank test เป็นขอบเขตบังคับเพียงเพราะมีในหนังสือ

Lecture 5 Tree-Based & Ensemble Models

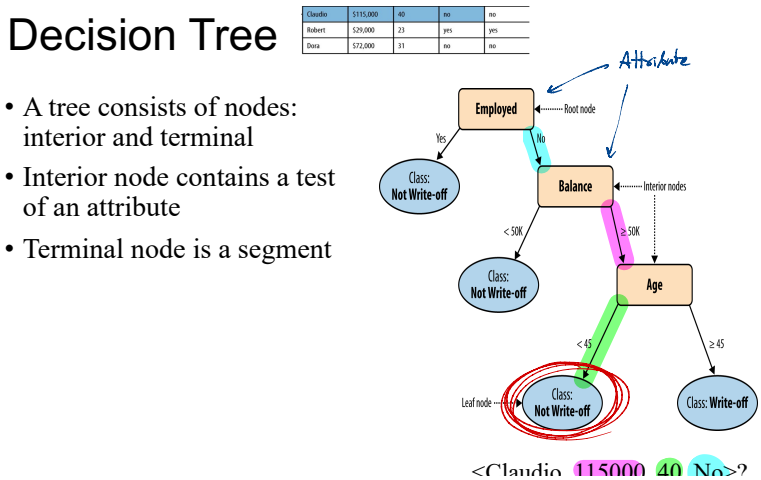
แหล่งเนื้อหา: L5 PDF p.1–12 | ขอบเขตอ่านสอบ Lecture 1–8

คำย่อที่ใช้ในบทนี้

IG คือ Information Gain. **RF** คือ Random Forest. **OOB** คือ Out-of-Bag คือแถวที่ไม่ถูกสุ่มเข้าชุด bootstrap ของต้นไม้นั้น. **AdaBoost** มาจาก Adaptive Boosting. **GBM** คือ Gradient Boosting Machine และ **GBDT** คือ Gradient Boosted Decision Trees. **XGBoost** มาจาก eXtreme Gradient Boosting. AUROC หมายถึง Area Under the Receiver Operating Characteristic Curve. Decision Tree แบ่ง feature space เป็นกลุ่มตาม target ผ่านชุดคำถามที่ internal nodes และให้คำตอบจาก leaf ความสามารถอธิบายเส้นทางเป็นจุดเด่น แต่ tree ที่ลึกมากอาจซับซ้อนและไม่เสถียรเมื่อ data เปลี่ยนเพียงเล็กน้อย

Tree induction

เริ่มจากข้อมูลทั้งหมดที่โหนดราก (root) เลือก feature และ threshold ที่แบ่ง target ได้ตามเกณฑ์ของโจทย์ แล้วส่งข้อมูลไปยังโหนดลูก (child nodes) ตามผลของแต่ละ branch ทำขั้นตอนนี้ซ้ำกับโหนดลูกจนถึงเงื่อนไขหยุด การเลือกในแต่ละรอบเป็นการตัดสินใจเฉพาะจุด (greedy) จึงไม่รับประกันว่าจะได้ต้นไม้ที่ดีที่สุดเมื่อมองทั้งต้น



รูป 5.1: เดินตาม test ของ attribute จาก root ถึง leaf เพื่อทำนาย record ใหม่ (L5 p.10)

อ่านภาพทีละจุด **1.** เริ่มที่ root แล้วอ่านคำถามของ attribute ที่ node ปัจจุบัน จากนั้นเลือก branch ของ sample **2.** ทำต่อจนถึง leaf แล้วจึงอ่าน target ที่ทาย กิ่งที่ sample ไม่ผ่านไม่มีผลโดยตรงต่อคำตอบครั้งนี้ แต่ยังเป็นส่วนของ trained tree Classification leaf ให้ class หรือ empirical class proportions. Regression leaf ให้ค่าตัวเลข เช่น mean ของ response ใน leaf ภายใต้ squared error สิ่งที่แบ่งคือ predictors ไม่ใช่รู้ target ของข้อมูลใหม่ล่วงหน้า

> **ต้องจำ** Root และ internal และ leaf เป็นตำแหน่งในโครงสร้าง ส่วน split criterion เป็นเหตุผลเลือกคำถาม และ prediction rule เป็นวิธีตอบจาก leaf ทั้งสามต้องแยกให้ได้

ฝึกให้ทำได้ ได้รับ tree และ sample ใหม่แล้ว trace path พร้อมเงื่อนไขแต่ละ branch ไม่เลือก leaf จาก class ที่อยากได้

ระวัง: L5 p.1 กล่าวถึงรอบสอบแยกในสไลด์ต้นฉบับ ขอบเขตของเล่มนี้รวม Lecture 1–8 จึงอ่านบทนี้เป็นส่วนของ Midterm ด้วย

5.1 Entropy, purity และ Information Gain

แหล่งเนื้อหา: L5 PDF p.13–20

Node จะบริสุทธิ์ (pure) เมื่อทุกตัวอย่างในโหนดมี class เดียวกัน Entropy ใช้วัดความไม่แน่นอนของ class: หาก class ปะปนกันมาก entropy จะสูง; หากมี class เดียว entropy จะเป็นศูนย์ ให้ p_k เป็นสัดส่วนของตัวอย่างใน class ที่ k และ K เป็นจำนวน class:

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k, \qquad 0 \log_2 0 := 0.$$

ในสูตรนี้ S คือชุดข้อมูลที่อยู่ในโหนด, K คือจำนวน class และ p_k คือสัดส่วนข้อมูลที่อยู่ใน class ที่ k การใช้ log ฐาน 2 ทำให้ entropy มีหน่วยเป็น bit หาก class ใดไม่มีข้อมูล เทอม $0 \log_2 0$ กำหนดให้มีค่าเป็นศูนย์ตามลิมิตของฟังก์ชัน สำหรับโจทย์ที่มีสอง class โหนดซึ่งมี class เดียวจะมี entropy เท่ากับ 0 ส่วนโหนดที่แบ่งสอง class เท่ากันจะมี entropy เท่ากับ 1 เมื่อใช้ log ฐาน 2 หากใช้ฐาน log อื่น ค่า entropy จะเปลี่ยนสเกล แต่การเปรียบเทียบ candidate split จะได้ลำดับเดิมเมื่อใช้ฐานเดียวกันทั้งหมด

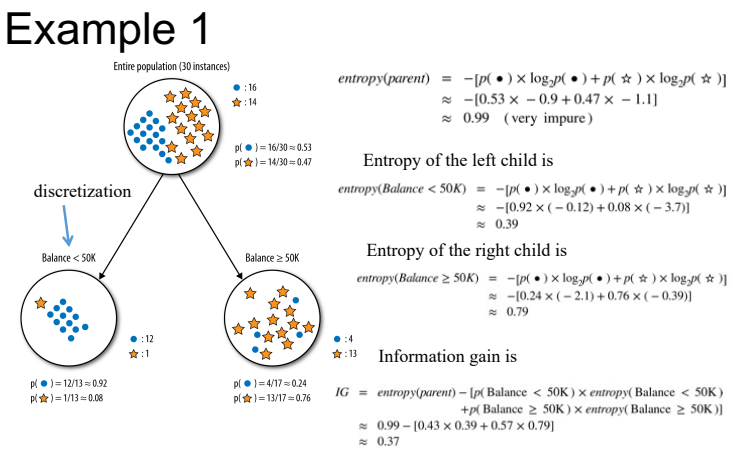
$$IG = H(parent) - \sum_{c} \frac{n_c}{n} H(child_c).$$

$H(parent)$ คือ entropy ก่อนแบ่ง ส่วน $H(child_c)$ คือ entropy ของ child ลำดับที่ c จำนวน n_c คือจำนวนแถวใน child นั้น และ n คือจำนวนแถวใน parent ก่อนแบ่ง ดังนั้น n_c/n จึงเป็นน้ำหนักของ child แต่ละกลุ่ม ผลรวมทางขวาคือ entropy หลังแบ่งที่ถ่วงตามจำนวนแถวแล้ว ส่วน Information Gain (IG) คือ entropy ที่ลดลงหลัง split อย่าเฉลี่ย entropy ของ child โดยให้น้ำหนักเท่ากัน หากจำนวนแถวของแต่ละ child ไม่เท่ากัน

ตัวอย่างคำนวณทีละขั้น

โหนดแม่มี Yes 6 และ No 4 แบ่งเป็นลูก A ที่มี Yes 4, No 0 และลูก B ที่มี Yes 2, No 4

1. แม่: $p_Y = 6/10 = 0.6, p_N = 4/10 = 0.4$ จึง $H_P = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.970951$
2. ลูก A เป็น class เดียวจึง $H_A = 0$ ส่วนลูก B มี $p_Y = 2/6 = 1/3, p_N = 4/6 = 2/3$ จึง $H_B = -(1/3)\log_2(1/3) - (2/3)\log_2(2/3) \approx 0.918296$
3. น้ำหนักลูก A และ B คือ $4/10$ กับ $6/10$ ได้ entropy หลัง split $= (4/10)(0) + (6/10)(0.918296) \approx 0.550978$
4. $IG = H_P - H_{after} \approx 0.970951 - 0.550978 = 0.419973$ bits



รูป 5.2: ตัวอย่าง Balance ของอาจารย์คำนวณ entropy ของ child แล้วถ่วงตามสัดส่วน เทียบ Residence ในหน้า 20 (L5 p.19)
อ่านภาพทีละจุด **1.** จุดน้ำเงิน และ ดาวเป็นสอง classes กลุ่มซ้ายมี 12:1 และขวามี 4:13 รวม parent 16:14
2. คำนวณ entropy ภายในกลุ่มด้วยตัวส่วน 13 และ 17 แล้วถ่วงด้วย 13/30 และ 17/30 ดูขั้นแทนค่าจาก counts ในหัวข้อขยาย ไม่ใช้เฉลี่ยสอง entropy แบบครึ่งต่อครึ่ง

ต้องจำ เลือก IG สูงสุดเทียบ candidate ที่ node เดียวกัน เพราะลดความไม่แน่นอนมากที่สุด ไม่เลือก entropy child ต่ำสุดของ child เดียวโดยละเลย child อื่น

5.2 คำนวณ Balance กับ Residence จาก counts ในภาพต้นฉบับ

แหล่งเนื้อหา: L5 PDF p.19–20. คำนวณซ้ำจากตัวเลขในภาพ

ในภาพจุดสีน้ำเงินมี 16 และดาวสีส้ม 14 รวม 30 จึง entropy ของ parent เป็น $H(16/30, 14/30) = 0.996792$ ค่าสไลด์ประมาณ 0.99 มาจาก rounding เราคง full precision เพื่อเทียบ candidate

Split และ Child	จุด	ดาว	ขนาด	Entropy
Balance $< 50K$	12	1	13	$H(12/13, 1/13) = 0.391244$
Balance $\geq 50K$	4	13	17	$H(4/17, 13/17) = 0.787127$
Residence OWN	7	1	8	0.543564
Residence RENT	4	6	10	0.970951
Residence OTHER	5	7	12	0.979869

1. ตรวจจำนวน child รวม parent: Balance $(12+4, 1+13) = (16, 14)$. Residence $(7+4+5, 1+6+7) = (16, 14)$
2. Balance weighted entropy $= (13/30)(0.391244) + (17/30)(0.787127) = 0.615577$ จึง $IG = 0.381214$
3. Residence weighted entropy $= (8/30)(0.543564) + (10/30)(0.970951) + (12/30)(0.979869) = 0.860548$ จึง $IG = 0.136243$
4. เลือก Balance เพราะ IG มากกว่า โดยใช้ counts ไม่ใช้ความรู้สึกว่ากิ่งสามกิ่งน่าจะแยกดีกว่าสองกิ่ง

ระวัง: ภาพต้นฉบับได้ Balance IG ประมาณ 0.37 เมื่อปัดค่าหลายขั้น ค่าคำนวณจาก counts ได้ประมาณ 0.3812. ranking เดิมไม่เปลี่ยน ความต่างนี้ต้องแยกจากการเปลี่ยนข้อมูล

ต้องจำ คำนวณ probability ภายในแต่ละ child ก่อน แล้วค่อยถ่วง child ด้วย n_c/n. ตัวส่วนสองชนิดนี้ทำหน้าที่ต่างกัน

5.3 Gini และ Regression-tree RSS

แหล่งเนื้อหา: L5 PDF p.21–22, 39–42

สำหรับ classification อีกเกณฑ์หนึ่งคือ Gini impurity ให้ p_k เป็นสัดส่วนของ class ที่ k ในโหนด ส่วน K คือจำนวน class; ค่า Gini ใกล้ศูนย์หมายถึงโหนดมี class เด่นชัด:

$$G(S) = \sum_k p_k(1 - p_k) = 1 - \sum_k p_k^2.$$

Binary pure node มี $G = 0$. balanced node มี 0.5 ดังนั้น Gini กับ entropy scale ไม่เท่ากันและไม่ใช่ probability ว่า model ทั้งระบบจะผิดตรง ๆ Split impurity ใช้ weighted sum $\frac{n_L}{n}G_L + \frac{n_R}{n}G_R$ สูตรในสไลด์บางรูปใช้ counts $n_L G_L + n_R G_R$ ซึ่งให้ลำดับเดียวกันเมื่อ parent size คงที่ สำหรับ regression แต่ละ leaf ทำนายด้วยค่าเฉลี่ยของ target ภายใน leaf และเลือก split ที่ทำให้ผลรวมความคลาดเคลื่อนกำลังสองของลูกสองฝั่งต่ำที่สุด ในสูตร y_i คือค่าจริงของแถวที่ i และ $\bar{y}_L, \bar{y}_R$ คือค่าเฉลี่ยของ target ใน leaf ซ้ายและขวา:

$$RSS = \sum_{i \in L}(y_i - \bar{y}_L)^2 + \sum_{i \in R}(y_i - \bar{y}_R)^2.$$

ตัวอย่างคำนวณทีละขั้น

ใช้ targets $y = (1, 2, 8, 9)$ และเปรียบเทียบก่อนแบ่งกับแบ่งสองแถวแรกและสองแถวท้าย

1. ก่อนแบ่ง $\bar{y} = (1+2+8+9)/4 = 5$ จึง $RSS = (1-5)^2 + (2-5)^2 + (8-5)^2 + (9-5)^2 = 16+9+9+16 = 50$
2. ลูกซ้ายทาย mean $(1+2)/2 = 1.5$ จึง $RSS_L = (1-1.5)^2 + (2-1.5)^2 = 0.25 + 0.25 = 0.5$
3. ลูกขวาทาย mean $(8+9)/2 = 8.5$ จึง $RSS_R = (8-8.5)^2 + (9-8.5)^2 = 0.5$
4. RSS หลัง split $= 0.5 + 0.5 = 1$ ลดลง $50 - 1 = 49$

ทำไม mean เหมาะกับ squared error

อนุพันธ์ของ $\sum_i (y_i - c)^2$ ตาม c เป็น $-2\sum_i (y_i - c)$ ตั้งศูนย์ได้ $c = \bar{y}$ หาก loss เปลี่ยน prediction rule ที่ optimal อาจเปลี่ยนด้วย

> **ต้องจำ** Tree classification ใช้ impurity ของ class proportions. regression ใช้ residual ของ numeric target ไม่เอา class entropy มาคำนวณ response ต่อเนื่องโดยตรง

ระวัง: Gini คำนวณ $0 \cdot 1 = 0$ ได้ตามปกติ ส่วน entropy จัดการ $0 \log 0$ ด้วย limit ไม่ถือว่า node pure เกิด infinity

5.4 Regression Tree: ไล่ทุก threshold และหาค่า leaf

แหล่งเนื้อหา: L5 PDF p.39–42. ตัวอย่างฝึกใหม่

ข้อมูลเรียง $x = 1, 2, 3, 4$ กับ $y = 1, 2, 8, 9$ Candidate thresholds ใช้ midpoint $1.5, 2.5, 3.5$ เมื่อไม่ split $\bar{y} = 5$, RSS50

s	$\bar{y}_L$	$\bar{y}_R$	RSS_L	RSS_R	รวม
1.5	1	$19/3$	0	$86/3$	28.6667
2.5	1.5	8.5	0.5	0.5	1
3.5	$11/3$	9	$86/3$	0	28.6667

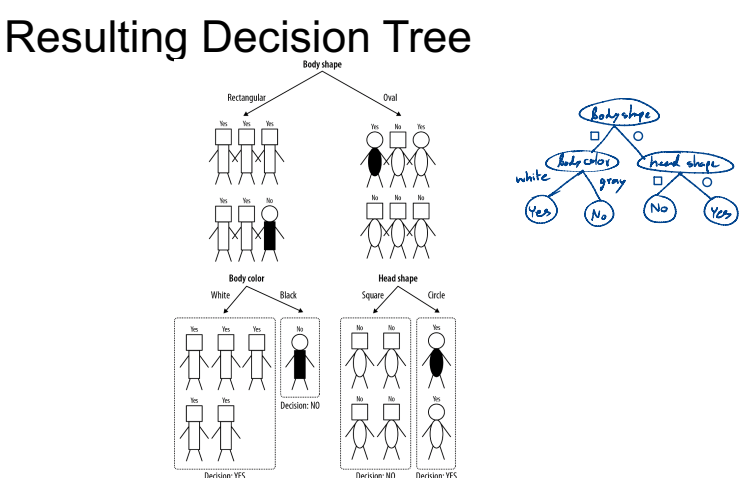
ตัวอย่าง threshold 1.5: Right $(2, 8, 9)$ mean $19/3$, residual $(-13/3, 5/3, 8/3)$ ยกกำลังสองรวม $(169+25+64)/9 = 86/3$ จุดเดี่ยวซ้ายมี RSS0 แต่รวมแล้วยังแพ้ 2.5 Prediction $x = 2.2$ ตกซ้ายของ 2.5 จึง 1.5 ส่วน $x = 100$ ตกขวาจึง 8.5 นี่แสดง extrapolation แบบ leaf mean ที่ไม่ต่อแนวโน้มขึ้นตาม x

> **ต้องจำ** Regression split ลดผลรวม RSS ของทั้งสองข้าง ไม่เลือกเพราะมี child เดียว pure หรือ RSS0. leaf mean คำนวณจาก target ของ records ใน leaf

5.5 ตัวอย่าง Write-off และการหยุด tree

แหล่งเนื้อหา: L5 PDF p.12, 23–27. Extra Reference: tree regularization

สไลด์แสดงข้อมูลรูปร่างของหัว, ร่างกาย และสี โดย target คือ write-off ไม่ใช่ label รูปร่าง เริ่มเลือก body-shape เพราะให้ IG สูงสุด แล้วแยกกลุ่ม oval ด้วย head-type และ rectangular ด้วย body-color ผลคือชุดเงื่อนไขที่บอกได้ว่าทำไม sample ไปถึง class นั้น



รูป 5.3: Tree จากลำดับ partition ของสไลด์: attribute ที่ใช้ใน branch หนึ่งไม่จำเป็นต้องใช้ในอีก branch (L5 p.27)

อ่านภาพทีละจุด **1.** Body shape เป็นคำถามแรก: Rectangular ไปถาม Color ส่วน Oval ไปถาม Head shape

2. Leaf เป็นผล Write-off ไม่ใช่ label ของ shape สองกิ่งสามารถใช้คนละ attribute และเกิด tie ของ candidate บางกิ่งได้ตามตาราง counts ที่ขยายไว้

Stopping และ regularization

ถ้า leaf pure แต่ขนาดเล็กมากอาจจำ noise การคุม max depth, min samples per leaf และ minimum improvement ลด capacity ได้ Pruning ตัดส่วนที่ประโยชน์ต่อ validation ไม่คุ้ม complexity โดยเป็นคำอธิบายเสริมจาก Géron Ch.6/ ISLP §8.1 ไม่แต่งว่าอาจารย์ระบุ pruning ออกสอบ

Interpolation และ Extrapolation

Regression tree แบบ leaf mean ตอบ piecewise constant ข้อมูลนอกช่วง feature ยังตก leaf หนึ่งและได้ค่าเดิม ไม่ยืด trend เชิงเส้นเหมือน parametric line จึง extrapolate pattern ต่อเนื่องได้จำกัด ตามข้อสังเกตใน L5 p.2

> **ต้องจำ** Decision Tree เป็น supervised segmentation ไม่ใช่ clustering ธรรมดา เพราะ criterion อาศัย target label อธิบาย path ได้ไม่แปลว่า tree เสถียรเสมอ

ฝึกให้ทำได้ เทียบผลเพิ่ม depth: training error มักลดได้ แต่ validation error อาจลดแล้วเพิ่ม ให้เลือกจาก validation ไม่ใช่ความ pure อย่างเดียว

5.6 Write-off: สร้างตารางจากรูปแล้ว induction จนครบต้นไม้

แหล่งเนื้อหา: L5 PDF p.12,23–27. ลำดับคนจากซ้ายไปขวา

รูปคนเป็นหนึ่ง record ต่อหนึ่งคน ตัวแปร target เป็น Yes และ No เหนือหัว ไม่ใช่สีตัวคน แปลงภาพเป็นตารางต่อไปนี้ คำ Gray ในหน้า 23–26 และ Black ในหน้า 12/27 ใช้แทนกลุ่มตัวทึบกลุ่มเดียวกันในตัวอย่างนี้

คน	Head	Body	Color	Write-off
1	Square	Oval	White	No
2	Circular	Oval	Gray	Yes
3	Square	Rectangular	White	Yes
4	Square	Rectangular	White	Yes
5	Square	Rectangular	White	Yes
6	Circular	Rectangular	Gray	No
7	Square	Rectangular	White	Yes
8	Square	Oval	White	No
9	Square	Oval	White	No
10	Square	Rectangular	White	Yes
11	Square	Oval	White	No
12	Circular	Oval	White	Yes

Parent มี 7 Yes,5 No. $H = 0.979869$ คำนวณ candidate ทุกตัวที่ root จากตาราง ไม่ใช้ counts เฉพาะกิ่งที่ชอบ

Candidate	Counts ของ child	Weighted H	IG
Head	Square:5Y4N. Circular:2Y1N	0.972881	0.006988
Body	Rectangular:5Y1N. Oval:2Y4N	0.784159	0.195710
Color	White:6Y4N. Gray:1Y1N	0.975792	0.004076

Body ชนะจึงเป็น root ถึงแม้จะยังมี impurity ทั้งสองกิ่ง ต่อไปคำนวณใหม่เฉพาะ records ที่อยู่ในกิ่งนั้น

กิ่ง Rectangular: IDs 3,4,5,6,7,10

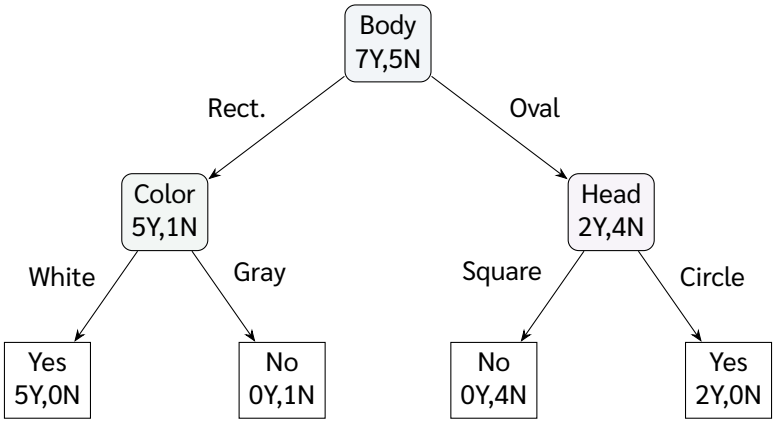
Parent $5Y1N$, $H = 0.650022$ ถ้าแยก Color: White $5Y0N$, Gray $0Y1N$ ได้ weighted entropy 0 และ $IG = 0.650022$ ถ้าแยก Head ก็ให้ groups เดียวกันบนข้อมูลกิ่งนี้และได้ gain เท่ากัน ภาพเลือก Color เป็นหนึ่งในคำตอบที่ tie ได้ ไม่ควรอ้างว่า Color เป็นทางเลือก unique จาก counts นี้

กิ่ง Oval: IDs 1,2,8,9,11,12

Parent $2Y4N$, $H = 0.918296$ แยก Head: Square $0Y4N$, Circular $2Y0N$ ได้ gain 0.918296 แยก Color: White $1Y4N$, Gray $1Y0N$ ได้ weighted entropy $(5/6)H(1/5, 4/5) = 0.601607$, gain 0.316689 จึงเลือก Head

Trace prediction แบบแสดงเงื่อนไข

คนใหม่ Head=Square,Body=Rectangular,Color=White: root ดู Body จากนั้น Rectangular จากนั้น ดู Color จากนั้น White จากนั้น Yes ห้ามวิ่งไป Oval เพราะ Head=Square ก่อน ส่วน Circular และ Oval และ White: Body=Oval จากนั้น Head=Circular จากนั้น Yes สีไม่ถูกถามในเส้นทางนี้



รูป 5.4: Tree วาดใหม่พร้อม counts: อ่านคำถามที่ node และเงื่อนไขบนลูกศร กิ่งซ้ายใช้ Color กิ่งขวาใช้ Head. counts ลูกทุกกิ่งต้องรวมเท่ากับ parent

> **ต้องจำ** เลือก candidate ที่ node ปัจจุบัน ไม่ reuse IG จาก root. tie อาจทำให้มี tree หลายแบบที่ fit train เท่ากัน และ pure training leaves ไม่รับประกัน generalization

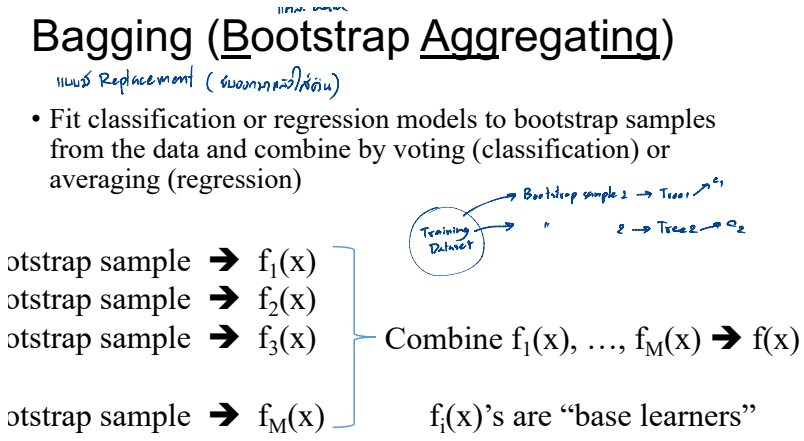
5.7 Bagging, Bootstrap และ Out-of-bag

แหล่งเนื้อหา: L5 PDF p.31–48, 52

Bagging (Bootstrap Aggregating) สุ่มแถวจากชุดฝึกแบบใส่คืน (with replacement) เพื่อสร้างชุดฝึกหลายชุด จากนั้นฝึก base learner แยกจากกัน แล้วรวมคำทำนายของทุกตัว สำหรับ classification อาจรวมด้วยการโหวตหรือเฉลี่ยความน่าจะเป็น ส่วน regression มักเฉลี่ยค่าทำนาย

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x).$$

Bootstrapping เปิดโอกาสให้ record ซ้ำและให้บาง record ไม่ถูกเลือก Record ที่ไม่อยู่ในชุดของ tree นั้นคือ out-of-bag (OOB) ใช้ประเมินด้วยเฉพาะ trees ที่ไม่เคยฝึก record นั้น ไม่ใช้ทุก tree รวมกันแล้วเรียก OOB



รูป 5.5: ข้อมูลแต่ละ bootstrap sample สร้าง base learner แล้วรวมผล ต่างจาก boosting ที่ learner ถัดไปขึ้นกับความผิดของชุดก่อน (L5 p.47)

อ่านภาพทีละจุด **1.** แต่ละทางแยกใช้ bootstrap sample ของ training pool แล้ว fit learner แยกกัน

2. ลูกศรที่รวมด้านท้ายคือ aggregate prediction ไม่ใช่ส่ง residual ของ tree ก่อนเข้า tree ถัดไป. record อาจซ้ำใน sample และเป็น OOB บางต้น

อ่านเพิ่มจากตำรา ISLP Ch.5 และ §8.2: โอกาส record หนึ่งไม่ถูกเลือกใน n draws คือ $(1-1/n)^n \approx e^{-1} = 0.368$ จึงมี unique records ประมาณ 63.2% ต่อ bootstrap sample แม้ sample size เท่ากับ n

> **ต้องจำ** Bagging เหมาะกับ learner variance สูงเช่น deep trees และมีผลหลักลด variance มากกว่าลด bias. sampling with replacement ไม่ใช่แบ่ง fold แบบไม่ซ้ำ

ทดสอบตัวเอง: ข้อมูล A, B, C, D สุ่ม B, B, A, D ถูกต้องเป็น bootstrap. C เป็น OOB สำหรับ learner นี้ แต่ไม่ใช่ OOB ของทุก learner เสมอ

5.8 Bootstrap และ OOB: ตารางผู้เรียนที่ใช้ทำนายแต่ละ record

แหล่งเนื้อหา: L5 PDF p.31–48,52. ตัวอย่างฝึกใหม่

Training pool $\{A, B, C, D\}$ ให้ samples ขนาด 4 ทุกต้น ตารางนี้แสดงสมาชิกที่มีซ้ำและ OOB ที่ไม่ถูกเลือกเลย

Tree	Bootstrap training sample	OOB
1	A,A,B,D	C
2	B,B,C,C	A,D
3	A,C,D,D	B

ถ้า record A มี target 0 และ trees 1,2,3 ทาย $(0, 1, 0)$ OOB prediction ของ A ใช้เฉพาะ tree 2 จึง 1 และผิด ไม่ใช้ majority 0 จากสามต้นเพราะต้น 1 และ 3 เคยเห็น A แล้ว หาก D ใช้เฉพาะ tree 2 เช่นกัน ส่วน C ใช้ต้น 1 B ใช้ต้น 3 สำหรับ $n = 4$ โอกาสไม่เลือก record หนึ่ง $= (3/4)^4 = 0.316406$. expected unique records $= 4[1 - (3/4)^4] = 2.734375$ ไม่เท่ากับ 63.2%exact เพราะ 63.2%เป็น large-n limit

Variance floor ที่อธิบายเหตุผลของ Random Forest

ถ้า $\sigma^2 = 4$, $\rho = 0.5$, $B = 10$ ได้ variance $= 0.5(4) + 0.5(4)/10 = 2.2$ เพิ่ม B เป็น 100 ได้ 2.02 ถ้าลด correlation เป็น 0.1 และ $B = 10$ ได้ 0.76 จึงเห็นว่า diversity สำคัญ ไม่ใช่เพิ่มต้นอย่างเดียวแล้ว variance จะเป็นศูนย์

> **ต้องจำ** OOB เป็นความสัมพันธ์ระหว่าง record กับ tree ไม่ใช่ชุด test ใหม่ที่แยกถาวร. forest ยังคงต้องมี evaluation protocol ของ task

5.9 Random Forest: ลด correlation ของ trees

แหล่งเนื้อหา: L5 PDF p.34–57

Random Forest เพิ่มการสุ่มตัวแปรในแต่ละ node ก่อนหา split แต่ละครั้งจะพิจารณาเพียงบาง predictors แล้วเลือก split ที่ดีที่สุดจากกลุ่มย่อยนั้น เมื่อใช้ร่วมกับ bootstrap จึงช่วยลดโอกาสที่ต้นไม้ทุกต้นจะเลือกตัวแปรเด่นตัวเดียวกันและทำให้คำทำนายสัมพันธ์กันมากเกินไป

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

อ่านเพิ่มจากตำรา สูตรนี้เป็น intuition จาก Géron Ch.7/ISLP §8.2 เมื่อทุก learner มี variance เท่ากันและ pairwise correlation ρ เดียวกัน ไม่ใช่สมมติฐานจริงของทุก forest แสดงว่าถ้า ρ สูง การเพิ่ม trees อย่างเดียวเหลือ variance floor

ผลของ hyperparameters

More trees มักลด variance ของ aggregate แล้วเข้าสู่จุด diminishing returns แต่เพิ่มเวลา และ memory. depth และ min-leaf คุมความซับซ้อนราย tree. max-features คุม diversity และ strength ของแต่ละ tree การสุ่มน้อยเกินทำให้ learners อ่อนเกินไปได้

Feature importance และ evaluation

สไลด์ code แสดง min-samples-leaf, max-depth, class-weight และ importance ต้องแยก learned importance จากค่าที่เราตั้ง การ ranking ใน model ไม่ใช่ causal proof และ feature ที่สัมพันธ์กันอาจแบ่ง importance กัน ใช้ ROC และ AUC กับ score ไม่ hard labels ถ้าต้องการ curve หลาย threshold

> **ต้องจำ** Bagging สุ่ม observations. Random Forest เพิ่ม feature subsampling ที่ node อย่าจำว่าเลือก feature subset เดียวคงที่ทั้ง forest เสมอ

ระวัง: ข้อดี categorical และ missing data ในสไลด์เป็นคุณสมบัติของ tree และ forest บาง implementation ไม่ถือว่า API ทุกชนิดรับ string และ missing ได้โดยไม่ preprocessing

ทำไมสุ่ม features ใน Random Forest ช่วย bagging

สมมติทุก tree มี variance σ^2 และ correlation ระหว่างคู่เท่ากับ ρ. ให้ $\bar{f} = B^{-1}\sum_b \hat{f}_b$ และใช้ $\mathrm{Cov}(\hat{f}_b, \hat{f}_c) = \rho\sigma^2$ เมื่อ $b \neq c$

$$\begin{aligned}\mathrm{Var}(\bar{f}) &= \frac{1}{B^2}\left[\sum_b \mathrm{Var}(\hat{f}_b) + \sum_{b\neq c}\mathrm{Cov}(\hat{f}_b, \hat{f}_c)\right] \\ &= \frac{1}{B^2}[B\sigma^2 + B(B-1)\rho\sigma^2] = \sigma^2\left[\rho + \frac{1-\rho}{B}\right].\end{aligned}$$

ถ้า $B = 100$ และ $\rho = 0.6$, variance เหลือ $0.604\sigma^2$ แม้เพิ่ม trees มาก ถ้า trees ไม่สัมพันธ์กันจะเหลือ $0.01\sigma^2$. เมื่อ feature ที่แรงที่สุดครอง root ทุกต้น bagged trees อาจตอบคล้ายกัน การสุ่ม candidate features ที่แต่ละ split เปิดทางให้ต้นไม้ต่างกันและลด correlation แลกกับบางต้นที่อ่อนลง ต้องวัดบน validation และ OOB ตามขั้นตอนที่ไม่ leak สูตรนี้เป็นแบบจำลองเพื่อทำความเข้าใจ ไม่ใช่ค่าจริงที่ต้องใช้ทำนาย variance ของ forest ทุกชุดข้อมูล

อ่านเพิ่มจากตำรา ISLP §8.2, printed p.347 (PDF p.354): อธิบายกรณี strong predictor ทำให้ bagged trees คล้ายกัน และการสุ่ม features ช่วย decorrelate เชื่อมภาพ ensemble ใน L5 p.31–52

5.10 Boosting และ AdaBoost

แหล่งเนื้อหา: L5 PDF p.58–61

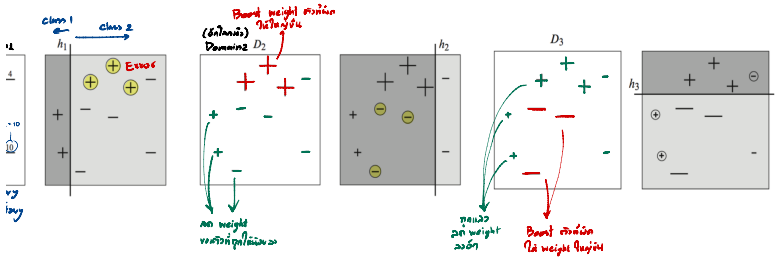
Boosting สร้างแบบจำลองรวมเป็นลำดับ ผู้เรียนรอบใหม่จะเน้นส่วนที่แบบจำลองรวมรอบก่อนยังทำได้ไม่ดี ใน AdaBoost จะเพิ่มน้ำหนักให้ตัวอย่างที่ผู้เรียนทำนายผิด แล้วรวมผู้เรียนแต่ละรอบด้วยน้ำหนักของผู้เรียนเอง ต้องแยกน้ำหนักตัวอย่างออกจากน้ำหนักที่ใช้รวมคำทำนาย

$$H(x) = \mathrm{sign}\left(\sum_{t=1}^{T}\alpha_t h_t(x)\right).$$

Weighted error, learner weight และ update น้ำหนักของ samples เป็นคนละชนิด weight ไม่สับสนกับ coefficient ของ feature หรือ random-forest vote น้ำหนัก sample ที่สูงทำให้ learner รอบหน้าให้ความสำคัญ record นั้นมากขึ้น

AdaBoost (Schapire and Freund, 2012)

- Fit additive model $\sum_t c_t h_t(x)$
- In each stage, introduce a weak learner to compensate the shortcoming of existing weak learner



รูป 5.6: AdaBoost เปลี่ยนความสำคัญของ observations เพื่อให้ learner ถัดไปแก้ความผิดก่อนหน้า (L5 p.60)

อ่านภาพทีละจุด **1.** อ่านน้ำหนัก observations ที่เปลี่ยนจาก learner รอบก่อน: คนที่ผิดสำคัญขึ้นในรอบถัดไป

2. Ensemble รวม learner ด้วย learner weights ซึ่งคนละชนิดกับ sample weights. weighted error ต้องใช้ distribution ปัจจุบันก่อนคำนวณน้ำหนัก learner ใหม่

อ่านเพิ่มจากตำรา Géron Ch.7 / ISLP §8.2: สำหรับ binary AdaBoost หนึ่ง convention ใช้ $\alpha_t = \frac{1}{2}\log[(1-e_t)/e_t]$ และ $D_{t+1}(i) \propto D_t(i)e^{-\alpha_t y_i h_t(x_i)}$ สูตรนี้เสริม intuition ไม่ใช่ claim ว่าสไลด์กำหนดให้จำ derivation

ตัวอย่างคำนวณทีละขั้น

ให้ weighted error $e = 0.2$ ตาม binary AdaBoost convention ในสูตรด้านบน

1. $1 - e = 0.8$ จึง $\alpha = \frac{1}{2}\ln[(1-e)/e] = \frac{1}{2}\ln(0.8/0.2) = \frac{1}{2}\ln 4 \approx 0.693147$
2. จุดผิดมี $y_i h_t(x_i) = -1$ จึง factor $e^{-\alpha(-1)} = e^{\alpha} = 2$
3. จุดถูกมี $y_i h_t(x_i) = +1$ จึง factor $e^{-\alpha} = 1/2 = 0.5$
4. หลังคูณแต่ละ weight ต้องหารด้วยผลรวม weights ใหม่ Z_t เพื่อให้ distribution รวมเป็น 1 อีกครั้ง ดูตัวอย่างสองรอบถัดไปสำหรับการ normalize ครบทุกแถว

> **ต้องจำ** Boosting เป็น sequential learning เพื่อแก้ข้อผิดพลาด ไม่ใช่สร้าง trees อิสระแล้วเฉลี่ย. weak learner ไม่ได้หมายถึงทุกต้นต้องสุ่มทายอย่างเดียว

ระวัง: Boosting อาจไวต่อ mislabeled samples และ outliers ข้อความ “ดีบน imbalanced data” ใน L5 p.58 ไม่ใช่รับประกันว่าชนะ bagging ทุกชุด

5.11 AdaBoost สองรอบ: sample weights, learner weights และ vote

แหล่งเนื้อหา: L5 PDF p.58–61. สูตร binary convention เสริมจาก Géron Ch.7

ใช้ labels $y = (+1, +1, -1, -1)$ และ $D_1 = (0.25, 0.25, 0.25, 0.25)$ Learner $h_1 = (+1, -1, -1, -1)$ ผิดเฉพาะคน 2

1. $e_1 = 0.25$, $\alpha_1 = \frac{1}{2}\log[(1-e_1)/e_1] = \frac{1}{2}\log 3 = 0.549306$
2. คนถูกคูณ $e^{-\alpha_1} = 1/\sqrt{3}$. คนผิดคูณ $e^{\alpha_1} = \sqrt{3}$
3. ก่อน normalize weights $(0.144338, 0.433013, 0.144338, 0.144338)$ ผลรวม $Z_1 = 0.866025$
4. $D_2 = (1/6, 1/2, 1/6, 1/6)$ จึงคน 2 สำคัญขึ้นในรอบถัดไป ไม่ได้เปลี่ยน label ของคน 2
5. ให้ $h_2 = (+1, +1, +1, -1)$ ผิดคน 3 weighted error $e_2 = 1/6$ ไม่ใช่ $1/4$ เพราะ weights รอบนี้ไม่เท่ากัน
6. $\alpha_2 = \frac{1}{2}\ln[(1-1/6)/(1/6)] = \frac{1}{2}\ln 5 \approx 0.804719$ คนถูกคูณ $1/\sqrt{5}$ ส่วนคน 3 ที่ผิดคูณ $\sqrt{5}$
7. Unnormalized weights เป็น $(1/(6\sqrt{5}), 1/(2\sqrt{5}), \sqrt{5}/6, 1/(6\sqrt{5}))$ ผลรวม $Z_2 = 5/(6\sqrt{5}) + \sqrt{5}/6 = \sqrt{5}/3$
8. หารแต่ละตัวด้วย Z_2: ได้ $(1/10, 3/10, 1/2, 1/10) = (0.1, 0.3, 0.5, 0.1)$ ซึ่งรวมเป็น 1

คน	y	$\alpha_1 h_1$	$\alpha_2 h_2$	sign ของผลรวม
1	+1	+0.549306	+0.804719	+1 ถูก
2	+1	-0.549306	+0.804719	+1 ถูก
3	-1	-0.549306	+0.804719	+1 ผิด
4	-1	-0.549306	-0.804719	-1 ถูก

แม้แต่ละ learner ผิด 1 คนเหมือนกัน แต่ α ไม่เท่ากันเพราะ weighted errors ต่างกัน Ensemble สองรอบ รอบยังผิดคน 3 การเพิ่มรอบไม่ได้รับประกัน training error ลดทุกครั้งในทุก convention

ระวัง: ถ้า $e_t = 0.5$ ได้ $\alpha_t = 0$ ไม่ช่วย weighted vote ถ้า error 0 formula ไป infinity ต้องมี stopping และ numerical handling อย่าแทน log 0 โดยไม่มีการจัดการ

ต้องจำ แยก $D_t(i)$ ของ sample ออกจาก α_t ของ learner และ recompute weighted error ด้วย D_t ปัจจุบันทุกครั้ง

5.12 Gradient Boosting: residual เป็น negative gradient

แหล่งเนื้อหา: L5 PDF p.62–68

ให้ ensemble $F_{t-1}(x)$ และ loss $L(y, F)$ สร้าง pseudo-residual

$$r_{it} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{t-1}(x_i)}, \quad F_t(x) = F_{t-1}(x) + \nu\, h_t(x).$$

Fit learner h_t กับ residual และ negative gradient แล้วเพิ่ม correction ด้วย learning rate ν สำหรับ $L = \frac{1}{2}(y-F)^2$ ได้ residual $r = y - F$ แต่ถ้าใช้ squared loss ไม่มีครึ่งจะเป็น $2(y-F)$ ตามสไลด์ p.63 มี factor 2 ต้องอ่าน convention ก่อนคำนวณ

ตัวอย่างคำนวณทีละขั้น

ให้ $y = (3, 5, 7)$ ใช้ $F_0 = \bar{y}$ และ $\nu = 0.5$ สมมติ learner ใหม่ fit residual ได้ตรงทุกจุด

1. $F_0 = (3+5+7)/3 = 5$ จึง $r_1 = y - F_0 = (3-5, 5-5, 7-5) = (-2, 0, 2)$
2. เมื่อ $h_1 = r_1$, $F_1 = F_0 + 0.5 h_1 = (5-1, 5+0, 5+1) = (4, 5, 6)$
3. Residual ใหม่ $r_2 = y - F_1 = (3-4, 5-5, 7-6) = (-1, 0, 1)$
4. SSE เดิม $= (-2)^2 + 0^2 + 2^2 = 8$ ส่วน SSE ใหม่ $= (-1)^2 + 0^2 + 1^2 = 2$

สมมติฐานว่า fit residual ได้ตรงใช้เฉพาะตัวอย่างนี้ ในตัวอย่าง stump แบบสองรอบถัดไปจะคำนวณ leaf means ที่ไม่เท่ากับ residual ทุกแถว

Gradient boosting algorithm

Algorithm 2 Gradient boosting.

let F_0 be a "dummy" constant model
for $m = 1, \ldots, M$
  for each pair $(\boldsymbol{x}_i, y_i)$ in the training set
    compute the *pseudo-residual* $R(y_i, F_{m-1}(\boldsymbol{x}_i))$ = negative gradient of the loss
  train a regression sub-model h_m on the pseudo-residuals
  add h_m to the ensemble: $F_m(\boldsymbol{x}) = F_{m-1}(\boldsymbol{x}) + \eta \cdot h_m(\boldsymbol{x})$
return the ensemble F_M

รูป 5.7: Algorithm ของอาจารย์: คำนวณ pseudo-residual, train learner, แล้วเพิ่มเข้า ensemble เป็นรอบ (L5 p.65)

อ่านภาพทีละจุด **1.** อ่าน algorithm ตามลำดับ initialize จากนั้น negative gradient จากนั้น fit base learner จากนั้น update ensemble
2. Index ของ prediction ต้องเป็นรอบก่อนหน้าในการหา residual และเป็นรอบปัจจุบันหลัง update ถ้าเปลี่ยน loss จะไม่ใช้ y–F โดยอัตโนมัติ

> **ต้องจำ** Gradient Boosting ใช้ gradient ของ loss ตาม prediction และ function ไม่ใช่ gradient ของ input. residual เท่ากับ negative gradient เฉพาะ loss และ convention ที่เหมาะสม

ฝึกให้ทำได้ แสดง initialization, residual, fitted correction, learning-rate scaling และ updated prediction ตามลำดับ อย่าใช้ original residual ตลอดทุก round

5.13 Gradient Boosting สองรอบ: learner ไม่ได้ fit residual exact

แหล่งเนื้อหา: L5 PDF p.62–68. ตัวอย่าง stump ฝึกใหม่
ให้ $x = (1, 2, 3, 4)$, $y = (1, 2, 8, 9)$. $L_i = \frac{1}{2}(y_i - F_i)^2$, $F_0 = \bar{y} = 5$, $\nu = 0.5$ ฝึก stump โดยเลือก threshold 2.5 ในทั้งสองรอบ และใช้ mean residual ต่อ leaf

คน	y	F_0	r_1	h_1	F_1	r_2	h_2	F_2
1	1	5	-4	-3.5	3.25	-2.25	-1.75	2.375
2	2	5	-3	-3.5	3.25	-1.25	-1.75	2.375
3	8	5	3	3.5	6.75	1.25	1.75	7.625
4	9	5	4	3.5	6.75	2.25	1.75	7.625

1. Residual 1 $= (-4, -3, 3, 4)$. left mean -3.5, right mean 3.5 เป็น h_1 ไม่ใช่ copy residual ทุกจุด
2. $F_1 = F_0 + 0.5h_1 = (3.25, 3.25, 6.75, 6.75)$. SSE $= 2(2.25^2 + 1.25^2) = 13.25$ จากเดิม 50
3. คำนวณ $r_2 = y - F_1$ ใหม่ แล้ว fit leaf means ± 1.75
4. $F_2 = F_1 + 0.5h_2$. SSE $= 2(1.375^2 + 0.375^2) = 4.0625$

Stump แต่ละรอบให้ piecewise correction ดังนั้นสองคนใน leaf เดียวกันมี correction เท่ากันแต่ residual ไม่เท่ากัน การฝึกต้นลึกกว่าอาจแก้ได้ละเอียดกว่าโดยเพิ่ม capacity ถ้าใช้ loss ไม่มีครึ่ง negative gradient มี factor 2 ต้องปรับการคำนวณตามนั้น

> **ต้องจำ** ทุก round อิง prediction ของ ensemble ล่าสุด. learning rate คูณ correction ก่อนบวก ไม่คูณทั้ง F_{t-1} โดยไม่ได้ระบุ algorithm

5.14 XGBoost และสรุปการเลือก Ensemble

แหล่งเนื้อหา: L5 PDF p.69–76

สไลด์อธิบาย XGBoost เป็น optimized gradient boosting ที่เพิ่ม regularization, sparsity awareness, parallelization และ cache-aware access พร้อม DMatrix เพื่อจัดข้อมูลให้ efficient ค่าที่เห็นใน code ได้แก่ eta, max-depth, objective=binary:logistic และ number of rounds เป็นตัวอย่าง ไม่ใช่ optimal settings ที่ต้องใช้ทุกงาน

5.15 XGBoost: เห็น leaf weight และ split gain จาก regularization

แหล่งเนื้อหา: L5 PDF p.69–76. สมการ second-order เสริมจาก XGBoost Official Tutorial ไม่ใช่สูตรที่สไลด์บังคับจำ

แกนใน lecture คือ optimized gradient boosting กับ regularization เพื่อเข้าใจว่า “regularized” เปลี่ยนต้นไม้ได้อย่างไร ใช้ตัวอย่าง objective ของรอบใหม่ที่ approximate ด้วย gradient g_i และ Hessian h_i

$$\widetilde{J} = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

ถือว่าโครงสร้างต้นไม้คงที่และ $H_j + \lambda > 0$ อนุพันธ์ของ objective ต่อ leaf weight คือ $G_j + (H_j + \lambda) w_j = 0$ จึง $w_j^* = -G_j/(H_j + \lambda)$ แทนกลับในส่วนของ leaf นี้

$$\begin{aligned} G_j w_j^* + \tfrac{1}{2}(H_j + \lambda)(w_j^*)^2 + \gamma &= -\frac{G_j^2}{H_j + \lambda} + \frac{1}{2}\frac{G_j^2}{H_j + \lambda} + \gamma \\ &= -\frac{1}{2}\frac{G_j^2}{H_j + \lambda} + \gamma. \end{aligned}$$

Split gain คือค่า objective ของ parent ลบค่า objective รวมของลูกทั้งสอง โทษจำนวน leaves เพิ่มอีกหนึ่งจึงเหลือ $-\gamma$

$$\text{Gain} = \tfrac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma.$$

แทนค่าจาก squared error แบบเดียวกับตัวอย่างก่อน

ใช้ $L = \frac{1}{2}(y - F)^2$: $g = F - y$, $h = 1$. ที่ $F = 5$ ได้ $g = (4, 3, -3, -4)$ แบ่ง 2.5 ได้ $G_L = 7, H_L = 2, G_R = -7, H_R = 2, G = 0, H = 4$

1. $\lambda = 1$ ให้ weight ของ leaf ซ้าย $-7/3$, weight ของ leaf ขวา $7/3$ ขนาดเล็กกว่า residual means ± 3.5 เมื่อไม่มี penalty
2. ถ้า $\gamma = 0.5$: gain $= \frac{1}{2}(49/3 + 49/3 - 0) - 0.5 = 95/6 \approx 15.8333 > 0$ จึง split คุ้มใน objective นี้
3. ถ้า $\gamma = 17$: gain $= 49/3 - 17 = -2/3 < 0$ จึงไม่ split ตามเงื่อนไข gain บวก
4. ถ้า learning rate $\eta = 0.3$ correction จริงเป็น ± 0.7 หลัง shrink ไม่ใช่ $\pm 7/3$ เต็ม ๆ

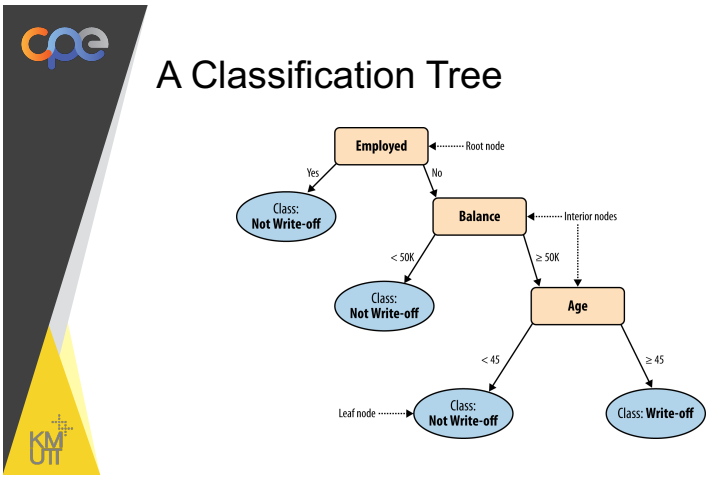
ระวัง: ตัวอย่างนี้เป็น second-order derivation เพิ่มเพื่อเข้าใจ ไม่อ้างว่าแนวสอบใน L5 ระบุให้จำสูตร gain เนื้อหา XGBoost พื้นฐานและการอ่าน parameter ยังเป็นแกนบังคับอ่าน

> **ต้องจำ** Regularization ของ leaf weight λ กับ penalty จำนวน leaves γ และ learning rate η มีหน้าที่ต่างกัน ไม่ใช้ Gini มาแทน gain นี้โดยไม่เปลี่ยน objective

อ่านเพิ่มจากตำรา แหล่งของ objective และ leaf weight และ gain ส่วนขยาย: XGBoost, Introduction to Boosted Trees, Structure Score และ Learn the Tree Structure: `https://xgboost.readthedocs.io/en/stable/tutorials/model.html` (ตรวจ 4 ตุลาคม 2026). ตัวเลขตัวอย่างและ derivation ทีละขั้นเรียบเรียงใหม่

5.16 เดิน tree ของ Claudio และอ่านผลรวมหลาย trees

แหล่งเนื้อหา: L5 PDF p.7–10,38–46

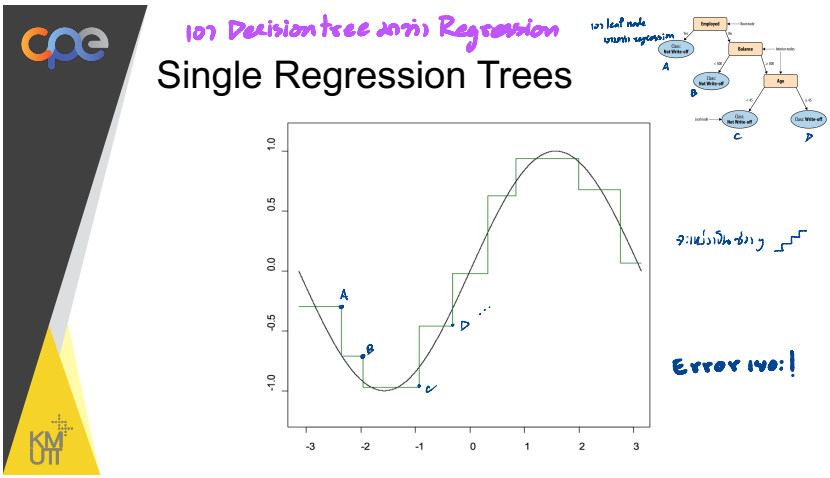


รูป 5.8: Claudio เดินจาก root ถึง leaf ใน decision tree

อ่านภาพทีละจุด **1.** เริ่มถาม Employed? ก่อน หาก No จึงดู Balance แล้วจึง Age ไม่อ่าน feature โดยสลับลำดับเอง

2. Claudio: Balance 115,000, Age 40, Employed No จากนั้น Balance อย่างน้อย 50,000 จากนั้น Age ต่ำกว่า 45 จากนั้น leaf Not Write-off

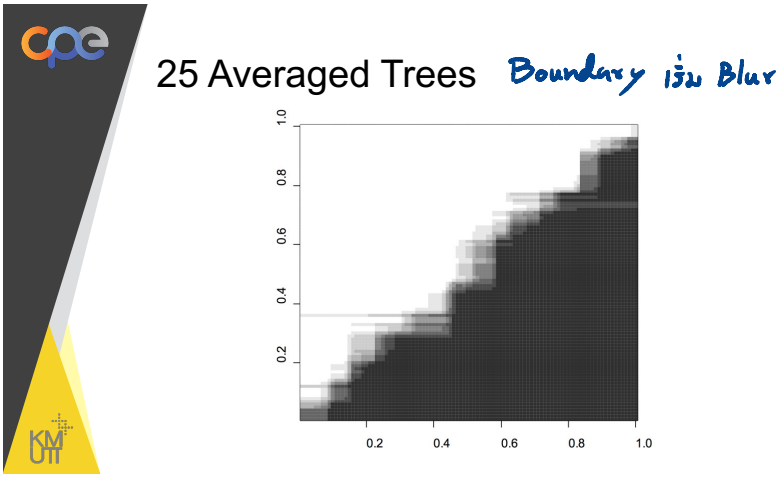
จุด threshold ต้องอ่าน inequality ทุกครั้ง ค่า age 45 ไป branch ≥ 45 ไม่ใช่ < 45 และ prediction ของ leaf ใช้สมาชิกที่ผ่าน ทุกเงื่อนไขก่อนหน้า ไม่ใช่ทุก records ของ dataset ที่มี age ตรงกัน



รูป 5.9: Single regression trees หน้า 40: เห็นความแปรปรวนของเส้นที่ fit

อ่านภาพทีละจุด **1.** แต่ละ tree ให้ตัวเลขจาก leaf เช่น mean outcome เปลี่ยน training samples แล้วเส้นที่ fit อาจเปลี่ยนมาก จึงเป็นแรงจูงใจให้เฉลี่ยหลาย trees

2. Forest เฉลี่ยตัวเลขเพื่อ prediction. scatter ของ predictions อาจช่วยวิเคราะห์ความไม่แน่นอน แต่ sample variance ระหว่าง trees ไม่ใช่ calibrated prediction interval อัตโนมัติ



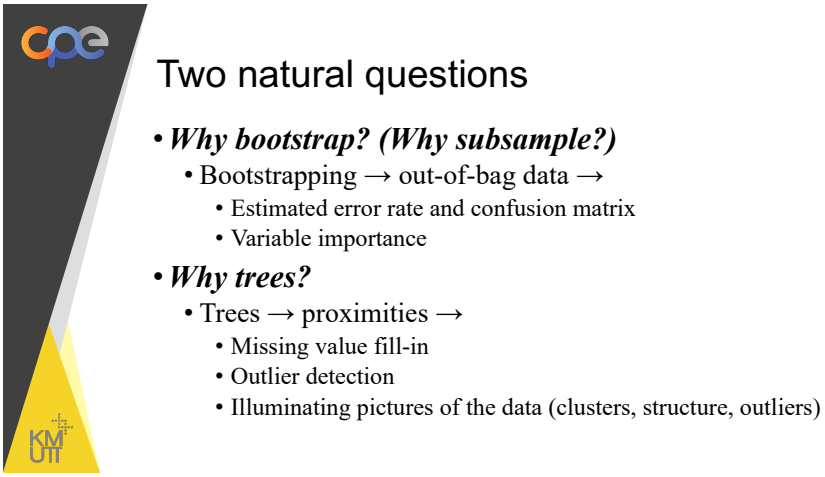
รูป 5.10: Classification forest และสัดส่วน votes ในภาพต้นทาง

อ่านภาพทีละจุด 1. ถ้า 25 trees มี 1 vote ต่อ class หนึ่ง สัดส่วน votes เป็น $1/25$ ไม่ใช่ $1/$จำนวน features

2. Hard vote กับ mean leaf probabilities เป็นวิธีรวมที่ต่างกัน ตัวอย่าง trees ให้ probabilities 0.49,0.49,1: hard votes เลือก class 0 แต่ mean 0.66 เลือก class 1

5.17 Forest proximity, missing values และการอ่าน code

แหล่งเนื้อหา: L5 PDF p.49–57

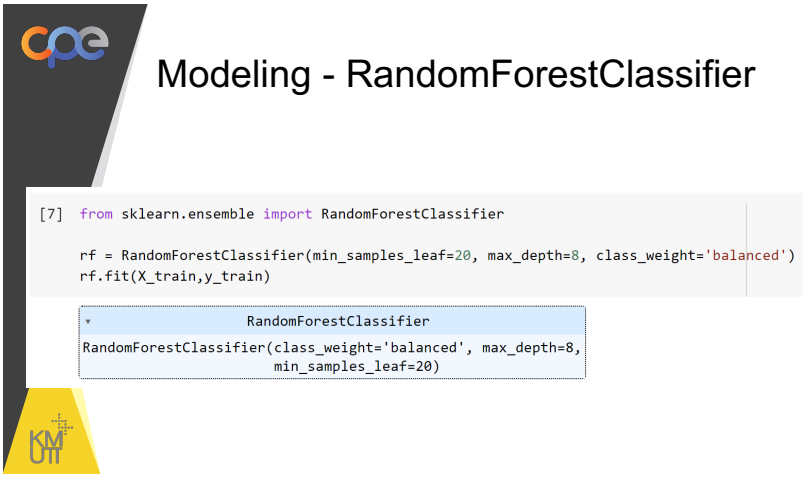


รูป 5.11: Two natural questions หน้า 52: bootstrap และ OOB และ tree proximity

อ่านภาพทีละจุด 1. bullet ฝั่ง bootstrap เชื่อม OOB กับ error rate และ confusion และ importance ฝั่ง trees เชื่อม proximity กับ imputation และ outliers และ structure

2. นิยาม proximity ของคู่ (i, j) คือสัดส่วน trees ที่ไป terminal node เดียวกัน เป็นความใกล้ตาม model ต้องระบุจะใช้ทุก tree หรือเฉพาะ OOB

เช่น 10 trees มี 7 trees ที่คู่ข้อมูลไป leaf เดียวกัน proximity=0.7 ใช้เป็นแนวทางสำรวจ similarity, outliers หรือ imputation ได้ แต่ไม่ใช่ proof ว่าสองตัวอย่างมี labels เดียวกัน การจัดการ categorical และ missing ต้องสอดคล้องกับ implementation และ preprocessing ไม่ควรท่องว่า random forest ทุก library รับ string หรือ NaN ได้โดยอัตโนมัติ



รูป 5.12: Modeling RandomForestClassifier หน้า 54: fit และ predict ด้วย parameters ที่กำหนด

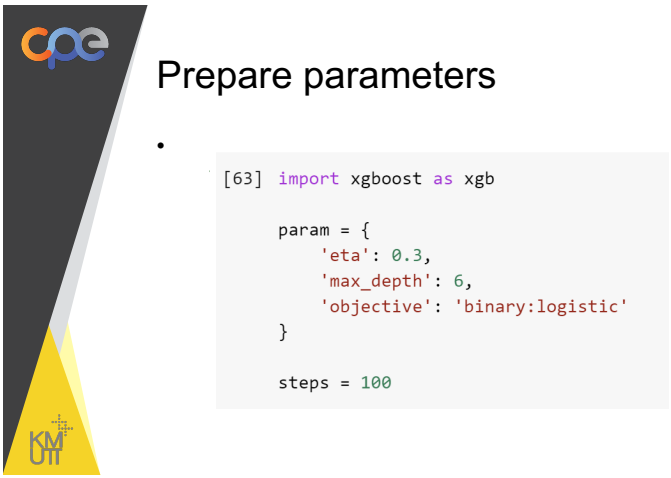
อ่านภาพทีละจุด **1.** อ่าน parameters ก่อน fit: max depth 8, min samples leaf 20 และ class weight balanced ทำหน้าที่คนละอย่าง

2. Shape train $(1029, 44)$ จากขั้น load หน้า 53 หมายถึง 1,029 samples/44 features การ fit ใช้ training labels และ predict คืน class และ score ตาม method ที่เรียก

Parameter และ ผล	ตีความ
max depth 8	จำกัดความลึกของแต่ละ tree ไม่ใช่จำนวน trees
min samples leaf 20	leaf ต้องมี samples ขั้นต่ำ ลดการ fit กิ่งที่ข้อมูลน้อย
class weight balanced	ถ่วง contribution ตาม classes ไม่เท่ากับเพิ่มจำนวน bootstrap draws ของ rare class โดยอัตโนมัติ
Feature importance	วัดการใช้ feature ตามนิยาม importance ของ model อาจกระจายระหว่าง features ที่ correlated ไม่ใช่ causal effect
AUC 0.82 ในตัวอย่าง	ค่าจากการประเมินตัวอย่างนั้น ไม่ใช่ข้อสรุป RF ชนะทุก dataset

5.18 อ่าน XGBoost workflow และเทียบกับ GBM อย่างมีเงื่อนไข

แหล่งเนื้อหา: L5 PDF p.62–76



รูป 5.13: XGBoost parameters ในตัวอย่างหน้า 70

อ่านภาพทีละจุด **1.** objective binary:logistic ให้ probability ของ binary target. learning rate และ eta ทำ shrinkage ของ contribution ต่อรอบ

2. max depth คือ complexity ของ tree ต่อรอบ ส่วนจำนวน boosting rounds คือจำนวนขั้นต่อเนื่อง ไม่ใช่ epochs ของ NN

DMatrix เป็นโครงสร้างเก็บ features และ labels และสนับสนุนการประมวลผลของ XGBoost ไม่ใช่อีกชนิดของ predic-

tion model การ cache และ parallel computation ภายในขั้นไม่ทำให้ boosting rounds เป็นอิสระ: ขั้นถัดไปยังอาศัย current predictions และ gradients จากขั้นก่อน RF เฉลี่ย trees ที่ฝึกด้วย randomness เพื่อลด variance. GBM และ AdaBoost เพิ่ม learners ตาม error ของ ensemble ปัจจุบันแต่ใช้สิ่งที่เน้นต่างกัน AdaBoost เปลี่ยน sample distribution ตามการจำแนกผิด ขณะที่ squared-error GBM fit negative gradient ซึ่งคือ residual ค่าตัวอย่าง GBM และ XGB AUC 0.79 เทียบ RF 0.82 ในสไลด์เป็นผลตัวอย่าง ไม่ใช่ลำดับความสามารถทั่วไปของ algorithms

ต้องจำ จำ workflow fit จากนั้น predict score จากนั้น evaluate บอกว่าค่าจาก examples เป็นหลักฐานเฉพาะชุดนั้น และแยก class weights, sample weights, residuals กับ feature importance

ประเด็น	Bagging และ Random Forest	Boosting
training	learners จาก samples ต่างกัน fit แยก	learner ใหม่แก้ข้อผิด ensemble เดิม
combine	vote และ average	additive weighted correction
ผลหลัก	ลด variance และ correlation	ลด residual และ loss แบบเป็นลำดับ
hyperparam.	trees, depth, features, leaf size	learning rate, rounds, learner depth

Checklist

- คำนวณ entropy และ weighted IG และ Gini และ RSS และเลือก split ได้
- Trace path และแยก classification และ regression leaf ได้
- อธิบาย bootstrap, OOB และ feature randomness ได้
- แยก bagging และ RF และ AdaBoost และ Gradient Boosting ได้
- อัปเดต residual หนึ่งรอบและบอก factor 2 ตาม loss ได้
- อธิบาย learning-rate × number-of-rounds trade-off ได้
- อ่าน feature importance โดยไม่สรุป causal effect ได้

ต้องจำ Lecture 5 หน้าแรกบอกมีคำนวณและให้ดู XGBoost เพิ่ม จึงเก็บหัวข้อนี้ตามระดับสไลด์ LightGBM อยู่ใน annotation เป็นตัวอย่าง ไม่มี derivation ของ model ในเอกสารนี้จึงไม่สร้างบทอธิบายลึกแทนอาจารย์

อ่านเพิ่มจากตำรา อ่านต่อ Géron Ch.6–7, ISLP Ch.8. บทนี้อยู่ในขอบเขต Lecture 1–8 และมีโจทย์ในชุดฝึก

Lecture 6 Neural Network Basics

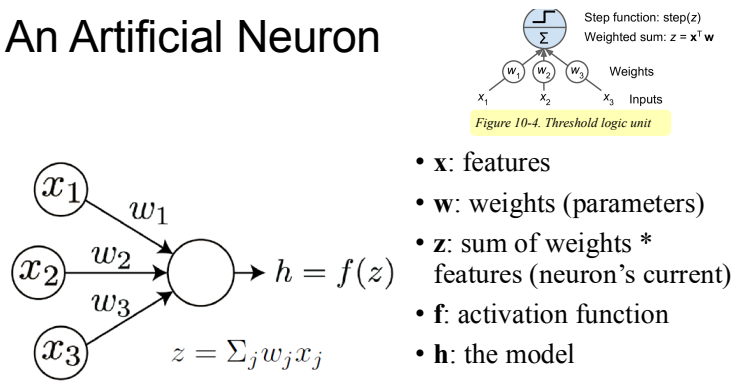
แหล่งเนื้อหา: L6 PDF p.1–19 | ขอบเขตอ่านสอบ Lecture 1–8

คำย่อที่ใช้ในบทนี้

NN คือ Neural Network, **ANN** คือ Artificial Neural Network และ **DNN** คือ Deep Neural Network. **ReLU** คือ Rectified Linear Unit. **BCE** คือ Binary Cross-Entropy และ **CCE** คือ Categorical Cross-Entropy. **Adam** มาจาก Adaptive Moment Estimation. **BN** หรือ BatchNorm คือ Batch Normalization. SGD ใช้ชื่อเต็มตามบทที่ 2 ส่วน tanh คือ hyperbolic tangent. Artificial neuron รับ features, รวมด้วย weights และ bias แล้วผ่าน activation:

$$z = \sum_j w_j x_j + b, \qquad h = f(z).$$

w, b เป็น learned parameters. activation type, number of units และ layers, batch size และ learning rate เป็น hyperparameters เปลี่ยน task ได้จาก output และ loss ไม่ใช่ถือว่า neuron ทุกตัวทำ binary classification จริงเสมอ



รูป 6.1: Neuron แยก weighted sum กับ activation: z คือ input score ส่วน h คือ output หลังแปลง (L6 p.5)

อ่านภาพทีละจุด **1.** ขาเข้าทุกเส้นมี input คูณ weight แล้วรวมที่ weighted-sum node พร้อม bias

2. Activation เปลี่ยน z เป็น output ของ neuron ไม่ใช่เพิ่ม weight อีกเส้น. bias เป็น learned offset ไม่ใช่ feature ที่อ่านจาก sample

ตัวอย่างคำนวณทีละขั้น

ตัวอย่าง loan ใน L6 p.12 ใช้ $x = (0.44, 0.63, 0, 0.60)$ และ $w = (0.7, 0.6, -0.1, -0.2)$ โดยไม่มี bias เพิ่มในสูตรนี้

1. คูณแต่ละคู่ได้ $0.44(0.7) = 0.308$, $0.63(0.6) = 0.378$, $0(-0.1) = 0$, $0.60(-0.2) = -0.12$
2. รวมเป็น $z = 0.308 + 0.378 + 0 - 0.12 = 0.566$
3. $p = \sigma(z) = 1/(1 + e^{-0.566}) \approx 0.637840$ ปัดสองตำแหน่งเป็น 0.64
4. ใช้ threshold 0.5 จึงทาย good class ตาม label ที่ตัวอย่างกำหนด

> **ต้องจำ** Neuron เดียวที่ sigmoid output และไม่มี hidden layer เทียบได้กับ Logistic Regression เครือข่ายหลาย hidden layers พร้อม nonlinear activation ทำสิ่งที่กว้างกว่า

ระวัง: คำว่า "2-layer" ในบางสไลด์นับ input+output ต่างจาก convention ที่นับเฉพาะ trainable layers ต้องบอกวิธีนับก่อนตอบ

6.1 MLP, layer sizes และจำนวน parameters

แหล่งเนื้อหา: L6 PDF p.14–19, 43, 52–54

Fully connected layer ที่มี n_{in} input units และ n_{out} output units มี $n_{in}n_{out}$ weights และ n_{out} biases จึงรวม

$$N_{params} = (n_{in} + 1)n_{out}.$$

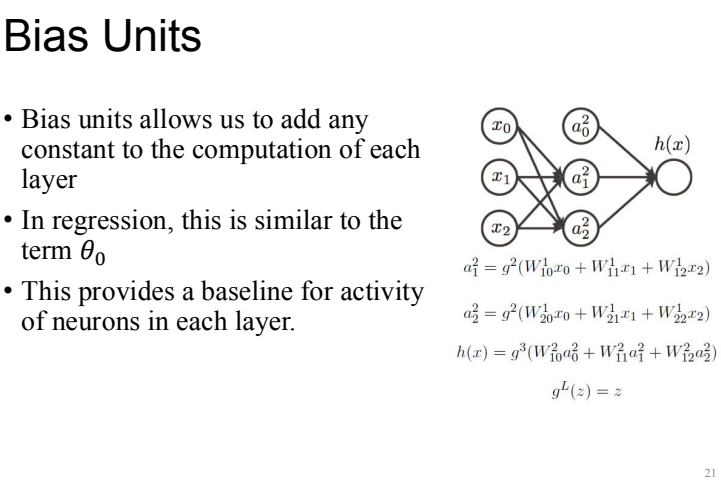
สำหรับหลาย layers รวมตามคู่ขนาดติดกัน ไม่เพิ่ม bias ของ input เป็น trainable unit ที่มี output เองอีกครั้ง การวาด bias node เป็นการเขียน constant 1 ให้เชื่อมไป layer ถัดไป ไม่ใช่ hidden feature ที่ต้องคำนวณ activation

ตัวอย่างคำนวณทีละขั้น

เครือข่ายมี input 3 หน่วย ตามด้วย hidden 4 หน่วย, hidden 2 หน่วย และ output 1 หน่วย

ชั้น	Weights	Biases	รวม
Hidden 1	$3(4) = 12$	4	16
Hidden 2	$4(2) = 8$	2	10
Output	$2(1) = 2$	1	3

รวม weights $= 12 + 8 + 2 = 22$, biases $= 4 + 2 + 1 = 7$ จึง parameters ทั้งหมด $= 22 + 7 = 29$ หรือ $(3 + 1)4 + (4 + 1)2 + (2 + 1)1 = 29$



รูป 6.2: Bias units ให้ baseline แต่ละ layer คล้าย intercept ใน regression นับเชื่อมให้ตรงกับ model ที่โจทย์แสดง (L6 p.19)

อ่านภาพทีละจุด **1.** ดู edges ระหว่าง units ของสอง layers ที่ติดกันเพื่อ count weights
2. Bias node ให้ค่า 1 แต่มี trainable coefficient แยกต่อ output unit ของ layer ถัดไป ไม่ count bias unit เป็น hidden feature ที่มี activation อีกชั้น

เหตุผลที่ต้อง nonlinear activation

ถ้าทุก layer เป็น affine transform จะยุบ composition เป็น affine transform เดียว จึงไม่เกิด nonlinear boundary แม้เพิ่มจำนวน layer การเพิ่ม units และ layers เพิ่ม capacity แต่เพิ่มความเสี่ยง overfitting และต้นทุนด้วย

> **ต้องจำ** จำนวน units ไม่เท่ากับจำนวน parameters ต้องนับทุก edge และ weight และ bias ตาม architecture

ฝึกให้ทำได้ ได้รับรูป network แล้วระบุ input และ hidden และ output, convention ของจำนวน layers และ total trainable parameters ได้

6.2 Activation functions และ output ตาม task

แหล่งเนื้อหา: L6 PDF p.6–8, 44, 55–57

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}, \quad \mathrm{ReLU}(z) = \max(0, z).$$

Sigmoid output อยู่ $(0, 1)$ เหมาะ binary probability. Tanh อยู่ $(-1, 1)$ และ zero-centered. ReLU มี gradient 1 ด้านบวกและ 0 ด้านลบ จึงคำนวณง่ายแต่ neuron อาจ inactive ถาวรในข้อมูลที่เห็น (dying ReLU)

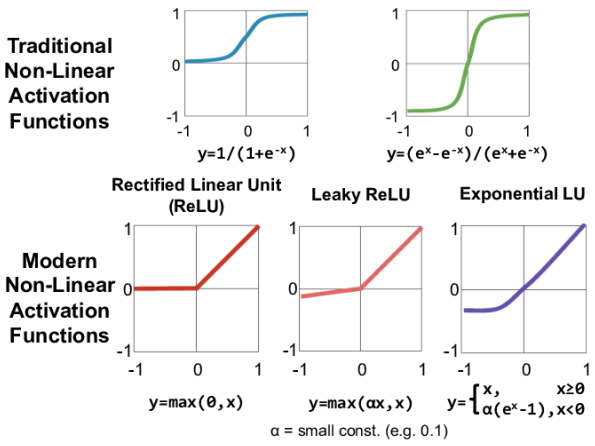
$$\sigma'(z) = \sigma(z)(1-\sigma(z)), \quad \tanh'(z) = 1 - \tanh^2(z).$$

ทั้ง sigmoid และ tanh saturate ที่ขนาด $|z|$ ใหญ่ ทำให้ gradient เล็ก ถ้าคูณหลาย layers เกิด vanishing gradients ในการส่งสัญญาณย้อนกลับ ReLU ช่วยลดปัญหาบางส่วนแต่ไม่รับประกัน gradient ดีทุก architecture

เลือก output activation และ loss

Regression ที่ไม่จำกัด range ใช้ linear output กับ MSE. Binary class ใช้หนึ่ง sigmoid output กับ binary cross-entropy. Multiclass mutually exclusive ใช้ softmax กับ categorical cross-entropy โดย

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad \sum_k p_k = 1.$$



รูป 6.3: ช่วงค่าและ shape ของ activation ต่างกันจึงมีผลต่อ gradient และ output meaning (L6 p.56)

อ่านภาพทีละจุด **1.** แกนนอนคือ pre activation z แกนตั้งคือ output a: sigmoid และ tanh มี saturation ส่วน ReLU มีส่วนเป็น 0 2. อ่านความชันของกราฟเพื่อเข้าใจ derivative และ gradient. output range ใช้เลือก task ส่วน hidden nonlinearities ใช้เพิ่ม representation capacity

> **ต้องจำ** Hidden activation เลือกเพื่อ nonlinear representation. output activation เลือกตาม support และความหมายของ target อย่าสลับหน้าที่

ระวัง: L6 p.57 บอก ReLU ควรใช้ hidden layer เป็นแนวแนะนำของสไลด์ ไม่ใช่ข้อห้ามสากลสำหรับ output ที่บังคับค่าบวก

6.3 Softmax + Cross-entropy: probability, loss และ gradient

แหล่งเนื้อหา: L6 PDF p.44,55–57; derivation เสริม

สำหรับ 3 classes ให้ logits $z = (\log 2, 0, 0)$ และ target class 2 แบบ one-hot $y = (0, 1, 0)$ จะได้ $e^z = (2, 1, 1)$ จึง $p = (0.5, 0.25, 0.25)$ และ $L = -\log p_2 = \log 4 \approx 1.386294$

$$L = -\sum_j y_j \ln p_j = -\sum_j y_j z_j + \ln\left(\sum_r e^{z_r}\right) \quad (\sum_j y_j = 1),$$
$$\frac{\partial L}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_r e^{z_r}} = p_k - y_k,$$
$$\nabla_z L = (0.5 - 0, 0.25 - 1, 0.25 - 0) = (0.5, -0.75, 0.25).$$

ถ้า $\eta = 0.1$ และคิดอัปเดต logits โดยตรงเพื่อเห็นทิศ จะได้ $z' = z - 0.1(p - y) = (\log 2 - 0.05, 0.075, -0.025)$ การฝึกจริงอัปเดต weights ผ่าน chain rule ไม่ใช่ทุก network มี logits เป็น free parameters โดยตรง

Numerical stability ที่ทำด้วยมือได้

ลบค่าคงที่ $c = \max_k z_k$ จากทุก logit ไม่เปลี่ยน softmax เพราะทั้งเศษและส่วนคูณด้วย e^{-c} เหมือนกัน เช่น $(1000, 1001, 999)$ ใช้ $(-1, 0, -2)$ ได้ probabilities เดียวกันและไม่ overflow ไม่สลับเป็น sigmoid 3 ตัวเพราะ sigmoid outputs ไม่บังคับ sum 1

> **ต้องจำ** Softmax ใช้ probability ของ class จริงใน loss; gradient รวมทุก class เป็น 0 แต่ไม่ได้แปลว่าแต่ละ gradient เป็น 0

6.4 Forward propagation และ Backpropagation

แหล่งเนื้อหา: L6 PDF p.20–33, 37–42. Extra Reference: chain rule

Forward pass คำนวณค่าของแต่ละชั้นไปข้างหน้าจนได้คำทำนาย แล้วนำไปหาค่า loss ส่วน backpropagation ใช้กฎลูกโซ่ (chain rule) ไล่อนุพันธ์ย้อนกลับ เพื่อบอกว่าพารามิเตอร์แต่ละตัวมีผลต่อ loss เท่าใด จากนั้น optimizer ใช้ gradient เหล่านั้นปรับ weights ทั้งสามขั้นตอน—forward, backward และ update—ทำหน้าที่ต่างกัน

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f^{(l)}(z^{(l)}).$$

โดย l ระบุหมายเลขชั้น, $a^{(l-1)}$ คือค่าจากชั้นก่อนหน้า, $W^{(l)}$ คือ weights, $b^{(l)}$ คือ bias, $z^{(l)}$ คือผลรวมเชิงเส้นก่อน activation และ $a^{(l)}$ คือผลหลังผ่านฟังก์ชัน $f^{(l)}$

อ่านเพิ่มจากตำรา ISLP §10.7.1 และ Géron Ch.10: สำหรับ scalar chain $x \to z = wx + b \to a = f(z) \to L$,

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w} = \frac{\partial L}{\partial a} f'(z) x.$$

นี่เป็น derivation เพิ่มเพื่อเข้าใจ training ของสไลด์ ไม่ใช่เปลี่ยน notation ของอาจารย์โดยเงียบ

ตัวอย่างคำนวณทีละขั้น

ให้ $x = 2, y = 1$ ใช้ linear activation $a = z = wx + b$ และ $L = \frac{1}{2}(a - y)^2$

1. ถ้า $w = 0.5, b = 0$, $a = 0.5(2) = 1 = y$ จึง $L = 0$ และ gradients เป็น 0
2. เปลี่ยนจุดเริ่มเป็น $w = 0, b = 0$ จะได้ $a = 0$ และ $L = \frac{1}{2}(0 - 1)^2 = 0.5$
3. Chain rule ให้ $\partial L/\partial a = a - y = -1$, $\partial a/\partial z = 1$, $\partial z/\partial w = x = 2$ จึง $\partial L/\partial w = (-1)(1)(2) = -2$ และ $\partial L/\partial b = (-1)(1)(1) = -1$
4. ใช้ $\eta = 0.1$: $w' = 0 - 0.1(-2) = 0.2$, $b' = 0 - 0.1(-1) = 0.1$
5. Forward ใหม่ $a' = 0.2(2) + 0.1 = 0.5$ จึง $L' = \frac{1}{2}(0.5 - 1)^2 = 0.125 < 0.5$

ต้องจำ Forward ส่งค่าที่ model คำนวณ. backward ส่ง derivatives. update เปลี่ยน parameter ไม่มีการส่ง label ไปแทน output ของ hidden neuron โดยตรง

ระวัง: L6 p.28 เขียน gradient ชี้ลง แต่จริงเป็น negative gradient ที่ชี้ลงตามสูตร update. L6 p.23 มีสมการ weighted sum ที่อ่านต่างเครื่องหมาย ให้ยึดนิยาม sum of weights×features และใช้ convention สอดคล้อง

6.5 Forward และ Backprop สองชั้นแบบคำนวณทุก parameter

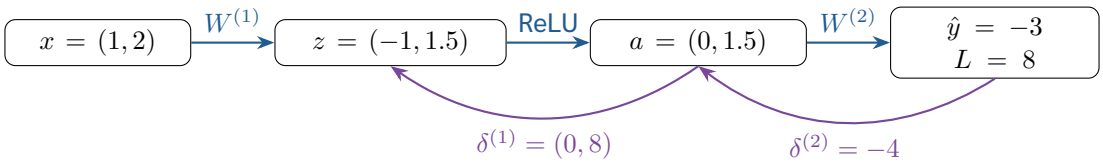
แหล่งเนื้อหา: L6 PDF p.20–33,37–42. ตัวอย่างฝึกใหม่ตาม chain rule

ใช้ input $x = (1, 2)^T$. hidden 2 units เป็น ReLU. output 1 unit เป็น linear และ $L = \frac{1}{2}(\hat{y} - y)^2$, $y = 1$

$$W^{(1)} = \begin{bmatrix} 1 & -1 \\ 0.5 & 0.5 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 1 & -2 \end{bmatrix}, \quad b^{(2)} = 0.$$

Forward: เก็บค่าที่จะใช้ย้อนกลับ

1. คำนวณทีละ hidden unit: $z_1^{(1)} = 1(1) + (-1)(2) + 0 = -1$ และ $z_2^{(1)} = 0.5(1) + 0.5(2) + 0 = 1.5$ จึง $z^{(1)} = (-1, 1.5)^T$
2. $a^{(1)} = \text{ReLU}(z^{(1)}) = (0, 1.5)^T$ เก็บทั้งก่อนและหลัง activation เพราะ derivative ใช้ sign ของ z
3. $\hat{y} = W^{(2)}a^{(1)} + b^{(2)} = 1(0) - 2(1.5) = -3$
4. Error $\hat{y} - y = -4$, loss $\frac{1}{2}(-4)^2 = 8$ ระบุ signed error ก่อนยกกำลังสองเพื่อไม่เสียทิศ gradient



รูป 6.4: ลูกศรน้ำเงินส่งค่า forward; สีม่วงส่ง derivative ย้อนกลับ ไม่ใช้ label ไปแทน hidden activation

Backward: จาก loss กลับหา output layer

เพราะ output เป็น linear $\delta^{(2)} = \partial L/\partial z^{(2)} = -4$ จึง

$$\nabla W^{(2)} = \delta^{(2)}(a^{(1)})^T = (0, -6), \quad \nabla b^{(2)} = -4.$$

Hidden ได้รับ $\partial L/\partial a^{(1)} = (W^{(2)})^T\delta^{(2)} = (-4, 8)^T$ แล้วคูณ derivative ของ ReLU $(0, 1)^T$ แบบ elementwise ไม่ใช่ dot product

$$\delta^{(1)} = (-4, 8)^T \odot (0, 1)^T = (0, 8)^T.$$

$$\nabla W^{(1)} = \delta^{(1)}x^T = \begin{bmatrix} 0 & 0 \\ 8 & 16 \end{bmatrix}, \quad \nabla b^{(1)} = (0, 8)^T.$$

แถวแรก gradient 0 เพราะ ReLU ที่ไม่ active ใน sample นี้ ไม่ใช่ proof ว่า neuron นี้ไม่มีประโยชน์กับทุก sample ส่วน hidden unit 2 เชื่อม input 2 มากกว่า input 1 สองเท่าใน gradient เพราะ $x_2 = 2x_1$

Update พร้อมกันจากค่าเก่า แล้ว forward ใหม่

ใช้ $\eta = 0.01$ โดยทุก gradient มาจาก forward เดิม อัปเดตแต่ละองค์ประกอบ

$$w_{21}^{(1)\,new} = 0.5 - 0.01(8) = 0.42, \qquad w_{22}^{(1)\,new} = 0.5 - 0.01(16) = 0.34,$$
$$b_2^{(1)\,new} = 0 - 0.01(8) = -0.08, \qquad w_2^{(2)\,new} = -2 - 0.01(-6) = -1.94,$$
$$b^{(2)\,new} = 0 - 0.01(-4) = 0.04.$$

แถว hidden แรกและ output weight ที่ต่อจาก activation ศูนย์มี gradient ศูนย์จึงคงค่าเดิม ได้เมทริกซ์หลังอัปเดต

$$W_{new}^{(1)} = \begin{bmatrix} 1 & -1 \\ 0.42 & 0.34 \end{bmatrix}, \quad b_{new}^{(1)} = (0, -0.08)^T, \quad W_{new}^{(2)} = (1, -1.94), \quad b_{new}^{(2)} = 0.04.$$

Forward ใหม่ hidden 2 $= 0.42 + 0.34(2) - 0.08 = 1.02$; output $= -1.94(1.02) + 0.04 = -1.9388$; loss $= 0.5(-2.9388)^2 = 4.31827272 < 8$ ไม่ใช้ weight ใหม่ของ output ตอนหา gradient hidden ใน backward รอบเดิม

> **ต้องจำ** ท่องลำดับ δ_{out} แล้ว output gradients แล้ว hidden δ แล้ว hidden gradients แล้ว การอัปเดตพร้อมกัน; shape ของ gradient ต้องตรง parameter และเก็บ activation จาก forward ก่อน

6.6 Epoch, Batch, tensor shape และ training loop

แหล่งเนื้อหา: L6 PDF p.34–41, 45–50, 77–78

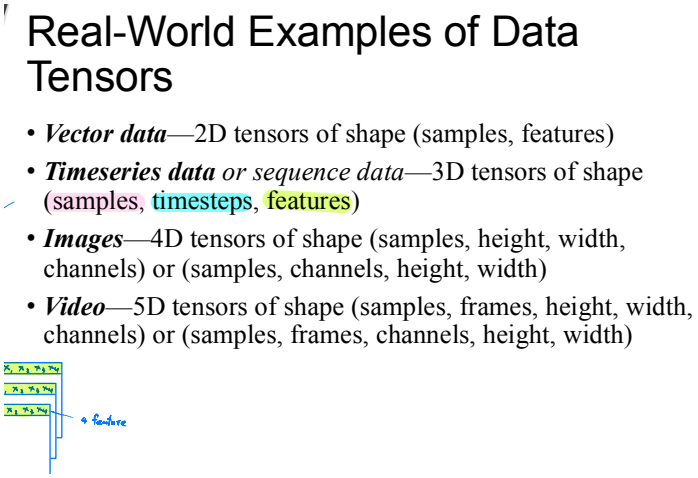
หนึ่ง epoch หมายถึงการใช้ตัวอย่างฝึกครบหนึ่งรอบ ส่วน batch คือกลุ่มตัวอย่างที่นำมาคำนวณ gradient ในการอัปเดตหนึ่งครั้ง ดังนั้นจำนวน epoch คือจำนวนรอบผ่านข้อมูล แต่จำนวน steps คือจำนวนครั้งที่อัปเดตพารามิเตอร์ ซึ่งโดยทั่วไปไม่เท่ากัน

$$\text{steps per epoch} = \lceil N/b \rceil$$

เมื่อเก็บ batch สุดท้ายที่ไม่เต็ม ถ้า drop last batch ใช้ floor และ sample ส่วนท้ายไม่ได้ถูกใช้ในรอบนั้น ตัวอย่าง $N = 1000, b = 64$ มี 16 steps และ epoch. 10 epochs มี 160 updates ไม่ใช่ 10 updates

Tensor axes ตามสไลด์

Vector dataset: (samples, features). sequence: (samples, timesteps, features). image: (samples, height, width, channels). video เพิ่ม frames axis คำว่า 4D tensor คือ 4 axes ไม่ได้แปลว่า pixel แต่ละตัวเป็น four-dimensional feature ของ application เสมอ



รูป 6.5: Shapes ของข้อมูลตามชนิด input แกน sample ไม่ใช่แกน feature และ channels ไม่ใช่ number of classes (L6 p.41)

อ่านภาพทีละจุด **1.** นับ axes ไม่ใช่นับค่าที่อยู่ในภาพ: images มักมี sample,height,width,channels

2. หนึ่ง RGB image มี 3 channels ไม่ใช่ 3 samples และแกน timesteps ของ sequence ต่างจาก features ต้องให้ tensor shape ตรงกับ layer ที่รับ

อ่าน code training จากหน้าสไลด์

Model definition กำหนด layers และ activations. compile เลือก optimizer และ loss และ metrics. fit ให้ข้อมูล, epochs, batch-size, validation. evaluate และ predict ใช้หลัง fit Metrics เช่น accuracy ไม่ใช่ loss ที่ optimizer ลดโดยตรงเสมอ โค้ดในเอกสารเป็นตัวอย่างของเวอร์ชันที่สอน ไม่อ้างว่าทดสอบกับ framework ปัจจุบันแล้ว

> **ต้องจำ** Weights update จาก batch ไม่ใช่จาก epoch definition. batch size 32 ในสไลด์เป็น starting suggestion ไม่ใช่ค่าบังคับที่ดีที่สุด

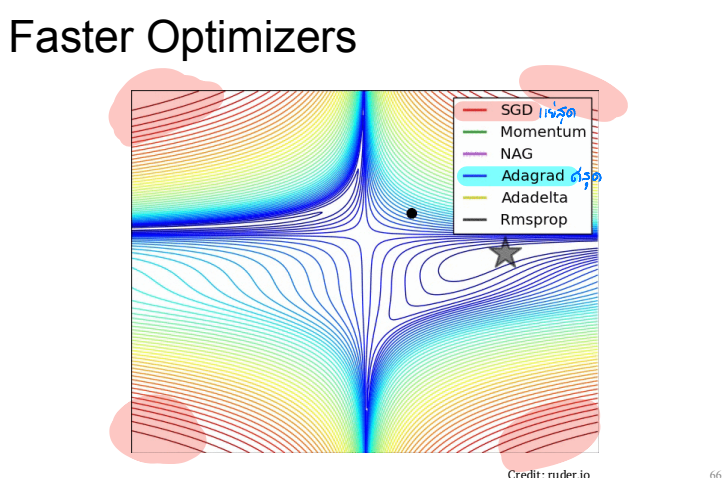
6.7 Initialization และ optimizers

แหล่งเนื้อหา: L6 PDF p.24, 58–66

Hidden neurons ที่เริ่ม weights เท่ากันทั้งหมดอาจได้รับ gradients เหมือนกันและเรียน feature ซ้ำ จึงใช้ random initialization เพื่อ break symmetry ไม่ได้แปลว่า bias ทุกตัวห้ามเริ่ม 0 Random เล็กเกินหรือใหญ่เกินมีผลต่อ signal และ gradient จึงมี initialization ที่คำนึง fan-in และ fan-out

Optimizers ในสไลด์

Momentum สะสมทิศการก้าวที่ผ่านมา ลดการสั่นและเร่งในทิศสม่ำเสมอ. Nesterov ประเมิน gradient ที่จุด look-ahead. AdaGrad ปรับ learning rate ราย parameter จาก gradient history. Adam ใช้ moving averages ของ gradient และ squared gradient เพื่อ scale update



รูป 6.6: ตัวอย่างเส้นทาง optimizer บน loss landscape ต่างกัน รูปไม่ได้พิสูจน์ optimizer ตัวเดียวดีที่สุดทุกงาน (L6 p.64)

อ่านภาพทีละจุด **1.** แต่ละเส้นทางเป็น update rule ที่ต่างกันบน loss landscape ที่วาดเดียวกัน

2. การสั่นข้ามแนวแคบกับการสะสมทิศช่วยเข้าใจ Momentum และ adaptive steps ภาพตัวอย่างนี้ไม่ให้หลักฐานว่า optimizer หนึ่งชนะทุก task

ความเข้าใจเชื่อม L2

ทุกวิธีใช้ gradient ของ objective เดียวกันได้ แต่ update rule ต่างกัน Hyperparameters มีผลต่อ convergence การเลือก Adam ไม่ได้แก้ data leakage, label ผิด หรือ loss ไม่ตรง task โดยอัตโนมัติ

> **ต้องจำ** ต้องรู้ mechanism และผลของ learning rate มากกว่าท่องชื่อ optimizer. random initialization เป็นการตั้งค่าก่อนฝึก ไม่ใช่ dropout

อ่านเพิ่มจากตำรา Géron Ch.11: Glorot และ He initialization เลือก scale ตาม layer และ activation ในฉบับนี้รักษาระดับสไลด์โดยไม่บังคับจำสูตร variance ทุก initializer

6.8 Optimizer trace: SGD, Momentum และ Adam ก้าวต่างกันอย่างไร

แหล่งเนื้อหา: L6 PDF p.58–66; สูตรฝึกเสริม

ให้ scalar weight $w_0 = 1$, gradients ที่ได้รับสองรอบ $g_1 = 2, g_2 = -1$; $\eta = 0.1$ เปรียบเทียบ update จาก ลำดับ gradients เดียวกันเพื่อแยกกลไก ไม่อ้างว่า weights ที่ต่างกันจะสร้าง ลำดับ gradients เดียวกันใน training จริง

SGD

$w_1 = 1 - 0.1(2) = 0.8$; $w_2 = 0.8 - 0.1(-1) = 0.9$ ไม่มี memory ของ gradient เก่า

Momentum convention $v_t = \beta v_{t-1} + g_t$

ให้ $\beta = 0.9, v_0 = 0$: $v_1 = 2, w_1 = 0.8$; $v_2 = 0.9(2) - 1 = 0.8$, $w_2 = 0.8 - 0.1(0.8) = 0.72$ แม้ gradient ปัจจุบันลบ แต่ memory ยังทำให้ step ลด weight นี่คือเหตุผลที่ทิศ step ไม่ต้องตรงกับ $-g_t$ ของรอบปัจจุบันเสมอ

Adam รอบแรกและเหตุผลของ bias correction

ใช้ $m_t = 0.9m_{t-1} + 0.1g_t$, $v_t = 0.999v_{t-1} + 0.001g_t^2$, เริ่ม 0 รอบแรก $m_1 = 0.2, v_1 = 0.004$ หากยังไม่ correct จะเล็กเพราะเริ่ม 0 ต้อง $\hat{m}_1 = 0.2/(1-0.9) = 2$, $\hat{v}_1 = 0.004/(1-0.999) = 4$

$$w_1 = 1 - 0.1\frac{2}{\sqrt{4}+\epsilon} \approx 0.9.$$

Root Mean Square (RMS) scale ในส่วนช่วยแยกขนาด step ราย parameter แต่ไม่ได้รับประกัน global optimum Adam ไม่แก้ loss และ data ผิดโดยอัตโนมัติ ถ้าโจทย์ใช้ momentum แบบ $v = \beta v + (1-\beta)g$ ต้องแทน factor ตาม convention

ต้องจำ Optimizer รับ gradient จาก backprop; Momentum มี ประวัติ gradient อันดับหนึ่ง; Adam มี first และ second moment พร้อม bias correction และ epsilon เพื่อเสถียรภาพ

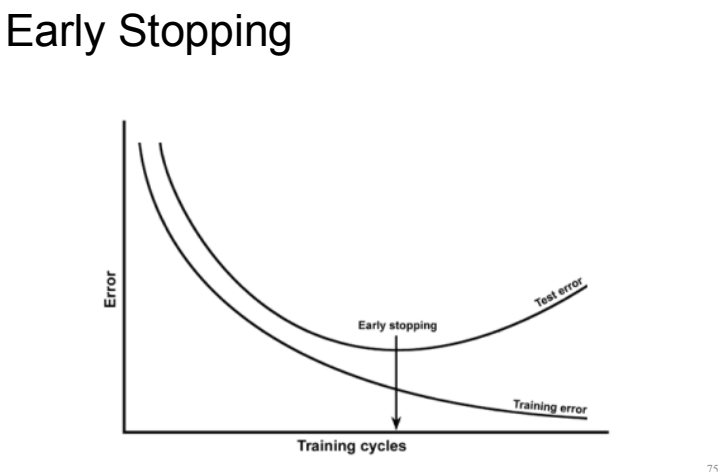
6.9 Overfitting, Early stopping, L1/L2 และ Dropout

แหล่งเนื้อหา: L6 PDF p.67–76

เครือข่ายมี degrees of freedom มาก จึงจำ training examples ได้ง่าย การทดสอบบนข้อมูลเดียวกับที่ train ไม่วัด generalization ต้องใช้ validation เพื่อติดตาม model selection และ test ที่กันไว้

Early stopping

เก็บ parameter ที่ validation performance ดีที่สุดและหยุดเมื่อไม่ดีขึ้นตาม patience ไม่จำเป็นต้องหยุดทันทีที่หนึ่งจุดแกว่ง และการหยุดที่ epoch ล่าสุดไม่เท่ากับ restore weights ของ epoch ดีที่สุดเสมอ



รูป 6.7: Train error ยังลดได้ขณะที่ validation error เริ่มเพิ่ม เลือก checkpoint จาก validation ไม่ใช่ training minimum (L6 p.73)

อ่านภาพทีละจุด **1.** ดู minimum ของ validation error และทิศ training error ใน epoch เดียวกัน

2. ถ้า train ลดต่อแต่ validation เพิ่มคือสัญญาณ overfit ใช้ best checkpoint ของ validation พร้อม patience ไม่ใช้ epoch ที่ train ต่ำสุดหรือ test เพื่อตัดสิน

L1/L2 เพิ่ม penalty บน weights ตามบท 4. Dropout สุ่มปิด units ชั่วคราวใน training step ลดการพึ่งพา units ชุดเดิม ใน inference โดยทั่วไปไม่สุ่มปิดเช่น training และมีการ scale เพื่อรักษา expected activation

ตัวอย่างคำนวณทีละขั้น

Dropout rate 0.25 หมายถึงโอกาสปิดหน่วย 0.25 และ keep probability $q = 1 - 0.25 = 0.75$

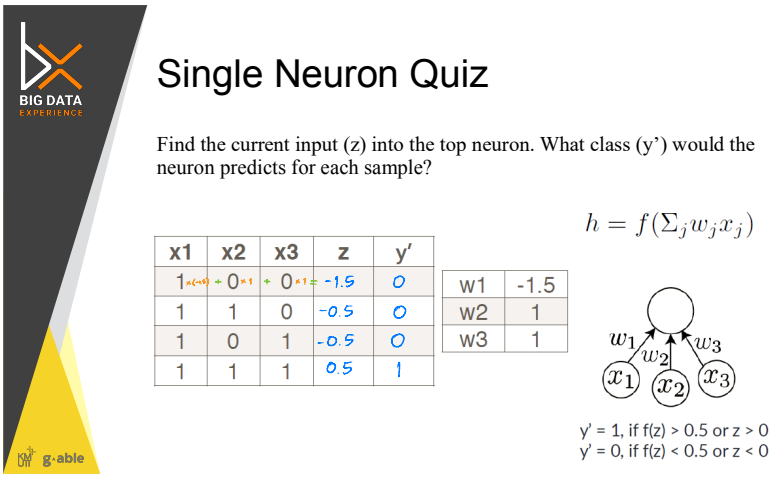
1. Inverted dropout คูณ activation ของหน่วยที่เก็บไว้ด้วย $1/q = 1/0.75 = 4/3$
2. ถ้า activation เดิม $a = 3$ ผล training เป็น $3(4/3) = 4$ ด้วย probability 0.75 หรือเป็น 0 ด้วย probability 0.25
3. Expected activation $E[\tilde{a}] = 0.75(4) + 0.25(0) = 3$ เท่า activation เดิม
4. ตอน inference ไม่สุ่ม mask และใช้ $a = 3$ โดยไม่คูณ 0.75 ซ้ำตาม convention นี้

ต้องจำ Dropout ไม่ลบ neuron ถาวร, ไม่เหมือน pruning และไม่เพิ่ม training samples. Early stopping ใช้ validation ไม่ใช้ test เพื่อตัดสิน epoch

ระวัง: L2 penalty กับ weight decay อาจต่างกันสำหรับ adaptive optimizers ข้อความ “L2(weight decay)” ในสไลด์อ่านในบริบทที่สอน ไม่ขยายว่าเหมือนกันทุก algorithm

6.10 Neuron AND และ Quiz นับ weights ในภาพจริง

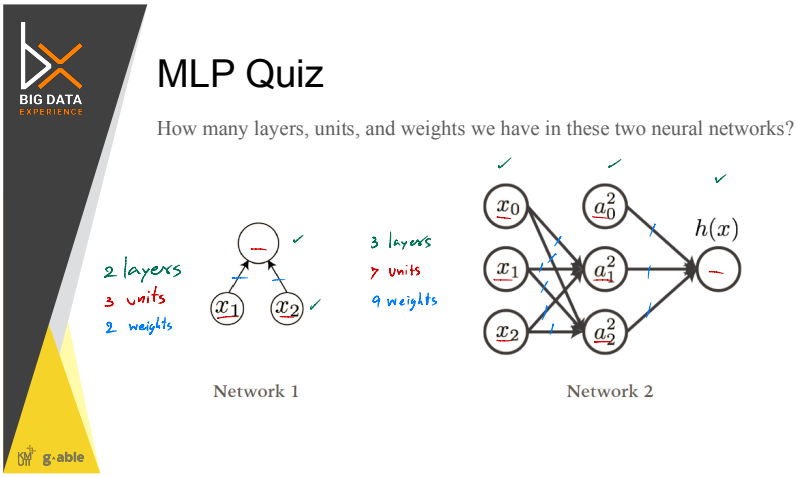
แหล่งเนื้อหา: L6 PDF p.6–19



รูป 6.8: AND neuron ในหน้า 8: bias input เป็นค่าคงที่ 1

อ่านภาพทีละจุด **1.** weights $(-1.5, 1, 1)$ ใช้กับ $(1, x_2, x_3)$ ดังนั้น $z = -1.5 + x_2 + x_3$
2. ใช้ threshold $z \geq 0$ ให้ AND สำหรับ binary inputs ถ้าใช้ sigmoid แล้ว threshold 0.5 class เหมือนกัน แต่ probability ไม่เป็น 0/1 ตรง ๆ

x_2	x_3	z	Class
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	1

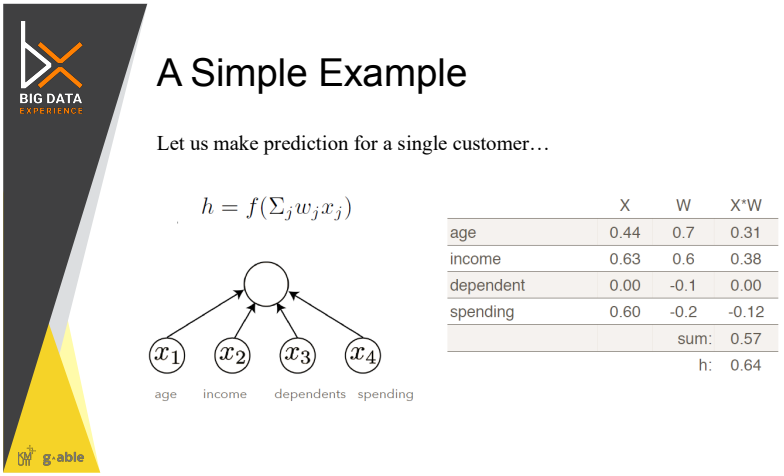


รูป 6.9: MLP quiz หน้า 18: bias nodes กับ learned units

อ่านภาพทีละจุด **1.** รูปซ้ายมี 2 input units จากนั้น 1 output unit จึง 2 weights ตามเส้นที่วาด ไม่เพิ่ม bias ที่รูปไม่ให้

2. รูปขวาวาด input bias x_0 และ hidden bias a_0: 3×2 edges เข้า hidden learned units +3 edges เข้า output =9 parameters

รูปขวาวาด 7 nodes รวม constants แต่ data features จริงมี 2, hidden learned units มี 2 และ output มี 1 จึงเขียน count $(2+1)2+(2+1)1=9$ หากถาม “จำนวน neuron” ต้องระบุรวม bias node หรือไม่ ห้ามใช้ 3 hidden learned units เพราะ $a_0=1$ ไม่ได้คำนวณ activation จาก input



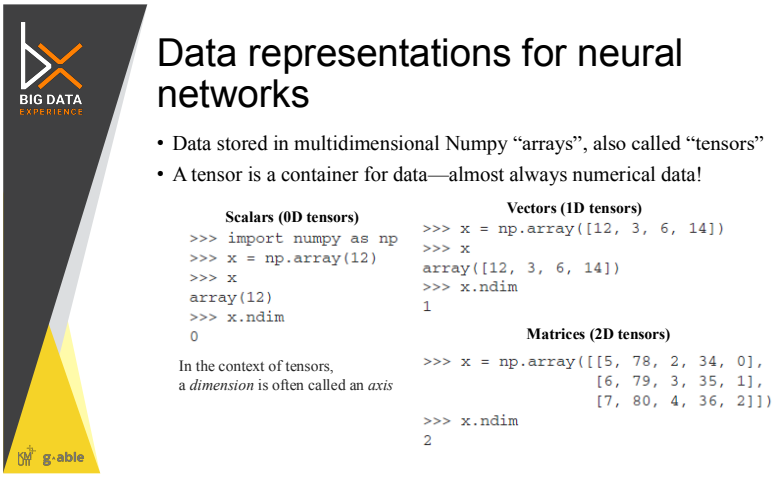
รูป 6.10: Loan features ส่งเข้า neuron ในตัวอย่างต้นทาง

อ่านภาพทีละจุด **1.** Vector features ต้องเรียงตรงกับ weights เช่น amount, term, income ตามที่นิยาม

2. สเกลและหน่วยเปลี่ยน weighted sum. feature มากไม่ได้หมายถึง probability มากเสมอ เพราะ sign ของ weights และ bias ยังมีผล

6.11 Tensor rank, loss log base และการอ่าน training code

แหล่งเนื้อหา: L6 PDF p.34–50

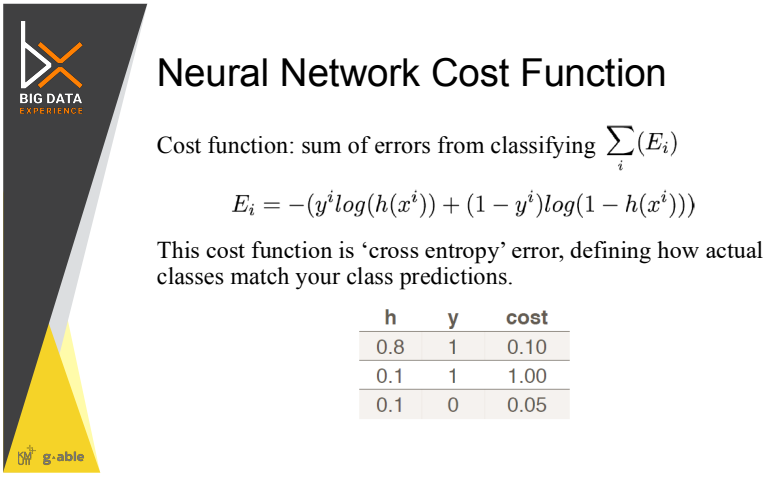


รูป 6.11: Tensor examples และ ndim: อย่าสับสน dimensions กับจำนวน entries

อ่านภาพทีละจุด **1.** Scalar เช่น 12 มี ndim 0. vector 4 entries มี ndim 1 ไม่ใช่ 4 dimensions ในความหมาย tensor rank

2. Matrix 2×3 มี ndim 2. ภาพ RGB หนึ่งภาพมี rank 3 แต่ batch ของภาพเพิ่ม sample axis จึง rank 4

Video batch อาจ shape (N, T, H, W, C) มี rank 5 โดย T เป็น frame และ time axis คำว่า “dimension” จึงต้องบอกว่าพูดถึง tensor rank หรือ feature-space dimension



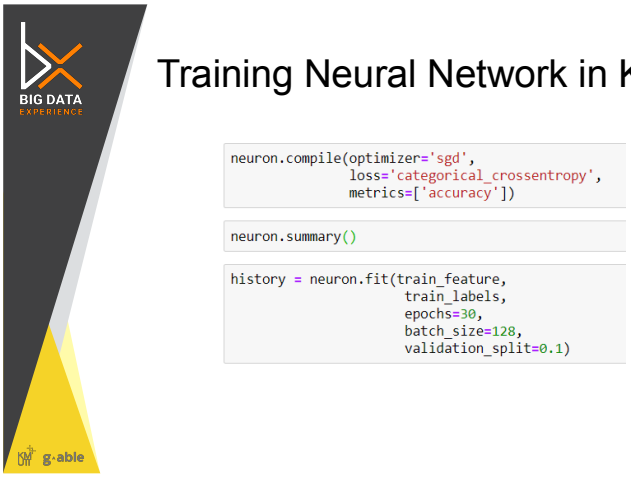
รูป 6.12: Cost table หน้า 42: ตัวเลขสอดคล้องกับ log ฐาน 10

อ่านภาพทีละจุด **1.** ที่ $(p, y) = (0.8, 1)$ ค่า 0.10 ใกล้ $-\log_{10}(0.8) = 0.09691$ ส่วน $(0.1, 1)$ ได้ 1.00

2. เล่มนี้ใช้ natural log ใน BCE เพื่อให้ derivative ตามมาตรฐานที่เขียนไว้ ค่า loss จึงต่างด้วย positive scale ไม่ใช่ต่าง objective ordering

p	y	BCE ฐาน 10	BCE natural log
0.8	1	0.096910	0.223144
0.1	1	1.000000	2.302585
0.1	0	0.045757	0.105361

เพราะ $\log_{10} x = \ln x / \ln 10$ gradient จึงเป็น $(p - y)/\ln 10$ ถ้าใช้ loss ฐาน 10 ไม่ใช่ $p - y$ เท่าเดิม Fixed learning rate ทำให้ขนาด update ต่างกัน แม้ minimum ของ unregularized loss ไม่เปลี่ยน หากมี regularizer ต้อง scale relative strength ให้สอดคล้องด้วย



รูป 6.13: Compile และ fit ในหน้า 50: ต้องตรวจคู่ output–loss กับหน้าก่อน

อ่านภาพทีละจุด **1.** หน้า 49 ใช้ Dense 1 sigmoid สำหรับ binary outcome คู่ที่สอดคล้องคือ binary cross-entropy

2. หน้า 50 เขียน categorical cross-entropy ซึ่งไม่ตรงกับ binary single-output setup นี้ จึงไม่ควรคัด code สองหน้าไปรันโดยไม่แก้คู่ output และ labels และ loss

สำหรับ mutually exclusive K classes ใช้ K-unit softmax กับ one-hot categorical cross-entropy หรือ integer-label sparse version ที่เหมาะสม ส่วนหลาย binary labels อิสระใช้ sigmoid หลาย units กับ binary loss. K units ไม่ได้แปลว่า softmax ถูกเสมอ Epochs 30 หมายถึงผ่าน training data 30 รอบ. batch size 128 กำหนด samples ต่อ update. validation split ไม่ใช้ validation labels เพื่อ update weights

6.12 Activation เพิ่มเติม: Leaky ReLU และ ELU

แหล่งเนื้อหา: L6 PDF p.55–60

Leaky ReLU $f(z) = z$ เมื่อ $z \geq 0$ และ az เมื่อ $z < 0$, $0 < a < 1$. derivative negative branch เป็น a จึงไม่เป็นศูนย์ทั้งหมดอย่าง ReLU ส่วน ELU ใช้ $f(z) = z$ เมื่อ $z \geq 0$ และ $\alpha(e^z - 1)$ เมื่อ $z < 0$, derivative αe^z ใน negative branch และมี negative saturation ต่างจาก leaky linear branch Neither รับประกันว่า optimization ไม่ติดปัญหาใด ๆ การเลือก activation ต้องเข้ากับ depth, initialization และ task Batch Normalization normalize activations ในขั้น training แล้วเรียน scale γ และ shift β. inference ใช้ stored statistics ตามการตั้งค่า จึงต้องแยก trainable parameters กับ moving statistics วิธี train เร็วขึ้นในหน้า 60 ยังรวมการใช้ pretrained layers ไม่ใช่เปลี่ยน optimizer อย่างเดียว

6.13 Nesterov และ AdaGrad: สูตรพร้อมขั้น update ที่ไม่ข้าม

แหล่งเนื้อหา: L6 PDF p.61–66. derivation อธิบายเพิ่มเติม

กำหนด momentum เป็น displacement: $v_t = \rho v_{t-1} - \eta g(\theta_{t-1})$, $\theta_t = \theta_{t-1} + v_t$ สำหรับ Nesterov ใน convention นี้ ประเมิน gradient ที่ look-ahead $\theta_{t-1} + \rho v_{t-1}$ แล้ว $v_t = \rho v_{t-1} - \eta g(\theta_{t-1} + \rho v_{t-1})$ ไม่ใช่ใช้สูตรสอง convention ปะปนจนบวก gradient ผิดทิศ ตัวอย่าง $J = \theta^2/2$, $\theta_0 = 2, v_0 = 0, \rho = 0.9, \eta = 0.1$: ก้าว 1 ทั้งสองได้ $v_1 = -0.2, \theta_1 = 1.8$ ก้าว 2 momentum ใช้ $g = 1.8$ จึง $v_2 = -0.36, \theta_2 = 1.44$. Nesterov มองที่ $1.8 - 0.18 = 1.62$ จึง $v_2 = -0.342, \theta_2 = 1.458$ ต่างกันที่จุดประเมิน ไม่ใช่ Nesterov รับประกัน loss ต่ำกว่าทุกก้าว AdaGrad สะสม $s_t = s_{t-1} + g_t \odot g_t$ และ $\theta_t = \theta_{t-1} - \eta g_t / (\sqrt{s_t} + \epsilon)$ แบบ elementwise ตัวอย่าง gradients $(2, 1)$ สองก้าว, $\eta = 0.1, \epsilon = 0$: $s_1 = (4, 1)$ ทำ update $(-0.1, -0.1)$. $s_2 = (8, 2)$ ทำ update $(-0.070711, -0.070711)$ ตัวสะสมไม่ลืมอดีตจึง effective steps อาจเล็กลงมาก ส่วน RMSProp และ Adam ใช้ exponential averages. Adam ยังต้องแก้ bias ตอนต้นของ moving averages

ต้องจำ ระบุ convention ของ velocity ก่อน. AdaGrad มี denominator ต่อ parameter และสะสม squared gradients ไม่ใช่สะสม gradient ที่บวก และ ลบหักล้าง

6.14 Early stopping และ Dropout: trace สถานะการฝึก

แหล่งเนื้อหา: L6 PDF p.67–76; ตัวอย่างฝึกใหม่

Validation loss แต่ละ epoch $(0.8, 0.6, 0.55, 0.57, 0.56, 0.58)$; patience 2, นับเมื่อไม่ดีขึ้นจาก best ที่บันทึก และ restore best weights Epoch 3 เป็น best 0.55; epoch 4 ไม่ดีขึ้นครั้ง 1; epoch 5 ไม่ดีขึ้นครั้ง 2 จึงหยุดและคืน weights ของ epoch 3 ไม่รอ epoch 6 และไม่คืน epoch 5 เพียงเพราะเป็นล่าสุด

Epoch	Validation loss	bad count	Checkpoint
1	0.80	0	best=1
2	0.60	0	best=2
3	0.55	0	best=3
4	0.57	1	best ยัง 3
5	0.56	2	หยุดและ คืน weights ของ epoch 3

Inverted Dropout keep probability $q = 0.75$ สำหรับ activation a: training output $\tilde{a} = ma/q$, $m \sim Bernoulli(q)$ จึง $E[\tilde{a}] = a$ ถ้า $a = 3$ ให้ output 4 ด้วย probability 0.75 และ 0 ด้วย 0.25; inference output 3 ไม่คูณ 0.75 ซ้ำเมื่อใช้ inverted dropout convention

ต้องจำ Early stopping อาศัย validation history; dropout mask สุ่มเฉพาะ training ตาม protocol; rate ที่โจทย์ให้ต้องแยกว่า drop probability หรือ keep probability

BatchNorm: แยกสถิติของ batch จากค่าที่เก็บไว้

สำหรับ activation หนึ่งช่องใน training mini-batch สมมติ $z = (2, 4)$ จะได้ $\mu_B = 3$, $\sigma_B^2 = [(2-3)^2 + (4-3)^2]/2 = 1$. ถ้า ε เล็กจนละได้, normalized values คือ $(-1, 1)$; เมื่อ trainable scale $\gamma = 2$ และ shift $\beta = 1$ output เป็น $(-1, 3)$. γ, β เรียนจาก gradient แต่ moving mean และ variance สะสมเพื่อใช้ตอน inference จึงไม่คำนวณ mean จาก sample ทดสอบเพียงหนึ่งตัวแล้วอ้างว่าเป็น training batch เดิม จำนวนค่าใน BatchNorm 64 channels คือ 128 trainable กับ 128 moving statistics ตาม convention ในเล่ม; $\varepsilon > 0$ ช่วยไม่ให้หารศูนย์

อ่านเพิ่มจากตำรา Géron, Hands-On Machine Learning Ch.11, PDF p.359–361: แยก mini-batch mean และ variance, normalization, scale และ shift และสถิติที่ใช้เวลา inference เชื่อม L6 p.58–60 และ L7 p.61–62

6.15 Lecture 6: สรุปและ Checklist

แหล่งเนื้อหา: L6 PDF p.5–78

Pipeline ที่ต้องอธิบาย

เลือก input shape และ target จากนั้น กำหนด architecture จากนั้น initialize weights จากนั้น forward จากนั้น loss จากนั้น backprop gradients จากนั้น optimizer update จากนั้น validation แล้ว เลือก checkpoint จากนั้น test

Formula recap

$$z = w^T x + b, \quad a = f(z), \quad N_{dense} = (n_{in} + 1)n_{out}, \quad w \leftarrow w - \eta \frac{\partial L}{\partial w}.$$

Checklist

- คำนวณ neuron weighted sum, activation และ predicted class ได้
- นับ parameters รวม bias และอธิบาย layer convention ได้
- แยก linear และ sigmoid และ tanh และ ReLU และ softmax กับ task ได้
- อธิบาย forward และ backprop และ optimizer และ chain rule ได้
- นับ epoch และ step และ batch และอ่าน tensor shapes ได้
- อธิบาย symmetry problem กับ zero initialization ได้
- เปรียบเทียบ Momentum และ Nesterov และ AdaGrad และ Adam เชิงกลไกได้
- อ่าน train และ validation curve และเลือก regularization ได้

ต้องจำ สิ่งสำคัญคือความสัมพันธ์ของ architecture, loss, gradients และ generalization ไม่ใช่เพียงจำว่า neural network มีหลาย layers

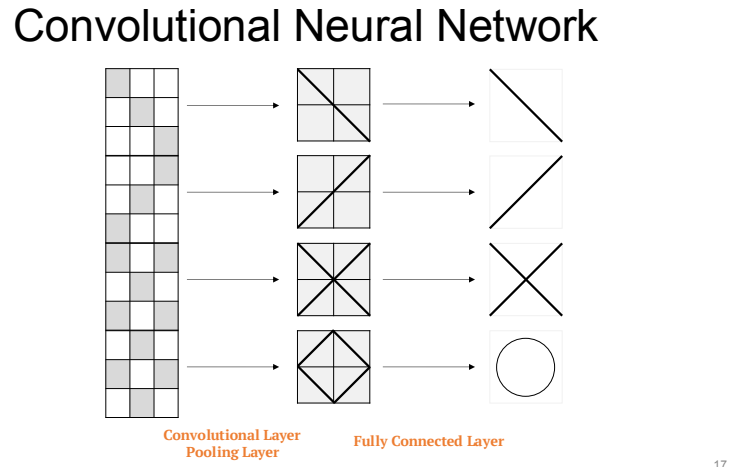
อ่านเพิ่มจากตำรา อ่านต่อ ISLP Ch.10 เฉพาะ single และ multilayer network, fitting, dropout. Géron Ch.10–11 ไม่เพิ่ม RNN เป็นเนื้อหาบังคับเพราะไม่ได้อยู่ใน Lecture นี้

Lecture 7 Convolutional Neural Networks

แหล่งเนื้อหา: L7 PDF p.1–18 | ขอบเขตอ่านสอบ Lecture 1–8

คำย่อที่ใช้ในบทนี้

CNN คือ Convolutional Neural Network. **Conv** ใช้เรียก convolutional layer แบบสั้น และ **FC** คือ Fully Connected layer. **RGB** คือ Red, Green, Blue ซึ่งเป็นสามช่องสี. **VGG** มาจาก Visual Geometry Group ส่วน **ResNet** คือ Residual Network. H, W และ C ใน shape หมายถึง height, width และ channels. ReLU และ BatchNorm ใช้ชื่อเต็มตามบทที่ 6. CNN ใช้ local connectivity และ parameter sharing เพื่อเรียน pattern ของภาพ เช่น edges และ texture ใน layer ต้นและส่วนประกอบที่ซับซ้อนขึ้นใน layer ถัดไป ต่างจาก Dense ที่เชื่อมทุก pixel โดยไม่รักษาโครงสร้าง spatial เป็นหลัก Convolution filter เป็น matrix ของ learned weights ที่เลื่อนไปตาม input แต่ละตำแหน่งคูณค่า pixel กับ weight แล้วรวมได้ output หนึ่งค่า Output หลายตำแหน่งรวมเป็น feature map และ filter หลายตัวสร้างหลาย maps



รูป 7.1: ตัวอย่างสไลด์เชื่อม Convolution, Pooling และ Fully Connected จาก pattern ท้องถิ่นสู่ class output (L7 p.18)

อ่านภาพทีละจุด **1.** ตาม spatial maps จาก convolution ไป pooling แล้ว vector และ classifier: แต่ละส่วนเปลี่ยน shape คนละแบบ

2. Conv เรียน filters, Pool รวมค่า, Fully connected เชื่อม features ไป outputs ดู parameter count แยกจากจำนวน feature-map pixels

Receptive field

Receptive field คือบริเวณ input ที่มีอิทธิพลต่อ output unit หนึ่ง เมื่อซ้อน layers unit ลึกจะรวมข้อมูลจากพื้นที่ใหญ่ขึ้นโดยอ้อม Feature hierarchy ไม่ได้หมายความว่าเราต้องกำหนด filter ตรวจเส้นทุกตัวเอง เพราะ weights เรียนจาก loss ผ่าน backprop

> **ต้องจำ** Kernel size กำหนดพื้นที่ท้องถิ่น. filter count กำหนด output channels. stride กำหนดระยะเลื่อน ทั้งสามคนละ quantity

ระวัง: Convolution kernel ใน CNN ไม่ใช่ kernel trick ของ SVM แม้ใช้คำเดียวกัน

7.1 ตัวอย่างคำนวณ filter ของอาจารย์

แหล่งเนื้อหา: L7 PDF p.4–14, 19–23

สไลด์ใช้ pattern 2×2 ที่มีค่า ± 1 และ filter

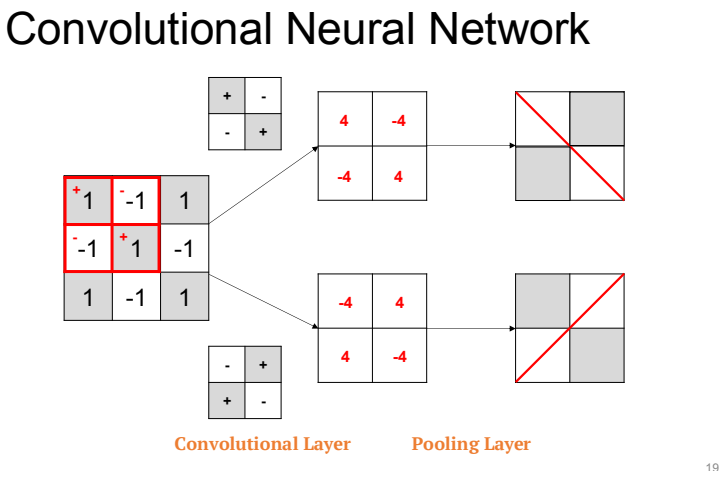
$$K = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

ถ้า input เป็น $X = K$ ผลรวมของผลคูณตามตำแหน่งเท่ากับ 4 ถ้า input เป็น $-K$ จะได้ -4 และถ้า input ทุก pixel เป็น 1 จะได้ 0 เครื่องหมายบวกหรือลบจึงบอกทิศของ diagonal pattern ตามการเข้ารหัสนี้ ไม่ได้แทนความน่าจะเป็น

ตัวอย่างคำนวณทีละขั้น

ใช้ patch $X = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$ และ filter $K = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

1. คูณตำแหน่งที่ตรงกัน: บนซ้าย $1(1) = 1$, บนขวา $1(-1) = -1$, ล่างซ้าย $(-1)(-1) = 1$, ล่างขวา $1(1) = 1$
2. รวมได้ $c = 1 - 1 + 1 + 1 = 2$ โดยไม่มี bias เพิ่ม
3. ReLU ให้ $\max(0, c) = \max(0, 2) = 2$ ถ้า c ติดลบ ReLU จึงค่อยเปลี่ยนเป็น 0 หลังคำนวณ signed sum แล้ว



รูป 7.2: เลื่อน filter บน input 3×3 ด้วย stride 1 ได้ output 2×2 ทั้งสอง filters ให้ maps ต่างกัน (L7 p.20)

อ่านภาพทีละจุด **1.** วาง filter บน patch ซ้ายบนก่อน multiply-sum แล้วเลื่อนไปตำแหน่งที่ stride อนุญาต
2. สอง filters ให้สอง feature maps ต่อ input เดียว ไม่รวมให้เหลือ map เดียวโดยไม่ระบุ operation. signed responses ต้องผ่าน activation ก่อนตีความขั้นถัดไป

ตัวอย่าง 3×3

ให้ $X = \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix}$ ใช้ filter ด้านบน, no padding, stride 1 จะได้ $\begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}$ เพราะ patch สี่ตำแหน่งสลับเป็น $K, -K, -K, K$

> **ต้องจำ** คูณตามตำแหน่งแล้ว sum ทั้งหมด ไม่ใช่ matrix multiplication ธรรมดา ต้องคำนวณแยกแต่ละ window

อ่านเพิ่มจากตำรา Géron Ch.14: neural-network libraries มักใช้ cross-correlation โดยไม่ flip filter แต่เรียก convolution ตามธรรมเนียม ตัวอย่างนี้ยึดวิธีเลื่อน filter ในสไลด์

7.2 Convolution trace ทุก window แล้ว ReLU และ Pooling

แหล่งเนื้อหา: L7 PDF p.4–23,40–42. ตัวเลขฝึกใหม่

ใช้ input และ filter ดังนี้ ไม่มี bias, valid padding, stride 1 และไม่ flip filter ซึ่งเป็น cross-correlation convention ของ CNN ในตัวอย่าง

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 0 & 1 & 1 \\ 1 & 2 & 2 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}.$$

แต่ละ window ให้ $a - b + d$ เมื่อ patch เขียน $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ แต่ยังควรแสดงผลคูณช่อง $c \cdot 0$ เพื่อไม่ลืมตำแหน่ง filter

Output	Patch	ผลรวม	หลัง ReLU
$(1,1)$	$\left[\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right]$	$1-2+0+1=0$	0
$(1,2)$	$\left[\begin{smallmatrix} 2 & 0 \\ 1 & 3 \end{smallmatrix}\right]$	$2-0+0+3=5$	5
$(1,3)$	$\left[\begin{smallmatrix} 0 & 1 \\ 3 & 2 \end{smallmatrix}\right]$	$0-1+0+2=1$	1
$(2,1)$	$\left[\begin{smallmatrix} 0 & 1 \\ 2 & 0 \end{smallmatrix}\right]$	$0-1+0+0=-1$	0
$(2,2)$	$\left[\begin{smallmatrix} 1 & 3 \\ 0 & 1 \end{smallmatrix}\right]$	$1-3+0+1=-1$	0
$(2,3)$	$\left[\begin{smallmatrix} 3 & 2 \\ 1 & 1 \end{smallmatrix}\right]$	$3-2+0+1=2$	2
$(3,1)$	$\left[\begin{smallmatrix} 2 & 0 \\ 1 & 2 \end{smallmatrix}\right]$	$2-0+0+2=4$	4
$(3,2)$	$\left[\begin{smallmatrix} 0 & 1 \\ 2 & 2 \end{smallmatrix}\right]$	$0-1+0+2=1$	1
$(3,3)$	$\left[\begin{smallmatrix} 1 & 1 \\ 2 & 0 \end{smallmatrix}\right]$	$1-1+0+0=0$	0

$$C = \begin{bmatrix} 0 & 5 & 1 \\ -1 & -1 & 2 \\ 4 & 1 & 0 \end{bmatrix}, \quad A = \begin{bmatrix} 0 & 5 & 1 \\ 0 & 0 & 2 \\ 4 & 1 & 0 \end{bmatrix}.$$

ต่อด้วย maxpool 2×2,stride 1,valid ได้ $P = \begin{bmatrix} 5 & 5 \\ 4 & 2 \end{bmatrix}$ เช่น window บนขวาใช้ $(5, 1, 0, 2)$ จึง max 5 output shape 3×3 แล้ว 2×2 จำนวน channels ยัง 1 และไม่ได้มี 4 pool weights ที่เรียนรู้

ขยายเป็นหลาย channels

ถ้า input RGB filter หนึ่งตัวมี kernel 2×2 สามแผ่น ต้องคำนวณผลรวมของทั้งสาม channels และบวก bias เพียงหนึ่งตัวจึงได้ output pixel หนึ่งค่า จำนวน output channels กำหนดด้วยจำนวน filters ไม่ใช่ input channels คูณ filters แล้วเก็บทุกแผ่นแยกถาวร

> **ต้องจำ** Trace patch ตำแหน่งเดิมในทุก channel จากนั้น multiply-sum รวม channel จากนั้น bias จากนั้น activation จากนั้น pool. output หนึ่ง filter คือหนึ่ง feature map

7.3 Output dimension: padding, stride และ kernel

แหล่งเนื้อหา: L7 PDF p.34–39, 43–46. formula จาก Explanation เสริม

ให้ภาพขาเข้ามีความสูง H และความกว้าง W; kernel มีขนาด $K_h \times K_w$; padding แต่ละด้านมีขนาด P_h, P_w; และ stride เป็น S_h, S_w ตามลำดับ โดยสูตรนี้ยังไม่รวม dilation ขนาดผลลัพธ์ของแต่ละแกนคำนวณได้ดังนี้:

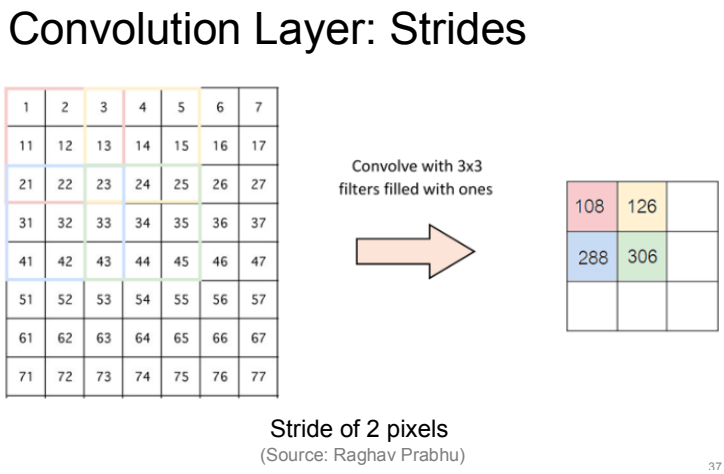
$$H_{out} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

Output channels เท่ากับ number of filters เมื่อ standard convolution หนึ่ง filter ครอบคลุม input channels ทั้งหมด ไม่คำนวณแต่ละ RGB แล้วทิ้งอีก channel

ตัวอย่างคำนวณทีละขั้น

ให้ input $28 \times 28 \times 1$, kernel 5×5, stride $s = 1$, padding $p = 2$ และ 40 filters

1. ความสูง output $= \lfloor (28 + 2(2) - 5)/1 \rfloor + 1 = 27 + 1 = 28$ และความกว้างคำนวณเหมือนกัน
2. 40 filters ให้ 40 output channels จึง shape $28 \times 28 \times 40$
3. ถ้าเปลี่ยนเป็น valid ($p = 0$) ขนาดแต่ละแกนเป็น $\lfloor (28 - 5)/1 \rfloor + 1 = 24$ จึง shape $24 \times 24 \times 40$



รูป 7.3: stride 2 ในสไลด์คำนวณเฉพาะหน้าต่างที่วางได้ จึงต้องดู stride และ border พร้อมกัน (L7 p.38)

อ่านภาพทีละจุด **1.** ตำแหน่ง window ที่ใช้ห่างกันตาม stride 2 และต้องไม่ออกนอก valid input

2. นับได้เท่าไรคือ output count ตาม floor formula ไม่ปัดเศษขึ้นทุกกรณี ถ้าเพิ่ม padding ต้องนับ pixel ขอบที่เติมด้วย

> **ต้องจำ** ไม่ลืม floor และ+1. padding เพิ่ม spatial extent ไม่เพิ่ม channel. filter count ไม่ใช่ kernel width

ระวัง: “SAME” กับ stride>1 ไม่แปลว่า output เท่า input เสมอ โดยทั่วไป target $\lceil H/S \rceil$. สูตร symmetric padding ด้านบนใช้เมื่อโจทย์กำหนด pad แต่ละด้านชัด

7.4 Parameter sharing, channels และ receptive field

แหล่งเนื้อหา: L7 PDF p.3, 36, 43–46

Convolution มาตรฐานที่ใช้ filter C_{in} channels และสร้าง output C_{out} channels มีพารามิเตอร์ดังนี้ หนึ่ง filter มี $K_h K_w C_{in}$ weights และ bias หนึ่งค่า แล้วคูณจำนวน filter ทั้งหมด:

$$N_{params} = (K_h K_w C_{in} + 1) C_{out}$$

เมื่อแต่ละ filter มี bias หนึ่งตัว จำนวน weights ไม่ขึ้นกับ H, W เพราะ ใช้ weights เดิมซ้ำ ชุดเดียวกันทุก location ต่างจาก Dense ที่จำนวน input pixel เพิ่มทำให้ connections เพิ่ม

ตัวอย่างคำนวณทีละขั้น

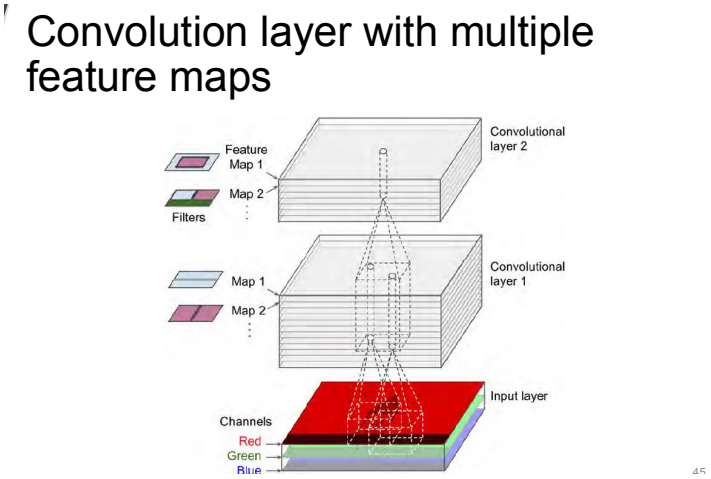
Conv ใช้ kernel 3×3, input 3 channels และ 16 filters โดยมีหนึ่ง bias ต่อ filter

1. หนึ่ง filter มี weights $3(3)(3) = 27$ ค่า และ bias 1 ค่า รวม 28 parameters
2. 16 filters มี parameters $28(16) = 448$ ค่า
3. ขยายภาพจากด้าน 32 เป็น 64 แต่ยังใช้ kernel และ channels เดิม จึงยังมี 448 parameters ส่วน activations และจำนวนการคำนวณเพิ่มตามขนาด output

Spatial receptive field เมื่อซ้อน layers

สำหรับ stride 1, 3×3 สอง conv layers ติดกัน unit ของ layer 2 มอง input 5×5 ได้โดยอ้อม ไม่ใช่ 6×6 และไม่ใช่ 3×3 คงเดิม หากมี stride และ pool การคำนวณต้อง ติดตาม effective jump ด้วย

อ่านเพิ่มจากตำรา Intuition เสริม: ให้ jump $j_0 = 1$ และ receptive field $r_0 = 1$, แต่ละ layer $r_l = r_{l-1} + (k_l - 1)j_{l-1}$ และ $j_l = j_{l-1}s_l$ สูตรนี้ช่วยตรวจ local-connectivity แต่ไม่ถือเป็นสูตรบังคับจากสไลด์



รูป 7.4: Filter ครอบคลุมหลาย input maps แล้วสร้าง output map หนึ่งชุด หลาย filters จึงได้หลาย output maps (L7 p.46)

อ่านภาพทีละจุด **1.** หนึ่ง filter มีแผ่น kernel ครบทุก input channels แล้วรวม contribution ให้หนึ่ง output map
2. ใช้ filters หลายตัวจึงได้หลาย output channels. weights แชร์ทุกตำแหน่งแต่ไม่แชร์ระหว่าง filters โดยอัตโนมัติ

> **ต้องจำ** หนึ่ง feature map มีหลาย activations แต่ weights ร่วมกัน. bias หนึ่งต่อ filter ไม่ใช่หนึ่งต่อ output pixel

7.5 Receptive field: track jump และพื้นที่ที่มองเห็น

แหล่งเนื้อหา: L7 PDF p.24–39. สูตร trace เสริม

กำหนด $r_0 = 1, j_0 = 1$ และ $r_l = r_{l-1} + (k_l - 1)j_{l-1}$, $j_l = j_{l-1}s_l$ ค่า j คือช่องว่างใน input เดิมระหว่าง output units ติดกัน ไม่ใช่ stride ของ layer ปัจจุบันอย่างเดียว

Layer	k	s	Receptive field r	Jump j
Input	-	-	1	1
Conv 3	3	1	3	1
Pool 2	2	2	4	2
Conv 3	3	1	8	2
Conv 3	3	1	12	2

หลัง pool output unit เห็น 4×4 ใน input เดิม. conv ถัดไปเพิ่ม $(3-1)\cdot 2 = 4$ จึงเห็น 8×8 ไม่ใช่ 6×6 ขนาดนี้เป็น nominal receptive field ภาพขอบอาจมี padding ร่วมอยู่ จึงไม่ใช่ทุกตำแหน่งเห็น pixel จริงครบเท่ากัน

> **ต้องจำ** เมื่อผ่าน stride หรือ pool ให้ อัปเดต jump ด้วย มิฉะนั้นคำนวณ field ของ layer ลึกตำเกินจริง

7.6 Parameter sharing ทำให้ gradient ต้องรวมทุกตำแหน่ง

แหล่งเนื้อหา: L7 PDF p.19–39. คำอธิบาย chain rule เสริมจาก NN

Filter weight หนึ่ง ตัว ถูก ใช้ ซ้ำ หลาย patch ดัง นั้น gradient ต่อ weight นั้น เป็น ผล รวม contributions ทุก ตำแหน่ง $\partial L/\partial K_{ab} = \sum_{i,j}(\partial L/\partial C_{ij})X_{i+a,j+b}$ ภาย ใต้ cross-correlation convention ที่ ระบุ ใช้ input 3×3 $X = \left[\begin{smallmatrix}1&2&3\\4&5&6\\7&8&9\end{smallmatrix}\right]$,kernel 2×2,bias 0,ไม่มี activation และสมมติ upstream gradient 2×2 ทุกค่าคือ 1 จะได้

$$\nabla K = \begin{bmatrix}1+2+4+5 & 2+3+5+6\\ 4+5+7+8 & 5+6+8+9\end{bmatrix} = \begin{bmatrix}12 & 16\\ 24 & 28\end{bmatrix}.$$

Bias gradient $= 1+1+1+1 = 4$ ไม่ใช่ 1 เพราะ bias แชร์สี่ตำแหน่ง ถ้ามี ReLU ก่อน loss upstream gradient ต้องผ่าน mask ก่อนรวม และ MaxPool ส่ง gradient ไปตำแหน่งที่ได้ max ตาม tie convention

> **ต้องจำ** Weights แชร์จึง count น้อย แต่ได้รับ gradient สะสมหลายตำแหน่ง. sharing ไม่ได้หมายความว่าใช้ gradient จาก patch แรกอย่างเดียว

Receptive field โตได้แม้ kernel เดิมยังเล็ก

ถ้า Conv สองชั้นต่อกัน แต่ละชั้นใช้ 3×3, stride 1, no dilation, หนึ่ง output ของชั้นแรกเห็น input 3×3 ส่วนหนึ่ง output ของชั้นที่สองรวมเพื่อนบ้านชั้นแรก 3×3 จึงรับอิทธิพลจาก input เดิม 5×5 (เพิ่มด้านละหนึ่ง pixel). การนับนี้ไม่เท่ากับ kernel ที่เรียนเป็น 5×5 หนึ่งชั้น: สองชั้นมี activation ระหว่างกลางและ parameter count ต่างกัน ภาพ receptive fields ของสไลด์ L7 p.3 และ CNN pipeline p.19–31 จึงอ่านเป็นพื้นที่ที่มีอิทธิพลสะสม ไม่ใช่บอกว่า weight หนึ่งตัวต่อทุก pixel ในพื้นที่ 5×5

อ่านเพิ่มจากตำรา Géron, Hands-On Machine Learning Ch.14, PDF p.459–461: local receptive fields และการซ้อน convolutional layers เพื่อเรียน pattern หลายระดับ

7.7 Pooling, Flatten และ Fully Connected

แหล่งเนื้อหา: L7 PDF p.18–31, 40–42

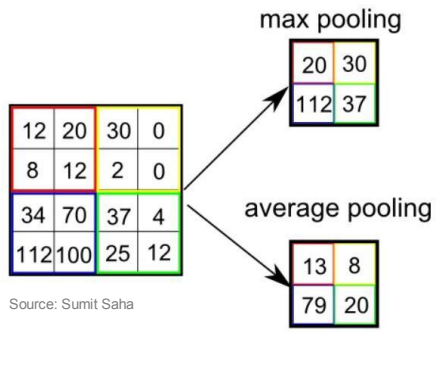
Pooling ลด spatial size ด้วย aggregate แต่ละ window Maxpool เลือกค่าสูงสุด. Average pooling หาค่าเฉลี่ย. Sum pool รวม ไม่ต้อง เรียน weights แบบ standard fixed pooling และโดยปกติไม่เปลี่ยน จำนวน channels

ตัวอย่างคำนวณทีละขั้น

ใช้ input $\begin{bmatrix}1&3&2&0\\4&2&8&1\\5&6&7&3\\2&9&1&4\end{bmatrix}$ กับ pool 2×2, stride 2 จึงมีหน้าต่างไม่ซ้อนกัน 4 ตำแหน่ง

หน้าต่าง	Max	Average
บนซ้าย $(1,3,4,2)$	4	$(1+3+4+2)/4 = 2.5$
บนขวา $(2,0,8,1)$	8	$(2+0+8+1)/4 = 2.75$
ล่างซ้าย $(5,6,2,9)$	9	$(5+6+2+9)/4 = 5.5$
ล่างขวา $(7,3,1,4)$	7	$(7+3+1+4)/4 = 3.75$

จัดผลตามตำแหน่งเดิมได้ max-pool $\begin{bmatrix}4&8\\9&7\end{bmatrix}$ และ average-pool $\begin{bmatrix}2.5&2.75\\5.5&3.75\end{bmatrix}$



รูป 7.5: Pooling ในสไลด์: max กับ average ใช้ window เดียวกันแต่ summary ต่างกัน (L7 p.41)

อ่านภาพทีละจุด **1.** Window ของ max และ average เท่ากัน ให้เปรียบเทียบค่าที่อ่านจาก pixels เดียวกัน

2. Max เก็บ response ที่เด่น ส่วน average ใช้ทุกค่า ทั้งคู่ลด spatial size ตาม stride และคง จำนวน channels ไม่ใช้จำนวน class มากำหนด output channels ของ pool

Flatten เปลี่ยน shape ให้ spatial dimensions รวมเป็น vector แต่ไม่เพิ่ม learned parameters หาก output $6 \times 6 \times 32$ จะได้ 1152 features ต่อ Dense 64 มี $(1152 + 1)64 = 73792$ parameters จึงเห็นว่าชั้น Dense อาจครอง parameters ส่วนใหญ่

สิ่งที่เสียและได้

Downsampling ลด computation ของ layers หลังและเพิ่มความทนต่อการเลื่อนเล็กน้อยบางชนิด แต่เสีย รายละเอียดเชิง spatial การ pool แรงมากอาจทำลายข้อมูลตำแหน่งสำคัญ ไม่รับประกัน translation invariance สมบูรณ์

> **ต้องจำ** Pooling ไม่ใช่ dropout. Flatten ไม่ใช่ fully connected รูป output เปลี่ยนไม่จำเป็นต้องมี trainable parameters

7.8 Shape และ parameters: ตารางจาก input จนถึง loss

แหล่งเนื้อหา: L7 PDF p.60–62. architecture ตัวอย่างในสไลด์

กำหนด bias หนึ่งต่อ filter หรือ dense unit. Pool 2×2,stride 2,valid. dense output 10 ใช้ softmax multiclass classification

Layer	สูตร shape และ คำอธิบาย	output shape	Parameters
Input	ภาพ gray 28×28	28×28×1	0
Conv 1	40 filters,k 5,s 1,same	28×28×40	$(25 + 1)40 = 1040$
Pool 1	ลดแต่ละแกนครึ่ง	14×14×40	0
Conv 2	32 filters,k 3,s 1,valid	12×12×32	$(9 \cdot 40 + 1)32 = 11552$
Pool 2	$\lfloor (12 - 2)/2 \rfloor + 1$	6×6×32	0
Flatten	$6 \cdot 6 \cdot 32$	1152	0
Dense	64 units	64	$(1152 + 1)64 = 73792$
BatchNorm	64 features. scale และ shift และ statistics	64	$4 \cdot 64 = 256$
Output	10 classes	10	$(64 + 1)10 = 650$

Total parameters $= 1040 + 11552 + 73792 + 256 + 650 = 87290$. trainable $= 87162$, non-trainable $= 128$ จาก moving mean และ variance ของ BatchNorm ส่วน ReLU,softmax,pool,flatten,dropout ไม่มี learned weights ใน setup นี้

หาก input ขยายเป็น 32×32 โดย layers เดิม

Conv 1 แบบ same แล้ว 32,Pool แล้ว 16,Conv 2 แบบ valid แล้ว 14,Pool แล้ว 7,Flatten $7 \cdot 7 \cdot 32 = 1568$ Conv parameters คงเดิม แต่ Dense 64 เป็น $(1568+1)64 = 100416$ ทำให้ total 113914 (trainable 113786, non-trainable 128) ไม่ตอบว่า architecture เดิม parameters ทุก layer คงเดิมเพราะ convolution ที่แชร์ weights อย่างเดียว

Same padding กับ stride มากกว่า 1

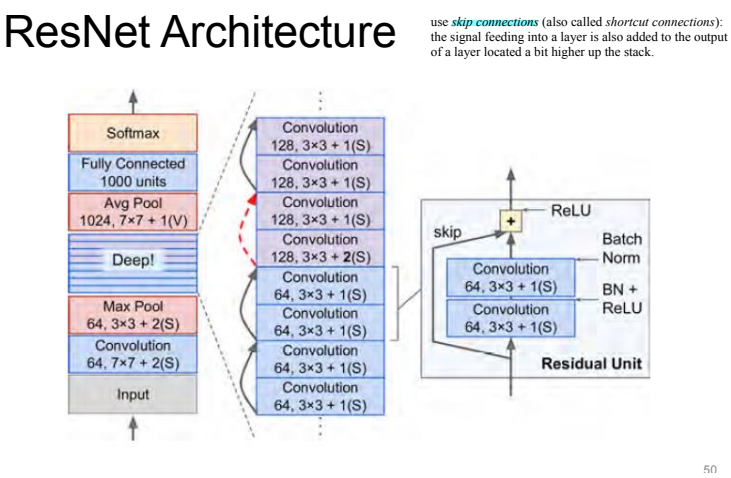
สูตรทั่วไป $n_{out} = \lfloor (n + 2p - k)/s \rfloor + 1$ ใช้เมื่อ padding ซ้าย และ ขวาเท่ากัน สำหรับ same มักตั้ง output $\lceil n/s \rceil$ และเลือก total padding $p_{tot} = \max[(n_{out} - 1)s + k - n, 0]$ ถ้า $n = 6, k = 3, s = 2$ ได้ output 3,total padding 1 จึงต้อง asymmetric padding หนึ่งด้าน ไม่แทน $p = 0.5$ เป็นจำนวน pixel จริงโดยไม่อธิบาย

> **ต้องจำ** มี input size ใหม่ต้อง track ทุก layer จน Flatten และ Dense. output shape นับ activation,parameter count นับ ค่าที่เรียนรู้ สองจำนวนนี้ไม่ใช่กัน

7.9 Architecture: LeNet, AlexNet, GoogLeNet, ResNet

แหล่งเนื้อหา: L7 PDF p.47–51

Architecture	แกนที่ควรจำตามสไลด์
LeNet-5 (1998)	conv และ pool หลายชั้นแล้ว classifier ใช้ tanh และ output แบบ RBF ใน original architecture ไม่เรียกทุก version ว่า softmax
AlexNet (2012)	ใหญ่ และ ลึกขึ้น ใช้ ReLU และการฝึกภาพขนาดใหญ่ สไลด์ระบุ output 1000 classes
GoogLeNet (2014)	Inception modules รวม branches หลาย kernel sizes และ 1×1 เพื่อลด channels และ parameters
ResNet (2015)	skip และ shortcut connections ให้ block เรียน residual $F(x)$ แล้วรวมกับ input x



รูป 7.6: ResNet เพิ่ม shortcut เพื่อส่ง representation และ gradient ข้าม block ต้องมี shapes ที่รวมกันได้ (L7 p.51)

อ่านภาพทีละจุด **1.** ตามทางยาวผ่าน block ได้ F(x) แล้วดูทาง shortcut ที่นำ x มารวมตอนท้าย

2. เครื่องหมายบวกคือ elementwise addition จึงต้อง shapes ตรงกัน หรือมี projection ที่ระบุไว้ ไม่ใช่ concat สอง tensors และไม่ใช่ average votes

Residual connection คืออะไร

$y = F(x) + x$ ไม่ใช่ average predictions ของสอง model หรือ dropout หาก shapes ไม่เท่ากันอาจใช้ projection ที่ shortcut การมี shortcut ช่วย training ของ deep network ไม่ได้แปลว่า gradient หายไม่ได้อีกต่อไป

สิ่งที่ไม่ต้องท่องโดยไม่มีเหตุ

ตารางจำนวน maps และ kernel และ stride ในสไลด์ควรอ่านเป็น การไล่ architecture ไม่ต้องนำทุกตารางตัวเลขมาเป็นข้อสอบจำใหม่ หากโจทย์ให้ sizes ให้ใช้ สูตร shape ตรวจเส้นทาง และแยก trainable layer จาก fixed operation

> **ต้องจำ** จุดเด่น Inception=หลาย scale และ branches จุดเด่น ResNet=residual และ skip connections ไม่ใช้คำว่า ensemble แทนทุก architecture ที่มี branch

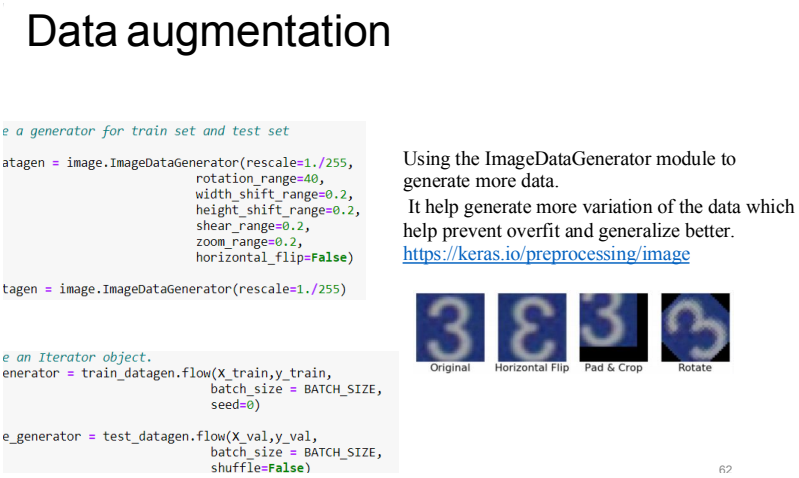
7.10 Transfer Learning, Data Augmentation และ training

แหล่งเนื้อหา: L7 PDF p.52–64

Transfer Learning reuse ความรู้จาก source task และ model เพื่อ target task ที่เกี่ยวข้อง มีสองทางในสไลด์: training source model เองแล้ว reuse หรือเลือก pretrained model แล้วเปลี่ยน classifier และ tune ให้ตรง target

Freeze กับ Fine-tune

การ freeze คือหยุดปรับพารามิเตอร์ของชั้นที่เลือก แต่ยังส่งข้อมูลผ่านชั้นเหล่านั้นใน forward pass เพื่อคำนวณ features ส่วน fine-tuning คือเปิดให้ปรับพารามิเตอร์บางชั้นด้วยข้อมูลของงานใหม่ มักเริ่มด้วย learning rate ที่เล็กกว่าการฝึกใหม่ทั้งหมด เพื่อไม่ให้เปลี่ยน representation ที่เรียนมาก่อนเร็วเกินไป Data augmentation เปลี่ยน training examples ให้มี variation ที่ label ยังถูก เช่น rotate และ shift ที่เหมาะกับ task ไม่ใช้ transformation ที่ทำให้ semantic เปลี่ยน และไม่สร้าง random augmentation ของ test เพื่อช่วยเลือก checkpoint โดยไม่กำหนด protocol



รูป 7.7: Augmentation ในสไลด์ช่วยให้ model เห็น variation แต่ validation และ test ต้องมี evaluation protocol คงที่ (L7 p.63)

อ่านภาพทีละจุด **1.** แต่ละภาพเป็นการเปลี่ยนจาก training image เดิมเพื่อให้เห็น variation โดยยังคง label

2. ตรวจ semantic ก่อนใช้ augmentation เช่นหมุนตัวเลขอาจเปลี่ยน class. pipeline validation และ test ต้องคง protocol และไม่ใช้เลือก transformation จาก testscore

ตัวอย่าง shape จาก Keras model ในสไลด์

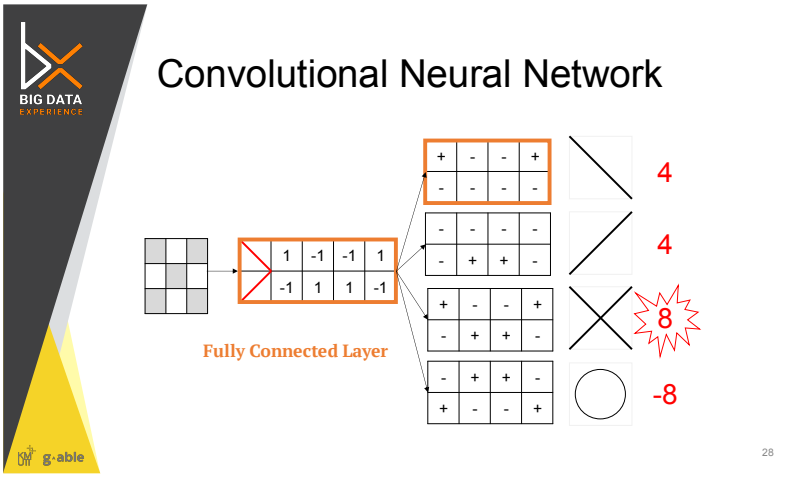
Input 28×28×1 จากนั้น conv 40, k 5, same แล้ว 28×28×40 แล้ว pool 2 แล้ว 14×14×40 แล้ว conv 32, k 3, valid แล้ว 12×12×32 แล้ว pool 2 แล้ว 6×6×32 แล้ว flatten 1152 แล้ว Dense 64 แล้ว output 10 ระบุ binary และ multiclass loss ให้ตรง output

> **ต้องจำ** Freeze ไม่หยุด forward. pretrained classifier 1000 classes ไม่ใช่ output ที่ใช้ได้ตรงกับ target 10 classes เสมอ. augmentation ไม่เพิ่ม labels ใหม่ตามใจ

อ่านเพิ่มจากตำรา ISLP §10.3 และ Géron Ch.14: visual features ชั้นต้น มัก transfer ได้ดีเมื่อ input domains สัมพันธ์กัน แต่ถ้า domain ต่างมากต้อง validate ไม่ถือว่าดีกว่า scratch เสมอ

7.11 ภาพจำแนก X และ Stride หน้า 38: เติมทุกช่องคำนวณ

แหล่งเนื้อหา: L7 PDF p.4–31,38

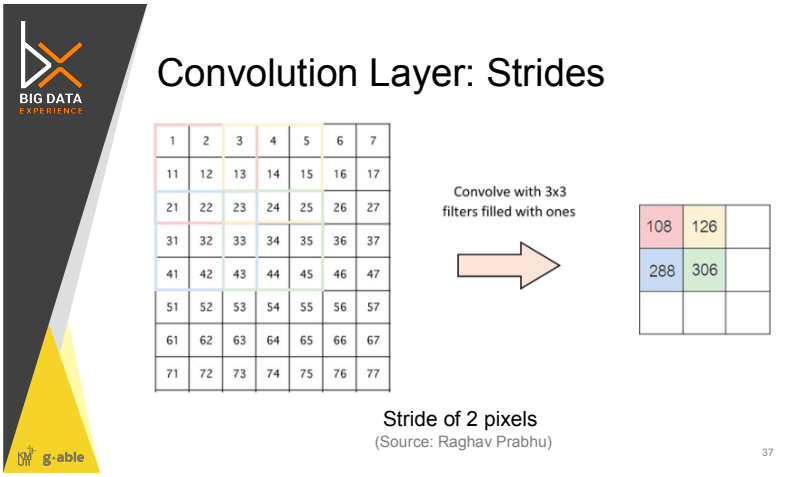


รูป 7.8: CNN pipeline ในภาพ X detector: feature maps จากนั้น รวมข้อมูล จากนั้น score

อ่านภาพทีละจุด 1. Filter ตรง pattern ให้ dot product สูง ส่วน pattern ตรงข้ามให้ค่าติดลบ เช่น checkerboard แบบสองมิติ×2 กับ filter เดียวกันได้ 4 และตรงข้ามได้−4

2. Flatten ไม่เรียน filter ใหม่. score ของ classifier ไม่เป็น probability จนระบุ output transformation เช่น sigmoid และ softmax

ภาพแรกที่ลอง weights ± 1 สี่ช่องมี $2^4 = 16$ combinations ในโจทย์สาธิต แต่ training CNN จริงเรียน real-valued weights ด้วย gradients ไม่ได้ enumerate แค่ 16 choices หาก $X = [1,-1,1;-1,1,-1;1,-1,1]$ กับ filter $[1,-1;-1,1]$ valid stride 1 ได้ feature map $[4,-4;-4,4]$ ส่วน filter ตรงข้ามได้ $[-4,4;4,-4]$ ต่อด้วย activation และ pooling ต้องระบุ window และ stride ให้ครบก่อนคำนวณขั้นถัดไป



รูป 7.9: Stride 2 หน้า PDF38: input 8×7 และ filter 3×3 ones

อ่านภาพทีละจุด 1. สี่ช่องสีเป็นตัวอย่างบาง windows ไม่ใช่ output ทั้งหมด ด้านขวาจริงวาด grid 3×3

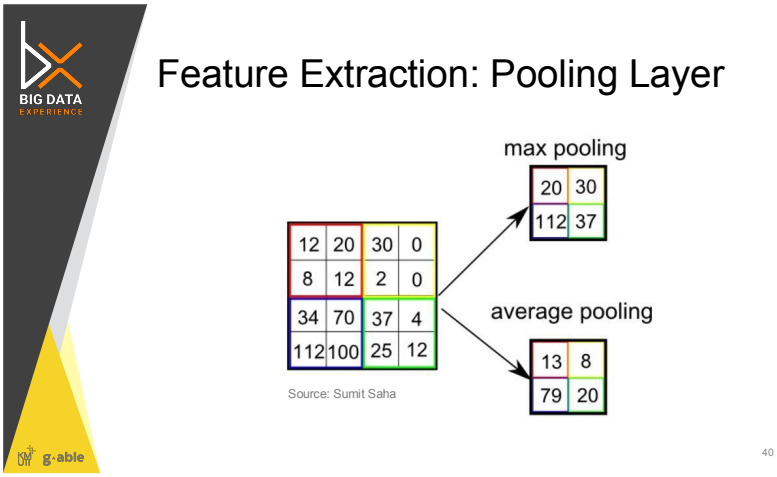
2. เลื่อนซ้ายบนของ patch ทีละ 2 ทั้งแกนนอนและตั้ง. patch ที่ยื่นเกิน input ใช้ไม่ได้เมื่อ valid

Input row $r = 0,\dots,7$ มีค่า $(10r+1,\dots,10r+7)$ จึง output shape $\lfloor (8-3)/2 \rfloor + 1 = 3$ แถว และ $\lfloor (7-3)/2 \rfloor + 1 = 3$ คอลัมน์ Top-left patch sums $1+2+3+11+12+13+21+22+23 = 108$ เลื่อนขวาสองช่องได้ 126. เลื่อนลงสองแถวเพิ่มทุกค่า 20 จึง sum เพิ่ม 180 ได้ 288

$$C = \begin{bmatrix} 108 & 126 & 144 \\ 288 & 306 & 324 \\ 468 & 486 & 504 \end{bmatrix}.$$

7.12 Pooling ต้นฉบับ: ค่าเฉลี่ยที่ปัดเศษต้องอ่านให้ถูก

แหล่งเนื้อหา: L7 PDF p.40–42



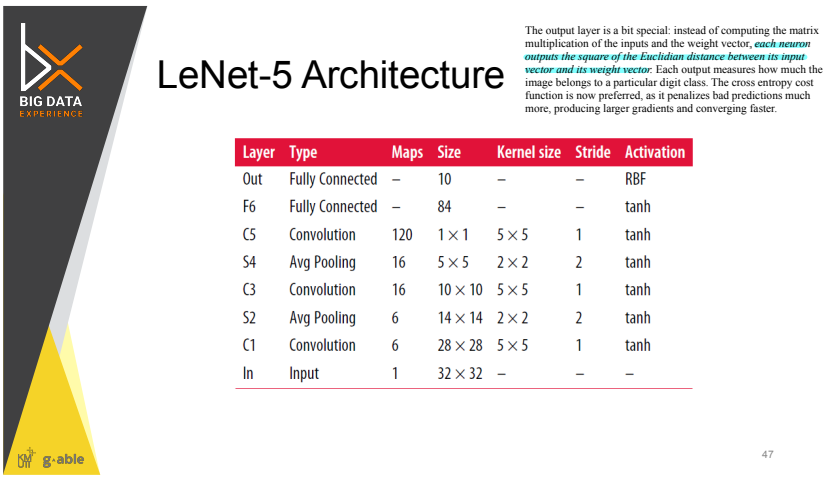
รูป 7.10: Max pool และ average pool ของ input เดียวกันในหน้า 41

อ่านภาพทีละจุด **1.** แบ่ง 2×2 ไม่ซ้อนกัน windows: max เลือกค่าสูงสุด ไม่ได้บวกทั้ง patch
2. Average pool ล่างขวาของ $(37, 4, 25, 12)$ คือ $78/4 = 19.5$. ภาพเขียน 20 เป็นค่าปัด ไม่ใช่ผล exact

ผล max $[20, 30; 112, 37]$ และ mean $[13, 8; 79, 19.5]$ ใช้ stride 2 output 2×2. เรียก 19.5 ว่า 20 ได้เมื่อระบุ rounding เท่านั้น Standard pooling ไม่มี learned weights ส่วน backprop max ส่ง gradient ตาม argmax และ average กระจาย gradient $1/4$ ต่อช่องใน window นี้

7.13 Architecture ในภาพจริง: shapes และทางเดินที่ต่างกัน

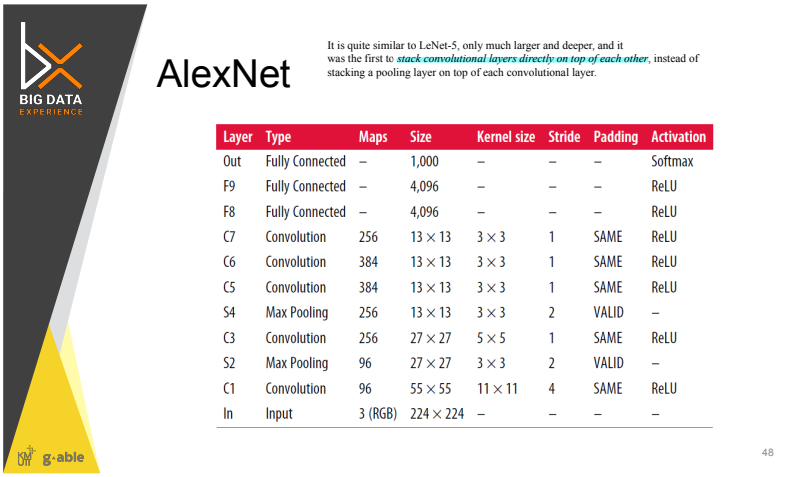
แหล่งเนื้อหา: L7 PDF p.47–51



รูป 7.11: LeNet-5 ในตารางต้นฉบับ

อ่านภาพทีละจุด **1.** ทางเดิน spatial 32 แล้ว 28 แล้ว 14 แล้ว 10 แล้ว 5 แล้ว 1 ตรงกับ conv 5 valid และ subsampling 2 ตามโครงสร้างที่แสดง
2. LeNet ประวัติศาสตร์มี activation และ subsampling และ output แบบของตน อย่าสรุปทุก architecture ใช้ ReLU กับ softmax เหมือนโมเดลฝึกใหม่

โครงสร้างของ LeNet ที่แสดงมี 6 feature maps หลัง C1,16 หลัง C3 และ 120 หลัง C5 จากนั้น 84 และ 10 outputs บางการเชื่อมต่อ และ subsampling ของ original LeNet ไม่ใช่ dense connectivity และ pooling convention ที่ใช้ในโจทย์ CNN สมัยใหม่ จึงนับ parameters ต้องอ่านรายละเอียด connectivity ก่อน

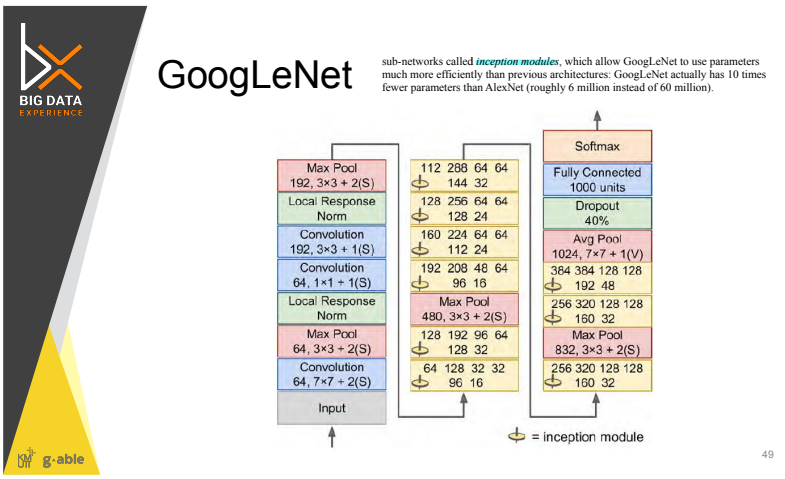


รูป 7.12: AlexNet table: ตรวจ padding เมื่ออ่าน output size

อ่านภาพทีละจุด **1.** หาก input 224,k 11,stride 4 แล้ว valid จะได้ 54. ถ้า padding 2 ทุกด้านได้ 55

2. คำว่า SAME ตาม convention output $\lceil 224/4 \rceil$ ให้ 56 จึงไม่อ้าง “224, SAME, 55” พร้อมกันโดยไม่อธิบาย padding ที่ใช้

ตัวอย่าง 227 กับ valid ได้ 55 เช่นกัน จึงแยกการย่อ architecture ในสไลด์จากเงื่อนไขคำนวณ shape ของโจทย์ ต้องใช้ $\lfloor (n + p_L + p_R - k)/s \rfloor + 1$ พร้อมค่า padding ที่กำหนดจริง



รูป 7.13: GoogLeNet และ Inception: branches ขนานรวม channel

อ่านภาพทีละจุด **1.** 1×1,3×3,5×5 และ pooling branches ดู input ในหลายสเกล. 1×1 ช่วยปรับ และ ลด channels ก่อน convolutions ที่แพงกว่า

2. Concatenate ต้องให้ spatial dimensions เข้ากันและเพิ่ม channels ต่างจาก ResNet addition ที่ shapes ต้องตรงหรือมี projection

7.14 Model summary หน้า 62: เติม Batch Normalization ที่เคยตก

แหล่งเนื้อหา: L7 PDF p.60–62

Layer (type)	Output Shape	Param #
conv1_1 (Conv2D)	(None, 28, 28, 40)	1040
max_pooling2d_1 (MaxPooling2	(None, 14, 14, 40)	0
dropout_1 (Dropout)	(None, 14, 14, 40)	0
conv1_2 (Conv2D)	(None, 12, 12, 32)	11552
max_pooling2d_2 (MaxPooling2	(None, 6, 6, 32)	0
dropout_2 (Dropout)	(None, 6, 6, 32)	0
flatten_1 (Flatten)	(None, 1152)	0
dense_1 (Dense)	(None, 64)	73792
batch_normalization_1 (Batch	(None, 64)	256
dropout_3 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 10)	650

Total params: 87,290
Trainable params: 87,162
Non-trainable params: 128

61

รูป 7.14: Summary จริงในหน้า 62: Total 87290, Trainable 87162, Non-trainable 128

อ่านภาพทีละจุด 1. แถว Batch Normalization มี 256 quantities สำหรับ 64 features: scale 64+shift 64+moving mean 64+moving variance 64

2. 128 ตัวแรก trainable,128 statistics ไม่ update ด้วย optimizer gradient แบบ weights จึงต้องแยก totals ทั้งสาม

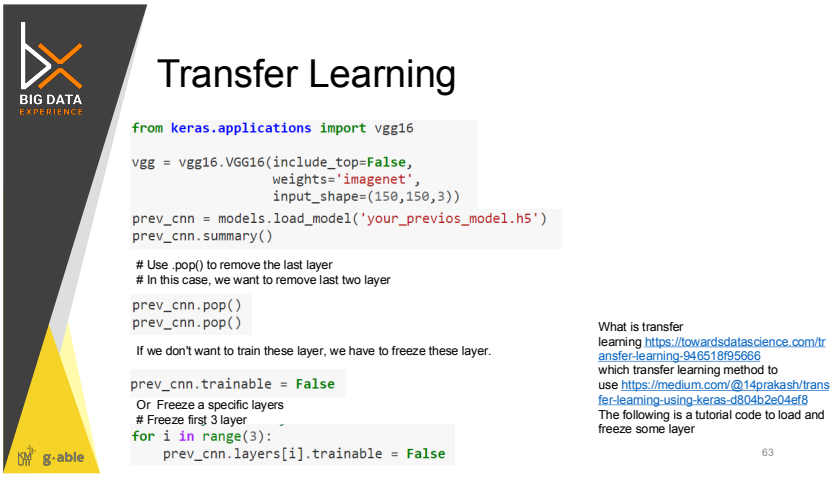
Batch norm กับ Dense 64 ทำ $\hat{a} = (a - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$ แล้ว $\gamma\hat{a} + \beta$ ระหว่าง train. running statistics ใช้สำหรับ inference ตาม setup ปกติ Dropout 3 แถวใน summary ไม่มี learned parameters แต่เปลี่ยนพฤติกรรม train และ inference ไม่ควรนับว่าไม่มีผลต่อ model เพราะ count 0

ต้องจำ จำ BatchNorm เมื่อมี affine 64 features: total 256, trainable 128, stored statistics 128. ผลรวมจริงของ architecture หน้า 62 คือ 87290 ไม่ใช่ 87034 ที่ละ BN

7.15 Develop CNN กับใช้ pretrained model: ขั้นตอนการฝึกจริง

แหล่งเนื้อหา: L7 PDF p.52–64

ขั้น	Develop CNN	Pretrained approach
1	เตรียม train และ validation และ test และ preprocessing	เลือก pretrained backbone ให้ input และ preprocessing เข้ากัน
2	กำหนด conv และ pool และ head และ output ตาม task	ตัด head เดิม เช่น include top false แล้วต่อ head ใหม่
3	สุ่ม initialization แล้ว train parameters	freeze backbone ฝึก head จากนั้นอาจ fine-tune บางชั้นด้วย learning rate เหมาะสม
4	เลือก model ด้วย validation, test เมื่อจบ	ประเมิน บน test โดย ไม่ เลือก architecture และ epoch ซ้ำจาก test



รูป 7.15: Pretrained VGG example และ shape ของภาพที่กำหนด

อ่านภาพทีละจุด **1.** input 150×150×3 ไม่ใช่ 28×28 gray ของ CNN ก่อนหน้า การใช้ backbone ไม่ทำให้ input channels เดิมใช้ได้โดยไม่ปรับ
2. include top false เอา classifier เดิมออก แต่ยังต้องจัดการ spatial feature tensor และ head ให้เข้ากัน Freeze หยุดเรียน weights ไม่ได้หยุด forward computation

Augmentation ฝึกต้องคง labels ตามสมมติฐานของงาน. random flips ไม่เหมาะกับทุกข้อมูล Validation และ test ควรใช้ preprocessing ที่กำหนดคงที่ ไม่สุ่ม distortions แล้วเปรียบเทียบคะแนนคนละเงื่อนไข การ freeze BN กับโหมด training และ inference เกี่ยวข้องกับ statistics ด้วย จึงต้องอ่าน setup ไม่สรุปว่า freeze weights หยุด moving statistics ทุกกรณี

7.16 Lecture 7: evaluation และ Checklist

แหล่งเนื้อหา: L7 PDF p.65–69. เชื่อม L1 p.82–87

CNN classifier ยังคำนวณ confusion matrix, precision และ recall เช่นเดียวกับ classifier อื่น Accuracy สูงไม่ได้รับประกันว่าจะพบ class ที่มีจำนวนน้อย ค่าผิดแต่ละแบบอาจมีต้นทุนต่างกัน จึงต้องกำหนด positive class และ threshold ก่อนคำนวณ

Checklist คำนวณ

- ทำ elementwise multiply-sum ทุก patch ได้
- Trace H, W, C ผ่าน conv และ pool และ flatten และ dense ได้
- ใช้ floor และ+1 ใน สูตร output ถูกได้
- นับ Conv parameters และ Dense parameters รวม bias ได้
- แยก kernel size และ filter count และ stride และ padding ได้
- คำนวณ max และ average pooling ได้
- อธิบาย parameter sharing และ receptive field และ feature hierarchy ได้
- แยก Inception กับ residual connection ได้
- อธิบาย freeze และ fine-tune และ augmentation และ evaluation ได้

ต้องจำ สูตรที่ควรจำสำหรับฝึก: output spatial size, Conv parameters, Dense parameters ก่อนคำนวณอ่าน padding, stride, channels และ bias convention

ข้อผิดพลาดที่พบบ่อย และวิธีตรวจคำตอบ

1. **นับ RGB เป็นตัวอย่างสามตัว:** ภาพหนึ่งภาพมี shape $H \times W \times 3$ โดย 3 คือจำนวน channels ของตัวอย่างเดิม ไม่ใช่จำนวนตัวอย่างแยกกัน ใน convolution หนึ่ง filter จึงมี weights $K_h K_w C_{in}$ ค่า เพราะ filter ต้องอ่านทุก input channel ก่อนรวมเป็น output หนึ่ง channel
2. **คิดว่าขนาดภาพกำหนดจำนวน weights ของ Conv:** ถ้า kernel 3×3 รับ 3 channels และมี 16 filters จะได้ $(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$ parameters เมื่อเพิ่มด้านภาพจาก 32 เป็น 64 จำนวนนี้ยังเป็น 448 เพราะใช้ weights ชุดเดิมซ้ำทุกตำแหน่ง สิ่งที่เพิ่มคือจำนวนตำแหน่งคำนวณและ activations
3. **คำนวณ output shape โดยไม่แทนค่า padding:** คำนวณแต่ละแกนด้วย $\left\lfloor \frac{H+2P-K}{S} \right\rfloor + 1$ ก่อน เช่น $H = 28, K = 5, S = 1$ แบบ valid มี $P = 0$ จึงได้ 24; แบบ same เมื่อ $P = 2$ ได้ 28 อย่าตัด floor ทิ้งเมื่อหารไม่ลงตัว
4. **นับ pooling เป็นชั้นที่มี weights:** max pooling และ average pooling แบบมาตรฐานใช้กติกาคงที่ ไม่ได้เรียน weights และคงจำนวน channels ไว้ เช่น $28 \times 28 \times 40$ เมื่อ pool 2×2 ด้วย stride 2 จะเป็น $14 \times 14 \times 40$
5. **ลืมคูณขนาด Flatten ตอนนับ Dense:** shape $6 \times 6 \times 32$ มี $6 \cdot 6 \cdot 32 = 1{,}152$ inputs ถ้าส่งเข้า Dense 64 units และแต่ละ unit มี bias จะมี $(1{,}152 + 1) \cdot 64 = 73{,}792$ parameters ให้คูณทุก input กับทุก unit แล้วบวก bias ต่อ unit
6. **ใช้ class ที่ตัดสินแล้วสร้าง ROC:** ROC ต้องไล่ threshold บน score หรือ probability เพื่อดูคู่ (FPR, TPR) หลายจุด ถ้าใช้ hard label ที่ตัดสินด้วย threshold เดียว จะเหลือจุดเดียวและประเมิน ranking ของโมเดลไม่ได้
7. **เปลี่ยน input แล้วไม่ไล่ shape ต่อ:** หลังแก้ขนาดภาพหรือจำนวน filters ให้เขียน shape หลังทุก Conv และ Pool จนถึง Flatten แล้วคำนวณ input ของ Dense ใหม่ ขนาดที่เปลี่ยนในชั้นต้นส่งผลต่อจำนวน Dense parameters ในชั้นท้าย

อ่านเพิ่มจากตำรา บทนี้ขยายเฉพาะ worked examples, architectures และ training ตาม Lecture ไม่รวม object detection หรือ segmentation ขั้นสูงจาก textbook ซึ่งอยู่นอกขอบเขตสไลด์

Lecture 8 Dimensionality Reduction

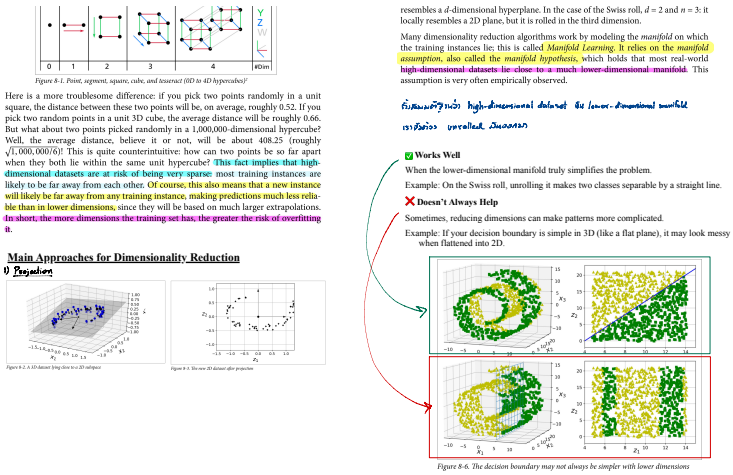
แหล่งเนื้อหา: L8 PDF p.1–8 | ขอบเขตอ่านสอบ Lecture 1–8

คำย่อที่ใช้ในบทนี้

PCA คือ Principal Component Analysis. **PC** คือ Principal Component และ **PVE** คือ Proportion of Variance Explained. **MDS** คือ Multidimensional Scaling. **LLE** คือ Locally Linear Embedding. **ISOMAP** มาจาก Isometric Mapping. SVD คือ Singular Value Decomposition ตามบทที่ 2. คำว่า component หมายถึงแกนใหม่ที่ประกอบจาก features เดิม ไม่จำเป็นต้องตรงกับ feature เดิมหนึ่งตัว. การลดมิติ (dimensionality reduction) แทนข้อมูลที่มี features จำนวนมากด้วยชุดตัวแปรที่มีจำนวนน้อยลง โดยพยายามเก็บโครงสร้างสำคัญ เช่น ความแปรปรวนหรือความสัมพันธ์ระหว่างเพื่อนบ้าน เป้าหมายอาจเป็นการแสดงภาพ ลดเวลาและหน่วยความจำ หรือตัดตัวแปรซ้ำซ้อน แต่ไม่ได้รับประกันว่าจะแยก class ได้ดีขึ้นทุกชุดข้อมูล

Curse of dimensionality

เมื่อ dimension สูงพื้นที่ที่ต้องครอบคลุมเพิ่มเร็ว ข้อมูลจำนวนเดิมจะ sparse และ distance อาจสะท้อนความคล้ายคลึงได้ไม่ดี การลด dimension เป็นหนึ่งแนวทาง แต่ข้อมูลอาจสูญเสีย signal ที่มี variance ต่ำและสำคัญต่อ target



รูป 8.1: Projection กับ Manifold learning มีเป้าหมายรักษาโครงสร้างต่างกัน. flatten Swiss roll อาจช่วยหรือทำให้ boundary ซับซ้อนกว่าเดิมได้ (L8 p.5)

อ่านภาพทีละจุด **1.** Projection ฉายลง subspace เชิงเส้น ส่วน manifold method พยายามรักษาโครงสร้างบนผิวที่โค้ง

2. Swiss roll อาจทับชั้นเมื่อฉายตรง ทำให้จุดคนละส่วนใกล้กัน. flatten ก็ไม่รับประกัน boundary ง่ายขึ้นถ้าตัว target ขึ้นกับตำแหน่งต่างแบบ

Feature selection กับ feature extraction

Selection เลือก columns เดิมบางตัว. PCA สร้าง linear combinations ของ features เดิม Component ไม่ใช่ feature เดิมที่ importance สูงที่สุดหนึ่งตัว แม้บาง loading ใหญ่ก็ตาม

> **ต้องจำ** PCA เป็น unsupervised ในขั้น fit เพราะไม่ใช้ labels แต่ใช้เป็น preprocessing ให้ supervised model ภายหลังได้

อ่านเพิ่มจากตำรา ISLP §12.2 และ Géron Ch.8: compression ที่รักษา variance ไม่จำเป็นต้องรักษาความสามารถทำนาย target ต้อง evaluate pipeline ทั้งหมด

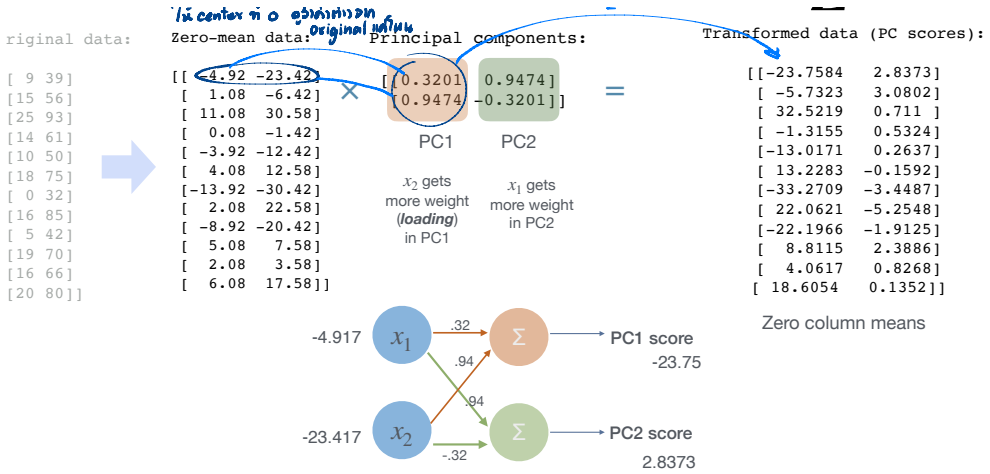
8.1 PCA: axis, score, loading และ centering

แหล่งเนื้อหา: L8 PDF p.6, 9–12, 18–19

PCA หาแกนตั้งฉากกันและเรียงแกนตามความแปรปรวนของข้อมูลหลังฉายลงบนแกนนั้น แกนแรก (PC1) เก็บความแปรปรวนได้มากที่สุด แกนที่สอง (PC2) เก็บได้มากที่สุดภายใต้เงื่อนไขว่าต้องตั้งฉากกับ PC1 และทำเช่นนี้ต่อไปสำหรับแกนถัดมา ให้ $X \in \mathbb{R}^{n\times d}$ เป็นข้อมูลดิบที่มี n แถวและ d ตัวแปร, μ เป็นเวกเตอร์ค่าเฉลี่ยรายคอลัมน์, และ $X_c = X - \mathbf{1}\mu^T$ เป็นข้อมูลที่ลบค่าเฉลี่ยแล้ว เมทริกซ์ $P = [p_1 \cdots p_d]$ รวบรวมแกนหลักที่มีความยาวหนึ่งหน่วยไว้เป็นคอลัมน์ เมื่อนำข้อมูลที่ center แล้วคูณกับ P จะได้พิกัดใหม่:

$$Z = X_c P, \qquad z_{ij} = x_{c,i}^T p_j.$$

Loading คือ coefficient ของ direction vector. Score คือ coordinate ของ observation บน direction นั้น ไม่ใช่ทั้งคู่เรียกว่า eigenvalue



รูป 8.2: ตัวอย่างใน Lecture 8 คูณข้อมูลที่ center แล้วด้วย loading matrix P เพื่อหา scores (L8 p.10)

อ่านภาพทีละจุด **1.** ลบค่าเฉลี่ย μ ออกจากข้อมูล X ก่อน แล้วคูณด้วยเมทริกซ์ loading P ผลลัพธ์คือ scores ของ observations **2.** ถ้า $X - \mu$ มีขนาด $n \times d$ และเก็บแกนไว้ r แกน จะได้ P ขนาด $d \times r$ และ score matrix ขนาด $n \times r$ ทิศของ eigenvector และ score สามารถกลับเครื่องหมายพร้อมกันได้โดยไม่เปลี่ยนแกนที่แทน

ตัวอย่างคำนวณทีละขั้น

ให้ $x = (3, 5)$, training mean $\mu = (1, 1)$ และ unit PC1 $p_1 = (1, 1)^T/\sqrt{2}$

1. Center ก่อน: $x_c = x - \mu = (3 - 1, 5 - 1) = (2, 4)$
2. Score คือ $z_1 = x_c p_1 = 2/\sqrt{2} + 4/\sqrt{2} = 6/\sqrt{2} = 3\sqrt{2}$
3. Projection ใน centered space: $z_1 p_1^T = (3\sqrt{2})(1, 1)/\sqrt{2} = (3, 3)$

ค่านี้ยังไม่ใช่ reconstruction ในหน่วยข้อมูลเดิม ต้องบวก μ กลับในหัวข้อ reconstruction

> **ต้องจำ** ก่อนทำ PCA ให้ลบค่าเฉลี่ยของแต่ละ feature ออกจากข้อมูล (center) ตามนิยาม covariance ที่ใช้ในบทนี้ Principal Component (PC) เป็นเวกเตอร์หน่วย ส่วน score อาจเป็นบวกหรือลบได้ หากกลับเครื่องหมายของ PC พร้อมกับ score ตำแหน่งเชิงเรขาคณิตที่แทนจะไม่เปลี่ยน

ระวัง: ใน L8 p.9–10 อาจเห็นเครื่องหมายของ PC ต่างกัน ทั้ง p และ $-p$ เป็นแกนเดียวกัน จึงไม่ถือว่าเป็นคำตอบ PCA คนละชุด

8.2 Covariance matrix และ eigendecomposition

แหล่งเนื้อหา: L8 PDF p.13–18

เมทริกซ์ covariance ของข้อมูลที่ center แล้วคำนวณจากผลคูณ $X_c^T X_c$ โดยหารด้วย $n-1$ เมื่อใช้ sample covariance ในสูตร Σ_{jk} คือ covariance ระหว่างตัวแปรคอลัมน์ที่ j และ k ส่วน $\bar{x}_j, \bar{x}_k$ คือค่าเฉลี่ยของสองคอลัมน์นั้น:

$$\Sigma = \frac{1}{n-1} X_c^T X_c, \quad \Sigma_{jk} = \frac{1}{n-1} \sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k).$$

Diagonal คือ variance ของ feature. off-diagonal คือ covariance เมื่อตัวแปรเพิ่ม และ ลดร่วมกัน Covariance positive แสดง linear co-variation ไม่ใช่ causal relationship และ covariance 0 ไม่รับประกัน independence ทุก distribution

$$\Sigma p_j = \lambda_j p_j, \quad \lambda_1 \geq \cdots \geq \lambda_d \geq 0, \quad P^T \Sigma P = \text{diag}(\lambda_1, \ldots, \lambda_d).$$

Eigenvectors เป็น directions. Eigenvalues เป็น variance ของ PC scores และ sum eigenvalues เท่ากับ total variance ของ ข้อมูลเดิม เมื่อใช้ทุก components

ทำไมแกน PCA จึงเป็น eigenvector

ให้ p เป็น column vector ที่มี $p^T p = 1$ แล้ว scores ของ centered data คือ $z = X_c p$ มี mean ศูนย์ ดังนั้น

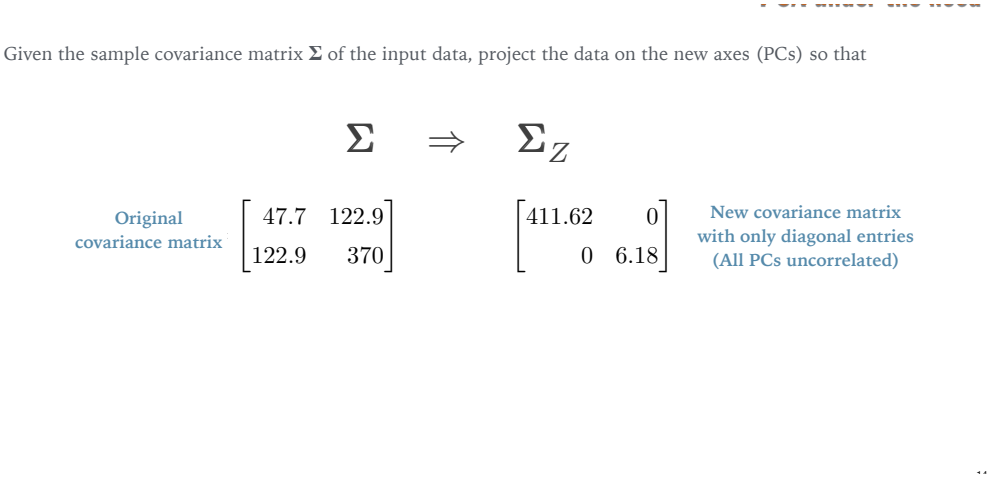
$$\begin{aligned} \text{Var}(z) &= \frac{z^T z}{n-1} = \frac{p^T X_c^T X_c p}{n-1} = p^T \Sigma p, \\ \mathcal{L}(p, \lambda) &= p^T \Sigma p - \lambda(p^T p - 1), \\ \nabla_p \mathcal{L} &= 2\Sigma p - 2\lambda p = 0 \quad \Rightarrow \quad \Sigma p = \lambda p, \\ p^T \Sigma p &= p^T(\lambda p) = \lambda(p^T p) = \lambda. \end{aligned}$$

จึงเลือก unit eigenvector ของ eigenvalue ที่มากที่สุดเป็น PC1 เพราะ projected variance เท่ากับ eigenvalue นั้น แกนถัดไปเลือกให้ตั้งฉากกับแกนที่เลือกแล้วและเรียง eigenvalues จากมากไปน้อย

ตัวอย่างคำนวณทีละขั้น

ให้ $\Sigma = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

1. $\det(\Sigma - \lambda I) = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1) = 0$ จึง eigenvalues 3 และ 1
2. สำหรับ $\lambda = 3$ ได้ $-p_x + p_y = 0$ จึง direction $(1, 1)$ หาร norm $\sqrt{1^2 + 1^2} = \sqrt{2}$ ได้ $p_1 = (1, 1)^T / \sqrt{2}$
3. สำหรับ $\lambda = 1$ ได้ $p_x + p_y = 0$ จึง $p_2 = (1, -1)^T / \sqrt{2}$
4. ตรวจ $p_1^T p_2 = (1 - 1)/2 = 0$ และ total variance $= 3 + 1 = 4$



รูป 8.3: หลังเปลี่ยน basis covariance ของ PC scores เป็น diagonal จึง uncorrelated ตาม variance ที่เรียง (L8 p.16)
อ่านภาพทีละจุด **1.** Diagonal เดิมเป็น feature variances และ off-diagonal เป็น covariances หลัง rotate แกน PC จะได้ diagonal
2. ค่าบน diagonal ใหม่คือ eigenvalues ตาม variance ของ scores ไม่ใช่ทุกค่าต้อง 1. decorrelation ไม่ใช่ proof ของ independence

> **ต้องจำ** PCA หมุนแกน เพื่อ decorrelate และ order variance ไม่ได้บังคับ variance ทุก component เป็น 1 การทำ whitening เป็นอีกขั้นหนึ่ง

ระวัง: L8 p.14 ตาราง covariance มี index ซ้ำหนึ่งช่องและ p.17 พิมพ์ label λ_1 ซ้ำสำหรับค่าที่สอง ควรอ่านเป็น symmetric covariance และ λ_2 ตาม matrix ไม่จำ typo

8.3 Proportion of Variance Explained และเลือก r

แหล่งเนื้อหา: L8 PDF p.11, 17, 21, 23–24

$$\text{PVE}_j = \frac{\lambda_j}{\sum_{k=1}^{d} \lambda_k}, \qquad \text{PVE}_{1:r} = \frac{\sum_{j=1}^{r} \lambda_j}{\sum_{k=1}^{d} \lambda_k}.$$

โดย λ_j คือ eigenvalue ของ principal component ตัวที่ j, d คือจำนวน components ทั้งหมด และ r คือจำนวนแกนที่เก็บไว้ สูตรแรกบอกสัดส่วนความแปรปรวนของแกนหนึ่ง ส่วนสูตรที่สองรวมสัดส่วนตั้งแต่ PC1 ถึง PCr หากกำหนด target variance 95% เลือก r น้อยสุดที่ cumulative PVE ถึง 0.95 ไม่ใช่เลือก components ที่แต่ละตัวมี PVE>0.95 และไม่เอา PVE ของ retained components normalize ใหม่แล้วอ้างว่ารักษาทั้งหมด

ตัวอย่างคำนวณทีละขั้น

ให้ eigenvalues เรียงจากมากไปน้อยเป็น $(8, 3, 1)$

1. Total variance $= 8 + 3 + 1 = 12$
2. เก็บ PC1: $PVE_1 = 8/12 = 2/3 \approx 0.666667$ หรือ 66.67%
3. เก็บสองแกน: $PVE_2 = (8 + 3)/12 = 11/12 \approx 0.916667$ หรือ 91.67%
4. เก็บครบสามแกน: $PVE_3 = 12/12 = 1$ ดังนั้นเกณฑ์อย่างน้อย 90% ใช้ 2 components แต่เกณฑ์ 95% ต้องใช้ 3

ตัวอย่าง L8 ใช้ eigenvalues ประมาณ 411.6 กับ 6.18 จึงเก็บ PC1 ได้ประมาณ 98.52% variance (ค่ารวมจาก rounding อาจต่างเล็กน้อย) แต่ไม่ได้แปลว่ารักษา 98.52%classification accuracy

Scaling เปลี่ยนผล PCA

Covariance PCA ให้ feature variance มากมีอิทธิพลมาก หากความต่างมาจากหน่วยวัดควรพิจารณา standardization ก่อน fit แต่ถ้า variance เป็น signal จริงการ standardize อาจเปลี่ยนความหมาย ต้องตัดสินตามบริบทและทำบน train only

ต้องจำ Explained variance เป็นสัดส่วน variation ใน X ไม่ใช่ variance ของ target, probability หรือ accuracy

ฝึกให้ทำได้ คำนวณ PVE เลือกจำนวน components อธิบายข้อแลกเปลี่ยนระหว่างการบีบอัดข้อมูลกับข้อมูลที่สูญเสีย และแยก PVE ออกจากคุณภาพการทำนายได้

8.4 PCA scaling: หน่วยวัดทำให้แกนเปลี่ยนได้

แหล่งเนื้อหา: L8 PDF p.11–18. intuition ฝึกเสริม

ถ้า covariance $\left[\begin{smallmatrix}100 & 0\\ 0 & 1\end{smallmatrix}\right]$ ให้ PC1 $(1,0)$,PVE $100/101 \approx 99.01\%$ หาก feature 1 variance สูงเพราะวัดหน่วย cm และ feature 2 หน่วย m มาก variance อาจสะท้อนหน่วยแทน signal เมื่อ standardize เป็น variance 1 ทั้งคู่และยัง uncorrelated จะได้ identity matrix และ eigenvalues $(1,1)$ ไม่มี PC1 ที่ unique Standardization ต้อง หา mean และ standard deviation จาก train เท่านั้น แล้วใช้ค่าเดิมกับ validation และ test หาก สัญญาณที่ช่วยแยก class อยู่ feature 2 ที่ variance ต่ำ การเก็บ PC1 ของข้อมูลก่อน standardize อาจทิ้ง signal ทั้งหมด แม้ PVE99%

ต้องจำ PVE สูงไม่รับประกัน classification ดี และ eigenvalue ซ้ำทำให้ basis ภายใน subspace นั้นเลือกได้หลายแบบ

8.5 Reconstruction และตัวอย่าง PCA ครบวงจร

แหล่งเนื้อหา: L8 PDF p.19–23

เมื่อเก็บ r directions $P_r \in \mathbb{R}^{d\times r}$:

$$Z_r = X_c P_r, \qquad \hat{X}_c = Z_r P_r^T, \qquad \hat{X} = Z_r P_r^T + \mathbf{1}\mu^T.$$

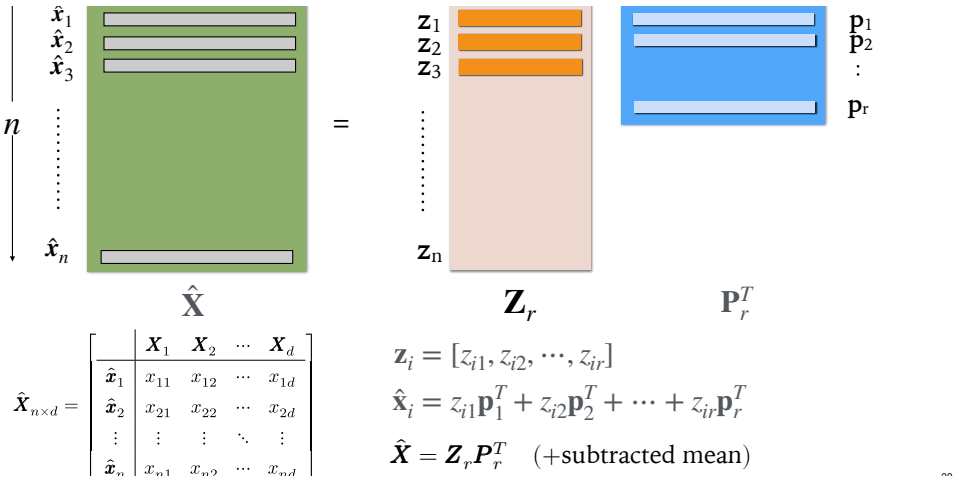
ถ้าเก็บทุก components และ orthonormal basis reconstruction คืน ข้อมูลที่ center แล้ว ได้ครบ ถ้าทิ้ง components ได้ projection โดยสูญเสีย variation ตั้งฉากกับ subspace ที่เก็บไว้ ห้ามลืม mean เมื่อคืน coordinates ในข้อมูลเดิม

ตัวอย่างคำนวณทีละขั้น

ใช้ $x=(3,5)$, $\mu=(1,1)$, $p_1=(1,1)^T/\sqrt{2}$ จากตัวอย่าง projection

1. $x_c=(2,4)$ และ $z_1=(2+4)/\sqrt{2}=3\sqrt{2}$
2. Centered reconstruction $\hat{x}_c=z_1p_1^T=(3,3)$
3. คืน mean: $\hat{x}=\mu+\hat{x}_c=(1,1)+(3,3)=(4,4)$
4. Residual $e=x-\hat{x}=(3-4,5-4)=(-1,1)$ จึง squared error $\|e\|_2^2=(-1)^2+1^2=2$
5. ตรวจ orthogonality: $ep_1=(-1+1)/\sqrt{2}=0$ ดังนั้นส่วนที่ทิ้งตั้งฉากกับแกนที่เก็บ

หากเก็บ PC2 เพิ่มด้วย score $z_2=(2-4)/\sqrt{2}=-\sqrt{2}$ ส่วนเพิ่มเป็น $z_2p_2^T=(-1,1)$ จึงคืน centered data $(3,3)+(-1,1)=(2,4)$ และคืน raw data $(3,5)$ ได้ exact



รูป 8.4: Reduced scores คูณ P_r^T กลับเป็น dimension เดิม แต่ข้อมูลที่ถูกทิ้งไม่ได้ถูกสร้างคืนอย่าง exact (L8 p.22)

อ่านภาพทีละจุด **1.** Reduced scores มีเพียง components ที่เก็บ นำคูณ transpose ของ retained directions แล้วคืน mean **2.** Output กลับมี dfeatures แต่เป็น projection approximation ข้อมูลที่ทิ้งไม่ได้กลับมา exact และ residual ตั้งฉาก subspace ที่เก็บ

> **ต้องจำ** Reconstruction กลับ dimension เดิมไม่แปลว่าเก็บ information เดิมทั้งหมด. transpose ใช้เพราะ columns ของ P orthonormal ไม่ใช่ inverse ของ matrix รูปสี่เหลี่ยมผืนผ้า ทั่วไป

อ่านเพิ่มจากตำรา ISLP §12.2: PCA ให้ rank-r approximation ที่ลด squared reconstruction error ในเงื่อนไข centered linear orthogonal projection จึงเชื่อม variance สูงสุด กับ projection error ต่ำสุด

8.6 PCA จากข้อมูลดิบ: covariance, eigenvectors, scores และ reconstruction

แหล่งเนื้อหา: L8 PDF p.9–23. ชุดตัวเลขฝึกใหม่ครบวงจร

มี 4 observations สอง features $X = \begin{bmatrix} 4 & 3 \\ 0 & 1 \\ 3 & 4 \\ 1 & 0 \end{bmatrix}$ ต้องลดเหลือ 1 component ใช้ sample covariance ตัวส่วน $n-1=3$ และไม่ standardize ในโจทย์นี้

1. หา mean และ center ทุกแถว

$\mu = (2, 2)$ จึง $X_c = \begin{bmatrix} 2 & 1 \\ -2 & -1 \\ 1 & 2 \\ -1 & -2 \end{bmatrix}$ แต่ละ column รวม 0 ตรวจได้ก่อนทำขั้นต่อไป

คน	x_{c1}	x_{c2}	x_{c1}^2	x_{c2}^2	$x_{c1}x_{c2}$
1	2	1	4	1	2
2	-2	-1	4	1	2
3	1	2	1	4	2
4	-1	-2	1	4	2
รวม	0	0	10	10	8

$$\Sigma = \tfrac{1}{3}X_c^T X_c = \begin{bmatrix} 10/3 & 8/3 \\ 8/3 & 10/3 \end{bmatrix}.$$

2. หา eigenvalues ด้วย characteristic polynomial

$$\det(\Sigma - \lambda I) = (10/3 - \lambda)^2 - (8/3)^2 = 0,$$
$$\lambda^2 - \frac{20}{3}\lambda + \frac{100 - 64}{9} = \lambda^2 - \frac{20}{3}\lambda + 4 = 0,$$
$$(\lambda - 6)(\lambda - 2/3) = 0 \quad \Rightarrow \quad \lambda_1 = 6,\ \lambda_2 = 2/3.$$

สำหรับ λ_1 สมการ $(-8/3)p_x + (8/3)p_y = 0$ ให้ $p_x = p_y$ normalize ด้วย $p_x^2 + p_y^2 = 1$ ได้ $p_1 = (1,1)/\sqrt{2}$ ส่วน $p_2 = (1,-1)/\sqrt{2}$ eigenvectors ตั้งฉากเพราะ dot product 0. ตรวจ $\Sigma p_1 = 6p_1$ และ $\Sigma p_2 = (2/3)p_2$

3. Project ทุก observation และตรวจ variance ของ scores

คน	$z_1 = (x_{c1} + x_{c2})/\sqrt{2}$	$z_2 = (x_{c1} - x_{c2})/\sqrt{2}$	z_1^2	z_2^2
1	$3/\sqrt{2}$	$1/\sqrt{2}$	4.5	0.5
2	$-3/\sqrt{2}$	$-1/\sqrt{2}$	4.5	0.5
3	$3/\sqrt{2}$	$-1/\sqrt{2}$	4.5	0.5
4	$-3/\sqrt{2}$	$1/\sqrt{2}$	4.5	0.5
รวม	0	0	18	2

Sample variance ของ $z_1 = 18/3 = 6$ ตรง λ_1. $z_2 = 2/3$. covariance $\sum z_1 z_2/3 = 0$ PCA ทำให้ uncorrelated แต่ไม่ได้พิสูจน์ independent ทุก distribution

4. เลือกจำนวน components จาก PVE

Total variance $= 6 + 2/3 = 20/3$. PVE1 $= 6/(20/3) = 0.9$ ถ้าต้องอย่างน้อย 90%เลือก 1 component ได้ ถ้าต้อง 95%ต้อง 2 อย่าปัด 0.9 เป็น 1 และบอกไม่เสีย information

5. Reconstruct และหาความผิดทุกแถว

เก็บ p_1 อย่างเดียว $\hat{x} = \mu + z_1 p_1^T$ คน 1 มี $z_1 p_1^T = (1.5, 1.5)$ คืน mean ได้ $(3.5, 3.5)$

คน	Original	Reconstruction	Residual $x - \hat{x}$	Squared error
1	(4,3)	(3.5,3.5)	(0.5,-0.5)	0.5
2	(0,1)	(0.5,0.5)	(-0.5,0.5)	0.5
3	(3,4)	(3.5,3.5)	(-0.5,0.5)	0.5
4	(1,0)	(0.5,0.5)	(0.5,-0.5)	0.5

ผลรวม squared reconstruction error 2 ตรงกับ $(n-1)\lambda_2 = 3(2/3)$ ที่ทิ้งไป แต่ mean squared error ต่อ scalar feature $= 2/(4 \cdot 2) = 0.25$ ต่างจาก error ต่อ observation $= 2/4 = 0.5$ ต้องระบุ denominator ก่อนตอบ

6. ตรวจ geometry และ การเลือกเครื่องหมายได้สองแบบ

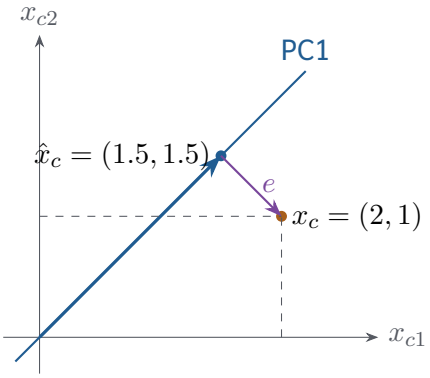
คน 1 residual $(0.5, -0.5)$ dot $p_1 = 0$ แสดงว่าความผิดตั้งฉาก แกนที่เก็บไว้ ถ้าใช้ $-p_1$ score ก็เป็น $-z_1$ และผลคูณ $(-z_1)(-p_1)^T$ เหมือนเดิม ไม่หักคะแนน eigenvector อีก sign เมื่อทุกขั้นสอดคล้อง

ต้องจำ PCA ครบขั้น: mean จากนั้น center จากนั้น covariance จากนั้น eigenpairs จากนั้น normalize และ order จากนั้น scores จากนั้น PVE จากนั้น reconstruct+mean จากนั้น residual. dimension ทุกขั้นและ denominator ต้องตรง

8.7 ชี้ภาพ PCA ด้วย projection และ residual จริง

แหล่งเนื้อหา: L8 PDF p.18–23. แผนภาพวาดใหม่จากตัวอย่างฝึก

ภาพต่อไปใช้ coordinates หลัง center ของคน 1 $x_c = (2, 1)$ โปรเจกต์บน PC1 ตามเส้น $x_2 = x_1$ ได้ $(1.5, 1.5)$ ส่วนที่เสียเป็น residual $(0.5, -0.5)$



รูป 8.5: ลูกศรน้ำเงินคือส่วนที่เก็บ สีม่วงคือส่วนที่ทิ้งซึ่งตั้งฉาก PC1. ทั้งภาพยังไม่บวก mean กลับ

1. อ่านจุดสีส้มเป็น observation เดิมหลัง center ไม่ใช่ loading
2. เส้นทแยง PC1 เป็น direction ส่วนระยะ signed ตามแกนนี้คือ score $3/\sqrt{2}$
3. จุดน้ำเงินเป็น projection ไม่ใช่ observation อีกคน และไม่จำเป็นอยู่ตรง sample จริง
4. คืน coordinates ในข้อมูลเดิม ต้องบวก mean $(2, 2)$ ให้ทั้ง projection ไม่เลื่อนเพียงแกนเดียว

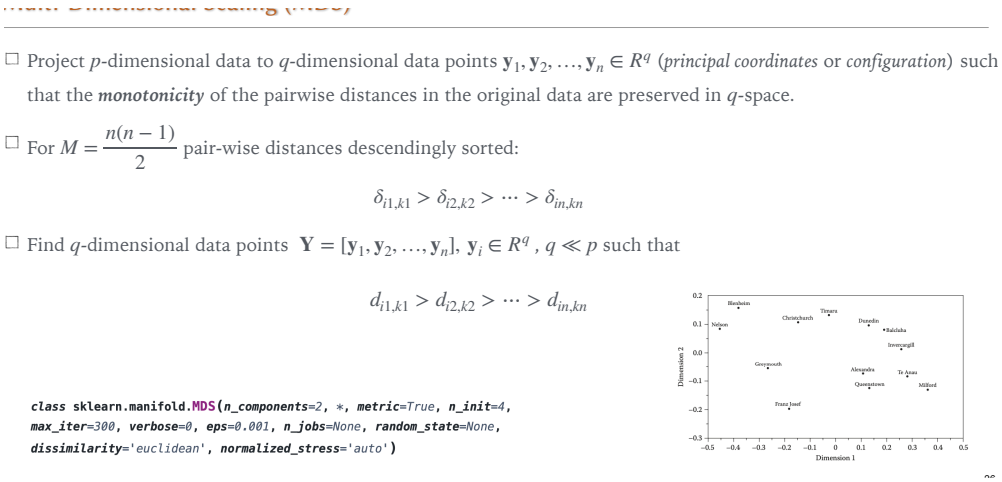
8.8 Manifold, distance และ MDS

แหล่งเนื้อหา: L8 PDF p.25–31

Manifold hypothesis มองว่าข้อมูล high-dimensional อาจอยู่ใกล้ low-dimensional structure ที่โค้ง เช่น Swiss roll การ project เชิงเส้นทับชั้นของ roll ทำให้คนละตำแหน่งบนผิวกลายเป็นใกล้กัน ต้องรักษา distance ที่เหมาะกับ structure

$$d_{Euc}(a, b) = \sqrt{\sum_k (a_k - b_k)^2}, \qquad d_{Man}(a, b) = \sum_k |a_k - b_k|.$$

Proximity matrix เก็บ pairwise dissimilarities ไม่จำเป็นต้องมี features ดิบเสมอ MDS หา coordinates มิติต่ำที่ให้ pairwise distances หรือ order ของ distances สอดคล้อง original โดย metricMDS กับ nonmetric MDS มี objective ต่างกัน



รูป 8.6: MDS เริ่มจาก distance และ dissimilarity แล้วหา configuration มิติต่ำ สไลด์เน้น monotonic relationship ของ distance (L8 p.28)

อ่านภาพทีละจุด **1.** Input ของ MDS เป็น pairwise distances และ dissimilarities ระหว่าง objects ไม่ต้องตีความแต่ละช่องเป็น feature

2. Output เป็น coordinates ที่รักษาระยะหรือ order ตามชนิด MDS Translation และ rotation และ reflection ที่ไม่เปลี่ยนระยะ ไม่ทำให้ embedding ผิด

ตัวอย่างคำนวณทีละขั้น

ใช้จุด $a = (1, 2)$ และ $b = (4, 6)$

1. ผลต่างรายแกน $a - b = (1 - 4, 2 - 6) = (-3, -4)$
2. Euclidean distance $\sqrt{(-3)^2 + (-4)^2} = \sqrt{9 + 16} = 5$
3. Manhattan distance $|-3| + |-4| = 3 + 4 = 7$

ตัวเลขต่างกันเพราะนิยาม geometry ต่างกัน ต้องเลือกตามวิธีที่โจทย์กำหนด

> **ต้องจำ** Embedding เป็น coordinates ใหม่ไม่ใช่ class labels. proximity matrix มี $n \times n$ ไม่ใช่ $n \times d$ features และ diagonal เป็น 0 สำหรับ distance ปกติ

ระวัง: สไลด์ MDS กล่าว preserve order สอดคล้อง nonmetric แนวคิด แต่ code parameter ในบางเวอร์ชันอาจใช้ metricMDS ต้องแยกแนวคิดไม่เปลี่ยนเป็น claim ว่า MDS ทุกแบบรักษาระยะ exact

8.9 MDS แบบสามจุด: distance matrix กับ embedding ไม่ใช่ features เดิม

แหล่งเนื้อหา: L8 PDF p.25–31. ตัวอย่างฝึกใหม่

ให้ $D = \begin{bmatrix} 0 & 1 & 3 \\ 1 & 0 & 2 \\ 3 & 2 & 0 \end{bmatrix}$ สำหรับ A,B,C วางหนึ่งมิติ $Y = (0, 1, 3)^T$ ได้ระยะ $|0-1| = 1$, $|0-3| = 3$, $|1-3| = 2$ ตรง D ทุกคู่พอดี ตัวอย่างนี้ทำได้เพราะระยะอยู่บนเส้นหนึ่งมิติอย่างสอดคล้อง ไม่ใช่ distance ทุก matrix สามารถฝังในหนึ่งมิติ ได้ exact

Coordinates ไม่ unique แต่ distances เหมือนเดิม

เลื่อนให้ centered ได้ $Y' = (-4/3, -1/3, 5/3)$. reflect เป็น $-Y'$ ก็ให้ distances เดิม Translation และ reflection จึงไม่ใช่ความผิดเมื่อ objective อาศัย distance ถ้า D ของสามคู่เท่ากับ 1 ทั้งหมด ไม่มีสามจุด distinct บนเส้นที่ pairwise distance ทั้งหมด 1 ได้ แต่ทำเป็น สามเหลี่ยมด้านเท่าในสองมิติ ได้

ต้องจำ MDS รักษา distances หรือ order ตามชนิด method แกน embedding ไม่ใช่ feature เดิมและ position ไม่ unique ใน sense ที่ไม่เปลี่ยน pairwise distance

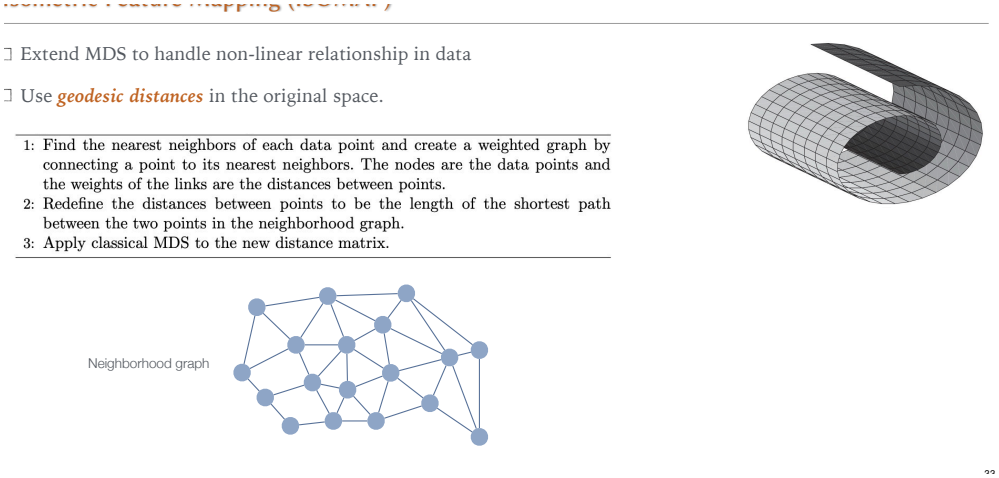
8.10 LLE และ ISOMAP: local structure กับ geodesic

แหล่งเนื้อหา: L8 PDF p.32–37

LLE หา neighbors ของแต่ละ point แล้ว approximatepoint เป็น weighted combination ของ neighbors โดย $\sum_j w_{ij} = 1$ และ weights ของจุดที่ไม่เป็น neighbors เป็น 0 จากนั้นหา embedding ที่รักษา weights เดิม:

$$\min_W \sum_i \left\| x_i - \sum_{j \in N(i)} w_{ij} x_j \right\|^2, \qquad \min_Y \sum_i \left\| y_i - \sum_j w_{ij} y_j \right\|^2.$$

ต้องมี constraints เพื่อกัน embedding ที่ยุบเป็นคำตอบเสื่อม เช่น coordinates หลัง center และ normalize covariance โดยเป็น Explanation เสริมจาก standard LLE ไม่อ้างว่าสอง objective เปล่าเพียงพอจะได้ embedding ที่มีความหมาย ISOMAP สร้าง neighborhood graph, ใช้ shortest-path distance ประมาณ geodesic บน manifold แล้วทำ MDS กับ distance นั้น ต่างจาก Euclidean distance ที่ลัดข้ามชั้น roll



รูป 8.7: ISOMAP ใช้เส้นทางใน neighbor graph เป็น distance ก่อน MDS (L8 p.35)

อ่านภาพทีละจุด **1.** จุดเชื่อมกับ neighbors เป็น graph แล้วระยะระหว่างจุดไกลคำนวณผ่าน shortest paths

2. Path length ประมาณ geodesic บน manifold ก่อน MDS ถ้า graph มี shortcut ข้าม fold ระยะหลายคู่ผิดได้ ไม่ใช่เพียงเส้นที่เพิ่มผิดเส้นเดียว

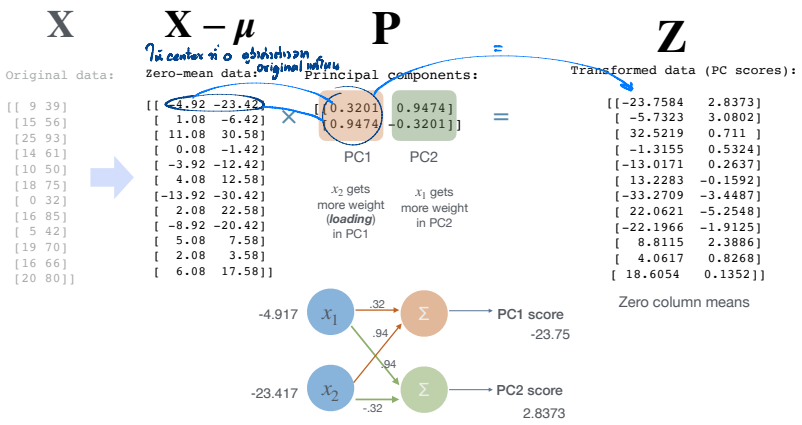
Effect ของ จำนวน neighbors

น้อยเกิน graph อาจ disconnected และ local estimates ไม่นิ่ง มากเกินเชื่อมจุดข้าม fold และทำให้ manifold structure ผิด จึงต้องดู geometry และ evaluate embedding ไม่เลือก neighbor count เพราะมีใน code ตัวอย่าง

ต้องจำ PCA=variance สูงสุด เชิงเส้น. MDS=distance configuration. LLE=local reconstruction weights. ISOMAP=geodesic distance ผ่าน graph

8.11 PCA ต้นฉบับ 12 แถว: คำนวณใหม่จาก raw data

แหล่งเนื้อหา: L8 PDF p.9–19



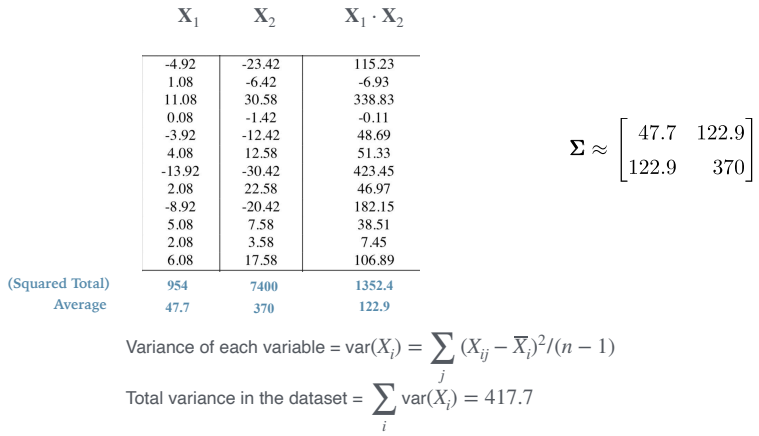
รูป 8.8: ข้อมูล 12 ตัวอย่างและ principal axes ในหน้า 10

อ่านภาพทีละจุด 1. อ่าน pair ของแต่ละแถวตามลำดับเดิมแล้ว center ด้วย training mean ไม่ใช้เลข eigenvalues ย้อนสร้างข้อมูลใหม่

2. Eigenvectors มี sign ambiguity: กลับเครื่องหมาย axis และ scores พร้อมกัน reconstruction เดิม แต่ห้ามกลับ axis แล้วเก็บ scores เก่า

Training mean $\bar{x} = (13.916667, 62.416667)$ และ centered cross-products ที่คำนวณจากข้อมูล 12 แถวจริงเป็น

$$X_c^T X_c = \begin{bmatrix} 524.916667 & 1352.416667 \\ 1352.416667 & 4070.916667 \end{bmatrix}, \quad \Sigma = \frac{X_c^T X_c}{11} = \begin{bmatrix} 47.719697 & 122.946970 \\ 122.946970 & 370.083333 \end{bmatrix}.$$



X_1	X_2	$X_1 \cdot X_2$
-4.92	-23.42	115.23
1.08	-6.42	-6.93
11.08	30.58	338.83
0.08	-1.42	-0.11
-3.92	-12.42	48.69
4.08	12.58	51.33
-13.92	-30.42	423.45
2.08	22.58	46.97
-8.92	-20.42	182.15
5.08	7.58	38.51
2.08	3.58	7.45
6.08	17.58	106.89
(Squared Total)	954	7400	1352.4
Average	47.7	370	122.9

$$\Sigma \approx \begin{bmatrix} 47.7 & 122.9 \\ 122.9 & 370 \end{bmatrix}$$

Variance of each variable = $\text{var}(X_i) = \sum_j (X_{ij} - \bar{X}_i)^2/(n-1)$

Total variance in the dataset = $\sum_i \text{var}(X_i) = 417.7$

รูป 8.9: หน้าคำนวณ covariance: ยืนยัน diagonal sums จาก raw data

อ่านภาพทีละจุด 1. ตัวเลข squared totals 954 และ 7400 ที่ปรากฏไม่สอดคล้องกับ raw 12 แถว. covariance สุดท้ายที่สไลด์ปัดเป็น 47.7 และ 370 ใกล้ค่าที่คำนวณใหม่

2. sample covariance หาร $n - 1 = 11$ ไม่ใช่ 12. cross-product 1352.416667 สอดคล้องกับค่าปัด 1352.4

Solve characteristic equation $\lambda^2 - \text{tr}(\Sigma)\lambda + \det \Sigma = 0$ ได้ eigenvalues $411.621854, 6.181176$ และเลือก normalized eigenvectors เป็น columns

$$P = \begin{bmatrix} 0.320082 & 0.947390 \\ 0.947390 & -0.320082 \end{bmatrix}, \quad Z = X_c P, \qquad PVE_1 = \frac{411.621854}{411.621854 + 6.181176} = 0.985206.$$

ตารางครบทุกตัวอย่าง: scores และ error เมื่อเก็บ PC1

ค่า $z_1 = x_c^T p_1$, $z_2 = x_c^T p_2$ และ $\hat{x} = \bar{x} + z_1 p_1$ Squared reconstruction error $\|x - \hat{x}\|^2 = z_2^2$ เพราะ orthonormal basis (ตัวเลขตารางปัด 3 ตำแหน่ง)

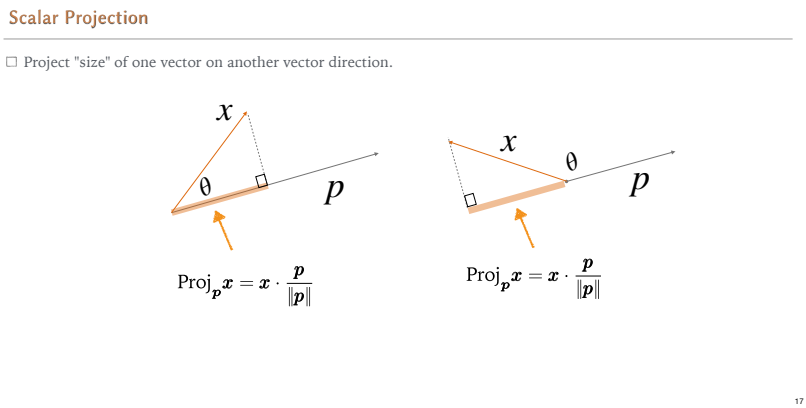
i	x_1	x_2	z_1	z_2	$\hat{x}_1, \hat{x}_2$	Error ^{2}
1	9	39	-23.758	2.837	$(6.312, 39.908)$	8.050
2	15	56	-5.732	3.080	$(12.082, 56.986)$	9.488
3	25	93	32.522	0.711	$(24.326, 93.228)$	0.506
4	14	61	-1.315	0.532	$(13.496, 61.170)$	0.283
5	10	50	-13.017	0.264	$(9.750, 50.084)$	0.070
6	18	75	13.228	-0.159	$(18.151, 74.949)$	0.025
7	0	32	-33.271	-3.449	$(3.267, 30.896)$	11.893
8	16	85	22.062	-5.255	$(20.978, 83.318)$	27.613
9	5	42	-22.197	-1.913	$(6.812, 41.388)$	3.658
10	19	70	8.811	2.389	$(16.737, 70.765)$	5.705
11	16	66	4.062	0.827	$(15.217, 66.265)$	0.684
12	20	80	18.605	0.135	$(19.872, 80.043)$	0.018

เช่นแถวแรก center $(-4.916667, -23.416667)$ ให้ $z_1 = -23.758447, z_2 = 2.837265$ Reconstruction เก็บ PC1 $\hat{x} \approx (6.312005, 39.908159)$ จึง residual ประมาณ $(2.687995, -0.908159)$ และ error 8.050070 ตรงกับ z_2^2 รวม 12 แถว error $= (n-1)\lambda_2 = 67.992937$ ไม่ใช่ λ_2 อย่างเดียว

ระวัง: หน้า 17 ใช้เครื่องหมายของ axes ต่างจากหน้า 10 ได้ แต่บรรทัด label eigenvalue ของ PC ที่สองต้องเป็น λ_2 ไม่ใช่ λ_1 ซ้ำ Mean และ cov และ eigenvalues ที่ปัดกลางทางอาจให้ scores ต่างเล็กน้อย เล่มนี้คำนวณจาก raw ก่อนปัดผล

8.12 Projection onto non-unit vector และความหมายของ score

แหล่งเนื้อหา: L8 PDF p.18–21



รูป 8.10: Dot product และ projection บนทิศทางที่กำหนด

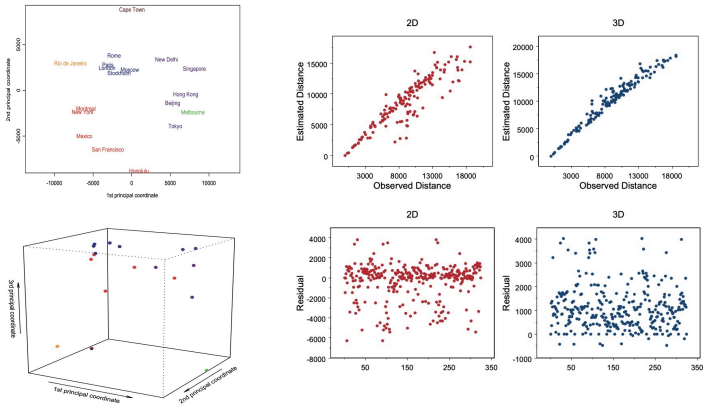
อ่านภาพทีละจุด **1.** Length ของ projection ที่มีเครื่องหมายคือ $x^T p/\|p\|$ เมื่อ p ไม่ unit ไม่ใช่ dot product โดยไม่ normalize **2.** Vector projection คือ $p(x^T p)/(p^T p)$. สำหรับ PCA unit axis ใช้ $z = x_c^T p$ และ projection vector zp ได้

ตัวอย่าง $x = (3, 1), p = (2, 1)$ มี dot 7,norm $\sqrt{5}$,signed scalar length $7/\sqrt{5}$, projected vector $(14/5, 7/5)$ อย่าให้ scalar 7 เป็นระยะยาวที่แท้จริงของ projection จากนั้นตรวจ residual $(0.2, -0.4)$ dot กับ p เป็น 0 แสดง orthogonality

8.13 อ่าน MDS plots, distances และ Swiss-roll comparisons

แหล่งเนื้อหา: L8 PDF p.24–38

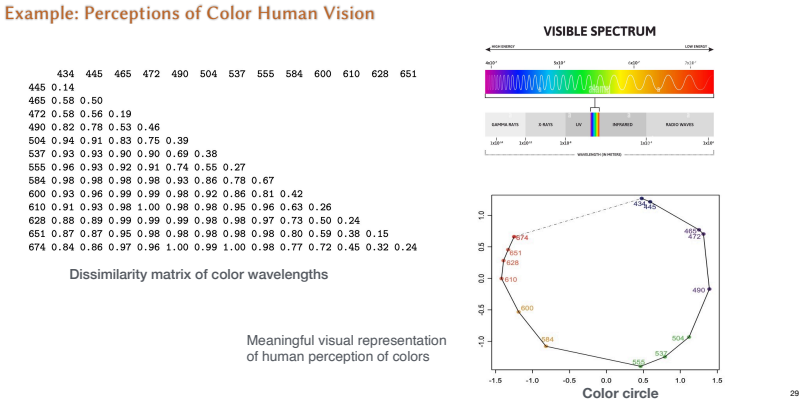
Distance ต้องเข้ากับข้อมูล: Euclidean $d_2 = \sqrt{\sum (x_j - z_j)^2}$,Manhattan $d_1 = \sum |x_j - z_j|$,Minkowski $d_p = (\sum |x_j - z_j|^p)^{1/p}$ สำหรับ $p \geq 1$. ordinal dissimilarities เน้นอันดับต่างจาก metric distances ที่มีขนาด meaningful ไม่ใช้รหัส category เป็น measurement โดยไม่มีเหตุผล



รูป 8.11: Airline distances และ Shepard plots ในหน้า 30

อ่านภาพทีละจุด 1. MDS จัด coordinates ให้ distances สัมพันธ์กับ observed dissimilarities. Shepard plot เทียบ dissimilarity ต้นทางกับ distance ใน embedding

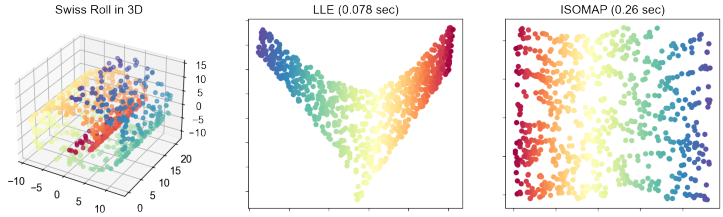
2. ตำแหน่งที่ใกล้เส้น และ ลำดับที่สอดคล้องช่วยวินิจฉัย fit แต่รูปสวยหรือ 3D โดยตัวมันเองไม่รับรอง generalization หรือ class accuracy



รูป 8.12: Perceptual color example: ปลายของ wavelength กับความใกล้เชิงการรับรู้

อ่านภาพทีละจุด 1. ตัวเลข wavelength เป็นแกนทางกายภาพ ไม่จำเป็นต้องมี distance เหมือน perceptual similarity

2. การ embed ตาม dissimilarities อาจได้โครงสร้างวนกลับใกล้กัน จึงไม่บังคับทุก manifold เป็น linear interval



รูป 8.13: Swiss-roll comparison: PCA, LLE, ISOMAP และวิธีอื่นในภาพ

อ่านภาพทีละจุด **1.** PCA ใช้ global linear variance. LLE รักษาการ reconstruct ด้วย neighbors. ISOMAP ใช้ shortest-path distances บน neighbor graph

2. runtime และรูปที่ได้ขึ้นกับ sample,neighbors,algorithm settings ไม่จำค่าความเร็วตัวอย่างเป็นอันดับตายตัวของทุก dataset

LLEcode ตัวอย่างใช้ neighbors 10 และ output components 2: neighbors กำหนด local reconstruction graph ส่วน components กำหนด embedding dimension ไม่ใช่คนละชื่อของจำนวน PCs ต้อง fit preprocessing และ embedding ตาม training protocol ถ้าใช้ต่อกับ prediction มิฉะนั้นข้อมูล test อาจมีส่วนเลือก representation โดยไม่ตั้งใจ

ต้องจำ PCA variance สูงไม่รับรองความสามารถ classify. MDS ใช้ dissimilarity,LLE ใช้ local weights,ISOMAP ใช้ geodesic graph distances รักษา neighbors และ geometry คนละความหมาย

8.14 LLE: หา weights และเห็นเหตุผลของ sum-to-one

แหล่งเนื้อหา: L8 PDF p.32–33. ตัวเลขฝึกเสริม

ให้จุดหนึ่ง $x_i = (1,0)$ และ neighbors $x_a = (0,0), x_b = (2,0)$ หา weights $w_a + w_b = 1$ และ minimize $\|x_i - w_a x_a - w_b x_b\|^2 = (1-2w_b)^2$ หา derivative $d(1-2w_b)^2/dw_b = 2(1-2w_b)(-2) = -4(1-2w_b)$ ตั้งเป็นศูนย์ได้ $1-2w_b = 0$ จึง $w_b = 0.5$ และ $w_a = 1-0.5 = 0.5$ แทนกลับ $w_a x_a + w_b x_b = 0.5(0,0) + 0.5(2,0) = (1,0) = x_i$ จึง squared reconstruction error 0 หาก embedding กำหนด neighbors $y_a = -1, y_b = 3$ เงื่อนไขรักษา weights ให้ $y_i = 0.5(-1) + 0.5(3) = 1$ ไม่ได้ copy coordinates x_i ตรง ๆ แต่รักษาวิธีประกอบจาก neighbors

ทำไม sum-to-one ทำให้ translation ไม่เปลี่ยน reconstruction error

เลื่อนทุก point ด้วย vector c:

$$(x_i + c) - \sum_j w_{ij}(x_j + c) = x_i - \sum_j w_{ij} x_j + c(1 - \sum_j w_{ij}).$$

เทอมสุดท้ายเป็น 0 เมื่อ ผลรวม weights เป็น 1 จึง error เดิม นอกจากนี้ weights ไม่จำเป็น nonnegative ใน standard LLE ทุกข้อ หาก point อยู่นอก convex hull ของ neighbors อาจต้องมี negative weight เพื่อ reconstruct

ระวัง: Embedding ทุก point เป็น 0 ทำให้ objective รักษา weights เป็น 0 ด้วย จึงต้องมี centering และ scale และ orthogonality constraints กัน collapse ไม่ถือว่า minimize objective เปล่าเพียงพอ

> **ต้องจำ** LLE ใช้ local reconstruction weights แล้วหา embedding ที่รักษา weights นั้น ต่างจาก PCA ที่ดู global variance และ ISOMAP ที่ดู geodesic distance

8.15 ISOMAP: จาก neighbor graph สู่ shortest-path matrix

แหล่งเนื้อหา: L8 PDF p.34–37. ตัวอย่างฝึกใหม่

สมมติ graph มี edges $A-B=1, B-C=2, C-D=1, A-D=10$ และไม่มี edges อื่น ค่า 10 อาจเป็น edge weight ในตัวอย่าง abstract graph ไม่จำเป็นเป็น Euclidean ของตำแหน่งที่กำหนด

1. ระยะเริ่มของ pairs ที่ไม่มี edge เป็น ∞ ไม่ใช่ 0. diagonal 0
2. $A-C$ ผ่าน B เป็น $1+2=3$. $B-D$ ผ่าน C เป็น $2+1=3$
3. $A-D$ เลือก min ของ edge ตรง 10 กับ path $A-B-C-D=4$ จึง shortest path 4

$$D_{geo} = \begin{bmatrix} 0 & 1 & 3 & 4 \\ 1 & 0 & 2 & 3 \\ 3 & 2 & 0 & 1 \\ 4 & 3 & 1 & 0 \end{bmatrix}.$$

นำ matrix นี้ไป MDS เช่น 1 D coordinates $(0,1,3,4)$ รักษา distances นี้ได้ ส่วน shortcut ผิดที่เพิ่ม $A-D=0.5$ จะทำให้ $A-D=0.5$ และ $A-C \le 0.5+1=1.5$ แทน 3 โครงสร้างเปลี่ยนหลายคู่ไม่ใช่คู่เดียว

> **ต้องจำ** ISOMAP ต้อง neighbor graph จากนั้น shortest paths จากนั้น MDS. neighbors มากเกินอาจลัด manifold น้อยเกินอาจ disconnect ไม่ตอบว่ามากที่สุดดีที่สุด

มอง PCA สองทางที่ให้คำตอบเดียวกัน

สไลด์ L8 p.9–23 เน้นเลือกแกนที่มี projected variance มากสุด อีกมุมหนึ่งคือเลือก r แกนให้ผลรวม squared reconstruction error บนข้อมูล training ที่ center แล้วต่ำสุด เพราะ eigenvectors เป็น orthonormal ถ้าเก็บ PC1 ของข้อมูลสองมิติที่มี sample eigenvalues $(9, 1)$ จะเก็บ $PVE = 9/(9+1) = 0.9$ และทิ้ง variance ของ PC2 เท่ากับ 1. สำหรับ n ตัวอย่าง ผลรวม squared reconstruction error เท่ากับ $(n-1)\lambda_2$ เมื่อใช้ sample covariance หาร $n-1$. อย่าสับสนกับค่า error ของตัวอย่างหนึ่ง หรือบอกว่า 90% variance แปลว่า 90% accuracy การกลับเครื่องหมายของ eigenvector พร้อม scores ไม่เปลี่ยน reconstruction

อ่านเพิ่มจากตำรา ISLP §12.2, printed p.509–510 (PDF p.515–516): PCA ในมุม low-rank approximation ที่ลด residual sum of squares เชื่อมสูตร score และ reconstruction ใน L8 p.20–23

8.16 Lecture 8: Pipeline และ Checklist

แหล่งเนื้อหา: L8 PDF p.1–38

PCA pipeline ที่ไม่ leak

แบ่ง train และ test จากนั้น หา mean และ scaler บน train จากนั้น fit PCA บน train แล้ว transform ด้วย basis เดิม แล้ว train downstream model แล้ว select r ด้วย validation แล้ว ประเมิน test แม้ PCA ไม่ใช้ label ก็ leak ได้ถ้า fit representation จาก test ที่ตั้งใจวัด performance ของข้อมูลใหม่

ประเด็น	PCA	Manifold methods
โครงสร้าง	global, linear subspace	อาจ nonlinear ผ่าน local และ distance
สิ่งรักษา	variance และ projection error	distances และ neighborhood relations ตาม method
การตีความ	loadings และ scores	coordinates อาจตีความ feature เดิมยากกว่า
failure	สัญญาณทำนายที่มี variance ต่ำ หาย	neighbors และ noise และ graph ไม่เหมาะ

Checklist

- แยก feature selection และ extraction และ loading และ score และ eigenvalue ได้
- Center ข้อมูลแล้วคำนวณ sample covariance ได้
- บอก orthogonality, variance order และ PC กับการเลือกเครื่องหมายได้สองแบบ ได้
- หา PVE และเลือก r จาก eigenvalues ได้
- Project และ reconstruct พร้อมคืน mean ได้
- แยก PCA จาก LDA ที่ใช้ labels ได้
- คำนวณ Euclidean และ Manhattan และอธิบาย proximity matrix ได้
- เปรียบเทียบ MDS และ LLE และ ISOMAP และ ข้อผิดพลาดจาก neighbors ได้

> **ต้องจำ** ตัวแปรที่ uncorrelated ไม่ได้แปลว่า independent เสมอไป โดยทั่วไปแล้ว component ที่มี variance สูงกว่าไม่ได้ทำนาย target ได้ดีกว่าเสมอ และการลดมิติไม่ได้ทำให้ทุกปัญหาง่ายขึ้น

อ่านเพิ่มจากตำรา อ่านต่อ ISLP §12.2, Géron Ch.8. ไม่มี Lecture 9 จึงไม่เพิ่ม clustering algorithm จาก textbook นอกขอบเขต L1–8 ที่แจ้ง

เมื่อเงื่อนไขเปลี่ยน ให้คาดผลของโมเดลอย่างไร

โจทย์ประยุกต์มักเปลี่ยนข้อมูล ต้นทุน หรือค่าตั้งของโมเดล แล้วถามว่าคำตอบจะเปลี่ยนไปทางใด ให้อ่านโจทย์เป็นลำดับ: **อะไรเปลี่ยน** → **อะไรคงเดิม** → **ตัวแปรใดในสมการได้รับผล** → **ผลที่คาดและข้อจำกัด**. ตารางนี้สรุปเหตุและผลที่ควรอธิบายได้ ไม่ใช่กฎที่รับประกันผลกับทุกชุดข้อมูล

บท	เปลี่ยนเงื่อนไข	ผลที่คาด	เหตุผลและสิ่งที่ต้องระวัง
1	สัดส่วนผู้ป่วยในประชากรลดลง แต่ความไวและอัตราผลบวกลวงของชุดตรวจคงเดิม	โอกาสเป็นโรคหลังผลตรวจบวกมักลดลง	Bayes ใช้ prevalence เป็น prior; อย่าสับสน sensitivity กับ posterior และ ตรวจ ต้นทุน ก่อน ตั้ง threshold
2	เพิ่ม learning rate โดยคงจุดเริ่มต้นและ gradient ไว้	แต่ละ update ก้าวไกลขึ้น	อาจเข้าใกล้คำตอบเร็วขึ้น หรือข้ามจุดต่ำสุดจนแกว่งได้ จึงต้องคำนวณ gradient ใหม่หลัง update
3	เพิ่ม C ใน soft-margin SVM	โมเดลลงโทษ margin violation มากขึ้น และมักยอมให้ training violation น้อยลง	ผล จริง ขึ้น กับ ข้อมูล และ optimum; ต้องแยก C ออกจาก γ และปรับ scale ของ features ก่อนเทียบ kernel
4	เพิ่ม λ ใน Ridge	ค่าสัมประสิทธิ์ที่ถูกลงโทษโดยทั่วไปหดเข้าหาศูนย์	bias มัก เพิ่ม และ variance มัก ลด แต่ไม่รับรองว่าทุก prediction จะลดลง; ตรวจว่า intercept ถูกลงโทษหรือไม่
5	เพิ่มความลึกสูงสุดของต้นไม้เดี่ยว	training fit มัก ดี ขึ้น แต่ ความแปรปรวนอาจสูงขึ้น	การเพิ่มความลึกเปิดทางให้ split มากขึ้น; ใน Random Forest การลดจำนวน features ต่อ split ลดความ คล้าย กัน ของ ต้นไม้ได้ แต่อาจเพิ่ม bias
6	ฝึกต่อหลัง validation loss ผ่านจุดต่ำสุด	training loss อาจลด แต่ผลกับข้อมูลใหม่อาจแย่ลง	เลือก checkpoint จาก validation ตามเกณฑ์ที่กำหนด แล้วใช้ test ประเมินเพียงครั้งเดียว
7	เพิ่ม stride ของ convolution โดยคง input, kernel และจำนวน filters	ความสูงและความกว้างของ output มักลดลง	ใช้ floor ใน สูตร output size; จำนวน channels ยัง เท่ากับ จำนวน filters และ จำนวน weights ของ kernel ไม่ได้ คูณ จำนวนตำแหน่ง
8	ลดจำนวน principal components ที่เก็บไว้	บีบอัดมากขึ้น แต่ PVE ไม่เพิ่มและ reconstruction error บน training set ไม่ลด	PCA รักษา ความ แปรปรวน ไม่ได้ รับ ประกัน accuracy; ต้องเลือกจำนวนแกนจาก validation ภายใน training fold

ต้องจำ อย่าตอบแค่ “เพิ่มแล้วดีขึ้น” ให้บอกทิศทางที่คาด กลไกในสูตร และเงื่อนไขที่ทำให้ผลนั้นอาจไม่เกิด

ฝึกให้ทำได้ เมื่ออ่านโจทย์ ให้ทำเครื่องหมายสิ่งที่เปลี่ยนหนึ่งอย่างและสิ่งที่คงเดิม จากนั้นคาดทิศทางก่อนแทนตัวเลข วิธีนี้ช่วยแยกความสัมพันธ์เชิงสาเหตุออกจากตัวเลือกที่ชวนเดา

แผนที่ตัวแปรและเงื่อนไขของสมการ Lecture 1–8

อ่านหน้านี้คู่กับการพิสูจน์และตัวอย่างในแต่ละบท: สัญลักษณ์ตัวเดียวอาจใช้ต่างหน้าที่ในคนละ Lecture โดยเฉพาะ n (จำนวนตัวอย่าง และ จำนวน features), p (ความน่าจะเป็น และ จำนวน features), b (intercept และ ขนาด mini-batch) และ C (penalty ของ SVM และ ตัวแปร censoring) ต้องดูนิยามของโจทย์ก่อนแทนค่า

8.17 Lecture 1: การเรียนรู้, Bayes และตัววัดผล

แหล่งเนื้อหา: L1 PDF p.15, 36–40, 49–58, 69–87

ใน $Y = f(X) + \varepsilon$, $X \in \mathbb{R}^d$ เป็นเวกเตอร์ predictors ของหนึ่ง observation, Y คือค่าจริง, f เป็นความสัมพันธ์ที่ยังไม่ทราบ, ε คือ random error ซึ่งสมมติ mean เป็นศูนย์เมื่ออธิบาย decomposition ในตัวอย่าง Experience หน้า 39 ใช้ m เป็นจำนวนแถว, $X \in \mathbb{R}^{m \times d}$ เป็น design matrix, $w \in \mathbb{R}^d$ เป็น weights, $y \in \mathbb{R}^m$ เป็น targets, $r = Xw - y$ เป็น residual ที่ใช้ใน gradient, $J(w) = r^T r/m$ เป็น training MSE หากมี intercept ต้องนับคอลัมน์ 1 รวมใน d. ผล $w^* = (X^T X)^{-1} X^T y$ ต้องมี full column rank และ $m \geq d$. Gradient ศูนย์หมายถึง $X^T r = 0$ ไม่ได้แปลว่า $r = 0$. อีกวิธีดูขนาดคือ $X^T r$ เป็น $d \times 1$ จึงนำไปลบจาก w ได้

หน้า 40 เปรียบเทียบ norm: สำหรับ $u = (u_1, \ldots, u_d)$, $\|u\|_1 = \sum_j |u_j|$ เป็นผลรวมค่าสัมบูรณ์ (Manhattan), $\|u\|_2 = \sqrt{\sum_j u_j^2}$ เป็นระยะ Euclidean จากศูนย์ และ $\|u\|_2^2 = \sum_j u_j^2$ เป็น squared norm ที่ใช้ใน least squares เช่น $u = (3, 4)$ ให้ $\|u\|_1 = 7$, $\|u\|_2 = 5$, $\|u\|_2^2 = 25$. อย่าแทน 5 ในสูตร $r^T r$.

Bayes: $P(D \mid +)$ คือ posterior โรคเมื่อผลบวก, $P(+ \mid D)$ คือ sensitivity, $P(D)$ คือ prior และ $P(+)$ รวมทั้ง true และ false positives. LDA score $\delta_k(x)$ เปรียบเทียบ class k โดยใช้ prior π_k, mean μ_k และ covariance ร่วม Σ. ต้องมี covariance ที่ inverse ได้ตามรูปสูตรนั้น ส่วน QDA อนุญาต Σ_k แยก class. Confusion matrix ใช้ TP, FP, FN, TN โดย **positive** ต้องนิยามจากโจทย์ก่อน. precision $TP/(TP + FP)$ และ recall $TP/(TP + FN)$ มีตัวส่วนคนละกลุ่ม

> **ต้องจำ** แยก $P(A \mid B)$ จาก $P(B \mid A)$, แยก norm กับ squared norm และเขียนขนาด X, w, y ก่อนทำ matrix algebra

8.18 Lecture 2: OLS, GD และ logistic

แหล่งเนื้อหา: L2 PDF p.4–16, 25–38

สไลด์บางหน้าใช้ n แทนจำนวน feature แต่ในการรวม error ใช้จำนวนแถวต่างหาก เล่มนี้กำหนด m แถว, d coefficients (รวม bias ถ้ามี). สำหรับ $J = \frac{1}{m} \sum_i (y_i - x_i^T w)^2$, $\nabla_w J = \frac{2}{m} X^T (Xw - y)$. ถ้าเปลี่ยนเป็น $\frac{1}{2m} \sum_i (\cdots)^2$, factor 2 หายแต่ minimizer OLS เดิมเมื่อไม่มี penalty. $\eta > 0$ ใน $w_{t+1} = w_t - \eta g_t$ คือ learning rate, t คือรอบ update และ g_t ต้องประเมินจาก w_t. Batch ใช้ m แถวต่อ update, stochastic ใช้หนึ่งแถว และ mini-batch ใช้ b แถว. b นี้ไม่ใช่ intercept

Logistic กำหนด $z_i = b + w^T x_i$ เป็น logit, $p_i = \sigma(z_i)$ เป็น probability ที่ทำนาย class 1, $y_i \in \{0, 1\}$ เป็น label. Mean binary cross-entropy ให้ $\partial J/\partial b = m^{-1} \sum_i (p_i - y_i)$ และ $\nabla_w J = m^{-1} \sum_i (p_i - y_i) x_i$ เมื่อ sigmoid อยู่คู่ BCE อย่าคูณ $p_i(1 - p_i)$ ซ้ำ หาก objective เป็นผลรวมแทน mean จะไม่มี $1/m$ ใน gradient ปรับทุก parameters พร้อมกันจากค่าก่อน update

> **ต้องจำ** ก่อนแทนสูตร ระบุ SSE และ MSE และ half-MSE และว่ารวม bias ใน X หรือแยก b. ทิศ update คือ $-\eta \nabla J$

8.19 Lecture 3: ระยะ, slack, dual และ kernel

แหล่งเนื้อหา: L3 PDF p.8–16, 19–34

ให้ label $t_i \in \{-1, +1\}$ และ $f(x_i) = w^T x_i + b$. ค่า $t_i f(x_i)$ เป็น signed functional score. $|f(x_i)|/\|w\|_2$ เป็น Euclidean distance ถึง decision boundary. Hard margin ต้อง $t_i f(x_i) \geq 1$ ทุกแถว. full margin width เท่ากับ $2/\|w\|_2$. Soft margin ใช้ $\xi_i \geq \max(0, 1 - t_i f(x_i))$ และ minimize $\frac{1}{2}\|w\|_2^2 + C\sum_i \xi_i$, โดย $C > 0$ เป็นโทษรวมของการละเมิด margin ไม่ใช่ความกว้าง margin โดยตรง. ξ_i ไม่มีหน่วยระยะทาง Euclidean จุดที่ $0 < t_i f(x_i) < 1$ ทายถูกแต่ละเมิด margin

Dual ใช้ $\alpha_i \in [0, C]$ เป็น multiplier รายแถว, $\sum_i \alpha_i t_i = 0$ เมื่อ bias ไม่ถูก penalize ค่า $0 < \alpha_i < C$ เป็น free support vector ที่ $t_i f(x_i) = 1$ สำหรับหา b. อย่าถือว่าทุก $\alpha_i = C$ ทายผิด. Kernel $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ คือ inner product ใน feature space, ส่วน $K(x, z) = e^{-\gamma\|x-z\|^2}$ ของ RBF ใช้ $\gamma > 0$ คุมความเฉพาะที่ ไม่ใช่ C. Scale features ก่อนเลือก hyperparameters ภายใน training fold

> **ต้องจำ** เวลาถาม distance ให้หาร $\|w\|$. เวลาถาม slack ให้เทียบกับ signed score 1. ทั้งสองไม่ใช่ตัวเลขเดียวกัน

8.20 Lecture 4: penalty, rate และ censoring

แหล่งเนื้อหา: L4 PDF p.5–16, 20–25, 26–43

Ridge $RSS + \lambda\sum_{j=1}^{d} w_j^2$ และ Lasso $RSS + \lambda\sum_{j=1}^{d} |w_j|$ ใช้ $\lambda \geq 0$ คุม penalty. convention ที่ไม่ลงโทษ intercept w_0 ต้องระบุ ถ้าใช้ $MSE + \lambda\, penalty$ ด้วยค่า λ เดิม ผล optimum อาจเปลี่ยน เพราะ relative scale ของ data loss เปลี่ยน. Interaction $w_{12}x_1x_2$ ทำให้ผลของ x_1 ต่อ prediction ขึ้นกับ x_2 แต่ model ยัง linear ใน coefficients

Poisson $Y_i \mid x_i \sim \text{Poisson}(\mu_i)$ โดย $Y_i \in \{0, 1, \ldots\}$, $\mu_i = E(Y_i \mid x_i) > 0$, $\log \mu_i = \beta_0 + x_i^T\beta$. e^{β_j} เป็นตัวคูณของ mean count เมื่อ x_j เพิ่มหนึ่งหน่วยและตัวอื่นคงที่. Survival ให้ T เป็นเวลา event ที่แท้จริง, C เป็นเวลาหยุดติดตาม, $U = \min(T, C)$ เป็นเวลาที่สังเกต, $\delta = I(T \leq C)$ ชี้ event. $S(t) = P(T > t)$, $f(t) = -S'(t)$, $h(t) = f(t)/S(t)$ เป็น instantaneous rate ไม่ใช่ probability ใน KM n_j คือจำนวนที่ยังเสี่ยง ก่อน เวลา event t_j, d_j คือจำนวน events ณ เวลานั้น, $\hat{S}(t) = \prod_{t_j \leq t}(1 - d_j/n_j)$. censoring ไม่เพิ่ม d_j แต่ลด risk set ของเวลาถัดไป หาก event และ censor เวลาเดียวกันต้องระบุ convention

> **ต้องจำ** แยก T, C, U, δ ก่อนตาราง KM แยก μ ค่าเฉลี่ยนับครั้งจาก probability และแยก λ penalty จาก coefficients

8.21 Lecture 5: impurity, sampling และ sequential updates

แหล่งเนื้อหา: L5 PDF p.13–22, 47–52, 58–76

ใน node S, $n = |S|$, n_k คือจำนวน class k, $p_k = n_k/n$, $H(S) = -\sum_k p_k \log_2 p_k$ (ให้ $0 \log 0 = 0$), $G(S) = 1 - \sum_k p_k^2$. ถ้าแบ่งเป็น child S_c, $IG = H(S) - \sum_c (|S_c|/|S|) H(S_c)$ ต้องถ่วงตาม จำนวนแถว ของ child. Regression tree ใช้ $\bar{y}_c$ ของแต่ละ leaf เพื่อให้ $\sum_{i \in c}(y_i - \bar{y}_c)^2$ ต่ำสุด

Bagging ใช้ B ต้นไม้จาก bootstrap samples แบบใส่คืน, $\hat{f}_b$ คือ prediction ต้นที่ b, $\hat{f}_{bag} = B^{-1}\sum_b \hat{f}_b$ สำหรับ regression. OOB ของแถวหนึ่งใช้เฉพาะต้นที่ไม่มีแถวนั้นใน bootstrap ของต้นนั้น. Random Forest เพิ่มการสุ่ม candidate features ในแต่ละ node. m_{try} ไม่ใช่จำนวนตัวอย่าง. AdaBoost ใช้ sample distribution $D_t(i)$ ในการหา weighted error e_t ของ learner h_t และใช้ α_t ถ่วง vote ตัวเลขสองชนิดนี้ต่างหน้าที่. GBM ใช้ ensemble F_{t-1} หา negative gradient r_{it} ตาม prediction, fit h_t, แล้ว $F_t = F_{t-1} + \nu h_t$ โดย ν เป็น shrinkage สำหรับ half-squared loss $r = y - F$, หากใช้ squared loss ที่ไม่มีครึ่ง derivative มี factor 2

ต้องจำ อย่าเฉลี่ย child แบบไม่ถ่วงจำนวน อย่าใช้ residual รอบเก่าใน boosting รอบใหม่. XGBoost learning rate และ rounds และ depth ไม่ใช่ค่าเดียวที่ถูกทุกโจทย์

8.22 Lecture 6: layer index, gradient และตัวนับรอบ

แหล่งเนื้อหา: L6 PDF p.5–18, 20–42, 55–76

สำหรับ layer ℓ, $a^{(0)} = x$, $z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}$, $a^{(\ell)} = g^{(\ell)}(z^{(\ell)})$. ถ้า layer ก่อนมี $d_{\ell-1}$ units และ layer นี้มี d_ℓ units, $W^{(\ell)}$ มีขนาด $d_\ell \times d_{\ell-1}$, $b^{(\ell)}$ ยาว d_ℓ. parameters รวม $(d_{\ell-1}+1)d_\ell$. ใน scalar backprop, $\delta^{(\ell)} = \partial L/\partial z^{(\ell)}$ ไม่ใช่ $\partial L/\partial a$. gradient ต่อ weight เป็น $\delta^{(\ell)}(a^{(\ell-1)})^T$ และต้องใช้ weights ก่อน update ในการส่ง δ กลับ

ข้อมูล n แถว batch size b ใช้ $\lceil n/b \rceil$ updates ต่อ epoch เมื่อเก็บ batch สุดท้ายที่ไม่เต็ม. epoch หมายถึงเห็น training set หนึ่งรอบ ไม่ใช่หนึ่ง update เสมอ. BCE แบบ natural log ให้ $\delta = p - y$ สำหรับ sigmoid output การใช้ $\log_{10}$ จะคูณ gradient ด้วย $1/\ln 10$ เมื่อไม่ได้เปลี่ยน scaling อื่น. BatchNorm ต่อ channel และ unit มี γ, β ที่ train และ moving mean และ variance ที่เก็บค่า แต่ dropout ไม่มี learned weights และระหว่าง inference ต้องใช้โหมดที่ถูกต้อง

ต้องจำ นับ weight matrix จาก input units ของ layer ก่อนคูณ output units ของ layer นี้ แล้วบวก biases ทุก gradient มาจาก forward pass ชุดเดียวกัน

8.23 Lecture 7: shape, channels, sharing และ pooling

แหล่งเนื้อหา: L7 PDF p.19–46, 60–64

ให้ input $H \times W \times C_{in}$, kernel สูง และ กว้าง K_h, K_w, padding สองข้าง P_h, P_w, stride $S_h, S_w > 0$, filters C_{out}. Output height คือ $\lfloor (H + 2P_h - K_h)/S_h \rfloor + 1$ และ width ใช้ W, P_w, K_w, S_w ในสูตรเดียวกัน ต้องมีช่องวาง kernel ได้อย่างน้อยหนึ่งตำแหน่ง. Standard convolution มี parameters $(K_h K_w C_{in} + 1)C_{out}$ เมื่อหนึ่ง bias ต่อ filter จำนวนนี้ไม่คูณ output height และ width เพราะ weight sharing จำนวน activations กลับขึ้นกับ spatial size สำหรับ max pooling แต่ละ window ส่ง gradient กลับที่ตำแหน่ง max หาก windows ซ้อนกัน gradients ที่ input เดียวต้อง บวก กัน และถ้ามี tie ต้องระบุ convention ในเล่มนี้ใช้ cross-correlation ไม่ flip kernel

ตัวอย่าง input $5 \times 5 \times 3$, filter 3×3, valid, stride 2, filters 4 ให้ spatial $\lfloor (5-3)/2 \rfloor + 1 = 2$, output $2 \times 2 \times 4$ และ parameters $(3 \cdot 3 \cdot 3 + 1)4 = 112$. ResNet addition ต้องมี shapes ที่บวกกันได้. Inception concat ต้องมี spatial sizes ตรงกันก่อนต่อ channels

ต้องจำ shape, จำนวน activations และจำนวน parameters เป็นคนละคำตอบ ตรวจ padding และ stride ของแต่ละแกนและจำนวน input channels ก่อนคูณ

8.24 Lecture 8: covariance, score, reconstruction และ neighbors

แหล่งเนื้อหา: L8 PDF p.9–23, 24–38

กำหนด $X \in \mathbb{R}^{n \times d}$ เป็น training rows, $\bar{x} \in \mathbb{R}^d$ คือ mean ราย column, $X_c = X - \mathbf{1}\bar{x}^T$, $\Sigma = X_c^T X_c/(n-1)$ เป็น sample covariance สำหรับ $n > 1$. Eigenvectors unit ที่เรียงตาม eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ เป็น columns ของ P. เลือก $r \leq d$, $P_r \in \mathbb{R}^{d \times r}$. scores $Z_r = X_c P_r \in \mathbb{R}^{n \times r}$ และ reconstruction $\hat{X} = Z_r P_r^T + \mathbf{1}\bar{x}^T$. สำหรับจุดใหม่ที่เป็น row vector $x \in \mathbb{R}^{1 \times d}$ ใช้ training $\bar{x}, P_r$ เดิม, $z = (x - \bar{x}^T)P_r \in \mathbb{R}^{1 \times r}$ และ $\hat{x} = \bar{x}^T + zP_r^T$. $PVE_r = \sum_{j \leq r} \lambda_j / \sum_{j \leq d} \lambda_j$ ต้องมี total variance มากกว่า 0. eigenvector กลับเครื่องหมายได้หาก score กลับด้วย

MDS รับ dissimilarities δ_{ij} ของคู่ตัวอย่างแล้วจัด low-dimensional coordinates ให้สอดคล้องกับระยะ ไม่จำเป็นต้องมี feature matrix เดิม. LLE ใช้ w_{ij} เฉพาะ neighbors ของ i และ $\sum_j w_{ij} = 1$ เพื่อ reconstruct จุดใน neighborhood ก่อนหา coordinates. w_{ij} ไม่ใช่ class probability. ISOMAP ประมาณ geodesic ด้วย shortest paths บน neighbor graph ถ้ามี edge ข้ามแผ่น Swiss roll ระยะอาจ shortcut. k จำนวน neighbors กับ r มิติ embedding เป็นคนละ hyperparameter

ต้องจำ center ด้วย training mean ทุกครั้ง, หาร $n-1$ เมื่อใช้ sample covariance, ใช้ P_r^T คืนมิติและบวก mean. PVE บน training ไม่รับรอง accuracy หรือ reconstruction ของจุดใหม่

ดัชนีเทียบต้นฉบับ Lecture 1–8 กับหัวข้ออ่าน

เลขหน้าด้านซ้ายเป็น**หน้า PDF ต้นฉบับ** ไม่ใช่เลข slide พิมพ์ท้ายภาพ ใช้ตารางนี้เปิดตรวจเนื้อหากลับไปกลับมา หน้า topic ด้านขวาพาไปคำอธิบายหลักของช่วงนั้น หัวข้อที่เกี่ยวข้องอาจต่อเนื่องอีกหลายหน้า ภาพต่อเนื่องของกระบวนการจัดรวมพร้อมคำอธิบาย ไม่ถือว่าภาพที่หน้าตาคล้ายกันเป็นเหตุให้ข้ามขั้นสมการ ตรวจต้นฉบับ 471 หน้าจากทั้งข้อความและภาพ overview และเปิดภาพรายละเอียดเพิ่มเติมสำหรับสมการ และ code และตัวอย่างสำคัญ รายการรายหน้า 471 แถวอยู่ใน source bundle พร้อม title ที่สกัดได้และตำแหน่งหัวข้อ ไม่ได้ใช้คำว่า “ตรวจแล้ว” แทนการระบุว่าหน้านั้นสอนอะไร หน้า administrative และ assignment แยกจากเนื้อหาที่ต้องจำ

Lecture 1 — 89 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–8	ข้อมูลรายวิชาและตัวอย่างการใช้ ML	2
9–13	นิยาม ML, AI และ ML และ DL, T และ P และ E	2
14–19	Statistical learning, prediction และ inference	4
20–30	ประเภทการเรียนรู้และ tasks พร้อมภาพ	2
31–35	Performance, regression metrics, evaluation	2
36–41	Linear task, performance, Experience derivation	7
42–43	Underfit และ overfit	10
44–48	Bayes classifier และ decision boundary	16
49–54	Probability ก่อน Bayes	11
55–58	โจทย์ rare disease ต้นฉบับ	12
59–68	Spam และ Naive Bayes และ smoothing	12
69–77	LDA,Gaussian scores และ priors	14
78–81	Covariance contours,LDA และ QDA boundary	16
82–87	Confusion,metrics,ROC,ตัวอย่าง corn	17
88–89	Assignment และ ปิดบท	1

Lecture 2 — 40 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–2	บันทึกแนวสอบรอบเดิมและปก	i
3–6	Linear regression model,residual	21
7–12	Normal equation,Cramer,coefficients	22
13–16	Matrix OLS และตัวอย่าง multiple regression	23
17–19	GD และ nonlinear least-squares,stationary point	32
20–30	GD ภาพต่อเนื่องและขั้นตอน	32
31–33	Batch และ SGD และ minibatch,learning rate	27
34–39	Logistic probability,BCE gradient,boundary	32
40	ปิดบท	21

Lecture 3 — 35 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–3	ปก และ overview และ SVM concept	35
4–7	Hyperplane,large margin,scaling	38
8–11	Distance และ hardmargin objective	36
12–15	Soft margin,slack,C	37
16	Quadraticprogramform	44
17–18	ตัวอย่าง Lagrange ต้นฉบับ	39
19–24	Primal และ dual,KKT และ support vectors	44
25–33	Nonlinear features และ kernels,explicitmap	45
34	Hingeloss	38
35	ปิดบท	35

Lecture 4 — 44 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–4	ปกและ linear และ nonlinear regression	47
5	Interaction	47
6–12	Shrinkage,Ridge และ Lasso และ Elastic Net	49
13–16	Embedded selection ด้วย L1 logistic	58
17–22	GLM และ Poisson distribution และ link	51
23–25	Likelihood,coefficient,mineexample	59
26–33	Survival,time-to-event,censoring	53
34–37	S,f,h และ cumulative hazard	54
38–39	Censored records และ event indicator	59
40–43	KM และ risk set และแก้เลขตัวอย่าง 26 คน	59
44	ปิดบท	47

Lecture 5 — 77 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–2	แนวสอบรอบเดิม และ ปก	i
3–12	Decision tree และ Claudio และ ตัวอย่าง Write-off	76
13–20	Entropy และ IG และ counts ต้นฉบับ	64
21–22	Gini และ index	65
23–27	Recursive induction และ branch thresholds	66
28–30	Tree classification และ evaluation และ ROC	17
31–37	Bagging และ random forest และ bias และ variance	69
38–46	เดิน tree และรวม regression และ classification trees	76
47–48	Bootstrap และ OOB	69
49–52	Proximity และ outliers และ missing และ variable importance	77
53–57	RFcode และ shape และ parameters และ AUC	77
58–61	Boosting และ AdaBoost	71
62–68	GBM และ residual และ code	78
69–76	XGBoost และ DMatrix และ rounds และ eta และ evaluation	78
77	ปิดบท	63

Lecture 6 — 78 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–4	ปก และ intro และ neuron context	80
5–8	Neuron และ sigmoid และ ANDquiz	89
9–13	Loan example และ feature vector และ weighted input	89
14–19	MLP และ bias nodes และ โจทย์นับ weights	89
20–33	Forward และ backprop และ gradient และ learning rate	83
34–41	Epoch และ batch และ tensors และ ndim	90
42	BCEcost และฐาน log ในตาราง	90
43–44	Dense และ activation และ output	82
45–50	Prepare และ compile และ fit และแก้ output-losspair	90
51–54	DNN และ depth และ architecture	81
55–57	Activation รวม Leaky ReLU และ ELU	92
58–60	Initialization และ BN และ pretraining	92
61–66	Momentum และ NAG และ AdaGrad และ RMSProp และ Adam	92
67–76	Regularization และ earlystop และ dropout	88
77–78	Code และ training parameters และ review	85

Lecture 7 — 69 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–3	ปกและ CNN และ แนวคิด weight sharing	95
4–14	Filter pattern และ เรียน weights,dot products	104
15–18	Pixels และ CNN structure	95
19–31	Convolution pipeline,Xdetector และ flatten และ scores	104
32–37	Conv layer และ shape และ padding และ stride	97
38–39	Stride 2 ทุก window,output 3x 3	104
40–42	Pooling และ average exact versus rounded	106
43–46	Channels และ parameter sharing และ shape	98
47–51	LeNet และ AlexNet และ Inception และ ResNet	106
52–59	DevelopCNN และ ขั้นตอน pretrained model	108
60–62	CNNcode และ summary และ BN parameters counts	107
63–64	Augmentation และ workflow ของ pretrained VGG	108
65–69	Evaluation และ ROC และ assignment	109

Lecture 8 — 39 หน้าต้นฉบับ

หน้า PDF	เนื้อหา และ หัวข้อที่ใช้ตรวจ	หน้าอ่านหลัก
1–3	ภาพรวม unsupervised และ ปก	111
4–8	Dimensionality และ PCA และ center และ variance	112
9–17	PCA จากข้อมูลดิบ 12 แถว,cov และ eigen และ scores,แก้ numbers	121
18–21	Projection และ score และ reconstruction	122
22–23	PC variance และ representation ที่ลดมิติแล้ว	114
24–31	MDS และ distances และ Shepard และ airline และ colors	123
32–33	LLE neighbors และ weights และ eigendecomposition	120
34–38	LLEcode และ Swiss roll และ การเปรียบเทียบ ISOMAP	123
39	Lab และ ปิดบท	111